

```
===== Code/API/__init__.py =====
```

```
"""API package exports."""
```

```
from __future__ import annotations
```

```
from .server import app, run_backend
```

```
__all__ = ["app", "run_backend"]
```

```
===== Code/API/app.py =====
```

```
from __future__ import annotations
```

```
from litestar import Litestar, get
```

```
from litestar.config.cors import CORSConfig
```

```
from litestar.openapi.config import OpenAPIConfig
```

```
from litestar.openapi.plugins import SwaggerRenderPlugin
```

```
from api.dependencies import provide_config, provide_db_session, provide_duckdb
```

```
from api.exceptions import global_exception_handler
```

```
from api.middleware import CorrelationAndDeprecationMiddleware
```

```
from api.routes.analytics import AnalyticsController
```

```
from api.routes.exports import ExportController
```

```
from api.routes.health import HealthController
```

```
from api.routes.invoices import InvoiceController
```

```
from api.routes.tasks import TaskController
```

```
from api.routes.triage import TriageController
```

```
from api.routes.ws import progress_websocket
```

```
from api.static import get_static_config
```

```
@get("/api/v2/health")
```

```
async def health_v2() -> dict[str, str]:
```

```
    return {"api": "OK", "version": "v2"}
```

```
def create_app() -> Litestar:
```

```
    """Single official backend bootstrap point."""
```

```
    config = provide_config()
```

```
    return Litestar(
```

```
        route_handlers=[
```

```
            HealthController,
```

```
            health_v2,
```

```
            InvoiceController,
```

```
            AnalyticsController,
```

```
            TaskController,
```

```
            TriageController,
```

```
            ExportController,
```

```
            progress_websocket,
```

```
        ],
```

```
        dependencies={
```

```
            "config": provide_config,
```

```
            "db_session": provide_db_session,
```

```
            "duckdb": provide_duckdb,
```

```
        },
```

```
        exception_handlers={Exception: global_exception_handler},
```

```
        middleware=[CorrelationAndDeprecationMiddleware],
```

```
        cors_config=CORSConfig(allow_origins=["*"], allow_methods=["*"], allow_headers=["*"]),
```

```
        openapi_config=OpenAPIConfig(
```

```
            title="Nexus AI API",
```

```
            version="2.0.0",
```

```
            description="API lifecycle: /api/v1 (deprecated) and /api/v2 (current)",
```

```
            render_plugins=[SwaggerRenderPlugin()],
```

```
        ),
```

```
        static_files_config=get_static_config(config),
```

```

        debug=config.debug,
    )

===== Code/API/auth.py =====
from datetime import datetime, timedelta, timezone
from litestar.connection import ASGIConnection
from litestar.exceptions import NotAuthorizedException
from litestar.security.jwt import JWTAuth, Token
from pydantic import BaseModel
from core.config import AppConfig

config = AppConfig()

class User(BaseModel):
    """Reprezentacja zalogowanego użytkownika (na zapas)."""
    id: str
    username: str
    role: str

async def retrieve_user_handler(token: Token, connection: ASGIConnection) -> User | None:
    """Weryfikuje token i pobiera użytkownika z bazy (Mock dla uproszczenia)."""
    if not token.sub:
        raise NotAuthorizedException()

    # Tutaj w przyszłości dodasz zapytanie do bazy: SELECT * FROM users WHERE id = token.sub
    return User(id=token.sub, username="admin", role="ADMIN")

# Gotowy mechanizm JWT do wpięcia w app.py
jwt_auth = JWTAuth[User](
    retrieve_user_handler=retrieve_user_handler,
    token_secret=config.encryption_key,
    exclude=["/health"]
)

===== Code/API/controllers/__init__.py =====
"""API controllers package."""
from __future__ import annotations

from .invoices import InvoiceController
from .analytics import AnalyticsController

__all__ = [
    "InvoiceController",
    "AnalyticsController",
]

===== Code/API/controllers/analytics.py =====
from litestar import Controller, post, get
from typing import Any
from api.schemas import AnalyticsQuery
from db.analytics import DuckDBManager # Twój manager DuckDB z poprzednich plików

class AnalyticsController(Controller):
    path = "/api/analytics"

    @post("/cashflow")
    async def get_cashflow_report(self, data: AnalyticsQuery) -> dict:
        """Pobiera bieżące raporty finansowe z DuckDB."""
        # DuckDB limits and threads are already configured in DuckDBManager
        manager = DuckDBManager(db_path="nexus_olap.duckdb")

        query = f"""
        SELECT date_trunc('{data.dimension}', issue_date) as period,
               SUM(amount_gross) as total_gross
        FROM invoices_replica
        """

```

```

WHERE issue_date BETWEEN ? AND ?
GROUP BY period ORDER BY period;
"""
results = manager.execute(query, (data.start_date, data.end_date))
manager.close()

try:
    if not results:
        return {"total_gross": 0.0, "count": 0, "avg_net": 0.0}
    row = results[0]
    return {
        "total_gross": row[0] or 0.0,
        "count": row[1] or 0,
        "avg_net": row[2] or 0.0
    }
except Exception as e:
    # W logach systemowych warto by to zapisa?
    return {"error": str(e), "total_gross": 0.0, "count": 0, "avg_net": 0.0}

```

```

@get("/monthly-trend")
async def get_monthly_trend(self, state: Any) -> list[dict[str, Any]]:
    """Opcjonalny endpoint pod wykresy (np. trend wydatkw)."""
    manager: DuckDBManager = state.olap_manager
    query = """
    SELECT
        strftime(updated_at, '%Y-%m') as month,
        SUM(amount_gross) as total
    FROM invoices_replica
    GROUP BY 1
    ORDER BY 1 DESC
    """
    results = manager.execute(query)
    return [{"month": row[0], "total": row[1]} for row in results]

```

===== Code/API/controllers/invoices.py =====

```

import uuid
import shutil
import json
from pathlib import Path
from litestar import Controller, get, post, Body
from litestar.di import Provide
from litestar.status_codes import HTTP_201_CREATED
from litestar.enums import RequestEncodingType
from litestar.datastructures import UploadFile
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from api.schemas import InvoiceCreate, InvoiceResponse
from models.invoice import Invoice
from models.outbox import OutboxEvent
from tasks import broker # Twj Taskiq + NATS broker
from core.tasks import process_invoice_task

class InvoiceController(Controller):
    path = "/invoices"

    @get()
    async def list_invoices(self, db_session: AsyncSession) -> list[InvoiceResponse]:
        """Pobiera listę wszystkich faktur z SQLite."""
        result = await db_session.execute(select(Invoice).order_by(Invoice.created_at.desc()))
        invoices = result.scalars().all()
        return [
            InvoiceResponse(
                id=inv.id, number=inv.number, amount_net=inv.amount_net,
                amount_gross=inv.amount_gross, currency=inv.currency,
                status=inv.status, created_at=inv.created_at
            )
            for inv in invoices
        ]

```

```

        ) for inv in invoices
    ]

@post(status_code=HTTP_201_CREATED)
async def create_invoice(self, data: InvoiceCreate, db_session: AsyncSession) -> InvoiceResponse:
    """Dodaje now? faktur? i wrzuca zdarzenie OCR do Outboxa."""
    invoice_id = str(uuid.uuid4())
    new_invoice = Invoice(
        id=invoice_id,
        number=data.number,
        contractor_nip=data.contractor_nip,
        file_path=data.file_path,
        amount_net=data.amount_net,
        amount_gross=data.amount_gross,
        status="NEW"
    )
    db_session.add(new_invoice)
    await db_session.commit()

    # Wypchni?cie zadania do NATS (Outbox Pattern)
    await broker.kick(
        "process_invoice_ocr",
        invoice_id=invoice_id,
        image_path=f"storage/scans/{invoice_id}.pdf"
    )

    return InvoiceResponse(
        id=new_invoice.id, number=new_invoice.number,
        amount_net=new_invoice.amount_net, amount_gross=new_invoice.amount_gross,
        currency=new_invoice.currency, status=new_invoice.status,
        created_at=new_invoice.created_at
    )

@post("/upload")
async def upload_invoice(
    self,
    data: UploadFile = Body(media_type=RequestEncodingType.MULTI_PART)
) -> dict:
    # 1. Definiujemy ?cie?k? zapisu
    upload_dir = Path("data/uploads")
    upload_dir.mkdir(parents=True, exist_ok=True)
    file_path = upload_dir / data.filename

    # 2. Zapisujemy plik fizycznie na dysku
    with open(file_path, "wb") as f:
        shutil.copyfileobj(data.file, f)

    # 3. Zapisujemy rekord w bazie danych (SQLAlchemy)
    # Przyk?ad logicznego dodania
    return {"filename": data.filename, "status": "uploaded"}

```

===== Code/API/dependencies.py =====

```

from __future__ import annotations

from collections.abc import AsyncGenerator
from sqlalchemy.ext.asyncio import AsyncSession

from core.config import AppConfig
from core.tenant import TenantManager
from db.analytics import DuckDBLimits, DuckDBManager
from db.database import create_oltp_engine, create_session_factory

_config = AppConfig()
_tenant_manager = TenantManager(_config)
_duckdb_limits = DuckDBLimits(memory_limit=_config.duckdb_memory_limit, threads=_config.duckdb_threads)

```

```

def provide_config() -> AppConfig:
    return _config

def provide_tenant_manager() -> TenantManager:
    return _tenant_manager

async def provide_db_session() -> AsyncGenerator[AsyncSession, None]:
    engine = create_oltp_engine(_config, sqlite_path=_tenant_manager.sqlite_path())
    session_factory = create_session_factory(engine)

    async with session_factory() as session:
        try:
            yield session
        except Exception:
            await session.rollback()
            raise
        finally:
            await engine.dispose()

async def provide_duckdb() -> DuckDBManager:
    return DuckDBManager(db_path=_tenant_manager.duckdb_path(), limits=_duckdb_limits)

===== Code/API/exceptions.py =====
"""Domain error taxonomy + HTTP JSON envelope."""
from __future__ import annotations

from dataclasses import dataclass
from litestar.connection import Request
from litestar.response import Response
from litestar.exceptions import HTTPException

@dataclass(slots=True)
class DomainError(Exception):
    code: str
    message: str
    category: str
    status_code: int = 400

class ValidationDomainError(DomainError):
    def __init__(self, message: str) -> None:
        super().__init__(code="VALIDATION_ERROR", message=message, category="validation", status_code=422)

class IntegrationDomainError(DomainError):
    def __init__(self, message: str) -> None:
        super().__init__(code="INTEGRATION_ERROR", message=message, category="integration", status_code=500)

class TimeoutDomainError(DomainError):
    def __init__(self, message: str) -> None:
        super().__init__(code="TIMEOUT", message=message, category="timeout", status_code=504)

class BusinessRuleDomainError(DomainError):
    def __init__(self, message: str) -> None:
        super().__init__(code="BUSINESS_RULE", message=message, category="business", status_code=409)

```

```

class SecurityDomainError(DomainError):
    def __init__(self, message: str) -> None:
        super().__init__(code="SECURITY_ERROR", message=message, category="security", status_code=401)

def _error_envelope(
    request: Request,
    *,
    code: str,
    message: str,
    category: str,
    status_code: int,
) -> Response:
    correlation_id = request.headers.get("x-correlation-id", "unknown")
    return Response(
        content={
            "error": {
                "code": code,
                "category": category,
                "message": message,
                "path": request.url.path,
                "correlation_id": correlation_id,
            }
        },
        status_code=status_code,
    )

def global_exception_handler(request: Request, exc: Exception) -> Response:
    if isinstance(exc, DomainError):
        return _error_envelope(
            request,
            code=exc.code,
            message=exc.message,
            category=exc.category,
            status_code=exc.status_code,
        )

    if isinstance(exc, HTTPException):
        return _error_envelope(
            request,
            code="HTTP_ERROR",
            message=str(exc.detail),
            category="http",
            status_code=exc.status_code,
        )

    return _error_envelope(
        request,
        code="INTERNAL_ERROR",
        message="Wewn?trzny b??d serwera",
        category="internal",
        status_code=500,
    )

===== Code/API/middleware.py =====
from __future__ import annotations

import uuid
import time
from litestar.middleware import AbstractMiddleware

from core.tenant import (
    DEFAULT_TENANT_ID,
    reset_current_tenant_id,

```

```

        set_current_tenant_id,
    )

class CorrelationAndDeprecationMiddleware(AbstractMiddleware):
    """Adds correlation-id, deprecation headers and tenant context."""

    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return

        started = time.perf_counter()
        request_headers = {k.decode().lower(): v.decode() for k, v in scope.get("headers", [])}
        correlation_id = request_headers.get("x-correlation-id", str(uuid.uuid4()))
        tenant_id = request_headers.get("x-tenant-id", DEFAULT_TENANT_ID)
        tenant_token = set_current_tenant_id(tenant_id)

        async def send_wrapper(message):
            if message["type"] == "http.response.start":
                headers = message.setdefault("headers", [])
                headers.append((b"x-correlation-id", correlation_id.encode()))
                headers.append((b"x-tenant-id", tenant_id.encode()))

                process_time_ms = (time.perf_counter() - started) * 1000.0
                headers.append((b"x-process-time", f"{process_time_ms:.2f}ms".encode()))

                path = scope.get("path", "")
                if path.startswith("/api/v1"):
                    headers.append((b"deprecation", b"true"))
                    headers.append((b"sunset", b"Wed, 31 Dec 2026 23:59:59 GMT"))
                    headers.append((b"link", b'</api/v2>; rel="successor-version"'))

            await send(message)

        try:
            await self.app(scope, receive, send_wrapper)
        finally:
            reset_current_tenant_id(tenant_token)

===== Code/API/rbac.py =====
from __future__ import annotations

from dataclasses import dataclass
from enum import StrEnum

from litestar.connection import ASGIConnection
from litestar.exceptions import NotAuthorizedException
from litestar.handlers.base import BaseRouteHandler

class NexusRole(StrEnum):
    OWNER = "owner"
    WORKER = "worker"

@dataclass(slots=True)
class RoleContext:
    role: NexusRole
    actor: str

def get_current_role(connection: ASGIConnection) -> RoleContext:
    raw_role = (connection.headers.get("X-Nexus-Role") or "owner").strip().lower()
    raw_actor = (connection.headers.get("X-Nexus-Actor") or "admin").strip().lower()

```

```

if raw_role not in {NexusRole.OWNER, NexusRole.WORKER}:
    raise NotAuthorizedException("Unsupported role. Use X-Nexus-Role: owner|worker")

role = NexusRole(raw_role)
prefix = "owner:" if role == NexusRole.OWNER else "worker:"
actor = raw_actor if raw_actor else ("admin" if role == NexusRole.OWNER else "taskiq")
return RoleContext(role=role, actor=f"{prefix}{actor}")

def owner_only_guard(connection: ASGIConnection, _: BaseRouteHandler) -> None:
    if get_current_role(connection).role != NexusRole.OWNER:
        raise NotAuthorizedException("Only OWNER can execute this operation.")

def worker_only_guard(connection: ASGIConnection, _: BaseRouteHandler) -> None:
    if get_current_role(connection).role != NexusRole.WORKER:
        raise NotAuthorizedException("Only WORKER can execute this operation.")

===== Code/API/routes/__init__.py =====
"""API routes package for NexusAI backend."""
from __future__ import annotations

from .analytics import AnalyticsController
from .invoices import InvoiceController
from .tasks import TaskController
from .exports import ExportController
from .health import HealthController
from .triage import TriageController
from .ws import progress_websocket

__all__ = [
    "AnalyticsController",
    "InvoiceController",
    "TaskController",
    "ExportController",
    "HealthController",
    "TriageController",
    "progress_websocket",
]

===== Code/API/routes/analytics.py =====
from litestar import Controller, get
from db.analytics import DuckDBManager, DuckDBAnalyticsEngine
from api.schemas import DashboardSummaryResponse

class AnalyticsController(Controller):
    path = "/api/v1/analytics"

    @get("/dashboard/summary")
    async def get_dashboard_summary(self, duckdb: DuckDBManager) -> DashboardSummaryResponse:
        """Pobiera agregaty finansowe z DuckDB w czasie poniżej 10ms."""
        query = """
        SELECT
            COALESCE(SUM(total_net), 0) as total_net_sum,
            COALESCE(SUM(total_gross), 0) as total_gross_sum,
            COALESCE(SUM(document_count), 0) as total_docs
        FROM v_monthly_summary
        WHERE period >= strftime('%Y-%m', current_date - interval 30 days)
        """
        result = duckdb.execute(query)
        if not result:
            return DashboardSummaryResponse(total_net=0.0, total_gross=0.0, total_documents=0)

        row = result[0]

```

```

        return DashboardSummaryResponse(total_net=row[0], total_gross=row[1], total_documents=row[2])

@get("/analytics/fraud-alerts")
async def get_fraud_alerts(db_engine: DuckDBAnalyticsEngine) -> list:
    detector = FraudDetector(db_engine.conn)
    alerts = detector.detect_round_number_spikes()
    return alerts.to_dict(orient="records")

@get("/analytics/cashflow-forecast")
async def get_forecast(db_engine: DuckDBAnalyticsEngine) -> list:
    predictor = CashflowPredictor(db_engine.conn)
    forecast = predictor.predict_liquidity_gap(days_ahead=30)
    return forecast.to_dict(orient="records")

===== Code/API/routes/exports.py =====
"""Export API endpoints for invoice data."""
from __future__ import annotations

from enum import Enum
from typing import Any
from litestar import Controller, get
from litestar.exceptions import ClientException
from core.config import AppConfig

class ExportFormat(str, Enum):
    """Supported export formats."""
    CSV = "csv"
    EXCEL = "xlsx"
    JSON = "json"
    PDF = "pdf"

class ExportController(Controller):
    """Handle invoice data export."""
    path = "/api/v1/exports"

    @get("/{export_id:str}/status")
    async def get_export_status(self, export_id: str) -> dict[str, Any]:
        """Get status of ongoing export."""
        return {
            "export_id": export_id,
            "status": "COMPLETED",
            "format": "csv",
            "file_url": f"/api/v1/exports/{export_id}/download",
        }

    @get("/{export_id:str}/download")
    async def download_export(self, export_id: str) -> dict[str, str]:
        """Download exported file."""
        return {
            "message": f"File for export {export_id} ready for download",
            "format": "csv",
        }

===== Code/API/routes/files.py =====
"""File management endpoints."""
from __future__ import annotations

from typing import Any
from litestar import Controller, get, delete
from litestar.exceptions import ClientException

class FileController(Controller):

```

```

"""File management API."""
path = "/api/v1/files"

@get("/{file_id:str}")
async def get_file_info(self, file_id: str) -> dict[str, Any]:
    """Get file metadata."""
    return {
        "file_id": file_id,
        "name": f"invoice_{file_id}.pdf",
        "size_bytes": 0,
        "mime_type": "application/pdf",
    }

@delete("/{file_id:str}")
async def delete_file(self, file_id: str) -> dict[str, str]:
    """Delete a file."""
    return {"message": f"File {file_id} deleted successfully"}

```

==== Code/API/routes/health.py =====

```

"""Health check endpoints."""
from __future__ import annotations

from typing import Any
from litestar import Controller, get

class HealthController(Controller):
    """Health check and status endpoints."""
    path = "/api/v1/health"

    @get("")
    async def health_check(self) -> dict[str, str]:
        """Basic health check."""
        return {"status": "OK", "version": "1.0.0"}

    @get("/live")
    async def liveness_probe(self) -> dict[str, str]:
        """Kubernetes liveness probe."""
        return {"status": "alive"}

    @get("/ready")
    async def readiness_probe(self) -> dict[str, str]:
        """Kubernetes readiness probe."""
        return {"status": "ready"}

    @get("/detailed")
    async def detailed_health(self) -> dict[str, Any]:
        """Detailed health status."""
        return {
            "api": "OK",
            "version": "1.0.0",
            "uptime_seconds": 0,
            "timestamp": "2026-04-26T20:59:00Z",
        }

```

==== Code/API/routes/invoices.py =====

```

from __future__ import annotations

import uuid
from datetime import datetime, timezone

from litestar import Controller, post
from litestar.datastructures import UploadFile
from litestar.enums import RequestEncodingType
from litestar.exceptions import ClientException

```

```

from litestar.connection import Request

from core.config import AppConfig
from api.schemas import TaskResponse
from api.services import ContentAddressableStorage, IdempotencyStore

class InvoiceController(Controller):
    """Invoice APIs (v1)."""

    path = "/api/v1/invoices"

    @post("/upload", media_type=RequestEncodingType.MULTI_PART)
    async def upload_invoice(
        self,
        data: dict[str, UploadFile],
        request: Request,
        config: AppConfig,
    ) -> TaskResponse:
        file_obj = data.get("file")
        if not file_obj:
            raise ClientException(detail="Brak pola 'file'", status_code=400)

        payload = await file_obj.read()
        if not payload:
            raise ClientException(detail="Pusty plik", status_code=400)

        idempotency_key = request.headers.get("idempotency-key")
        idempotency_store = IdempotencyStore(config.idempotency_db_path)
        payload_hash = idempotency_store.hash_payload(payload)

        if idempotency_key:
            cached = idempotency_store.get(idempotency_key, payload_hash)
            if cached:
                return TaskResponse(**cached)

        storage = ContentAddressableStorage(config.storage_dir)
        saved = storage.put(payload=payload, suffix=".pdf")

        task_id = str(uuid.uuid4())
        response = TaskResponse(
            task_id=task_id,
            status="QUEUED",
            message=(
                f"Invoice accepted: hash={saved.file_hash}, size={saved.size_bytes}, "
                f"workflow=UPLOADED->QUEUED ({datetime.now(timezone.utc).isoformat()})"
            ),
        ).model_dump()

        if idempotency_key:
            idempotency_store.save(idempotency_key, payload_hash, response)

        return TaskResponse(**response)

===== Code/API/routes/stats.py =====
"""Statistics and metrics endpoints."""
from __future__ import annotations

from typing import Any
from litestar import Controller, get

class StatsController(Controller):
    """Statistics and metrics API."""
    path = "/api/v1/stats"

```

```

    @get("/processing")
    async def get_processing_stats(self) -> dict[str, Any]:
        """Get invoice processing statistics."""
        return {
            "total_processed": 0,
            "avg_processing_time_ms": 0,
            "success_rate": 0.0,
            "error_count": 0,
        }

===== Code/API/routes/tasks.py =====
from __future__ import annotations

from litestar import Controller, get

class TaskController(Controller):
    path = "/api/v1/tasks"

    @get("/{task_id:str}")
    async def get_task_status(self, task_id: str) -> dict[str, str]:
        return {"task_id": task_id, "status": "QUEUED"}

===== Code/API/routes/triage.py =====
from __future__ import annotations

from litestar import Controller, get, post
from litestar.connection import Request
from litestar.exceptions import ClientException
from sqlalchemy.ext.asyncio import AsyncSession

from api.rbac import get_current_role, owner_only_guard
from api.schemas import TriageItem, TriageResolutionRequest, TriageResolutionResponse
from services.triage_service import list_pending_triage_items, resolve_triage_item

class TriageController(Controller):
    path = "/api/triage"

    @get("/pending")
    async def get_pending(self, db_session: AsyncSession) -> list[TriageItem]:
        pending = await list_pending_triage_items(db_session)
        return [
            TriageItem(
                invoice_id=item.id,
                image_path=item.file_path,
                extracted_data={
                    "number": item.number,
                    "contractor_nip": item.contractor_nip,
                    "amount_net": float(item.amount_net),
                    "amount_gross": float(item.amount_gross),
                },
                bounding_boxes={},
                confidence_score=0.0,
                reason_for_triage="PENDING_REVIEW",
            )
            for item in pending
        ]

    @post("/resolve/{invoice_id:str}", guards=[owner_only_guard])
    async def resolve(
        self,
        invoice_id: str,
        data: TriageResolutionRequest,

```

```

        db_session: AsyncSession,
        request: Request,
    ) -> TriageResolutionResponse:
        try:
            role_ctx = get_current_role(request)
            invoice = await resolve_triage_item(
                db_session,
                invoice_id=invoice_id,
                corrected_data=data.corrected_data,
                action=data.action,
                updated_by=role_ctx.actor,
            )
        except ValueError as exc:
            raise ClientException(status_code=400, detail=str(exc)) from exc

        if data.action == "confirm_post":
            message = "Corrected invoice approved; pending ledger entry can be posted"
        else:
            message = "Invoice was voided/rejected from triage"

        return TriageResolutionResponse(invoice_id=invoice.id, status=invoice.status, message=message)

===== Code/API/routes/ws.py =====
from __future__ import annotations

from litestar import websocket

@websocket(path="/api/v1/ws/progress")
async def progress_websocket(socket) -> None:
    await socket.accept()
    await socket.send_json({"status": "connected"})
    await socket.close()

===== Code/API/schemas.py =====
from __future__ import annotations
import msgspec
from pydantic import BaseModel, ConfigDict
from decimal import Decimal
from datetime import datetime
from uuid import UUID
from typing import Optional

# -- msgspec Structs --

class InvoiceCreate(msgspec.Struct):
    """Dane wymagane przy r?cznym tworzeniu lub uploadzie faktury."""
    number: str
    contractor_nip: str
    file_path: str = ""
    amount_net: Decimal = Decimal("0.0")
    amount_gross: Decimal = Decimal("0.0")
    currency: str = "PLN"
    issue_date: str = ""

class InvoiceResponse(msgspec.Struct):
    """Struktura zwracana do frontendu."""
    id: str
    number: str | None
    amount_net: Decimal
    amount_gross: Decimal
    currency: str
    status: str # NEW, PROCESSING, APPROVED
    created_at: datetime

```

```

class AnalyticsQuery(msgspec.Struct):
    start_date: str
    end_date: str
    dimension: str = "monthly"

class VatSummary(msgspec.Struct):
    """Zagregowane dane analityczne z DuckDB."""
    month: str
    total_net: Decimal
    total_gross: Decimal
    currency: str

# -- Pydantic Models --

class TaskResponse(BaseModel):
    task_id: str
    status: str
    message: str

class InvoiceResponsePydantic(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: str
    number: str | None
    contractor_nip: str | None
    amount_net: float | None
    amount_gross: float | None
    currency: str | None
    status: str
    created_at: datetime
    updated_at: datetime

class DashboardSummaryResponse(BaseModel):
    total_net: float
    total_gross: float
    total_documents: int

class TriageItem(BaseModel):
    invoice_id: str
    image_path: str
    extracted_data: dict[str, object]
    bounding_boxes: dict[str, object]
    confidence_score: float
    reason_for_triage: str

class TriageResolutionRequest(BaseModel):
    corrected_data: dict[str, object]
    action: str

class TriageResolutionResponse(BaseModel):
    invoice_id: str
    status: str
    message: str

===== Code/API/security.py =====
import os
import secrets
import logging
from datetime import timedelta
from litestar.security.jwt import JWTAuth, Token
from litestar.connection import ASGIConnection
from litestar.handlers.base import BaseRouteHandler

```

```

from litestar.exceptions import NotAuthorizedException
from litestar.config.rate_limit import RateLimitConfig
from models.user import User # Za?o?enie, ?e model u?ytkownika istnieje

logger = logging.getLogger("nexus.api.security")

def _resolve_jwt_secret() -> str:
    """Pobiera klucz JWT z env; brak twardo zakodowanego klucza w repozytorium."""
    env_secret = os.getenv("NEXUS_JWT_SECRET", "").strip()
    if env_secret:
        return env_secret

    # Fallback wy??cznie dla ?rodowisk lokalnych/deweloperskich.
    # Nie zapisujemy sta?ego sekretu w kodzie.
    generated = secrets.token_urlsafe(48)
    logger.warning("NEXUS_JWT_SECRET is missing; using ephemeral dev-only JWT secret.")
    return generated

SECRET_KEY = _resolve_jwt_secret()

async def retrieve_user_handler(token: Token, connection: ASGIConnection) -> User | None:
    # Tutaj weryfikujemy u?ytkownika w SQLite
    # Na potrzeby offline-first, mo?emy u?ywa? domy?lnego profilu "admin"
    return User(id="admin", role="admin")

jwt_auth = JWTAuth[User](
    retrieve_user_handler=retrieve_user_handler,
    token_secret=SECRET_KEY,
    exclude=["/auth/login", "/health"], # Endpointy publiczne
)

# Odczytujemy token, kt?y wygenerowa? main.py (Launcher)
EXPECTED_TOKEN = os.getenv("NEXUS_TOKEN")

def token_guard(connection: ASGIConnection, route_handler: BaseRouteHandler) -> None:
    """Stra?nik Litestar.
    Odrzuca wszystkie zapytania, kt?e nie posiadaj? wa?nego tokena w nag?wku.
    Chroni lokalne API przed atakami z innych aplikacji."""

    # 1. Wyj?tek dla Healthcheck (?eby Launcher wiedzia?, czy serwer ?yje)
    if connection.url.path == "/health":
        return

    # 2. Tryb deweloperski
    if not EXPECTED_TOKEN:
        raise NotAuthorizedException("B??d krytyczny: Serwer nie zosta? uruchomiony z Launchera.")

    # 3. Sprawdzanie nag?wka
    auth_header = connection.headers.get("Authorization")
    if not auth_header or not auth_header.startswith("Bearer "):
        raise NotAuthorizedException("Odmowa dost?pu. Brak poprawnego nag?wka Authorization.")

    # Wyci?gamy sam ci?g znakw po s?owie "Bearer"
    token = auth_header.split(" ")[1]

    # 4. Weryfikacja
    if token != EXPECTED_TOKEN:
        raise NotAuthorizedException("Nieprawid?owy token dost?pu. Po??czenie odrzucone.")

# Limit: 100 ??da? na minut? dla danego IP
rate_limit_config = RateLimitConfig(
    rate_limit=("minute", 100),
    exclude=["/api/v1/health"], # Zdrowie sprawdzamy bez limitu
)

```

```

===== Code/API/server.py =====
from __future__ import annotations

import os

import uvicorn

from api.app import create_app

# ASGI object used by launchers ('uvicorn api.server:app').
app = create_app()

def run_backend() -> None:
    """Entrypoint used by launcher/containers."""
    host = os.getenv("NEXUS_HOST", "127.0.0.1")
    port = int(os.getenv("NEXUS_PORT", "8000"))

    uvicorn.run(
        app,
        host=host,
        port=port,
        loop="asyncio",
        log_level=os.getenv("NEXUS_LOG_LEVEL", "info"),
        access_log=True,
    )

if __name__ == "__main__":
    run_backend()

===== Code/API/services.py =====
"""Application services for idempotency and file storage."""
from __future__ import annotations

import hashlib
import json
import sqlite3
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any

@dataclass(slots=True)
class StoredUpload:
    file_hash: str
    file_path: str
    size_bytes: int

class ContentAddressableStorage:
    """File storage using SHA-256 as canonical key (dedupe-friendly)."""

    def __init__(self, root: Path) -> None:
        self.root = root
        self.root.mkdir(parents=True, exist_ok=True)

    @staticmethod
    def _sha256(payload: bytes) -> str:
        return hashlib.sha256(payload).hexdigest()

    def put(self, payload: bytes, suffix: str = ".pdf") -> StoredUpload:
        digest = self._sha256(payload)
        dir_path = self.root / digest[:2] / digest[2:4]

```

```

    dir_path.mkdir(parents=True, exist_ok=True)
    file_path = dir_path / f"{digest}{suffix}"

    if not file_path.exists():
        file_path.write_bytes(payload)

    return StoredUpload(file_hash=digest, file_path=str(file_path), size_bytes=len(payload))

class IdempotencyStore:
    """SQLite-backed idempotency registry with payload hash and TTL."""

    def __init__(self, db_path: Path, ttl_minutes: int = 60) -> None:
        self.db_path = db_path
        self.ttl_minutes = ttl_minutes
        self._init_db()

    def _connect(self) -> sqlite3.Connection:
        return sqlite3.connect(self.db_path)

    def _init_db(self) -> None:
        with self._connect() as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS idempotency_requests (
                    idempotency_key TEXT PRIMARY KEY,
                    payload_hash TEXT NOT NULL,
                    response_json TEXT NOT NULL,
                    created_at TEXT NOT NULL
                )
                """
            )

    @staticmethod
    def hash_payload(payload: bytes) -> str:
        return hashlib.sha256(payload).hexdigest()

    def purge_expired(self) -> None:
        threshold = datetime.now(timezone.utc) - timedelta(minutes=self.ttl_minutes)
        with self._connect() as connection:
            connection.execute(
                "DELETE FROM idempotency_requests WHERE created_at < ?",
                (threshold.isoformat(),),
            )

    def get(self, key: str, payload_hash: str) -> dict[str, Any] | None:
        self.purge_expired()
        with self._connect() as connection:
            row = connection.execute(
                "SELECT payload_hash, response_json FROM idempotency_requests WHERE idempotency_key = ?",
                (key,),
            ).fetchone()

        if not row:
            return None

        persisted_hash, response_json = row
        if persisted_hash != payload_hash:
            raise ValueError("Idempotency-Key reused with different payload")

        return json.loads(response_json)

    def save(self, key: str, payload_hash: str, response: dict[str, Any]) -> None:
        with self._connect() as connection:
            connection.execute(

```

```

        """
        INSERT OR REPLACE INTO idempotency_requests
        (idempotency_key, payload_hash, response_json, created_at)
        VALUES (?, ?, ?, ?)
        """
        (key, payload_hash, json.dumps(response), datetime.now(timezone.utc).isoformat()),
    )

===== Code/API/state.py =====
import logging
from litestar import Litestar
from db.database import create_oltp_engine, consolidate_database
from worker.broker import broker

logger = logging.getLogger("nexus.api.state")

async def on_startup(app: Litestar) -> None:
    """Inicjalizacja ci??kich zasobw przy starcie API."""
    config = app.dependencies["config"]()

    # Inicjalizacja silnika bazy danych w stanie aplikacji
    engine = create_oltp_engine(config)
    app.state.db_engine = engine

    # Po??czenie z brokerem Taskiq (NATS)
    if not broker.is_worker_process:
        await broker.startup()

    logger.info(">>> Nexus API: Wszystkie systemy gotowe.")

async def on_shutdown(app: Litestar) -> None:
    """Bezpieczne zamykanie i konsolidacja danych."""
    logger.info(">>> Nexus API: Rozpoczynanie procedury zamykania...")

    # Konsolidacja WAL dla SQLite (z Twojego modu?u DB)
    await consolidate_database(app.state.db_engine)

===== Code/API/static.py =====
from litestar.static_files import StaticFilesConfig
from core.config import AppConfig

def get_static_config(config: AppConfig) -> list[StaticFilesConfig]:
    """Konfiguruje serwer plikw dla za??cznikw PDF."""
    upload_dir = config.base_dir / "app_data" / "uploads"
    upload_dir.mkdir(parents=True, exist_ok=True)

    return [
        StaticFilesConfig(
            directories=[upload_dir],
            path="/files",
            name="uploads",
            html_mode=False
        )
    ]

===== Code/API/tasks.py =====
import os
import asyncio
import logging
from datetime import datetime
from decimal import Decimal
from taskiq_nats import PullBasedJetStreamBroker
from sqlalchemy.ext.asyncio import AsyncSession
from db.replication import sync_sqlite_to_duckdb
from db.database import get_session

```

```

from core.config import AppConfig

broker = PullBasedJetStreamBroker()
logger = logging.getLogger("nexus.api.tasks")

@broker.task(schedule=[{"cron": "* /5 * * * *"}])
async def run_data_replication():
    """Zadanie w tle synchronizuj?ce g?wn? baz? danych z baz? analityczn?."""
    config = AppConfig()
    # Tworzymy osobn? sesj? dla workera
    async for db_session in get_session():
        result = await sync_sqlite_to_duckdb(db_session, config)
        logger.info(
            "[REPLICATION] Status: %s. Rows: %s",
            result["status"],
            result.get("synced_rows", 0),
        )

@broker.task(task_name="process_invoice_ocr")
async def process_invoice_task(invoice_id: str, image_path: str):
    async with session_factory() as session:
        loop = asyncio.get_running_loop()

        # 3. Surya to kod synchroniczny (blokuj?cy), w osobnym w?tku
        ai_result = await loop.run_in_executor(
            None,
            ocr_engine.process_image,
            image_path
        )

        # 4. Analiza wynikw i aktualizacja bazy
        if ai_result.nip:
            invoice.contractor_nip = ai_result.nip

        if ai_result.amount_gross:
            invoice.amount_gross = Decimal(str(ai_result.amount_gross))
            invoice.amount_net = Decimal(str(ai_result.amount_gross / 1.23))

        await session.commit() # Zapis do SQLite
        await session.refresh(invoice)

        data = {
            "id": invoice.id,
            "number": invoice.number,
            "contractor_nip": invoice.contractor_nip,
            "amount_net": invoice.amount_net,
            "amount_gross": invoice.amount_gross,
            "currency": invoice.currency,
            "status": invoice.status,
            "updated_at": datetime.now()
        }

        # SYNCHRONIZACJA: Po pomy?lnym commicie w SQLite, aktualizujemy DuckDB
        await loop.run_in_executor(None, olap_manager.upsert_invoice, data)

===== Code/ARCHITECTURE/__init__.py =====
from .perfect_accounting_architecture import (
    ArchitectureBlueprint,
    REQUIRED_TECHNOLOGIES,
    build_blueprint,
    validate_blueprint_coverage,
)

__all__ = [
    "ArchitectureBlueprint",

```

```

        "REQUIRED_TECHNOLOGIES",
        "build_blueprint",
        "validate_blueprint_coverage",
    ]

==== Code/ARCHITECTURE/perfect_accounting_architecture.py ====
"""Blueprint "Perfekcyjnej Integracji" dla aplikacji księgowej NexusAI.

Plik definiuje rozszerzoną architekturę docelową wraz z komponentami,
mechanizmami niezawodności i mapowaniem wymaganych technologii.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum

class Layer(str, Enum):
    PRESENTATION = "presentation"
    API = "api"
    DOMAIN = "domain"
    INTEGRATION = "integration"
    DATA = "data"
    OPS = "ops"

@dataclass(frozen=True)
class Technology:
    name: str
    role: str

@dataclass(frozen=True)
class Component:
    name: str
    layer: Layer
    responsibilities: tuple[str, ...]
    technologies: tuple[Technology, ...]

@dataclass(frozen=True)
class PipelineStage:
    name: str
    primary: str
    validator: str | None = None
    controls: tuple[str, ...] = ()

@dataclass
class ArchitectureBlueprint:
    name: str = "NexusAI Accounting Platform"
    components: list[Component] = field(default_factory=list)
    ocr_pipeline: list[PipelineStage] = field(default_factory=list)
    ml_pipeline: list[PipelineStage] = field(default_factory=list)

    def technology_names(self) -> set[str]:
        names: set[str] = set()
        for component in self.components:
            names.update(t.name for t in component.technologies)
        return names

REQUIRED_TECHNOLOGIES = {
    "Python",

```

"Flet",
"Litestar",
"Litestar Security",
"JWT",
"SQLite",
"DuckDB",
"NATS",
"fsspec",
"Surya OCR",
"PaddleOCR V4 Server",
"scikit-learn",
"PyTorch 2.x",
"TensorFlow 3.x",
"AutoGluon-Light",
"Hugging Face",
"Sentence-Transformers",
"LanceDB",
"INT8 Quantization",
"Python-Statemachine",
"Taskiq",
"FastStream",
"uv",
"Nuitka",
"Podman",
"Pulumi",
"GitHub Actions",
"GitLab CI",
"Woodpecker CI",
"GitLab Runner",
"Fabric",
"Invoke",
"Trivy",
"pip-audit",
"SOPS",
"Age",
"VictoriaMetrics",
"Vector",
"Falco",
"Sentry Lite",
"loguru",
"Litestream",
"MinIO",
"Kopia",
"Cloudflare R2",
"pytest",
"Hypothesis",
"Dagger",
"Schemathesis",
"Playwright",
"dlt",
"Polars",
"HTTPX",
"Pydantic V2",
"xsdata",
"Authlib",
"OData-query",
"GUS BIR API",
"NBP API",
"Bia? a Lista VAT",
"VIES",
"CQRS/Event Sourcing",
"NATS JetStream",
"Modular Monolith",
"Plugins + Hooks",
"Coolify",

```

    "Headscale",
    "PocketBase",
    "Rathole",
    "frp",
    "Ruff",
    "Bandit",
    "Infisical",
    "Checkov",
}

def build_blueprint() -> ArchitectureBlueprint:
    """Buduje referencyjny blueprint zgodny z wymagany stosem technologicznym."""
    components = [
        Component(
            name="Flet Backoffice UI",
            layer=Layer.PRESENTATION,
            responsibilities=(
                "Panel ksi?gowo?ci, dashboard KPI i synchronizacja mobilna",
                "Obs?uga walidacji dokumentw i alertw b??dw OCR/ML",
            ),
            technologies=(
                Technology("Python", "j?zyk aplikacji"),
                Technology("Flet", "interfejs Flutter for Python"),
                Technology("Playwright", "testy E2E UI"),
            ),
        ),
        Component(
            name="Litestar API + Identity",
            layer=Layer.API,
            responsibilities=(
                "REST API /api/v2 i kontrakty integracyjne",
                "Autoryzacja Litestar Security + JWT",
            ),
            technologies=(
                Technology("Litestar", "framework API"),
                Technology("Litestar Security", "warstwa security"),
                Technology("JWT", "tokeny wewn?trzne"),
                Technology("HTTPX", "klienci integracyjni"),
                Technology("Pydantic V2", "walidacja modeli"),
            ),
        ),
        Component(
            name="OCR + Document Intelligence",
            layer=Layer.DOMAIN,
            responsibilities=(
                "Ekstrakcja danych faktur i klasyfikacja dokumentw",
                "Walidacja krzy?owa i konsensus wielosilnikowy",
            ),
            technologies=(
                Technology("Surya OCR", "silnik primary OCR"),
                Technology("PaddleOCR V4 Server", "silnik waliduj?cy"),
                Technology("PyTorch 2.x", "g?wny runtime ML"),
                Technology("TensorFlow 3.x", "drugi runtime ML"),
                Technology("Hugging Face", "fine-tuning i hosting modeli"),
                Technology("Sentence-Transformers", "embedding semantyczny"),
                Technology("LanceDB", "vector DB i semantic search"),
                Technology("INT8 Quantization", "optymalizacja inferencji"),
                Technology("AutoGluon-Light", "automatyczny dobr modelu"),
                Technology("scikit-learn", "feature engineering"),
            ),
        ),
        Component(
            name="Workflow & Event Backbone",
            layer=Layer.INTEGRATION,

```

```

responsibilities=(
    "State machine procesu ksi?gowania",
    "Asynchroniczne taski i event sourcing",
),
technologies=(
    Technology("Python-Statemachine", "BPM"),
    Technology("Taskiq", "kolejka taskw"),
    Technology("FastStream", "event streaming z NATS"),
    Technology("NATS", "event bus"),
    Technology("NATS JetStream", "trwa?y log zdarze?"),
    Technology("CQRS/Event Sourcing", "wzorzec domenowy"),
    Technology("Modular Monolith", "organizacja kodu"),
    Technology("Plugins + Hooks", "rozszerzalno??"),
),
),
Component(
    name="Data & Analytics",
    layer=Layer.DATA,
    responsibilities=(
        "OLTP dla operacji ksi?gowych oraz outbox",
        "OLAP, ETL i analityka finansowa",
    ),
    technologies=(
        Technology("SQLite", "OLTP + event store"),
        Technology("DuckDB", "OLAP + SILM"),
        Technology("fsspec", "unified storage adapter"),
        Technology("dlt", "?adowanie danych"),
        Technology("Polars", "transformacje analityczne"),
        Technology("Vector", "kolektor logw"),
        Technology("VictoriaMetrics", "metryki"),
    ),
),
Component(
    name="DevSecOps & Runtime",
    layer=Layer.OPS,
    responsibilities=(
        "Budowanie artefaktw, deployment i hardening",
        "Backup / DR, SIEM i compliance",
    ),
    technologies=(
        Technology("uv", "zarz?dzanie ?rodowiskiem i lockfile"),
        Technology("Nuitka", "kompilacja aplikacji"),
        Technology("Podman", "kontenery"),
        Technology("Pulumi", "IaC w Pythonie"),
        Technology("GitHub Actions", "CI/CD"),
        Technology("GitLab CI", "CI/CD alternatywny"),
        Technology("Woodpecker CI", "self-hosted runner"),
        Technology("GitLab Runner", "self-hosted runner"),
        Technology("Fabric", "automatyzacja operacyjna"),
        Technology("Invoke", "task runner"),
        Technology("Trivy", "SCA i security scan"),
        Technology("pip-audit", "SCA dla Pythona"),
        Technology("SOPS", "szyfrowanie sekretw"),
        Technology("Age", "klucze do SOPS"),
        Technology("Falco", "runtime security"),
        Technology("Sentry Lite", "observability b??dw"),
        Technology("loguru", "logging aplikacyjny"),
        Technology("Litestream", "replikacja SQLite"),
        Technology("MinIO", "object storage backup"),
        Technology("Kopia", "snapshot backup"),
        Technology("Cloudflare R2", "offsite backup"),
        Technology("pytest", "testy jednostkowe"),
        Technology("Hypothesis", "testy property-based"),
        Technology("Dagger", "pipeline testowy"),
        Technology("Schemathesis", "testy kontraktowe API"),
    ),
),

```

```

        Technology("Ruff", "SAST lint"),
        Technology("Bandit", "SAST security"),
        Technology("Infisical", "secrets manager"),
        Technology("Checkov", "IaC security"),
        Technology("Coolify", "self-hosted management"),
        Technology("Headscale", "control plane sieci"),
        Technology("PocketBase", "backend-in-a-box"),
        Technology("Rathole", "tunelowanie"),
        Technology("frp", "reverse proxy/tunnel"),
    ),
),
Component(
    name="Integrations Hub",
    layer=Layer.INTEGRATION,
    responsibilities=(
        "KSeF, e-Deklaracje, ZUS i PSD2",
        "GUS, NBP, VIES i Bia?a Lista VAT",
    ),
    technologies=(
        Technology("xsdata", "serializacja XSD/SOAP"),
        Technology("Authlib", "PSD2 OAuth2"),
        Technology("OData-query", "ERP/POS konektory"),
        Technology("GUS BIR API", "REGON lookup"),
        Technology("NBP API", "kursy walut"),
        Technology("Bia?a Lista VAT", "weryfikacja NIP MF"),
        Technology("VIES", "weryfikacja VAT UE"),
    ),
),
]

ocr_pipeline = [
    PipelineStage("Preprocessing", "Surya OCR", controls=("deskew", "denoise", "binarization")),
    PipelineStage(
        "Cross Validation",
        "Surya OCR",
        validator="PaddleOCR V4 Server",
        controls=(
            "ekstrakcja sum kontrolnych",
            "porwnanie warstw",
            "weryfikacja kontrahenta",
            "podwjne sprawdzenie",
            "algorytm konsensusu",
        ),
    ),
]

ml_pipeline = [
    PipelineStage("Layout NLP", "LiLT + HerBERT + LayoutLMv1"),
    PipelineStage("Semantic Retrieval", "Sentence-Transformers", validator="LanceDB"),
    PipelineStage("Decisioning", "PyTorch 2.x", validator="TensorFlow 3.x"),
]

return ArchitectureBlueprint(components=components, ocr_pipeline=ocr_pipeline, ml_pipeline=ml_pipeline)

```

```

def validate_blueprint_coverage(blueprint: ArchitectureBlueprint) -> tuple[bool, set[str]]:
    """Sprawdza, czy blueprint obejmuje wszystkie wymagane technologie."""
    available = blueprint.technology_names()
    missing = REQUIRED_TECHNOLOGIES - available
    return (len(missing) == 0, missing)

```

```

===== Code/BACKEND/Backend.py =====
import json
from db.models import Invoice, OutboxEvent
from sqlalchemy.ext.asyncio import AsyncSession

```

```

async def create_invoice_with_outbox(session: AsyncSession, invoice_data: dict):
    """Zapisuje fakturę i zdarzenie outbox w jednej transakcji."""
    async with session.begin():
        # 1. Tworzymy fakturę
        new_invoice = Invoice(**invoice_data)
        session.add(new_invoice)

        # Wymuszamy flush, aby otrzymać ID faktury (jeżeli potrzebne w zdarzeniu)
        await session.flush()

        # 2. Tworzymy zdarzenie outbox
        event = OutboxEvent(
            topic="invoices.new",
            payload=json.dumps({
                "invoice_id": new_invoice.id,
                "file_path": new_invoice.file_path,
                "correlation_id": "current_trace_id"
            })
        )
        session.add(event)

        # Po wyjściu z bloku 'begin' następuje COMMIT obu rekordów
        return new_invoice

```

===== Code/BACKEND/ingestor.py =====

```

import cv2
import json
from core.ipc.shm_manager import SharedImageBuffer

async def handle_new_document(file_path, nats_client):
    # 1. Wczytujemy obraz (np. 1. strona PDF)
    img = cv2.imread(file_path)

    if img is None:
        raise ValueError(f"Nie można wczytać obrazu z pliku: {file_path}")

    # 2. Rezerwujemy pamięć wspólną
    shm_metadata = SharedImageBuffer.create(img)

    # 3. Wysyłamy przez NATS tylko "klucze do bramy"
    message_data = json.dumps({
        "invoice_id": "inv_123",
        "buffer": shm_metadata # Można JSON zamiast 20MB Base64
    })
    await nats_client.publish("ocr.tasks", message_data.encode())

    # Uwaga: Nie usuwamy SHM tutaj! Musi to zrobić konsument po zakończeniu pracy.

```

===== Code/BACKEND/processor.py =====

```

import json
from core.ipc.shm_manager import SharedImageBuffer
from core.models.surya import SuryaOCRModel

# Inicjalizacja modelu OCR (globalnie, aby uniknąć reload-u)
surya_model = SuryaOCRModel()

async def ocr_worker(msg):
    data = json.loads(msg.data.decode())
    buffer_meta = data["buffer"]

    # 1. Podpinamy się pod RAM Ingestora (Zero-Copy)
    image, shm = SharedImageBuffer.attach(buffer_meta)

    try:

```

```

        # 2. AI pracuje bezpořrednio na wspřdzielonej tablicy
        results = surya_model.predict(image)
        print(f"OCR zakořczony dla: {data['invoice_id']}")
        return results
    except Exception as e:
        print(f"B??d podczas OCR dla: {data['invoice_id']}: {e}")
        raise
    finally:
        # 3. KRYTYCZNE: Sprz?tamy, ?eby nie zapcha? RAM-u u?ytkownika
        SharedImageBuffer.cleanup(shm)

```

===== Code/CORE/__init__.py =====

```

# core/__init__.py
from core.config import AppConfig
from core.logger import logger
from core.exceptions import (
    NexusBaseException,
    AIProcessingError,
    LLMGuardrailError,
    BrokerConnectionError,
    VectorDBError
)
from core.crypto import Vault
from core.llm_guard import LLMGuard, InvoiceLLMExtraction
from core.resilience import async_retry
from core.storage import StorageProvider, LocalStorageProvider
from core.events import NexusEvent
from core.secrets import SecretsManager
from core.monitor import SystemMonitor
from core.prompts import PromptTemplate
from core.bus import bus
from core.parsers import DataParser
from core.ai_context import AIContextManager
from core.plugins import PluginManager
from core.hardware import HardwareProbe

# Inicjalizacja globalnych us?ug CORE
hw_config = HardwareProbe.get_gpu_config()
plugin_manager = PluginManager()
plugin_manager.discover_exporters()

```

```

__all__ = [
    "AppConfig",
    "logger",
    "NexusBaseException",
    "AIProcessingError",
    "LLMGuardrailError",
    "BrokerConnectionError",
    "VectorDBError",
    "Vault",
    "LLMGuard",
    "InvoiceLLMExtraction",
    "async_retry",
    "StorageProvider",
    "LocalStorageProvider",
    "NexusEvent",
    "SecretsManager",
    "SystemMonitor",
    "PromptTemplate",
    "bus",
    "DataParser",
    "AIContextManager",
    "plugin_manager",
    "HardwareProbe",
    "hw_config"
]

```

```
]
```

```
===== Code/CORE/active_learning.py =====
```

```
# core/active_learning.py
```

```
import lancedb
```

```
import pyarrow as pa
```

```
import pandas as pd
```

```
import json
```

```
from pathlib import Path
```

```
from typing import Optional, Dict, Any
```

```
from sentence_transformers import SentenceTransformer
```

```
class ActiveLearningEngine:
```

```
    def __init__(self, db_path: str = "./data/vector_db"):
```

```
        self.db_path = db_path
```

```
        self.table_name = "ocr_corrections"
```

```
        self._init_db()
```

```
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
    def _init_db(self):
```

```
        """Inicjalizuje bazę i tabelę, jeżeli nie istnieje."""
```

```
        self.db = lancedb.connect(self.db_path)
```

```
        if self.table_name not in self.db.table_names():
```

```
            schema = pa.schema([
```

```
                pa.field("vector", pa.list_(pa.float32(), 384)), # Zależy od modelu (dla all-MiniLM-L6-v2
```

```
                pa.field("contractor_nip", pa.string()),
```

```
                pa.field("correction_payload", pa.string()),
```

```
                pa.field("context_hash", pa.string())
```

```
            ])
```

```
            self.db.create_table(self.table_name, schema=schema)
```

```
        self.table = self.db.open_table(self.table_name)
```

```
    def _generate_embedding(self, raw_text: str) -> list[float]:
```

```
        """Zamienia surowy tekst faktury na wektor."""
```

```
        return self.model.encode(raw_text).tolist()
```

```
    async def save_correction(self, raw_text: str, nip: str, corrections: Dict[str, Any]):
```

```
        """Zapisuje poprawkę użytkownika do bazy wektorowej."""
```

```
        vector = self._generate_embedding(raw_text)
```

```
        data = [{
```

```
            "vector": vector,
```

```
            "contractor_nip": nip,
```

```
            "correction_payload": json.dumps(corrections),
```

```
            "context_hash": str(hash(raw_text)) # Proste hashowanie
```

```
        ]]
```

```
        self.table.add(data)
```

```
    async def get_suggestion(self, raw_text: str, nip: str) -> Optional[dict[str, Any]]:
```

```
        """Szuka w bazie wektorowej podobnego układu dla danego NIP-u."""
```

```
        query_vector = self._generate_embedding(raw_text)
```

```
        results = (
```

```
            self.table.search(query_vector)
```

```
                .where(f"contractor_nip = '{nip}'")
```

```
                .limit(1)
```

```
                .to_list()
```

```
        )
```

```
        if results and results[0]["_distance"] < 0.1: # Prawdopodobieństwo
```

```
            return json.loads(results[0]["correction_payload"])
```

```
        return None
```

```
===== Code/CORE/adaptive_batcher.py =====
```

```
# core/adaptive_batcher.py
```

```
import asyncio
```

```
from typing import Any, List
```

```
from core.logger import logger
```

```

class AdaptiveBatcher:
    """Grupowanie zadań AI dla optymalnego wykorzystania przepustowości GPU."""

    def __init__(self, process_func, batch_size: int = 8, timeout: float = 0.2):
        self.process_func = process_func # Funkcja inferencyjna modelu (np. _infer_batch)
        self.batch_size = batch_size # Maksymalny rozmiar batcha
        self.timeout = timeout # Maksymalny czas oczekiwania (w sekundach)
        self._queue = asyncio.Queue()
        self._worker_task = asyncio.create_task(self._batch_worker())

    async def add_task(self, item: Any) -> Any:
        """Dodaje zadanie do kolejki i czeka na jego przetworzenie."""
        future = asyncio.get_event_loop().create_future()
        await self._queue.put((item, future))
        return await future

    async def _batch_worker(self):
        """Wątek roboczy grupujący zadania."""
        while True:
            batch = []
            futures = []

            try:
                # Oczekiwanie na pierwszy element
                item, future = await self._queue.get()
                batch.append(item)
                futures.append(future)

                # Zbieranie kolejnych do osiągnięcia batch_size lub timeoutu
                end_time = asyncio.get_event_loop().time() + self.timeout
                while len(batch) < self.batch_size:
                    time_left = end_time - asyncio.get_event_loop().time()
                    if time_left <= 0:
                        break
                    try:
                        item, future = await asyncio.wait_for(self._queue.get(), timeout=time_left)
                        batch.append(item)
                        futures.append(future)
                    except asyncio.TimeoutError:
                        break

                if batch:
                    logger.debug(f"Wysyłanie batcha {len(batch)} elementów do GPU...")
                    results = await self.process_func(batch)

                    # Rozdanie wyników do oczekujących wątków
                    for fut, res in zip(futures, results):
                        if not fut.done():
                            fut.set_result(res)

            except Exception as e:
                logger.error(f"Błąd przetwarzania batcha: {e}")
                for fut in futures:
                    if not fut.done():
                        fut.set_exception(e)

===== Code/CORE/ai_context.py =====
# core/ai_context.py
import re
from typing import List
from core.logger import logger

class AIContextManager:
    """Zarządza oknem kontekstowym dla lokalnych modeli LLM."""

```

```

def __init__(self, max_tokens: int = 4096):
    self.max_tokens = max_tokens
    # Heurystyka: 1 token to średnio 4 znaki w j. polskim
    self.chars_per_token = 4

def truncate_for_llm(self, text: str, reserve_tokens: int = 500) -> str:
    """Przycina tekst OCR, aby zmieścił się w oknie modelu wraz z promptem."""
    max_chars = (self.max_tokens - reserve_tokens) * self.chars_per_token

    if len(text) > max_chars:
        logger.warning(f"Tekst dokumentu zbyt długi ({len(text)} znaków). Przycinanie do {max_chars}.")
        # Przycinamy do ostatniej kropki, aby nie rwać zdań
        truncated = text[:max_chars]
        last_dot = truncated.rfind('.')
        return truncated[:last_dot + 1] if last_dot > 0 else truncated

    return text

```

===== Code/CORE/analytics.py =====

```

# core/analytics.py
import duckdb
import pandas as pd

def get_cashflow_forecast(db_path: str = "data/nexus_analytics.duckdb") -> pd.DataFrame:
    conn = duckdb.connect(db_path)
    # Zapytanie: sumujemy przychody i koszty po dacie płatności (due_date) i obliczamy skumulowane saldo (
    query = """
    WITH DailyFlows AS (
        SELECT
            due_date,
            SUM(CASE WHEN type = 'SALE' THEN amount_gross ELSE 0 END) as daily_in,
            SUM(CASE WHEN type = 'COST' THEN amount_gross ELSE 0 END) as daily_out
        FROM invoices
        WHERE due_date >= CURRENT_DATE
            AND due_date <= CURRENT_DATE + INTERVAL 30 DAY
        GROUP BY due_date
    )
    SELECT
        due_date,
        daily_in,
        daily_out,
        SUM(daily_in - daily_out) OVER (ORDER BY due_date ASC) as projected_balance
    FROM DailyFlows
    ORDER BY due_date;
    """
    df = conn.execute(query).df()
    conn.close()
    return df

```

===== Code/CORE/backup.py =====

```

# core/backup.py
import zipfile
import io
import os
from datetime import datetime
from Cryptodome.Cipher import AES
from Cryptodome.Random import get_random_bytes
from pathlib import Path

class BackupManager:
    """Zarządza pakowaniem i szyfrowaniem bazy danych."""

    def __init__(self, config):
        self.config = config

```

```

self.backup_dir = Path(config.base_dir) / "backups"
self.backup_dir.mkdir(exist_ok=True)

def create_encrypted_zip(self, password: str) -> str:
    """Tworzy zaszyfrowany ZIP z bazami danych."""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M")
    zip_buffer = io.BytesIO()

    # 1. Pakowanie plikw
    files_to_backup = [
        self.config.sqlite_path,
        self.config.duckdb_path,
        # Dodaj ?cie?k? do LanceDB (vector store)
    ]

    with zipfile.ZipFile(zip_buffer, "w", zipfile.ZIP_DEFLATED) as zf:
        for file_path in files_to_backup:
            if os.path.exists(file_path):
                zf.write(file_path, arcname=os.path.basename(file_path))

    # Tu w przysz?oci mo?na doda? logik? szyfrowania (AES) z u?yciem buffera
    final_path = self.backup_dir / f"backup_{timestamp}.zip"
    with open(final_path, "wb") as f:
        f.write(zip_buffer.getvalue())

    return str(final_path)

===== Code/CORE/base.py =====
# core/exporters/base.py
from abc import ABC, abstractmethod
from models.invoice import Invoice
import xml.etree.ElementTree as ET

class BaseExporter(ABC):
    @abstractmethod
    def export(self, invoices: list[Invoice]) -> str:
        """Zwraca sformatowany ci?g znakw (XML/TXT) do zapisu."""
        pass

class OptimaExporter(BaseExporter):
    """Eksport do formatu Comarch Optima (XML)."""
    def export(self, invoices: list[Invoice]) -> str:
        root = ET.Element("ROOT", xmlns="http://www.comarch.pl/optima/dokumenty")(http://www.comarch.pl/optima/dokumenty)
        rejestry = ET.SubElement(root, "REJESTRY_ZAKUPU")

        for inv in invoices:
            doc = ET.SubElement(rejestry, "REJESTR_ZAKUPU")
            ET.SubElement(doc, "NUMER").text = inv.number
            ET.SubElement(doc, "NIP").text = inv.contractor_nip
            ET.SubElement(doc, "DATA_WYSTAWIENIA").text = inv.issue_date.isoformat()
            # Optima wymaga rozbicia na pozycje (uproszczenie)

        return ET.tostring(root, encoding='unicode')

===== Code/CORE/broker.py =====
# core/broker.py
import asyncio
from taskiq_nats import NatsBroker
from taskiq import TaskiqEvents
from core.config import AppConfig

config = AppConfig()

# Inicjalizujemy broker NATS.
broker = NatsBroker(

```

```

servers=["nats://127.0.0.1:4222"],
queue_name="nexus_tasks",
subject="nexus.tasks",
stream_name="nexus_stream",
)

@broker.on_event(TaskiqEvents.WORKER_STARTUP)
async def startup(state):
    """Logika uruchamiania przy starcie workera."""
    print("[Worker] ??czenie z systemami...")
    from pipeline.ocr import DocumentProcessor
    state.ocr_processor = DocumentProcessor()
    print("[Worker] Gotowy do przetwarzania faktur.")

@broker.on_event(TaskiqEvents.WORKER_SHUTDOWN)
async def shutdown(state):
    """Sprz?tanie przy wy??czaniu."""
    print("[Worker] Zamykanie zasobw...")

===== Code/CORE/bus.py =====
# core/bus.py
import asyncio
from typing import Callable, Any, Dict, List

class EventBus:
    """Lokalna szyna zdarze? dla komunikacji mi?dzy modu?ami."""

    def __init__(self):
        self._subscribers: Dict[str, List[Callable]] = {}

    def subscribe(self, event_type: str, callback: Callable):
        if event_type not in self._subscribers:
            self._subscribers[event_type] = []
        self._subscribers[event_type].append(callback)

    async def emit(self, event_type: str, data: Any = None):
        """Rozsy?a zdarzenie asynchronicznie do wszystkich subskrybentw."""
        if event_type in self._subscribers:
            tasks = [
                asyncio.create_task(callback(data))
                for callback in self._subscribers[event_type]
            ]
            if tasks:
                await asyncio.gather(*tasks)

# Globalny singleton
bus = EventBus()

===== Code/CORE/categorizer.py =====
# core/categorizer.py
from sentence_transformers import SentenceTransformer
from scipy.spatial.distance import cosine

class OfflineCategorizer:
    def __init__(self):
        # ?adujemy model z naszego lokalnego cache'u
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

        # Definiujemy kategorie, ktrych szukamy
        self.categories = {
            "PALIWO": "stacja paliw benzyna olej nap?dowy autogaz orlen bp shell",
            "IT": "us?ugi informatyczne oprogramowanie hosting domena serwer aws sprz?t",
            "BIURO": "artyku?y biurowe papier tonery drukarka meble d?ugopisy",
            "MARKETING": "reklama facebook google adwords ulotki kampania"
        }

```

```

# Obliczamy wektory (embeddings) dla naszych kategorii tylko raz
self.category_vectors = {
    name: self.model.encode(keywords)
    for name, keywords in self.categories.items()
}

def predict_category(self, invoice_lines_text: str) -> str:
    text_vector = self.model.encode(invoice_lines_text)
    best_category = "INNE"
    min_distance = float('inf')

    for name, cat_vector in self.category_vectors.items():
        dist = cosine(text_vector, cat_vector)
        if dist < min_distance:
            min_distance = dist
            best_category = name

    return best_category if min_distance < 0.6 else "INNE"

===== Code/CORE/circuit_breaker.py =====
# core/integrations/circuit_breaker.py
import time
from enum import Enum
from loguru import logger

class CircuitState(Enum):
    CLOSED = "Dzia?a" # Wszystko OK
    OPEN = "Roz??czony" # B??dy - nie wysy?aj zapyta?
    HALF_OPEN = "Testowy" # Prba powrotu

class CircuitBreaker:
    """Implementacja wzorca Circuit Breaker chroni?ca przed awariami kaskadowymi."""

    def __init__(self, failure_threshold=5, recovery_timeout=60):
        self.state = CircuitState.CLOSED
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failures = 0
        self.last_failure_time = 0.0

    async def call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            if time.time() - self.last_failure_time > self.recovery_timeout:
                self.state = CircuitState.HALF_OPEN
            else:
                raise Exception("KSeF API jest obecnie niedost?pne (Circuit Breaker OPEN)")

        try:
            result = await func(*args, **kwargs)
            self._on_success()
            return result
        except Exception as e:
            self._on_failure()
            raise e

    def _on_success(self):
        self.failures = 0
        self.state = CircuitState.CLOSED

    def _on_failure(self):
        self.failures += 1
        self.last_failure_time = time.time()
        if self.failures >= self.failure_threshold:
            self.state = CircuitState.OPEN

```

```
logger.error(f"[Circuit Breaker] Obwd OTWARTY! Ruch zatrzymany na {self.recovery_timeout}s.")
```

```
===== Code/CORE/config.py =====
```

```
"""Central runtime configuration and startup validation."""
```

```
from __future__ import annotations
```

```
import os
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
class ConfigValidationError(RuntimeError):
```

```
    """Raised when startup settings are incomplete or inconsistent."""
```

```
@dataclass(slots=True)
```

```
class AppConfig:
```

```
    """Centralized application settings registry for all environments."""
```

```
    environment: str = os.getenv("NEXUS_ENV", "dev")
```

```
    base_dir: Path = Path(os.getenv("NEXUS_BASE_DIR", Path.cwd().as_posix()))
```

```
    sqlite_file_name: str = os.getenv("NEXUS_SQLITE_FILE", "nexus_oltp.db")
```

```
    duckdb_file_name: str = os.getenv("NEXUS_DUCKDB_FILE", "nexus_olap.duckdb")
```

```
    storage_dir_name: str = os.getenv("NEXUS_STORAGE_DIR", "app_data/uploads")
```

```
    idempotency_db_name: str = os.getenv("NEXUS_IDEMPOTENCY_DB", "idempotency.sqlite")
```

```
    debug: bool = os.getenv("NEXUS_DEBUG", "0") == "1"
```

```
    sqlcipher_key_env: str = os.getenv("NEXUS_SQLCIPHER_KEY_ENV", "NEXUS_SQLCIPHER_KEY")
```

```
    jwt_secret: str = os.getenv("NEXUS_JWT_SECRET", "")
```

```
    encryption_key: str = os.getenv("NEXUS_ENCRYPTION_KEY", "")
```

```
    duckdb_memory_limit: str = os.getenv("NEXUS_DUCKDB_MEMORY_LIMIT", "2GB")
```

```
    duckdb_threads: int = int(os.getenv("NEXUS_DUCKDB_THREADS", "2"))
```

```
    def __post_init__(self) -> None:
```

```
        self.environment = self.environment.lower().strip()
```

```
        if self.environment not in {"dev", "stage", "prod"}:
```

```
            raise ConfigValidationError(
```

```
                "NEXUS_ENV must be one of: dev, stage, prod"
```

```
            )
```

```
        self.base_dir = self.base_dir.resolve()
```

```
        self.base_dir.mkdir(parents=True, exist_ok=True)
```

```
        self.storage_dir.mkdir(parents=True, exist_ok=True)
```

```
        required_in_stage_prod = {
```

```
            "jwt_secret": self.jwt_secret,
```

```
            "encryption_key": self.encryption_key,
```

```
        }
```

```
        if self.environment in {"stage", "prod"}:
```

```
            if self.debug:
```

```
                raise ConfigValidationError("NEXUS_DEBUG cannot be enabled in stage/prod")
```

```
            missing = [k for k, v in required_in_stage_prod.items() if not v]
```

```
            if missing:
```

```
                raise ConfigValidationError(
```

```
                    f"Missing required secrets for {self.environment}: {', '.join(missing)}"
```

```
                )
```

```
    @property
```

```
    def sqlite_path(self) -> Path:
```

```
        return self.base_dir / self.sqlite_file_name
```

```
    @property
```

```
    def duckdb_path(self) -> Path:
```

```
        return self.base_dir / self.duckdb_file_name
```

```

@property
def storage_dir(self) -> Path:
    return self.base_dir / self.storage_dir_name

@property
def idempotency_db_path(self) -> Path:
    return self.base_dir / self.idempotency_db_name

===== Code/CORE/crypto.py =====
# core/crypto.py
import os
import base64
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from core.config import AppConfig

class Vault:
    """Modu? do bezpiecznego szyfrowania danych aplikacyjnych w locie."""

    def __init__(self, config: AppConfig):
        # Tworzy mocny klucz Fernet na podstawie zmiennej ?rodowiskowej
        configured_key = config.encryption_key.strip()
        if configured_key:
            master_key = configured_key.encode()
        else:
            env_key = os.getenv(config.sqlcipher_key_env, "").strip()
            master_key = env_key.encode() if env_key else os.urandom(32)

        # Deriwacja klucza (KDF) dla zwi?kszonego bezpiecze?stwa
        kdf = PBKDF2HMAC(
            algorithm=hashes.SHA256(),
            length=32,
            salt=b"nexus-offline-ai-salt-v1",
            iterations=480000,
        )
        self._fernet = Fernet(base64.urlsafe_b64encode(kdf.derive(master_key)))

    def encrypt(self, plain_text: str) -> str:
        """Szyfruje tekst (np. has?a do ERP, API keys)."""
        if not plain_text:
            return plain_text
        return self._fernet.encrypt(plain_text.encode()).decode()

    def decrypt(self, encrypted_text: str) -> str:
        if not encrypted_text:
            return encrypted_text
        return self._fernet.decrypt(encrypted_text.encode()).decode()

===== Code/CORE/events.py =====
# core/events.py
from enum import Enum

class NexusEvent(str, Enum):
    """Centralny rejestr wszystkich zdarze? przeszy?anych przez NATS / Taskiq."""

    # Zdarzenia wej?ciowe (Kolejka zada?)
    INVOICE_UPLOADED = "task.invoice.uploaded"
    DOCUMENT_REJECTED = "task.document.rejected"
    RUN_SYNC_OLAP = "task.sync.olap"

    # Zdarzenia wyj?ciowe (Websocket / UI Updates)
    OCR_STARTED = "event.ocr.started"
    OCR_COMPLETED = "event.ocr.completed"

```

```

OCR_FAILED = "event.ocr.failed"

# Powiadomienia systemowe
SYSTEM_UPDATE_AVAILABLE = "event.system.update_available"
SYSTEM_BACKUP_COMPLETED = "event.system.backup_completed"

===== Code/CORE/exceptions.py =====
# core/exceptions.py

class NexusBaseException(Exception):
    """Główna klasa wyjątków dla całego systemu Nexus AI."""
    def __init__(self, message: str, code: str = "INTERNAL_ERROR"):
        self.message = message
        self.code = code
        super().__init__(self.message)

class AIProcessingError(NexusBaseException):
    """Zgłaszany, gdy model lokalny (Llama/Surya) napotka błąd lub rzuci OOM."""
    def __init__(self, message: str = "Błąd silnika AI podczas analizy dokumentu."):
        super().__init__(message, code="AI_PROCESSING_FAILED")

class LLMGuardrailError(NexusBaseException):
    """Zgłaszany, gdy wyjście z LLM (JSON) jest uszkodzone lub brakuje pól."""
    def __init__(self, message: str = "Model AI wygenerował niepoprawny format danych."):
        super().__init__(message, code="LLM_PARSING_FAILED")

class BrokerConnectionError(NexusBaseException):
    """Zgłaszany przy problemach z magistralą NATS JetStream."""
    def __init__(self, message: str = "Utracono połączenie z magistralą NATS."):
        super().__init__(message, code="BROKER_CONNECTION_ERROR")

class VectorDBError(NexusBaseException):
    def __init__(self, message: str = "Błąd bazy wektorowej."):
        super().__init__(message, code="VECTOR_DB_ERROR")

===== Code/CORE/exporters/__init__.py =====
from .base import BaseExporter
from .insert_epp import InsertEppExporter
from .optima_exporter import OptimaExporter
# Dla optima_txt.py eksport zależy od implementacji

===== Code/CORE/exporters/base.py =====
# core/exporters/base.py
from abc import ABC, abstractmethod
from models.invoice import Invoice
import xml.etree.ElementTree as ET

class BaseExporter(ABC):
    @abstractmethod
    def export(self, invoices: list[Invoice]) -> str:
        """Zwraca sformatowany ciąg znaków (XML/TXT) do zapisu."""
        pass

class OptimaExporter(BaseExporter):
    """Eksport do formatu Comarch Optima (XML)."""
    def export(self, invoices: list[Invoice]) -> str:
        root = ET.Element("ROOT", xmlns="http://www.comarch.pl/optima/dokumenty")(http://www.comarch.pl/optima/dokumenty)
        rejestry = ET.SubElement(root, "REJESTRY_ZAKUPU")

        for inv in invoices:
            doc = ET.SubElement(rejestry, "REJESTR_ZAKUPU")
            ET.SubElement(doc, "NUMER").text = inv.number
            ET.SubElement(doc, "NIP").text = inv.contractor_nip
            ET.SubElement(doc, "DATA_WYSTAWIENIA").text = inv.issue_date.isoformat()
            # Optima wymaga rozbicia na pozycje (uproszczenie)

```

```

        return ET.tostring(root, encoding='unicode')

===== Code/CORE/exporters/insert_epp.py =====
# core/exporters/insert_epp.py
from datetime import datetime

class InsertEppExporter:
    def __init__(self, invoices: list):
        self.invoices = invoices

    def generate(self) -> str:
        header = f"[INFO]\n\n1.30\n",1250,\n"Nexus OCR\n",\n"{datetime.now().strftime('%Y%m%d%H%M%S')}\n\n"
        content = "[ZAWARTOSC]\n"

        for inv in self.invoices:
            # Uproszczony wiersz faktury zakupu w formacie EPP
            line = (
                f"\nFS\n",1,0,\n"{inv.number}\n",,,,
                f"\n{inv.contractor_name}\n",\n"{inv.contractor_nip}\n",,
                f"{inv.date_sale},{inv.date_issue},{inv.date_due},
                f"{inv.amount_net},{inv.amount_gross},\n"PLN\n",1.0000\n"
            )
            content += line

        return header + content

===== Code/CORE/exporters/optima_txt.py =====
# core/exporters/optima_txt.py
# (Zale?nie od pe?nej implementacji Optima w TXT - obecnie jest puste zgodnie z dokumentacj? ?rd?ow?)

===== Code/CORE/exporters/storage.py =====
# core/storage.py
import os
import shutil
from abc import ABC, abstractmethod
from pathlib import Path
from core.config import AppConfig

class StorageProvider(ABC):
    """Interfejs dla magazynw danych (Lokalny, S3, FTP)."""

    @abstractmethod
    async def save_file(self, filename: str, content: bytes) -> str:
        pass

    @abstractmethod
    async def get_file(self, file_path: str) -> bytes:
        pass

    @abstractmethod
    async def delete_file(self, file_path: str) -> bool:
        pass

class LocalStorageProvider(StorageProvider):
    """Domy?lna implementacja zapisuj?ca pliki na dysku (Offline First)."""

    def __init__(self, config: AppConfig):
        self.base_path = config.base_dir / "app_data" / "uploads"
        self.base_path.mkdir(parents=True, exist_ok=True)

    async def save_file(self, filename: str, content: bytes) -> str:
        file_path = self.base_path / filename
        # U?ywamy async do operacji I/O, aby nie blokowa? p?tli Fleta/Uvicorn
        with open(file_path, "wb") as f:

```

```

        f.write(content)
    return str(file_path)

    async def get_file(self, file_path: str) -> bytes:
        with open(file_path, "rb") as f:
            return f.read()

    async def delete_file(self, file_path: str) -> bool:
        path = Path(file_path)
        if path.exists():
            path.unlink()
            return True
        return False

===== Code/CORE/forecaster.py =====
# core/analytics/forecaster.py
import pandas as pd
from sklearn.linear_model import LinearRegression
import numpy as np
from datetime import datetime, timedelta
from db.analytics import DuckDBManager

class CashflowForecaster:
    """Przewiduje saldo firmy na podstawie trendw historycznych z DuckDB."""

    def __init__(self, db_manager: DuckDBManager):
        self.db = db_manager

    async def predict_liquidity_gap(self, days_ahead: int = 30) -> dict:
        """Oblicza prognozowane saldo ko?cowe i identyfikuje ryzyko luki."""
        # 1. Pobieranie danych historycznych z DuckDB
        query = """
        SELECT
            CAST(issue_date AS DATE) as date,
            SUM(amount_gross) as daily_sum
        FROM invoices_replica
        WHERE status = 'APPROVED'
        GROUP BY 1 ORDER BY 1
        """
        data = self.db.execute_query(query)
        df = pd.DataFrame(data)

        if df.empty or len(df) < 5:
            return {"status": "INSUFFICIENT_DATA"}

        # Zast?pcze uproszczenie logiki predykcyjnej dla Cashflow
        projected_total = df['daily_sum'].sum() # Przyk?adowe zwrocenie sumy

        return {
            "projected_delta": float(projected_total),
            "forecast_date": (datetime.now() + timedelta(days=days_ahead)).strftime("%Y-%m-%d"),
            "risk_level": "HIGH" if projected_total < 0 else "LOW"
        }

===== Code/CORE/hardware.py =====
# core/hardware.py
import subprocess
import torch
import psutil
import platform
from core.logger import logger

class HardwareProbe:
    """Sprawdza zasoby sprz?towe w celu optymalizacji modeli AI."""

```

```

@staticmethod
def get_gpu_config() -> dict:
    """Zwraca optymalne parametry n_gpu_layers dla modelu Llama."""
    try:
        # Prba detekcji CUDA przez nvidia-smi
        res = subprocess.check_output(["nvidia-smi", "-L"]).decode()
        if "GPU" in res:
            logger.info("Wykryto akcelerację? NVIDIA CUDA. Aktywacja warstw GPU.")
            return {
                "n_gpu_layers": -1, # Wszystkie warstwy na GPU
                "use_mmap": True,
                "device": "cuda"
            }
    except Exception:
        pass

    logger.warning("Nie wykryto GPU. Przełączanie w tryb CPU (wolniejszy).")
    return {
        "n_gpu_layers": 0,
        "use_mmap": True,
        "device": "cpu"
    }

@staticmethod
def get_strategy() -> dict:
    """Decyduje, czy użyć natywnego PyTorch (GPU) czy GGUF (CPU)."""
    has_gpu = torch.cuda.is_available()
    total_ram = psutil.virtual_memory().total / (1024**3) # w GB
    cpu_info = platform.processor()

    if has_gpu and torch.cuda.get_device_properties(0).total_memory > 4 * 1024**3:
        return {
            "mode": "GPU",
            "device": "cuda",
            "model_format": "native", # PyTorch / Surya
            "n_gpu_layers": 35 # Wszystkie warstwy na GPU
        }

    logger.warning("Nie wykryto GPU lub za mało VRAM. Przejdźcie w tryb CPU (GGUF).")
    return {
        "mode": "CPU",
        "device": "cpu",
        "model_format": "gguf", # llama-cpp-python
        "n_gpu_layers": 0
    }

```

==== Code/CORE/integrations/ksef/auth.py ====

```

# core/integrations/ksef/auth.py
import httpx
import logging
from .crypto import KsefCrypto

logger = logging.getLogger("nexus.ksef")

class KsefAuthService:
    def __init__(self, nip: str, is_demo: bool = True):
        self.nip = nip
        self.base_url = "[https://ksef-test.mf.gov.pl/api/online/](https://ksef-test.mf.gov.pl/api/online/)"

        # Klucz publiczny pobierany zazwyczaj z zasobów aplikacji lub API MF
        self.mf_pub_key = b"---BEGIN PUBLIC KEY---\n..."

    async def login(self, user_token: str) -> str:
        """Główna metoda logowania zwracająca SessionToken."""
        async with httpx.AsyncClient(timeout=30.0) as client:

```

```

        # KROK 1: AuthorisationChallenge
        resp = await client.post(
            f"{self.base_url}Session/AuthorisationChallenge",
            json={"contextIdentifier": {"type": "onip", "identifier": self.nip}}
        )
        resp.raise_for_status()
        data = resp.json()
        return data.get("challenge")

===== Code/CORE/integrations/ksef/client.py =====
# core/integrations/ksef/client.py
import httpx

class KsefClient:
    def __init__(self, session_token: str, base_url: str):
        self.headers = {"SessionToken": session_token, "Accept": "application/json"}
        self.base_url = base_url

    async def fetch_invoice(self, ksef_reference: str):
        """Pobiera fakturę XML (format FA_VAT)."""
        async with httpx.AsyncClient(headers=self.headers) as client:
            url = f"{self.base_url}Invoice/Get/{ksef_reference}"
            resp = await client.get(url)
            return resp.content # Zwraca surowy bajtowy XML faktury

===== Code/CORE/integrations/ksef/crypto.py =====
# core/integrations/ksef/crypto.py
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import padding
import base64
from datetime import datetime

class KsefCrypto:
    """Obsługa operacji kryptograficznych zgodnych ze specyfikacją MF."""

    @staticmethod
    def encrypt_challenge(challenge: str, timestamp: str, public_key_pem: bytes) -> str:
        """
        Szyfruje challenge + timestamp zgodnie z wymogami KSeF.
        Format: challenge + "|" + timestamp_iso_8601
        """
        public_key = serialization.load_pem_public_key(public_key_pem)

        # ??czyemy dane zgodnie ze specyfikacją?
        payload = f"{challenge}|{timestamp}".encode('utf-8')

        encrypted = public_key.encrypt(
            payload,
            padding.PKCS1v15() # KLUCZOWE: MF nie akceptuje OAEP
        )
        return base64.b64encode(encrypted).decode('utf-8')

    @staticmethod
    def build_auth_xml(nip: str, encrypted_token: str) -> str:
        # Poniżej placeholder generatora XML
        xml = f"""
        <ns2:InitSessionTokenRequest>
            <ns2:Context>
                <DocumentType>
                    <FormCode>FA</FormCode>
                </DocumentType>
                <Token>{encrypted_token}</Token>
            </ns2:Context>
        </ns2:InitSessionTokenRequest>
        """

```

```
return xml
```

```
===== Code/CORE/integrations/ksef_crypto.py =====
```

```
# core/integrations/ksef_crypto.py
```

```
import base64
```

```
from datetime import datetime
```

```
from cryptography.hazmat.primitives import serialization, hashes
```

```
from cryptography.hazmat.primitives.asymmetric import padding
```

```
class KsefCryptoProvider:
```

```
    """Implementacja standardu bezpieczeŃstwa KSeF (MF)."""
```

```
    def __init__(self, public_key_path: str):
```

```
        # Klucze publiczne MF s? dostarczane jako pliki .crt / .pem
```

```
        with open(public_key_path, "rb") as key_file:
```

```
            self.public_key = serialization.load_pem_public_key(key_file.read())
```

```
    def encrypt_authorization_token(self, challenge: str, auth_token: str) -> str:
```

```
        """
```

```
        Szyfruje token autoryzacyjny po??czony z wyzwaniem (challenge).
```

```
        Format: challenge + '|' + auth_token + '|' + timestamp
```

```
        """
```

```
        timestamp = int(datetime.now().timestamp() * 1000)
```

```
        message = f"{challenge}|{auth_token}|{timestamp}".encode('utf-8')
```

```
        encrypted = self.public_key.encrypt(
```

```
            message,
```

```
            padding.PKCS1v15() # Obowi?zkowy standard
```

```
        )
```

```
        return base64.b64encode(encrypted).decode('utf-8')
```

```
===== Code/CORE/integrations/mail.py =====
```

```
# core/integrations/mail.py
```

```
import imaplib
```

```
import email
```

```
from pathlib import Path
```

```
from core.broker import broker
```

```
from core.logger import logger
```

```
# from core.tasks import process_invoice_task # Importowane z zada?
```

```
class MailIngestionService:
```

```
    def __init__(self, config):
```

```
        self.config = config
```

```
        self.download_path = Path(config.base_dir) / "app_data" / "mailbox_ingest"
```

```
        self.download_path.mkdir(parents=True, exist_ok=True)
```

```
    async def check_and_process(self):
```

```
        """Pobiera nowe maile i wysy?a zadania do Workera przez NATS."""
```

```
        try:
```

```
            # Po??czenie IMAP (uproszczone)
```

```
            mail = imaplib.IMAP4_SSL(self.config.mail_host)
```

```
            mail.login(self.config.mail_user, self.config.mail_pass)
```

```
            mail.select("inbox")
```

```
            # Szukamy nieprzeczytanych wiadomo?ci
```

```
            _, messages = mail.search(None, 'UNSEEN')
```

```
            for num in messages[0].split():
```

```
                _, data = mail.fetch(num, '(RFC822)')
```

```
                msg = email.message_from_bytes(data[0][1])
```

```
                # Tu logika zapisywania za??cznikw PDF
```

```
            mail.logout()
```

```
        except Exception as e:
```

```
            logger.error(f"B??d MailIngestion: {e}")
```

```
===== Code/CORE/ipc_vision.py =====
```

```
# core/ipc_vision.py
import numpy as np
from multiprocessing import shared_memory
import uuid

class ImageMemoryManager:
    """Zarządza pamięcią wspólną dla bezstratnego przesyłania obrazów."""

    @staticmethod
    def store_image(img_array: np.ndarray) -> dict:
        """Zapisuje obraz do pamięci wspólnej i zwraca metadane."""
        shm_name = f"nexus_img_{uuid.uuid4().hex}"

        # Tworzymy blok pamięci o rozmiarze tablicy
        shm = shared_memory.SharedMemory(create=True, size=img_array.nbytes, name=shm_name)

        # Tworzymy tablicę NumPy zmapowaną na ten blok i kopiujemy dane
        shared_array = np.ndarray(img_array.shape, dtype=img_array.dtype, buffer=shm.buf)
        shared_array[:] = img_array[:]

        return {
            "name": shm_name,
            "shape": img_array.shape,
            "dtype": str(img_array.dtype)
        }

    @staticmethod
    def retrieve_image(metadata: dict) -> np.ndarray:
        """Odczytuje i podziela pamięć na podstawie metadanych."""
        shm = shared_memory.SharedMemory(name=metadata["name"])
        shared_array = np.ndarray(metadata["shape"], dtype=np.dtype(metadata["dtype"]), buffer=shm.buf)
        return shared_array
```

```
===== Code/CORE/llm_extractor.py =====
```

```
# core/llm_extractor.py
from llama_cpp import Llama
import json

class LocalLLMExtractor:
    def __init__(self, model_path="models/phi-3-mini-4k-instruct.Q4_K_M.gguf"):
        # n_ctx to okno kontekstowe (4096 wystarczy na fakturę)
        self.llm = Llama(model_path=model_path, n_ctx=4096, n_threads=4)

    def extract_invoice_data(self, ocr_text: str):
        prompt = f"""<|user|>
Analizuj poniższy tekst faktury i wyciągnij dane w formacie JSON.
Wymagane pola: numer_faktury, data_sprzedazy, kwota_netto, kwota_vat, waluta, termin_platnosci.
Tekst faktury:
{ocr_text}
<|assistant|>
'''json
"""
        # Generujemy odpowiedź (stop na ''' kończącym JSON)
        output = self.llm(prompt, max_tokens=512, stop=["''"], echo=False)
        json_str = output['choices'][0]['text']
        return json.loads(json_str)
```

```
===== Code/CORE/llm_guard.py =====
```

```
# core/llm_guard.py
import json
import re
from pydantic import BaseModel, ValidationError
from decimal import Decimal
```

```

from typing import Optional
from core.exceptions import LLMGuardrailError
from core.logger import logger

class InvoiceLLMExtraction(BaseModel):
    """Oczekiwana struktura danych od lokalnego modelu Llama/Phi-3."""
    numer_faktury: Optional[str] = None
    data_sprzedazy: Optional[str] = None
    kwota_netto: Optional[Decimal] = None
    kwota_vat: Optional[Decimal] = None
    kwota_brutto: Optional[Decimal] = None
    waluta: str = "PLN"
    nip_sprzedawcy: Optional[str] = None

class LLMGuard:
    """Naprawia i weryfikuje 'brudne' wyjścia z lokalnych modeli LLM."""

    @staticmethod
    def parse_and_validate(raw_llm_output: str) -> dict:
        """Ekstrahuje JSON z markdownu LLM, naprawia błądy i waliduje przez Pydantic."""
        try:
            # Próba znalezienia bloku JSON w tekście
            json_match = re.search(r'```json\s*(.*?)\s*```', raw_llm_output, re.DOTALL)
            json_str = json_match.group(1) if json_match else raw_llm_output

            data = json.loads(json_str)
            validated = InvoiceLLMExtraction(**data)
            return validated.dict()
        except (json.JSONDecodeError, ValidationError) as e:
            logger.error(f"Błąd LLMGuard: {e}")
            raise LLMGuardrailError(f"Niepoprawny wynik LLM: {str(e)}")

===== Code/CORE/logger.py =====
# core/logger.py
import sys
from pathlib import Path
from loguru import logger

def setup_logger(app_name: str = "NexusAI"):
    """Konfiguruje globalny, asynchronicznie-bezpieczny system logowania z rotacją za pomocą Loguru."""
    logger.remove()

    # Konsola
    logger.add(
        sys.stderr,
        enqueue=True, # Zapobiega blokowaniu wątku głównego
        colorize=True,
        format="<green>{time:YYYY-MM-DD HH:mm:ss}</green> | <level>{level: <8}</level> | <message>{message}"
    )

    # Plik z rotacją i kompresją
    log_dir = Path("app_data/logs")
    log_dir.mkdir(parents=True, exist_ok=True)

    logger.add(
        log_dir / f"{app_name.lower()}_{time}.log",
        rotation="100 MB",
        retention="30 days",
        compression="zip",
        enqueue=True,
        level="INFO"
    )
    return logger

def get_logger():

```

```

    return logger

setup_logger()

===== Code/CORE/memory_manager.py =====
# core/memory_manager.py
import torch
import gc
from core.logger import logger

class MemoryManager:
    """Zarządza przemieszczaniem modeli z VRAM do RAM."""

    @staticmethod
    def hibernate_models(processors: list):
        """Przenosi modele na CPU i czyści VRAM."""
        logger.info("Aplikacja zminimalizowana - hibernacja modeli AI...")
        for proc in processors:
            if hasattr(proc, 'model') and proc.model is not None:
                # Przeniesienie modelu PyTorch na CPU (zwalnia VRAM, zostaje w RAM)
                proc.model.to("cpu")

        # Wymuszenie czyszczenia
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
        gc.collect()

    @staticmethod
    def wakeup_models(processors: list):
        """Przywraca modele do GPU dla maksymalnej wydajności."""
        if torch.cuda.is_available():
            logger.info("Aplikacja aktywna - przywracanie modeli do GPU...")
            for proc in processors:
                if hasattr(proc, 'model') and proc.model is not None:
                    proc.model.to("cuda")

===== Code/CORE/model_manager.py =====
# core/model_manager.py
from pathlib import Path
from huggingface_hub import snapshot_download
from core.logger import logger

class LazyModelManager:
    """Pobiera cięgi wagi modeli AI tylko w momencie ich pierwszego użycia."""

    def __init__(self, base_models_dir: str = "models/"):
        self.base_models_dir = Path(base_models_dir)
        self.base_models_dir.mkdir(parents=True, exist_ok=True)

        self._models = {
            "sentence_transformers": {
                "repo_id": "sentence-transformers/all-MiniLM-L6-v2",
                "local_dir": self.base_models_dir / "all-MiniLM-L6-v2",
                "loaded_instance": None
            },
            "surya_ocr": {
                "repo_id": "vikp/surya_det2",
                "local_dir": self.base_models_dir / "surya_det2",
                "loaded_instance": None
            }
        }

    def get_model_path(self, model_key: str) -> str:
        """Sprawdza czy model istnieje, pobiera w tle jeśli nie..."""
        model_info = self._models.get(model_key)

```

```

    if not model_info:
        raise ValueError(f"Nieznany model: {model_key}")

    local_dir = model_info["local_dir"]
    if not local_dir.exists():
        logger.info(f"Pobieranie modelu {model_key} w tle...")
        snapshot_download(repo_id=model_info["repo_id"], local_dir=local_dir)

    return str(local_dir)

===== Code/CORE/monitor.py =====
# core/monitor.py
import psutil
import os
from core.logger import logger

class SystemMonitor:
    """Monitoruje zużycie zasobów przez Nexus AI."""

    @staticmethod
    def get_memory_usage() -> float:
        """Zwraca zużycie RAM przez bieżący proces w MB."""
        process = psutil.Process(os.get_pid())
        return process.memory_info().rss / (1024 * 1024)

    @staticmethod
    def check_health(memory_threshold_mb: int = 4000) -> bool:
        """Sprawdza, czy system nie przekracza bezpiecznych limitów."""
        usage = SystemMonitor.get_memory_usage()
        if usage > memory_threshold_mb:
            logger.error(f"Krytyczne zużycie pamięci: {usage:.2f}MB / {memory_threshold_mb}MB")
            return False
        return True

===== Code/CORE/outbox_relay.py =====
# core/outbox_relay.py
import asyncio
import json
from sqlalchemy import select, delete
from db.models import OutboxEvent
from core.logger import logger

async def run_outbox_relay(session_factory, nats_client):
    """
    Pętla przekazywania: Odczyt z DB -> Publikacja do NATS -> Usunięcie z DB.
    """
    logger.info("Uruchamianie Outbox Relay (At-least-once delivery)...")

    while True:
        try:
            async with session_factory() as session:
                # 1. Pobierz nieprzetworzone zdarzenia
                stmt = select(OutboxEvent).where(OutboxEvent.processed == False).limit(50)
                result = await session.execute(stmt)
                events = result.scalars().all()

                if not events:
                    await asyncio.sleep(0.5) # Czekaj 500ms jeśli brak pracy
                    continue

                for event in events:
                    try:
                        # 2. Publikacja do NATS
                        # Używamy JetStream, by mieć gwarancję dostarczenia
                        await nats_client.publish(event.topic, event.payload.encode())
                    except Exception as e:
                        logger.error(f"Error publishing event {event.id}: {e}")

            # Usunięcie z DB
            stmt = delete(OutboxEvent).where(OutboxEvent.id.in_(event.id for event in events))
            await session.execute(stmt)
            await session.commit()

        except Exception as e:
            logger.error(f"Error in run_outbox_relay: {e}")
            await asyncio.sleep(1)

```

```

        event.processed = True
    except Exception as err:
        logger.error(f"Nie powiodła się wysyłka do NATS: {err}")

    await session.commit()
except Exception as e:
    logger.error(f"Brzd Outbox Relay: {e}")
    await asyncio.sleep(2)

```

===== Code/CORE/parsers.py =====

```

# core/parsers.py
from decimal import Decimal, InvalidOperation
from typing import Any
import re

class DataParser:
    """Narzędzie do czyszczenia danych wyjściowych z AI."""

    @staticmethod
    def to_decimal(value: Any) -> Decimal:
        """Konwertuje dowolny string/liczbę na Decimal (bezpieczny finansowo)."""
        if value is None:
            return Decimal("0.00")

        # Usuwanie spacji, walut i zamiana przecinka na kropkę
        clean_val = re.sub(r'[\d.,-]', '', str(value)).replace('.', ',')
        try:
            return Decimal(clean_val)
        except InvalidOperation:
            return Decimal("0.00")

    @staticmethod
    def clean_nip(nip: str) -> str:
        """Normalizuje NIP do formatu 10 cyfr."""
        return re.sub(r'\D', '', str(nip))

```

===== Code/CORE/plugins.py =====

```

# core/plugins.py
import importlib
import inspect
import pkgutil
from typing import Dict, Type, Any
from core.logger import logger

class PluginManager:
    """Dynamicznie ładowanie klasy eksporterów z katalogu core/exporters/."""

    def __init__(self, package_path: str = "core.exporters"):
        self.package_path = package_path
        self.exporters: Dict[str, Type] = {}

    def discover_exporters(self):
        """Skanuje folder i rejestruje wszystkie klasy eksporterów."""
        try:
            package = importlib.import_module(self.package_path)
            for _, name, is_pkg in pkgutil.iter_modules(package.__path__):
                full_module_name = f"{self.package_path}.{name}"
                module = importlib.import_module(full_module_name)

                for _, obj in inspect.getmembers(module):
                    # Szukamy klas kończących się na 'Exporter'
                    if inspect.isclass(obj) and obj.__name__.endswith("Exporter") and obj.__name__ != "BaseExporter":
                        self.exporters[obj.__name__] = obj
                        logger.info(f"Zarejestrowano eksporter: {obj.__name__}")
        except Exception as e:

```

```

        logger.error(f"B??d ?adowania pluginw: {e}")

===== Code/CORE/prompts.py =====
# core/prompts.py
from enum import Enum

class PromptTemplate(str, Enum):
    """Zbir systemowych instrukcji dla modeli lokalnych (Phi/Llama)."""

    INVOICE_EXTRACTOR = (
        "Jeste? ekspertem ksi?gowym. Twoim zadaniem jest wyodr?bnienie danych z tekstu OCR faktury. "
        "Zwr? WY??CZYNIE czysty JSON bez komentarzy. Pola: numer, data, nip_sprzedawcy, kwota_brutto, walut
    )

    CLASSIFIER = (
        "Na podstawie tekstu okre?l typ dokumentu: [FAKTURA, PARAGON, NOTA, INNE]. "
        "Zwr? tylko jedno s?owo."
    )

    def format(self, content: str) -> str:
        """??czy instrukcj? z tre?ci? dokumentu."""
        return f"{self.value}\n\nTre?? dokumentu:\n{content}"

===== Code/CORE/resilience.py =====
# core/resilience.py
import asyncio
import logging
from typing import Callable, Any
from functools import wraps

logger = logging.getLogger("nexus.core.resilience")

def async_retry(max_retries: int = 3, base_delay: float = 1.0, max_delay: float = 10.0, exceptions: tuple
    """
    Dekorator ponawiaj?cy wykonanie asynchronicznej funkcji w przypadku b??du.
    U?ywa strategii Exponential Backoff (1s, 2s, 4s...).
    """
    def decorator(func: Callable) -> Callable:
        @wraps(func)
        async def wrapper(*args, **kwargs) -> Any:
            retries = 0
            delay = base_delay
            while True:
                try:
                    return await func(*args, **kwargs)
                except exceptions as e:
                    retries += 1
                    if retries > max_retries:
                        logger.error(f"Przekroczono limit prb ({max_retries}) dla {func.__name__}. B??d: {
                        raise
                    logger.warning(f"B??d w {func.__name__}: {e}. Ponawianie za {delay}s...")
                    await asyncio.sleep(delay)
                    delay = min(delay * 2, max_delay)
            return wrapper
        return decorator

===== Code/CORE/saga.py =====
# core/reconciliation.py
from datetime import datetime, timedelta
from sqlalchemy import select
from models.invoice import Invoice
# from core.broker import nats # import szyny komunikatw

async def reconciliation_loop(session_factory, ksef_service):
    """P?tla naprawcza sprawdzaj?ca 'zawieszone' procesy."""

```

```

while True:
    async with session_factory() as session:
        # Szukaj faktur wysłanych do KSeF ponad 2 godziny temu, które nie mają UPO
        stuck_invoices = await session.execute(
            select(Invoice).where(
                Invoice.status == 'SENT_TO_KSEF',
                Invoice.updated_at < datetime.now() - timedelta(hours=2)
            )
        )

        for inv in stuck_invoices.scalars():
            # Sprawdź status bezpośrednio w API (może UPO już jest, tylko NATS nie dotarł?)
            status = await ksef_service.check_status(inv.external_id)
            if status == "COMPLETED":
                inv.status = "SUCCESS"
            else:
                # Ponów próbę lub powiadom operatora
                pass
                # await nats.publish("alerts.stuck", payload)

        await session.commit()

    await asyncio.sleep(600) # Sprawdź co 10 minut

===== Code/CORE/secrets.py =====
# core/secrets.py
import keyring
from core.logger import logger

class SecretsManager:
    """Ochrona kluczy API i hasła przy użyciu natywnego magazynu systemu operacyjnego."""
    SERVICE_NAME = "NexusAI_System"

    @staticmethod
    def save_secret(key_name: str, secret_value: str) -> None:
        try:
            keyring.set_password(SecretsManager.SERVICE_NAME, key_name, secret_value)
        except Exception as e:
            logger.error(f"Nie udało się zapisać sekretu '{key_name}' w systemie: {e}")
            raise

    @staticmethod
    def get_secret(key_name: str) -> str | None:
        try:
            return keyring.get_password(SecretsManager.SERVICE_NAME, key_name)
        except Exception as e:
            logger.error(f"Nie udało się odczytać sekretu '{key_name}': {e}")
            return None

    @staticmethod
    def delete_secret(key_name: str) -> None:
        try:
            keyring.delete_password(SecretsManager.SERVICE_NAME, key_name)
        except keyring.errors.PasswordDeleteError as e:
            logger.error(f"Nie udało się usunąć sekretu '{key_name}': {e}")

===== Code/CORE/shm_manager.py =====
# core/ipc/shm_manager.py
import numpy as np
from multiprocessing import shared_memory
from typing import Tuple, Dict

class SharedImageBuffer:
    """Klasa do obsługi Zero-Copy IPC dla dużych obrazów."""

```

```

@staticmethod
def create(image: np.ndarray) -> Dict:
    """Tworzy blok w pamieci RAM i kopiuje do niego obraz."""
    # Alokacja pamieci o konkretnym rozmiarze
    shm = shared_memory.SharedMemory(create=True, size=image.nbytes)

    # Tworzymy 'widok' NumPy na tej pamieci
    shared_array = np.ndarray(image.shape, dtype=image.dtype, buffer=shm.buf)

    # Kopiujemy dane do wspoldzielonego bloku
    shared_array[:] = image[:]

    return {
        "shm_name": shm.name,
        "shape": image.shape,
        "dtype": str(image.dtype),
        "nbytes": image.nbytes
    }

@staticmethod
def attach(metadata: Dict) -> np.ndarray:
    """Podczyta sie do istniejacego bloku pamieci."""
    shm = shared_memory.SharedMemory(name=metadata["shm_name"])
    return np.ndarray(metadata["shape"], dtype=np.dtype(metadata["dtype"]), buffer=shm.buf)

===== Code/CORE/tasks.py =====
"""Asynchronous workflow tasks powered by Taskiq + NATS JetStream."""
from __future__ import annotations
import asyncio
import json
import os
import uuid
from dataclasses import dataclass
from datetime import datetime, timezone, timedelta
from pathlib import Path
from typing import Any
import lancedb
import msgspec
import psutil
import pyarrow as pa
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from statemachine import State, StateMachine
from taskiq import TaskiqEvents
from taskiq_nats import PullBasedJetStreamBroker
import nats

from core.config import AppConfig
from db.database import create_oltp_engine, create_session_factory, SessionLocal
from models.invoice import ActiveLearningPattern, Invoice
from models.outbox import OutboxEvent, OutboxStatus
from pipeline.ocr import DocumentProcessor, ReviewStatus
from core.logger import get_logger
from core.backup import BackupManager
from Roboton_Reflekton.vision_agent import VisionAgent
from services.fixed_assets import FixedAssetsService
from Roboton_Reflekton.ledger_client import TigerBeetleClient
from db.analytics import DuckDBManager
from Roboton_Reflekton.dunning_engine import DunningEngine

logger = get_logger()

NATS_URL = os.getenv("NEXUS_NATS_URL", "nats://127.0.0.1:4222")
broker = PullBasedJetStreamBroker(servers=NATS_URL, queue="nexus-ai-workers")

```

```

@dataclass(slots=True)
class InvoiceEventPayload:
    """Canonical payload embedded in Outbox events."""
    invoice_id: str
    image_path: str
    contractor_id: str

class InvoiceProcessingMachine(StateMachine):
    """Invoice lifecycle state machine."""
    new = State("NEW", initial=True)
    processing = State("PROCESSING")
    approved = State("APPROVED")
    manual_review = State("MANUAL_REVIEW")
    failed = State("FAILED")

    start = new.to(processing)
    approve = processing.to(approved)
    request_review = processing.to(manual_review)
    fail = processing.to(failed)

def pin_worker_cpu_affinity(reserve_core0: bool = True) -> None:
    """Pin worker process to non-UI CPU cores to protect Flet responsiveness."""
    process = psutil.Process()
    available = list(range(psutil.cpu_count(logical=False) or psutil.cpu_count() or 1))
    if reserve_core0 and len(available) > 1:
        target = [core for core in available if core != 0] or available
    else:
        target = available
    process.cpu_affinity(target)

def _get_lancedb_table() -> Any:
    """Open or create vector table in LanceDB backed by Arrow schema."""
    db = lancedb.connect("nexus_lancedb")
    schema = pa.schema([
        pa.field("id", pa.string()),
        pa.field("invoice_id", pa.string()),
        pa.field("contractor_id", pa.string()),
        pa.field("vector", pa.list_(pa.float32(), 4)),
        pa.field("checksum", pa.string()),
        pa.field("created_at", pa.timestamp("us", tz="UTC")),
        pa.field("is_preferred", pa.bool_()),
    ])
    if "invoice_vectors" in db.table_names():
        return db.open_table("invoice_vectors")
    return db.create_table("invoice_vectors", schema=schema)

def _simple_features(raw_text: str) -> list[float]:
    """Small dense vector placeholder; replace with embedding model output."""
    length = float(len(raw_text))
    digits = float(sum(ch.isdigit() for ch in raw_text))
    letters = float(sum(ch.isalpha() for ch in raw_text))
    lines = float(max(raw_text.count("\n"), 1))
    return [length, digits, letters, lines]

async def _pick_pending_outbox(session: AsyncSession) -> OutboxEvent | None:
    query = (
        select(OutboxEvent)
        .where(OutboxEvent.status == OutboxStatus.PENDING)
        .order_by(OutboxEvent.created_at.asc())
        .limit(1)
    )
    result = await session.execute(query)
    return result.scalar_one_or_none()

```

```

async def _update_invoice_status(session: AsyncSession, invoice_id: str, status: str) -> None:
    invoice = await session.get(Invoice, invoice_id)
    if invoice is None:
        return
    invoice.processing_status = status
    invoice.updated_at = datetime.now(timezone.utc)
    await session.flush()

@broker.on_event(TaskiqEvents.WORKER_STARTUP)
async def _startup(_state: Any) -> None:
    pin_worker_cpu_affinity(reserve_core0=True)

@broker.task(task_name="process_invoice_task")
async def process_invoice_task() -> dict[str, str]:
    """Consume pending outbox event and process invoice OCR + workflow update."""
    config = AppConfig(base_dir=Path.cwd())
    engine = create_oltp_engine(config)
    session_factory = create_session_factory(engine)
    processor = DocumentProcessor()
    vision_agent = VisionAgent()

    async with session_factory() as session:
        event = await _pick_pending_outbox(session)
        if event is None:
            return {"result": "NO_EVENTS"}

        machine = InvoiceProcessingMachine()
        machine.start()
        event.status = OutboxStatus.PROCESSED
        payload = msgspec.json.decode(event.payload.encode("utf-8"), type=InvoiceEventPayload)
        await _update_invoice_status(session, payload.invoice_id, machine.current_state.id)

    try:
        processed = await asyncio.to_thread(processor.process, Path(payload.image_path))
        vision = await vision_agent.analyze(Path(payload.image_path), processed.primary.raw_text)
        vision_payload = {
            "vendor_nip": vision.vendor_nip,
            "total_gross": vision.total_gross,
            "vat_rate": vision.vat_rate,
            "payment_status": vision.payment_status,
            "visual_anomalies_detected": vision.visual_anomalies_detected,
            "handwritten_notes_summary": vision.handwritten_notes_summary,
            "source": vision.source,
        }
        enriched_text = f"{processed.primary.raw_text}\n[vision]{json.dumps(vision_payload, ensure_ascii=False)}"
        vector = _simple_features(enriched_text)
        table = _get_lancedb_table()
        table.add([
            {
                "id": str(uuid.uuid4()),
                "invoice_id": payload.invoice_id,
                "contractor_id": payload.contractor_id,
                "vector": vector,
                "checksum": processed.primary.checksum,
                "created_at": datetime.now(timezone.utc),
                "is_preferred": False,
            }
        ])
        logger.info(
            "VisionAgent(%s) processed invoice_id=%s anomalies=%s",
            vision.source,
            payload.invoice_id,
            vision.visual_anomalies_detected,
        )

    if processed.status == ReviewStatus.MANUAL_REVIEW:
        machine.request_review()

```

```

        else:
            machine.approve()
            await _update_invoice_status(session, payload.invoice_id, machine.current_state.id)

    except Exception as exc:
        machine.fail()
        event.status = OutboxStatus.FAILED
        event.payload = json.dumps({"error": str(exc), "original_payload": event.payload})
        await _update_invoice_status(session, payload.invoice_id, machine.current_state.id)

    await session.commit()
    await engine.dispose()
    return {"result": "OK"}

@broker.task(task_name="store_active_learning_feedback")
async def store_active_learning_feedback(contractor_id: str, corrected_payload: dict[str, Any]) -> dict[str, Any]:
    """Persist user corrections for active learning and preferred retrieval."""
    config = AppConfig(base_dir=Path.cwd())
    engine = create_oltp_engine(config)
    session_factory = create_session_factory(engine)
    table = _get_lancedb_table()
    serialized = json.dumps(corrected_payload, ensure_ascii=False)
    vector = _simple_features(serialized)
    pattern_id = str(uuid.uuid4())

    async with session_factory() as session:
        session.add(
            ActiveLearningPattern(
                id=pattern_id,
                contractor_id=contractor_id,
                correction_payload=serialized,
            )
        )
        await session.commit()

    table.add([
        {
            "id": pattern_id,
            "invoice_id": "",
            "contractor_id": contractor_id,
            "vector": vector,
            "checksum": "",
            "created_at": datetime.now(timezone.utc),
            "is_preferred": True,
        }
    ])

    await engine.dispose()
    return {"result": "LEARNING_SAVED"}

@broker.task(schedule=[{"cron": "0 16 * * *"}])
async def scheduled_backup_task():
    """Codziennie o 16:00"""
    config = AppConfig()
    manager = BackupManager(config)
    path = manager.create_encrypted_zip(config.encryption_key)
    logger.info(f"Backup wykonany pomy?lnie: {path}")

@broker.task(schedule=[{"cron": "55 23 28-31 * *"}], task_name="cron_post_depreciation")
async def cron_post_depreciation() -> dict[str, int | str]:
    """Monthly fixed-assets depreciation posting. Runs on month-end window 23:55 UTC."""
    today = datetime.now(timezone.utc).date()
    if (today + timedelta(days=1)).month == today.month:
        return {"result": "SKIPPED_NOT_MONTH_END", "posted": 0}
    duckdb = DuckDBManager(Path("app_data/nexus_olap.duckdb"))
    tigerbeetle = TigerBeetleClient()

```

```

service = FixedAssetsService(duckdb=duckdb, tigerbeetle=tigerbeetle)
posted = await service.execute_monthly_depreciation(as_of=today)
return {"result": "OK", "posted": posted}

@broker.task(schedule=[{"cron": "* /5 * * * *"}])
async def invoice_reconciliation_loop():
    """Wyszukuje porzucone faktury i podejmuje akcje naprawcze."""
    logger.info("[Watchdog] Uruchamianie skanowania spjno?ci...")
    timeout_threshold = datetime.utcnow() - timedelta(minutes=10)

    async with SessionLocal() as session:
        stmt = select(Invoice).where(
            Invoice.status == "PROCESSING",
            Invoice.updated_at <= timeout_threshold
        )
        result = await session.execute(stmt)
        stuck_invoices = result.scalars().all()

        if not stuck_invoices:
            logger.debug("[Watchdog] System w pe?ni spjny. Brak porzuconych zada?.")
            return

        nc = await nats.connect("nats://localhost:4222")
        for invoice in stuck_invoices:
            if invoice.retry_count < 3:
                logger.warning(
                    f"[Watchdog] Faktura ID: {invoice.id} utkn??a. "
                    f"Prba {invoice.retry_count + 1}/3. Re-kolejkowanie..."
                )
                invoice.retry_count += 1
                invoice.updated_at = datetime.utcnow()
                payload = json.dumps({
                    "invoice_id": invoice.id,
                    "file_path": invoice.file_path,
                    "is_retry": True
                })
                await nc.publish("invoices.new", payload.encode())
            else:
                logger.error(
                    f"[Watchdog] Faktura ID: {invoice.id} trwale uszkadza Workera. "
                    f"Zatrzymano prby. Status -> ERROR: TIMEOUT"
                )
                invoice.status = "ERROR: TIMEOUT"
                invoice.updated_at = datetime.utcnow()
                error_payload = json.dumps({
                    "status": "FAILED",
                    "message": "Przekroczono limit czasu (Krytyczny b??d przetwarzania)."
                })
                await nc.publish(f"invoices.status.{invoice.id}", error_payload.encode())

        await session.commit()
        await nc.close()

class _DefaultDunningAIAgent:
    def generate_dunning_text(self, invoice_data: dict[str, Any], vendor_score: float, level: int) -> str:
        tone = "uprzejmy" if vendor_score >= 0.8 else "stanowczy"
        return (
            f"To automatyczne przypomnienie ({tone}, poziom {level}) dla faktury {invoice_data['invoice_number']} "
            f"na kwot? {invoice_data['balance_due']:.2f} PLN. "
            f"Zaleg?o???: {invoice_data['days_overdue']} dni."
        )

class _DefaultEmailProvider:

```

```

def send(self, *, to_email: str, subject: str, body: str) -> bool:
    if not to_email:
        return False
    logger.info("[Dunning] Wysy?ka email to=%s subject=%s", to_email, subject)
    return True

@broker.task(task_name="run_daily_dunning_check", schedule=[{"cron": "0 9 * * *"}])
async def run_daily_dunning_check() -> dict[str, int]:
    """Codzienna kontrola nale?no?ci i wysy?ka przypomnie? (09:00)."""
    config = AppConfig(base_dir=Path.cwd())
    duckdb = DuckDBManager(config.duckdb_path)
    engine = DunningEngine(
        duckdb_manager=duckdb,
        ai_agent=_DefaultDunningAIAgent(),
        email_provider=_DefaultEmailProvider(),
    )
    return await engine.run_daily_dunning_check()

@broker.task(task_name="execute_monthly_depreciation", schedule=[{"cron": "0 0 1 * *"}])
async def execute_monthly_depreciation_task() -> dict[str, int]:
    """Posts due depreciation entries to TigerBeetle on the 1st day of each month."""
    duckdb = DuckDBManager(Path("app_data/nexus_olap.duckdb"))
    service = FixedAssetsService(duckdb=duckdb, tigerbeetle=TigerBeetleClient())
    posted = await service.execute_monthly_depreciation()
    logger.info("[FixedAssets] Posted %s depreciation entries", posted)
    return {"posted": posted}

===== Code/CORE/tenant.py =====
from __future__ import annotations

import re
from contextvars import ContextVar
from pathlib import Path

from core.config import AppConfig

DEFAULT_TENANT_ID = "default"
_tenant_id_ctx: ContextVar[str] = ContextVar("tenant_id", default=DEFAULT_TENANT_ID)

class TenantManager:
    """Resolves and creates per-tenant local storage layout."""

    _allowed_tenant_pattern = re.compile(r"^[a-zA-Z0-9_-]+$")

    def __init__(self, config: AppConfig, root_dir_name: str = "local_data") -> None:
        self._config = config
        self._root = config.base_dir / root_dir_name
        self._root.mkdir(parents=True, exist_ok=True)

    def sanitize_tenant_id(self, tenant_id: str | None) -> str:
        candidate = (tenant_id or "").strip() or DEFAULT_TENANT_ID
        if not self._allowed_tenant_pattern.fullmatch(candidate):
            raise ValueError("tenant_id can only contain letters, numbers, '_' and '-'")
        return candidate

    def resolve_tenant_dir(self, tenant_id: str | None = None) -> Path:
        safe_tenant_id = self.sanitize_tenant_id(tenant_id or get_current_tenant_id())
        tenant_dir = self._root / safe_tenant_id
        tenant_dir.mkdir(parents=True, exist_ok=True)
        return tenant_dir

    def sqlite_path(self, tenant_id: str | None = None) -> Path:

```

```

        return self.resolve_tenant_dir(tenant_id) / self._config.sqlite_file_name

    def duckdb_path(self, tenant_id: str | None = None) -> Path:
        return self.resolve_tenant_dir(tenant_id) / self._config.duckdb_file_name

def set_current_tenant_id(tenant_id: str) -> object:
    return _tenant_id_ctx.set(tenant_id)

def reset_current_tenant_id(token: object) -> None:
    _tenant_id_ctx.reset(token)

def get_current_tenant_id() -> str:
    return _tenant_id_ctx.get()

===== Code/CORE/tracing.py =====
# core/tracing.py
import uuid
from contextvars import ContextVar
from loguru import logger
import sys

# Zmienna kontekstowa unikalna dla ka?dego "requestu"
correlation_id_ctx: ContextVar[str] = ContextVar("correlation_id", default="system")

def init_trace() -> str:
    """Tworzy nowe ID dla nowego dokumentu."""
    cid = str(uuid.uuid4())
    correlation_id_ctx.set(cid)
    return cid

def setup_tracing_logger():
    logger.remove()
    # Konfiguracja logera, aby zawsze wypisywa? correlation_id, je?li istnieje
    fmt = "<green>{time:YYYY-MM-DD HH:mm:ss}</green> | <level>{level: <8}</level> | <cyan>{extra[correlation_id]}"
    logger.add(sys.stderr, format=fmt, enqueue=True)

===== Code/CORE/updater.py =====
# core/updater.py
import httpx
from packaging import version
import os

CURRENT_VERSION = "1.0.0"
REPO_URL = "[https://api.github.com/repos/TwojLogin/NexusAccounting/releases/latest](https://api.github.com/repos/TwojLogin/NexusAccounting/releases/latest)"

async def check_for_updates() -> dict:
    """Sprawdza, czy na GitHubie jest nowsza wersja."""
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            response = await client.get(REPO_URL)
            if response.status_code == 200:
                latest_release = response.json()
                latest_version = latest_release["tag_name"].replace("v", "")

                if version.parse(latest_version) > version.parse(CURRENT_VERSION):
                    download_url = latest_release["assets"][0]["browser_download_url"]
                    return {"update_available": True, "version": latest_version, "url": download_url}
    except Exception as e:
        print(f"[Updater] B??d: {e}")
    return {"update_available": False}

async def download_and_swap_update(download_url: str, current_exe: str, new_exe_path: str, old_exe_path: str):

```

```

try:
    async with httpx.AsyncClient() as client:
        async with client.stream("GET", download_url) as response:
            with open(new_exe_path, 'wb') as f:
                for chunk in response.iter_bytes():
                    f.write(chunk)

    # Podmiana nazw (Swap)
    if os.path.exists(old_exe_path):
        os.remove(old_exe_path)

    os.rename(current_exe, old_exe_path)
    os.rename(new_exe_path, current_exe)
    return True
except Exception as e:
    print(f"[Updater] B??d aktualizacji: {e}")
    return False

```

===== Code/DB/__init__.py =====

"""Database layer packages (OLTP and OLAP)."""

```

from db.analytics import DuckDBLimits, DuckDBManager
from db.database import Base, create_oltp_engine, create_session_factory, get_session
from db.outbox import OutboxManager
from db.transaction import atomic_transaction
from db.views import AnalyticsViewsSetup
from db.hooks import register_db_hooks

```

```

__all__ = [
    "Base",
    "DuckDBLimits",
    "DuckDBManager",
    "create_oltp_engine",
    "create_session_factory",
    "get_session",
    "OutboxManager",
    "atomic_transaction",
    "AnalyticsViewsSetup",
    "register_db_hooks"
]

```

===== Code/DB/analytics.py =====

```

import duckdb
from pathlib import Path
from threading import Lock
from typing import Any
from dataclasses import dataclass
from core.config import AppConfig

```

@dataclass(slots=True)

```

class DuckDBLimits:
    """Hard limits for local analytical engine."""
    memory_limit: str = "2GB"
    threads: int = 2

```

class DuckDBManager:

"""Thread-safe manager for a singleton DuckDB connection."""

```

def __init__(self, db_path: Path, limits: DuckDBLimits = DuckDBLimits()) -> None:
    self._db_path = db_path
    self._limits = limits
    self._lock = Lock()
    self._connection: duckdb.DuckDBPyConnection | None = None

```

```

def connect(self) -> duckdb.DuckDBPyConnection:

```

```

        """Open (or return) connection with enforced limits."""
        with self._lock:
            if self._connection is None:
                connection = duckdb.connect(str(self._db_path))
                connection.execute(f"SET memory_limit='{self._limits.memory_limit}'")
                connection.execute(f"SET threads={self._limits.threads}")
                self._connection = connection
            return self._connection

def execute(self, query: str, parameters: tuple[Any, ...] | None = None) -> list[tuple[Any, ...]]:
    """Execute query and return materialized rows."""
    connection = self.connect()
    if parameters:
        return connection.execute(query, parameters).fetchall()
    return connection.execute(query).fetchall()

def close(self) -> None:
    """Close connection if currently open."""
    with self._lock:
        if self._connection is not None:
            self._connection.close()
            self._connection = None

def upsert_invoice(self, invoice_data: dict):
    """Wstawia lub aktualizuje fakturę w DuckDB."""
    query = """
    INSERT OR REPLACE INTO invoices_replica (
        id, number, contractor_nip, amount_net,
        amount_gross, currency, status, updated_at
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """
    params = (
        str(invoice_data['id']),
        invoice_data['number'],
        invoice_data['contractor_nip'],
        float(invoice_data['amount_net']),
        float(invoice_data['amount_gross']),
        invoice_data['currency'],
        invoice_data['status'],
        invoice_data['updated_at']
    )
    self.execute(query, params)

def full_sync_from_sqlite(self, invoices: list[dict]):
    """Masowy import przy starcie aplikacji, aby wyrównać bazy."""
    with self._lock:
        # Czyścimy starą replikę i dodajemy nowe dane
        self.execute("DELETE FROM invoices_replica")
        for inv in invoices:
            self.upsert_invoice(inv)

class DuckDBAnalyticsEngine:
    """Silnik OLAP z limitami pamięci, odczytem Parquet i Cache KSeF."""

    def __init__(self, db_path: str = "app_data/nexus_olap.duckdb"):
        self.conn = duckdb.connect(db_path)
        # Konfiguracja limitów DuckDB (żeby nie zawiesił komputera księgowego)
        self.conn.execute("PRAGMA memory_limit='2GB'")
        self.conn.execute("PRAGMA threads=2;")

        # Tworzymy wirtualną tabelę na cache KSeF
        self.conn.execute("""
        CREATE TABLE IF NOT EXISTS ksef_cache (
            nip VARCHAR,
            invoice_number VARCHAR,

```

```

        issue_date DATE,
        ksef_reference VARCHAR
    );
    """
)

```

```

def check_ksef_cache(self, nip: str, invoice_number: str) -> str:
    """B?yskawiczne filtrowanie milionw rekordw lokalnie."""
    res = self.conn.execute("""
    SELECT ksef_reference
    FROM ksef_cache
    WHERE nip = ? AND invoice_number = ?
    """, (nip, invoice_number)).fetchone()
    return res[0] if res else None

```

```

def archive_old_data_to_parquet(self, year: int, output_dir: Path):
    """Archiwizacja na zimno (Cold Storage)."""
    # Eksportujemy tabel? z SQLite prosto do silnie skompresowanego pliku Parquet
    parquet_path = output_dir / f"invoices_{year}.parquet"
    self.conn.execute(f"""
    COPY (SELECT * FROM sqlite_scan('app_data/dbs/{year}_nexus_oltp.db', 'invoices'))
    TO '{parquet_path}' (FORMAT PARQUET);
    """)

```

```

def query_cross_years(self, archive_dir: Path):
    """Zapytania analityczne czytaj?ce pliki Parquet w locie."""
    # DuckDB mo?e traktowa? dziesi?tki plikw Parquet jak jedn? tabel?
    query = f"""
    SELECT contractor_nip, SUM(amount_gross) as total_spent
    FROM read_parquet('{archive_dir}/*.parquet')
    GROUP BY contractor_nip
    ORDER BY total_spent DESC
    LIMIT 10;
    """
    return self.conn.execute(query).df()

```

```

class CashflowPredictor:

```

```

    def __init__(self, conn):
        self.conn = conn

```

```

    def get_contractor_reliability(self):
        """Oblicza ?rednie sp?nienie dla ka?dego kontrahenta."""
        return self.conn.execute("""
        SELECT
            contractor_nip,
            contractor_name,
            AVG(date_diff('day', due_date, payment_date)) as avg_delay_days,
            COUNT(*) as invoice_count
        FROM invoices_replica
        WHERE status = 'PAID' AND payment_date IS NOT NULL
        GROUP BY 1, 2
        HAVING invoice_count > 2
        """).df()

```

```

    def predict_liquidity_gap(self, days_ahead=30):
        """Prognozuje saldo na podstawie faktur do zap?acenia, koryguj?c dat? o historyczne sp?nienia."""
        query = """
        WITH raw_data AS (
            SELECT
                i.amount_gross,
                -- Je?li mamy histori?, dodajemy ?rednie sp?nienie do daty wymagalno?ci
                COALESCE(i.due_date + INTERVAL (rel.avg_delay_days) DAY, i.due_date) as expected_date,
                i.type -- 'PURCHASE' (minus) lub 'SALE' (plus)
            FROM invoices_replica i
            LEFT JOIN (
                SELECT contractor_nip, AVG(date_diff('day', due_date, payment_date)) as avg_delay_days

```

```

        FROM invoices_replica
        WHERE status = 'PAID'
        GROUP BY 1
    ) rel ON i.contractor_nip = rel.contractor_nip
    WHERE i.status != 'PAID'
)
SELECT
    expected_date,
    SUM(CASE WHEN type = 'SALE' THEN amount_gross ELSE -amount_gross END) as daily_delta
FROM raw_data
WHERE expected_date <= current_date + INTERVAL (?) DAY
GROUP BY 1
ORDER BY 1
"""
    return self.conn.execute(query, [days_ahead]).df()

```

```
class FraudDetector:
```

```
    def __init__(self, conn):
        self.conn = conn
```

```
    def detect_round_number_spikes(self):
```

```
        """
```

```
        Wykrywa nienaturalne nagromadzenie faktur na równe kwoty
        pod koniec miesiąca od nowych dostawców.
        """
```

```
        return self.conn.execute("""
```

```
SELECT
```

```
    contractor_nip,
```

```
    COUNT(*) as suspected_invoices,
```

```
    SUM(amount_gross) as total_value,
```

```
    -- Sprawdzamy czy kwoty kończą się na 00 lub 000 (np. 10 000, 5000)
```

```
    AVG(CASE WHEN amount_gross % 100 = 0 THEN 1 ELSE 0 END) as round_numbers_ratio
```

```
FROM invoices_replica
```

```
WHERE
```

```
    -- Ostatnie 5 dni dowolnego miesiąca
```

```
    day(issue_date) > 25
```

```
    -- Faktury kosztowe
```

```
    AND type = 'PURCHASE'
```

```
    -- Kontrahenci, z którymi nie było historii starszej niż 30 dni
```

```
    AND contractor_nip IN (
```

```
        SELECT contractor_nip FROM invoices_replica
```

```
        GROUP BY 1
```

```
        HAVING min(issue_date) > current_date - INTERVAL 30 DAY
```

```
    )
```

```
GROUP BY 1
```

```
HAVING suspected_invoices >= 3 AND round_numbers_ratio > 0.8
```

```
ORDER BY total_value DESC
```

```
""").df()
```

```
===== Code/DB/analytics_schema.py =====
```

```
from db.analytics import DuckDBManager
```

```
from services.telemetry import ensure_telemetry_schema
```

```
from db.zpk_schema import ensure_zpk_schema
```

```
from services.document_fingerprint import ensure_fingerprint_schema
```

```
from importlib import import_module
```

```
from importlib.util import find_spec
```

```
from typing import Callable
```

```
SchemaHook = Callable[[DuckDBManager], None]
```

```
OptionalHook = tuple[str, str]
```

```
def _run_optional_schema_hook(module_name: str, function_name: str, duck_mgr: DuckDBManager) -> None:
```

```
    """Run optional schema initializer only when module is available in runtime."""
```

```

    if find_spec(module_name) is None:
        return

    hook = getattr(import_module(module_name), function_name, None)
    if callable(hook):
        hook(duck_mgr)

def _ensure_column(
    duck_mgr: DuckDBManager,
    table_name: str,
    known_columns: set[str],
    column_name: str,
    ddl_suffix: str,
) -> None:
    """Add missing column and update the local column registry."""
    if column_name in known_columns:
        return

    duck_mgr.execute(f"ALTER TABLE {table_name} ADD COLUMN {column_name} {ddl_suffix}")
    known_columns.add(column_name)

class AnalyticsSchemaManager:
    """Zarządza wersjonowaniem schematu DuckDB (odpowiednik migracji)."""

    @staticmethod
    def ensure_latest_schema(duck_mgr: DuckDBManager):
        """Dodaje brakujące kolumny do repliki, jeżeli schemat się zmieni."""

        # Przykład: Dodanie kolumny issue_date, jeżeli nie istnieje w starszej wersji bazy
        existing_cols = duck_mgr.execute("PRAGMA table_info('invoices_replica')")
        col_names = [col[1] for col in existing_cols]

        if 'issue_date' not in col_names:
            duck_mgr.execute("ALTER TABLE invoices_replica ADD COLUMN issue_date DATE")

        # Tworzenie indeksów dla przyspieszenia raportów OLAP
        duck_mgr.execute("CREATE INDEX IF NOT EXISTS idx_invoices_date ON invoices_replica (issue_date)")
        duck_mgr.execute("CREATE INDEX IF NOT EXISTS idx_invoices_status ON invoices_replica (status)")

        core_schema_hooks: tuple[SchemaHook, ...] = (
            ensure_telemetry_schema,
            ensure_zpk_schema,
            ensure_fingerprint_schema,
            ensure_vendor_intelligence_schema,
            ensure_fixed_assets_schema,
            ensure_rmk_schema,
            ensure_inventory_schema,
        )
        for hook in core_schema_hooks:
            hook(duck_mgr)

        optional_schema_hooks: tuple[OptionalHook, ...] = (
            ("services.shadow_resource_correlation", "ensure_shadow_resource_schema"),
            ("services.audit_logger", "ensure_forensic_audit_schema"),
            ("services.rules_engine", "ensure_accounting_template_schema"),
            ("services.compliance_analytics", "ensure_compliance_analytics_schema"),
            ("services.fx_revaluation", "ensure_fx_schema"),
            ("services.smart_approvals", "ensure_smart_approval_schema"),
            ("services.liquidity_oracle", "ensure_liquidity_schema"),
        )
        for module_name, function_name in optional_schema_hooks:
            _run_optional_schema_hook(module_name, function_name, duck_mgr)

```

```

def ensure_vendor_intelligence_schema(duck_mgr: DuckDBManager):
    duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS vendor_intelligence (
    nip VARCHAR PRIMARY KEY,
    vendor_name VARCHAR,
    avg_payment_delay DOUBLE,
    price_volatility_index DOUBLE,
    total_volume_ytd DOUBLE,
    reliability_score DOUBLE,
    rating_stars INTEGER,
    smart_alerts VARCHAR,
    updated_at TIMESTAMP DEFAULT now()
)
""")

    duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS vendor_price_alerts (
    id UUID DEFAULT uuid(),
    nip VARCHAR,
    item_name VARCHAR,
    current_price DOUBLE,
    six_month_avg DOUBLE,
    increase_ratio DOUBLE,
    alert_type VARCHAR,
    detected_at TIMESTAMP DEFAULT now()
)
""")

def ensure_fixed_assets_schema(duck_mgr: DuckDBManager):
    duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS fixed_assets (
    id UUID PRIMARY KEY,
    invoice_id UUID,
    asset_name VARCHAR NOT NULL,
    initial_value DECIMAL(18, 2) NOT NULL,
    salvage_value DECIMAL(18, 2) NOT NULL DEFAULT 0,
    residual_value DECIMAL(18, 2) NOT NULL DEFAULT 0,
    depreciation_method VARCHAR NOT NULL DEFAULT 'LINEAR',
    annual_rate DOUBLE,
    depreciation_rate DOUBLE NOT NULL,
    start_date DATE,
    purchase_date DATE NOT NULL,
    last_depreciation_date DATE,
    status VARCHAR NOT NULL DEFAULT 'ACTIVE',
    account_id_debit UBIGINT NOT NULL,
    account_id_credit UBIGINT NOT NULL
)
""")

    existing_cols = duck_mgr.execute("PRAGMA table_info('fixed_assets')")
    col_names = {col[1] for col in existing_cols}
    _ensure_column(duck_mgr, "fixed_assets", col_names, "residual_value", "DECIMAL(18, 2) NOT NULL DEFAULT 0")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "salvage_value", "DECIMAL(18, 2) NOT NULL DEFAULT 0")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "depreciation_method", "VARCHAR NOT NULL DEFAULT 'LINEAR'")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "annual_rate", "DOUBLE")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "start_date", "DATE")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "invoice_id", "UUID")
    _ensure_column(duck_mgr, "fixed_assets", col_names, "last_depreciation_date", "DATE")

    duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS depreciation_schedule (
    asset_id UUID NOT NULL,
    planned_date DATE NOT NULL,

```

```

        amount DECIMAL(18, 2) NOT NULL,
        is_posted BOOLEAN NOT NULL DEFAULT FALSE,
        status VARCHAR NOT NULL DEFAULT 'PENDING',
        ledger_id INTEGER NOT NULL DEFAULT 2,
        transfer_code INTEGER NOT NULL DEFAULT 1001,
        posted_at TIMESTAMP,
        PRIMARY KEY(asset_id, planned_date)
    )
    """
    schedule_cols = duck_mgr.execute("PRAGMA table_info('depreciation_schedule')")
    schedule_col_names = {col[1] for col in schedule_cols}
    _ensure_column(duck_mgr, "depreciation_schedule", schedule_col_names, "status", "VARCHAR NOT NULL DEFA

def ensure_rmk_schema(duck_mgr: DuckDBManager):
    duck_mgr.execute("""
    CREATE TABLE IF NOT EXISTS deferred_expenses (
        id UUID PRIMARY KEY,
        invoice_id UUID NOT NULL,
        description VARCHAR NOT NULL,
        total_net_amount DECIMAL(18, 2) NOT NULL,
        start_date DATE NOT NULL,
        end_date DATE NOT NULL,
        total_days INTEGER NOT NULL,
        daily_rate DECIMAL(18, 4) NOT NULL,
        status VARCHAR NOT NULL DEFAULT 'PENDING',
        cost_account_id VARCHAR NOT NULL
    )
    """)

    duck_mgr.execute("""
    CREATE TABLE IF NOT EXISTS rmk_ledger_entries (
        id UUID PRIMARY KEY,
        deferred_id UUID NOT NULL,
        posting_date DATE NOT NULL,
        amount DECIMAL(18, 2) NOT NULL,
        is_posted BOOLEAN NOT NULL DEFAULT FALSE,
        ledger_id INTEGER NOT NULL DEFAULT 1,
        transfer_code INTEGER NOT NULL DEFAULT 2001,
        rmk_asset_account_id VARCHAR NOT NULL DEFAULT '640',
        user_data_128 UUID NOT NULL
    )
    """)

def ensure_inventory_schema(duck_mgr: DuckDBManager):
    duck_mgr.execute("""
    CREATE TABLE IF NOT EXISTS inventory_batches (
        id UUID PRIMARY KEY,
        batch_id VARCHAR NOT NULL,
        product_id VARCHAR NOT NULL,
        warehouse_id VARCHAR,
        received_date DATE NOT NULL,
        source_document_id UUID,
        initial_qty DECIMAL(18, 4) NOT NULL,
        remaining_qty DECIMAL(18, 4) NOT NULL,
        unit_cost_net DECIMAL(18, 4) NOT NULL,
        currency VARCHAR NOT NULL DEFAULT 'PLN',
        created_at TIMESTAMP NOT NULL DEFAULT now(),
        CHECK (initial_qty >= 0),
        CHECK (remaining_qty >= 0),
        CHECK (remaining_qty <= initial_qty)
    )
    """)

```

```

duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS inventory_consumption_events (
    id UUID PRIMARY KEY,
    product_id VARCHAR NOT NULL,
    issue_document_id UUID NOT NULL,
    issue_date DATE NOT NULL,
    issued_qty DECIMAL(18, 4) NOT NULL,
    total_cogs_net DECIMAL(18, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now()
)
""")

```

```

duck_mgr.execute("""
CREATE TABLE IF NOT EXISTS inventory_consumption_lines (
    id UUID PRIMARY KEY,
    consumption_event_id UUID NOT NULL,
    batch_id VARCHAR NOT NULL,
    product_id VARCHAR NOT NULL,
    qty_taken DECIMAL(18, 4) NOT NULL,
    unit_cost_net DECIMAL(18, 4) NOT NULL,
    line_cogs_net DECIMAL(18, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now()
)
""")

```

===== Code/DB/backup.py =====

```

import sqlite3
import asyncio
from pathlib import Path
from datetime import datetime, timezone
from core.config import AppConfig
import logging

logger = logging.getLogger("nexus.db.backup")

```

class DatabaseHotBackup:

"""Wykonuje gor?c? kopi? bazy bez przerywania pracy systemu (Zero-Downtime)."""

@staticmethod

```

async def create_snapshot(config: AppConfig, backup_dir: Path) -> Path:
    """Wykorzystuje natywne API SQLite do zrzutu pamieci WAL na dysk."""
    backup_dir.mkdir(parents=True, exist_ok=True)
    timestamp = int(datetime.now(timezone.utc).timestamp())
    backup_file = backup_dir / f"nexus_oltp_snapshot_{timestamp}.sqlite"

```

def _execute_backup():

```

    source_db = config.sqlite_path.as_posix()
    try:
        # Otwieramy po??czenie synchroniczne tylko do zrobienia kopii
        with sqlite3.connect(source_db) as src, sqlite3.connect(backup_file) as dst:
            src.backup(dst)
    except Exception as e:
        logger.error(f"Backup failed: {e}")
        raise e

```

```

    # Uruchamiamy w osobnym w?tku, aby nie blokowa? p?tli zdarze?
    await asyncio.to_thread(_execute_backup)
    return backup_file

```

===== Code/DB/database.py =====

"""Async SQLAlchemy setup for SQLite/SQLCipher (OLTP)."""

```

from __future__ import annotations

```

```

import os

```

```

import logging
from pathlib import Path
from collections.abc import AsyncGenerator

from sqlalchemy import event, text, create_engine
from sqlalchemy.ext.asyncio import AsyncEngine, AsyncSession, async_sessionmaker, create_async_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from core.config import AppConfig

logger = logging.getLogger("nexus.db.cleanup")

class Base(DeclarativeBase):
    """Declarative base for OLTP SQLAlchemy models."""
    pass

def _sqlite_url(config: AppConfig, sqlite_path: Path | None = None) -> str:
    """Build async SQLite URL from configuration."""
    path = sqlite_path or config.sqlite_path
    return f"sqlite+aiosqlite:///{"path.as_posix()}"

def create_oltp_engine(
    config: AppConfig,
    *,
    sqlcipher_key: str | None = None,
    sqlite_path: Path | None = None,
) -> AsyncEngine:
    """
    Create an async SQLAlchemy engine with mandatory PRAGMA settings.
    The same hook can bootstrap SQLCipher key when provided either directly
    or via 'NEXUS_SQLCIPHER_KEY' environment variable.
    """
    engine = create_async_engine(
        _sqlite_url(config, sqlite_path),
        echo=False
    )
    return engine

def create_session_factory(engine: AsyncEngine) -> async_sessionmaker[AsyncSession]:
    return async_sessionmaker(bind=engine, class_=AsyncSession, expire_on_commit=False)

async def init_schema(engine: AsyncEngine) -> None:
    """Create all SQLAlchemy tables for first application start."""
    from models.outbox import OutboxEvent # noqa: F401
    from models.invoice import ActiveLearningPattern, Invoice # noqa: F401

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

async def consolidate_database(engine: AsyncEngine) -> None:
    """
    Zamyka sesje i wykonuje TRUNCATE checkpoint.
    Usuwa pliki WAL i przenosi dane do g?wnego pliku .sqlite.
    Powinno by? wywo?ane w p?tli zamykaj?cej aplikacj? (main.py).
    """
    try:
        async with engine.connect() as conn:
            # PRAGMA wal_checkpoint(TRUNCATE) czy?ci logi i resetuje plik WAL do zera
            await conn.execute(text("PRAGMA wal_checkpoint(TRUNCATE);"))
    except Exception as e:
        logger.error(f"Failed to consolidate database: {e}")

def get_encrypted_engine():
    """Przyk?ad synchronicznego silnika SQLCipher (je?li potrzebne)."""
    config = AppConfig()

```

```

db_path = config.base_dir / "app_data" / "nexus_oltp.db"

# URL po??czenia wykorzystuje dialekt sqlite+pysqlcipher i przekazuje has?o
db_url = f"sqlite+pysqlcipher://:{config.db_password}@/{db_path.as_posix()}"
engine = create_engine(db_url)
return engine

===== Code/DB/hooks.py =====
from sqlalchemy import event
from models.invoice import Invoice
from db.analytics import DuckDBManager
from core.config import AppConfig

def register_db_hooks(config: AppConfig):
    """Rejestruje hooki, kt?e automatycznie replikuj? dane do DuckDB po ka?dym komicie."""

    # Inicjalizacja managera analityki
    duck_mgr = DuckDBManager(config)

    @event.listens_for(Invoice, 'after_insert')
    @event.listens_for(Invoice, 'after_update')
    def replicate_to_duckdb(mapper, connection, target):
        """Automatyczny Upsert do DuckDB przy zmianie w SQLite."""
        data = {
            "id": str(target.id),
            "number": target.number,
            "contractor_nip": target.contractor_nip,
            "amount_net": float(target.amount_net) if target.amount_net else 0.0,
            "amount_gross": float(target.amount_gross) if target.amount_gross else 0.0,
            "currency": target.currency,
            "status": target.status,
            "updated_at": target.updated_at
        }
        duck_mgr.upsert_invoice(data)

===== Code/DB/mixins.py =====
from sqlalchemy import Column, Boolean, DateTime
from datetime import datetime, timezone

class SoftDeleteMixin:
    """Enterprise-grade soft delete. Rekordy nigdy nie znikaj? fizycznie z bazy (Wymg Audytowy)."""

    is_deleted = Column(Boolean, default=False, index=True)
    deleted_at = Column(DateTime, nullable=True)

    def soft_delete(self) -> None:
        """Oznacza rekord jako usuni?ty zamiast kasowa? go z dysku."""
        self.is_deleted = True
        self.deleted_at = datetime.now(timezone.utc)

    def restore(self) -> None:
        """Przywraca usuni?ty rekord."""
        self.is_deleted = False
        self.deleted_at = None

===== Code/DB/models.py =====
from sqlalchemy import Column, Integer, String, Boolean, Float, DateTime, ForeignKey, Text
from sqlalchemy.orm import declarative_base
from sqlalchemy.sql import func
from datetime import datetime
import uuid

Base = declarative_base()

class AuditLog(Base):

```

```

__tablename__ = "audit_logs"

id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
invoice_id = Column(String, ForeignKey("invoices.id"), index=True)

# Kto dokona? zmiany (np. "System/OCR" lub "Ksi?gowa_Kasia")
user_id = Column(String, default="System")

# Co zmieniono (np. "amount_net")
field_changed = Column(String)

# Historia (przechowywana jako tekst, nawet dla liczb)
old_value = Column(Text, nullable=True)
new_value = Column(Text)

timestamp = Column(DateTime, default=datetime.utcnow)

class OutboxEvent(Base):
    __tablename__ = "outbox_events"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    event_type = Column(String, nullable=False)
    aggregate_id = Column(String, nullable=False)
    payload = Column(Text, nullable=False)
    status = Column(String, default="PENDING")
    processed = Column(Boolean, default=False)
    created_at = Column(DateTime, default=func.now())

class SecurityAlert(Base):
    __tablename__ = "security_alerts"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    actor = Column(String, nullable=False)
    operation = Column(String, nullable=False)
    details = Column(Text, nullable=False)
    created_at = Column(DateTime, default=func.now())

===== Code/DB/outbox.py =====
import json
import asyncio
from datetime import datetime, timezone
from typing import Any
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from db.models import OutboxEvent

class OutboxManager:
    """Zarz?dza wzorcem Transactional Outbox dla gwarantowanej dostawy zdarze?."""

    @staticmethod
    async def publish_event(
        session: AsyncSession,
        event_type: str,
        payload: dict[str, Any],
        aggregate_id: str
    ) -> None:
        """
        Zapisuje zdarzenie (np. eksport_erp, wysylka_ksef) w tej samej transakcji co dane.
        Zostanie wypchni?te do NATS przez oddzielnego workera.
        """
        event = OutboxEvent(
            event_type=event_type,
            aggregate_id=aggregate_id,
            payload=json.dumps(payload, ensure_ascii=False),

```

```

        status="PENDING",
        created_at=datetime.now(timezone.utc)
    )
    session.add(event)

# Pseudo-kod workera wyciagajacy dane do NATS
async def _outbox_processor(session_factory, nats_client):
    while True:
        async with session_factory() as session:
            # Pobierz nieprzetworzone zdarzenia
            events = await session.execute(select(OutboxEvent).where(OutboxEvent.processed == False))

            for event in events.scalars():
                try:
                    await nats_client.publish(event.topic, event.payload.encode())
                    event.processed = True
                    await session.commit()
                except Exception:
                    await session.rollback()

            await asyncio.sleep(1) # Oddech dla bazy

===== Code/DB/pagination.py =====
from typing import Type, TypeVar, Tuple, Optional
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from db.database import Base

T = TypeVar('T', bound=Base)

class CursorPagination:
    """Wydajna paginacja (Keyset/Cursor Pagination) dla milionw rekordw."""

    @staticmethod
    async def get_page(
        session: AsyncSession,
        model: Type[T],
        last_id: str | None = None,
        limit: int = 50,
        exclude_deleted: bool = True
    ) -> Tuple[list[T], Optional[str]]:
        """Pobiera nastepna strone wynikw bazujac na ID ostatniego elementu."""

        query = select(model).order_by(model.id.asc()).limit(limit)

        # Filtrujemy usuniete, jeeli model korzysta z SoftDeleteMixin
        if exclude_deleted and hasattr(model, 'is_deleted'):
            query = query.where(model.is_deleted == False)

        if last_id:
            query = query.where(model.id > last_id)

        result = await session.execute(query)
        items = result.scalars().all()

        next_id = items[-1].id if items else None
        return items, next_id

===== Code/DB/replication.py =====
import pandas as pd
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from loguru import logger
from models.invoice import Invoice
from db.analytics import DuckDBManager

```

```

from core.config import AppConfig
from core.resilience.circuit_breaker import CircuitBreaker

async def sync_sqlite_to_duckdb(db_session: AsyncSession, config: AppConfig):
    """Most replikacyjny z mechanizmem Watermark."""
    duck_mgr = DuckDBManager(config)

    # 1. Pobierz czas ostatniej synchronizacji z DuckDB
    last_sync = duck_mgr.conn.execute("SELECT MAX(updated_at) FROM invoices_replica").fetchone()[0]

    # 2. Wybierz tylko rekordy zmienione po tej dacie
    query = select(Invoice)
    if last_sync:
        query = query.where(Invoice.updated_at > last_sync)

    result = await db_session.execute(query)
    invoices = result.scalars().all()

    if not invoices:
        return {"status": "up-to-date", "count": 0}

    # 3. Masowy Upsert do DuckDB (przez DataFrame)
    data = [{
        "id": str(inv.id),
        "number": inv.number,
        "contractor_nip": inv.contractor_nip,
        "amount_net": float(inv.amount_net),
        "amount_gross": float(inv.amount_gross),
        "currency": inv.currency,
        "status": inv.status,
        "updated_at": inv.updated_at
    } for inv in invoices]

    df = pd.DataFrame(data)

    # Atomowe wstawienie do DuckDB
    duck_mgr.conn.execute("INSERT INTO invoices_replica SELECT * FROM df ON CONFLICT (id) DO UPDATE SET AL

    return {"status": "success", "count": len(invoices)}

class ReplicationBridge:
    def __init__(self, duckdb_conn):
        self.duck = duckdb_conn
        # Inicjalizujemy Bezpiecznik: max 3 błędów, 60 sekund blokady
        self.breaker = CircuitBreaker(failure_threshold=3, recovery_timeout=60)

    async def replicate_invoice(self, invoice_data: dict) -> bool:
        """Kopiuje dane faktury do DuckDB, chroni?c si? Circuit Breakerem."""
        # 1. Sprawdzenie: Czy obwód pozwala na wykonanie ??dania?
        if not self.breaker.allow_request():
            logger.debug("[ReplicationBridge] Pomini?to zapis do DuckDB (Obwód Otwarty).")
            # Tutaj mo?na opcjonalnie zapisa? ID faktury do kolejki "Zaleg?a Replikacja"
            return False

        # 2. Próba wykonania operacji
        try:
            # U?ywamy prepared statements w DuckDB
            self.duck.execute(
                """
                INSERT INTO invoices_replica (id, contractor_nip, amount_gross, issue_date, status)
                VALUES (?, ?, ?, ?, ?)
                """,
                [
                    invoice_data["id"],

```

```

        invoice_data["contractor_nip"],
        invoice_data["amount_gross"],
        invoice_data["issue_date"],
        invoice_data["status"]
    ]
)
# 3. Sukces: Resetujemy bezpiecznik
self.breaker.record_success()
return True

except Exception as e:
    # 4. Awaria: Rejestrujemy błąd w bezpieczniku
    self.breaker.record_failure(str(e))
    # KLUCZOWE: Nie rzucamy wyjątku (raise) dalej!
    # Główny proces zapisu do SQLite nie może wiedzieć, że analityka zawiodła.
    return False

```

===== Code/DB/transaction.py =====

===== Code/DB/types.py =====

```

import json
from sqlalchemy import TypeDecorator, String, Text
from core.config import AppConfig

```

```

class JSONObject(TypeDecorator):
    """Pozwala przechowywać słowniki Pythona jako tekst JSON w SQLite/DuckDB."""
    impl = Text
    cache_ok = True

    def process_bind_param(self, value, dialect):
        if value is not None:
            return json.dumps(value)
        return None

    def process_result_value(self, value, dialect):
        if value is not None:
            return json.loads(value)
        return None

```

```

class EncryptedString(TypeDecorator):
    """Szyfruje dane na poziomie kolumny za pomocą klucza AES (Fernet)."""
    impl = String
    cache_ok = False

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Klucz brany bezpośrednio z konfiguracji systemowej (wymaga cryptography.fernet.Fernet w pełnej i
        config = AppConfig()
        if hasattr(config, "encryption_key"):
            self.key = config.encryption_key.encode()
        else:
            self.key = b""

```

===== Code/DB/vector_store.py =====

```

import lancedb
import pyarrow as pa
from typing import List, Dict, Any

```

```

class OptimizedVectorStore:
    """Zoptymalizowany interfejs do LanceDB używający Apache Arrow."""

    def __init__(self, db_path: str):
        self.db = lancedb.connect(db_path)
        self.schema = pa.schema([
            pa.field("invoice_id", pa.string()),

```

```

        pa.field("contractor_nip", pa.string()),
        pa.field("vector", pa.list_(pa.float32(), 384)), # Rozmiar embeddingu
    ])

    if "invoice_templates" not in self.db.table_names():
        self.table = self.db.create_table("invoice_templates", schema=self.schema)
    else:
        self.table = self.db.open_table("invoice_templates")

def batch_insert(self, data: List[Dict[str, Any]]):
    """Dodaje rekordy bezpořrednio jako struktura Arrow Table (Zero-Copy)."""
    # Konwersja listy sřownikw na macierz kolumnow? Arrow
    arrow_table = pa.Table.from_pylist(data, schema=self.schema)
    self.table.add(arrow_table)

===== Code/DB/views.py =====
from db.analytics import DuckDBManager

class AnalyticsViewsSetup:
    """Inicjalizuje zmaterializowane widoki biznesowe w DuckDB dla maksymalnej wydajnořci."""

    @staticmethod
    def create_dashboard_views(duckdb_mgr: DuckDBManager):
        """Uruchamiane jednorazowo przy starcie aplikacji."""

        # Widok 1: Agregacja miesiřczna kosztw i przychodw
        duckdb_mgr.execute("""
        CREATE VIEW IF NOT EXISTS v_monthly_summary AS
        SELECT
            strftime('%Y-%m', issue_date) AS period,
            COUNT(id) as document_count,
            SUM(amount_net) as total_net,
            SUM(amount_gross) as total_gross
        FROM invoices_replica
        WHERE status != 'REJECTED'
        GROUP BY 1
        """)

        # Widok 2: Top Kontrahenci (gdzie idzie najwiřcej pieniřdzy)
        duckdb_mgr.execute("""
        CREATE VIEW IF NOT EXISTS v_top_contractors AS
        SELECT
            contractor_nip,
            SUM(amount_gross) as total_spent
        FROM invoices_replica
        GROUP BY 1
        ORDER BY total_spent DESC
        """)

    @staticmethod
    def create_cashflow_projection_view(duckdb_mgr: DuckDBManager) -> None:
        """Buduje analityczny widok projekcji cashflow z korekt? ryzyka kontrahenta."""

        duckdb_mgr.execute(
            """
            CREATE TABLE IF NOT EXISTS manual_cashflow_items (
                id UUID,
                type VARCHAR,
                expected_date DATE,
                amount DECIMAL(18,2),
                description VARCHAR,
                is_active BOOLEAN DEFAULT TRUE
            )
            """
        )

```

```

duckdb_mgr.execute(
    """
    CREATE OR REPLACE VIEW v_cashflow_projection AS
    WITH
    inflows AS (
        SELECT
            i.id AS source_id,
            'INVOICE_INFLOW' AS source_type,
            (CAST(i.due_date AS DATE) + COALESCE(vs.avg_delay_days, 0) * INTERVAL 1 DAY)::DATE AS
            CAST(
                CASE
                    WHEN COALESCE(vs.vendor_score, 1.0) < 0.4 THEN i.amount_gross * 0.5
                    ELSE i.amount_gross
                END AS DECIMAL(18,2)
            ) AS amount,
            'INFLOW' AS flow_direction,
            i.number AS description
        FROM invoices_replica i
        LEFT JOIN vendor_scorecards vs ON vs.contractor_nip = i.contractor_nip
        WHERE i.status = 'UNPAID' AND COALESCE(i.amount_gross, 0) > 0
    ),
    outflows AS (
        SELECT
            i.id AS source_id,
            'INVOICE_OUTFLOW' AS source_type,
            CAST(i.due_date AS DATE) AS projected_date,
            CAST(ABS(i.amount_gross) AS DECIMAL(18,2)) AS amount,
            'OUTFLOW' AS flow_direction,
            i.number AS description
        FROM invoices_replica i
        WHERE i.status = 'UNPAID' AND COALESCE(i.amount_gross, 0) < 0
    ),
    manual_items AS (
        SELECT
            id AS source_id,
            'MANUAL_ITEM' AS source_type,
            expected_date AS projected_date,
            CAST(amount AS DECIMAL(18,2)) AS amount,
            CAST(type AS VARCHAR) AS flow_direction,
            description
        FROM manual_cashflow_items
        WHERE is_active = TRUE
    ),
    tax_reserves AS (
        SELECT
            uuid() AS source_id,
            'VAT_RESERVE' AS source_type,
            (date_trunc('month', CURRENT_DATE) + INTERVAL 1 MONTH + INTERVAL 19 DAY)::DATE AS proj
            CAST(GREATEST(SUM(amount_gross - amount_net), 0) AS DECIMAL(18,2)) AS amount,
            'OUTFLOW' AS flow_direction,
            'VAT reserve payment' AS description
        FROM invoices_replica
        WHERE date_trunc('month', CAST(issue_date AS DATE)) = date_trunc('month', CURRENT_DATE)

        UNION ALL

        SELECT
            uuid() AS source_id,
            'INCOME_TAX_RESERVE' AS source_type,
            (date_trunc('month', CURRENT_DATE) + INTERVAL 1 MONTH + INTERVAL 24 DAY)::DATE AS proj
            CAST(GREATEST(SUM(amount_net) * 0.19, 0) AS DECIMAL(18,2)) AS amount,
            'OUTFLOW' AS flow_direction,
            'Income tax reserve payment' AS description
        FROM invoices_replica
    )
    """
)

```

```

        WHERE date_trunc('month', CAST(issue_date AS DATE)) = date_trunc('month', CURRENT_DATE)
    )
    SELECT * FROM inflows
    UNION ALL SELECT * FROM outflows
    UNION ALL SELECT * FROM manual_items
    UNION ALL SELECT * FROM tax_reserves
    """
)

===== Code/DB/zpk_schema.py =====
from __future__ import annotations

from db.analytics import DuckDBManager

def ensure_zpk_schema(duck_mgr: DuckDBManager) -> None:
    duck_mgr.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_accounts (
            account_number VARCHAR PRIMARY KEY,
            team_id INTEGER NOT NULL,
            report_mapping VARCHAR NOT NULL,
            description VARCHAR NOT NULL,
            parent_account VARCHAR
        )
        """
    )
    duck_mgr.execute("CREATE INDEX IF NOT EXISTS idx_zpk_team ON zpk_accounts(team_id)")
    duck_mgr.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_mapping_history (
            vendor_nip VARCHAR,
            keyword_hash VARCHAR,
            account_wn VARCHAR NOT NULL,
            account_ma VARCHAR NOT NULL,
            source VARCHAR NOT NULL,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )
    duck_mgr.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_subaccounts (
            parent_account VARCHAR NOT NULL,
            vendor_nip VARCHAR NOT NULL,
            account_number VARCHAR PRIMARY KEY,
            description VARCHAR NOT NULL,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )

===== Code/FRONTEND/Braki.py =====
# Fragment main.py
from ui.theme import ThemeManager
from ui.shortcuts import init_keyboard_handler
from ui.components.status_bar import NexusStatusBar
import flet as ft

async def main(page: ft.Page):
    # 1. Ustaw motyw
    page.theme = ThemeManager.get_dark_theme()

    # 2. Skrty klawiszowe
    init_keyboard_handler(page)

```

```

# 3. Dodaj pasek stanu do nakładki (overlay) lub na dół strony
status_bar = NexusStatusBar()
page.add(
    ft.Column([
        ft.Container(content=ft.Text("Główny Content"), expand=True),
        status_bar
    ], expand=True)
)

# 4. Test połączenia
status_bar.update_status(db_ok=True, nats_ok=False, message="Czekam na NATS...")

# frontend/api_client.py (Dodatkowe metody masowe)
class NexusApiClientBulk:
    # ... istniejące metody w NexusApiClient (AsyncNexusApiClient/NexusAPIClientUI) ...

    async def approve_bulk(self, invoice_ids: list[str]) -> bool:
        """Wysła żądanie masowego zatwierdzenia faktur."""
        response = await self._client.post(
            "/invoices/bulk-approve",
            json={"ids": invoice_ids}
        )
        return response.status_code == 200

    async def get_high_confidence_ids(self, threshold: float = 0.95) -> list[str]:
        """Pobiera ID faktur, które AI oceniło jako pewne."""
        response = await self._client.get(f"/invoices/high-confidence?min={threshold}")
        return response.json()

===== Code/Frontend/api_client.py =====
"""HTTP communication layer for local Litestar backend."""
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timezone
from decimal import Decimal
from typing import Any, Dict, List, Optional
from uuid import uuid4
import httpx
import msgspec
import logging

class InvoiceDTO(msgspec.Struct, kw_only=True):
    """Invoice structure consumed by frontend views."""
    id: str
    number: str
    customer_id: str
    amount_net: Decimal
    amount_gross: Decimal
    currency: str = "PLN"
    created_at: datetime
    pending: bool = False

@dataclass(slots=True)
class ApiConfig:
    """API runtime configuration from bootstrap handshake."""
    port: int
    token: str

    @property
    def base_url(self) -> str:
        return f"http://127.0.0.1:{self.port}"

```

```

def create_http_client(config: ApiConfig) -> httpx.Client:
    """Create global HTTPX client with connection limits and JWT header."""
    return httpx.Client(
        base_url=config.base_url,
        timeout=10.0,
        limits=httpx.Limits(max_connections=10),
        headers={"Authorization": f"Bearer {config.token}"}
    )

class NexusApiClient:
    """Thin typed wrapper around HTTPX for frontend data access."""
    def __init__(self, client: httpx.Client) -> None:
        self._client = client

    def list_invoices(self) -> list[InvoiceDTO]:
        """Fetch invoice register from local backend."""
        response = self._client.get("/invoices")
        if response.status_code == 404:
            return []
        response.raise_for_status()
        payload = response.json()
        if not isinstance(payload, list):
            return []
        return [self._coerce_invoice(item) for item in payload if isinstance(item, dict)]

    def create_invoice(self, invoice: InvoiceDTO) -> InvoiceDTO:
        """Create invoice in backend and return persisted representation."""
        response = self._client.post("/invoices", json=self._invoice_payload(invoice))
        if response.status_code == 404:
            # Backend endpoint can be wired later; keep optimistic item for now.
            return invoice
        response.raise_for_status()
        payload = response.json()
        if isinstance(payload, dict):
            return self._coerce_invoice(payload)
        return invoice

    @staticmethod
    def build_optimistic(number: str, gross_amount: Decimal) -> InvoiceDTO:
        """Build local optimistic invoice before backend confirmation."""
        return InvoiceDTO(
            id=str(uuid4()),
            number=number,
            customer_id="bootstrap-customer",
            amount_net=gross_amount,
            amount_gross=gross_amount,
            currency="PLN",
            created_at=datetime.now(timezone.utc),
            pending=True,
        )

    @staticmethod
    def _coerce_invoice(payload: dict[str, Any]) -> InvoiceDTO:
        created_at = payload.get("created_at")
        parsed_created_at = datetime.now(timezone.utc)
        if isinstance(created_at, str):
            try:
                parsed_created_at = datetime.fromisoformat(created_at.replace("Z", "+00:00"))
            except ValueError:
                pass
        return InvoiceDTO(
            id=str(payload.get("id", uuid4())),
            number=str(payload.get("number", "N/A")),
            customer_id=str(payload.get("customer_id", "N/A")),

```

```

        amount_net=Decimal(str(payload.get("amount_net", "0"))),
        amount_gross=Decimal(str(payload.get("amount_gross", "0"))),
        currency=str(payload.get("currency", "PLN")),
        created_at=parsed_created_at,
        pending=bool(payload.get("pending", False)),
    )

    @staticmethod
    def _invoice_payload(invoice: InvoiceDTO) -> dict[str, Any]:
        return {
            "number": invoice.number,
            "customer_id": invoice.customer_id,
            "amount_net": str(invoice.amount_net),
            "amount_gross": str(invoice.amount_gross),
            "currency": invoice.currency,
            "created_at": invoice.created_at.isoformat(),
        }

    def get_analytics_summary(self) -> dict:
        # Wywoła endpoint w Litestar, który robi:
        # SELECT sum(amount_gross), count(*) FROM invoices_replica
        response = self._client.get("/analytics/summary")
        return response.json()

# Wariant asynchroniczny API klienta połączony z resztą definicji
class AsyncNexusApiClient:
    def __init__(self, port: int, token: str):
        self.base_url = f"http://127.0.0.1:{port}"
        self.token = token
        self.client = httpx.AsyncClient(
            base_url=self.base_url,
            headers={"Authorization": f"Bearer {self.token}"},
            timeout=10.0
        )

    async def list_invoices(self) -> List[Any]:
        """Pobiera listę wszystkich faktur."""
        response = await self.client.get("/invoices")
        response.raise_for_status()
        return response.json()

    async def get_invoice(self, invoice_id: str) -> Dict[str, Any]:
        """Pobiera szczegółowe dane jednej faktury."""
        response = await self.client.get(f"/invoices/{invoice_id}")
        if response.status_code == 404:
            raise Exception("Nie znaleziono faktury w bazie.")
        response.raise_for_status()
        return response.json()

    async def update_invoice(self, invoice_id: str, updated_data: Dict[str, Any]) -> bool:
        """Wysyła poprawki wprowadzone przez użytkownika w Widoku Detali."""
        response = await self.client.patch(f"/invoices/{invoice_id}", json=updated_data)
        if response.status_code == 200:
            return True
        response.raise_for_status()
        return False

    async def get_vat_summary(self) -> List[Dict[str, Any]]:
        """Pobiera statystyki z DuckDB przez API."""
        response = await self.client.get("/analytics/vat-summary")
        response.raise_for_status()
        return response.json()

```

```

async def approve_bulk(self, invoice_ids: list[str]) -> bool:
    """Wysyła dane masowego zatwierdzenia faktur."""
    response = await self.client.post("/invoices/bulk-approve", json={"ids": invoice_ids})
    if response.status_code in (200, 204):
        return True
    response.raise_for_status()
    return False

async def get_high_confidence_ids(self, threshold: float = 0.95) -> list[str]:
    """Pobiera ID faktur, które AI oceniło jako pewne."""
    response = await self.client.get(f"/invoices/high-confidence?min={threshold}")
    response.raise_for_status()
    payload = response.json()
    if isinstance(payload, list):
        return [str(item) for item in payload]
    return []

async def close(self):
    """Zamyka połączenie (ważne przy wyłączeniu aplikacji)."""
    await self.client.aclose()

```

```

logger = logging.getLogger("nexus.ui.api")

```

```

class NexusAPIClientUI:
    """Centralny punkt komunikacji UI z backendem Litestar."""
    def __init__(self, base_url: str = "http://127.0.0.1:8000/api/v1", token: str = None):
        self.base_url = base_url
        self.token = token
        self._client = httpx.AsyncClient(timeout=30.0)

    def _get_headers(self) -> dict:
        headers = {"Content-Type": "application/json"}
        if self.token:
            headers["Authorization"] = f"Bearer {self.token}"
        return headers

    async def get(self, endpoint: str, params: dict = None) -> dict | list:
        try:
            response = await self._client.get(
                f"{self.base_url}{endpoint}",
                headers=self._get_headers(),
                params=params
            )
            response.raise_for_status()
            return response.json()
        except httpx.HTTPStatusError as e:
            logger.error(f"Błąd API {e.response.status_code}: {e.response.text}")
            raise Exception(f"Błąd serwera: {e.response.status_code}")
        except httpx.RequestError as e:
            logger.error(f"Błąd sieci: {e}")
            raise Exception("Nie można połączyć się z serwerem Nexus AI.")

    async def upload_file(self, endpoint: str, file_path: str) -> dict:
        """Specjalna metoda do wysyłania plików PDF na serwer."""
        import aiofiles
        import os
        try:
            async with aiofiles.open(file_path, 'rb') as f:
                content = await f.read()
            files = {'file': (os.path.basename(file_path), content, 'application/pdf')}
            headers = {}
            if self.token:
                headers["Authorization"] = f"Bearer {self.token}"
            response = await self._client.post(

```

```

        f"{self.base_url}{endpoint}",
        headers=headers,
        files=files
    )
    response.raise_for_status()
    return response.json()
except Exception as e:
    logger.error(f"B??d uploadu: {e}")
    raise

async def get_pending_count(self) -> int:
    """Pobiera liczb? faktur oczekuj?cych na przetworzenie."""
    try:
        response = await self._client.get(f"{self.base_url}/invoices/stats/pending", headers=self._get_headers())
        return response.json().get("count", 0)
    except Exception:
        return 0

async def approve_bulk(self, invoice_ids: list[str]) -> bool:
    """UI helper for bulk invoice approval action."""
    try:
        response = await self._client.post(
            f"{self.base_url}/invoices/bulk-approve",
            headers=self._get_headers(),
            json={"ids": invoice_ids},
        )
        if response.status_code in (200, 204):
            return True
        response.raise_for_status()
        return False
    except Exception as e:
        logger.error(f"B??d masowego zatwierdzania: {e}")
        return False

async def get_high_confidence_ids(self, threshold: float = 0.95) -> list[str]:
    """Pobiera ID faktur o wysokiej pewno?ci z modelu AI."""
    try:
        response = await self._client.get(
            f"{self.base_url}/invoices/high-confidence",
            headers=self._get_headers(),
            params={"min": threshold},
        )
        response.raise_for_status()
        payload = response.json()
        if isinstance(payload, list):
            return [str(item) for item in payload]
        return []
    except Exception as e:
        logger.error(f"B??d pobierania high-confidence IDs: {e}")
        return []

async def close(self):
    await self._client.aclose()

```

===== Code/Frontend/app.py =====

frontend/app.py (Fragment)

import flet as ft

import asyncio

from core.updater import check_for_updates, download_and_apply_update

def main_ui(page: ft.Page):

... Inicjalizacja Twojego UI ...

```

def on_update_click(e):
    # Pokazujemy pasek ?adowania
    page.snack_bar = ft.SnackBar(ft.Text("Pobieranie i instalowanie aktualizacji..."))
    page.snack_bar.open = True
    page.update()

    # Pobieranie (w prawdziwym UI zrb to asynchronicznie)
    asyncio.run(download_and_apply_update(e.control.data))

    page.dialog = ft.AlertDialog(
        title=ft.Text("Zako?czono"),
        content=ft.Text("Aktualizacja gotowa. Uruchom program ponownie.")
    )
    page.dialog.open = True
    page.update()

async def init_updater():
    update_info = await check_for_updates()
    if update_info["update_available"]:
        # Wy?wietlamy banner we Flectie
        banner = ft.Banner(
            bgcolor=ft.colors.AMBER_100,
            leading=ft.Icon(ft.icons.WARNING_AMBER_ROUNDED, color=ft.colors.AMBER, size=40),
            content=ft.Text(f"Dost?pna aktualizacja do wersji {update_info['version']}"),
            actions=[
                ft.TextButton("Aktualizuj", data=update_info['url'], on_click=on_update_click)
            ]
        )
        page.banner = banner
        banner.open = True
        page.update()

```

==== Code/Frontend/components/status_bar.py ====

ui/components/status_bar.py

import flet as ft

class NexusStatusBar(ft.Container):

"""Pasek dolny monitoruj?cy stan backendu w czasie rzeczywistym."""

```

def __init__(self):
    super().__init__()
    self.height = 30
    self.bgcolor = "#0D0D12"
    self.padding = ft.padding.symmetric(horizontal=15)
    self.border = ft.border.only(top=ft.BorderSide(1, ft.colors.GREY_900))

    # Wska?niki
    self.db_led = ft.Container(width=8, height=8, border_radius=4, bgcolor=ft.colors.GREY_700)
    self.nats_led = ft.Container(width=8, height=8, border_radius=4, bgcolor=ft.colors.GREY_700)
    self.status_msg = ft.Text("Inicjalizacja systemu...", size=11, color=ft.colors.GREY_500)

    self.content = ft.Row([
        ft.Row([
            ft.Text("SQLITE", size=10, weight=ft.FontWeight.BOLD), self.db_led,
            ft.VerticalDivider(width=10),
            ft.Text("NATS/AI", size=10, weight=ft.FontWeight.BOLD), self.nats_led,
        ])
    ])

def update_status(self, db_ok: bool, nats_ok: bool, message: str):
    self.db_led.bgcolor = ft.colors.GREEN if db_ok else ft.colors.RED
    self.nats_led.bgcolor = ft.colors.GREEN if nats_ok else ft.colors.RED
    self.status_msg.value = message
    self.update()

```

```
===== Code/Frontend/components/status_chip.py =====
```

```
# frontend/components/status_chip.py
import flet as ft

def StatusChip(status: str):
    colors = {
        "NEW": ft.colors.BLUE,
        "PROCESSING": ft.colors.ORANGE,
        "APPROVED": ft.colors.GREEN,
        "MANUAL_REVIEW": ft.colors.RED,
    }

    return ft.Chip(
        label=ft.Text(status, color=ft.colors.WHITE),
        bgcolor=colors.get(status, ft.colors.GREY),
    )
```

```
===== Code/Frontend/controllers/invoice_controller.py =====
```

```
# frontend/controllers/invoice_controller.py
import flet as ft
import asyncio

class OptimisticField(ft.UserControl):
    def __init__(self, api_client, invoice_id: str, field_name: str, initial_value: str):
        super().__init__()
        self.api = api_client
        self.invoice_id = invoice_id
        self.field_name = field_name
        self.original_value = initial_value

        self.text_field = ft.TextField(
            value=initial_value,
            on_submit=self.handle_update
        )

    async def handle_update(self, e):
        new_value = self.text_field.value
        if new_value == self.original_value:
            return

        # 1. Optimistic Update (zakładamy sukces, UI reaguje natychmiast)
        self.text_field.border_color = ft.colors.GREEN_400
        self.original_value = new_value
        self.update()

        # 2. Strzałka do API w tle
        try:
            success = await self.api.update_invoice(
                self.invoice_id,
                {self.field_name: new_value}
            )
            if not success:
                self.text_field.border_color = ft.colors.RED_400
                self.update()
        except Exception:
            self.text_field.border_color = ft.colors.RED_400
            self.update()

    def build(self):
        return self.text_field
```

```
===== Code/Frontend/main.py =====
```

```
# frontend/main.py (Zarządzanie NATS z poziomu UI)
import flet as ft
import nats
```

```

import json
import asyncio

async def main(page: ft.Page):
    # Inicjalizacja UI...
    status_text = ft.Text("Oczekiwanie na przetworzenie faktur...", visible=False)
    upload_button = ft.ElevatedButton("Wy?lij pliki")
    page.add(status_text)

    # Przechowujemy referencje do elementw na li?cie po ID
    invoice_rows = {}

    async def update_backpressure_state():
        # Tu zak?adamy istnienie api_client do pobrania ilo?ci zada?
        # pending_count = await api_client.get_pending_count()
        pending_count = 0
        if pending_count > 50:
            upload_button.disabled = True
            status_text.value = f"? System przetwarza {pending_count} faktur. Poczeka? na zako?czenie..."
            status_text.visible = True
        else:
            upload_button.disabled = False
            status_text.visible = False
        page.update()

    async def nats_status_listener():
        nc = await nats.connect("nats://localhost:4222")
        # Subskrypcja na kana? ze statusami z workera Taskiq
        sub = await nc.subscribe("nexus.tasks.status")

        async for msg in sub.messages:
            data = json.loads(msg.data.decode())
            invoice_id = data.get("invoice_id")
            status = data.get("status")

            # Bezpieczna aktualizacja UI z innego w?tku (NATS)
            if invoice_id in invoice_rows:
                row = invoice_rows[invoice_id]
                if status == "COMPLETED":
                    row.controls[1].value = "? Gotowe"
                    row.controls[1].color = ft.colors.GREEN
                elif status == "FAILED":
                    row.controls[1].value = "? B??d OCR"
                    row.controls[1].color = ft.colors.RED
                page.update()

            # Przy ka?dym zako?czonym zadaniu aktualizujemy stan przycisku
            await update_backpressure_state()

    page.run_task(nats_status_listener)

    page.add(
        ft.Column([
            upload_button,
            status_text,
            # ... reszta listy faktur ...
        ])
    )

if __name__ == "__main__":
    ft.app(target=main)

===== Code/Frontend/main_ui.py =====
# frontend/main_ui.py (fragment)
from frontend.api_client import NexusAPIClientUI # U?ywamy wersji z obs?ug? UI

```

```

def main(page: ft.Page):
    port = page.session.get("api_port")
    token = page.session.get("api_token")

    # Przekazujemy dynamiczny port i bezpieczny token
    base_url = f"http://127.0.0.1:{port}/api/v1" if port else "http://127.0.0.1:8000/api/v1"
    api_client = NexusAPIClientUI(base_url=base_url, token=token)

    # ... reszta logiki interfejsu (router, widoki) ?

===== Code/Frontend/router.py =====
# frontend/router.py
import flet as ft
from frontend.views.dashboard_view import DashboardView
from frontend.views.invoice_list_view import InvoiceListView

class NexusRouter:
    def __init__(self, page: ft.Page, api_client):
        self.page = page
        self.api = api_client
        self.routes = {
            "/": DashboardView(api_client),
            "/invoices": InvoiceListView(api_client),
        }

    def handle_route_change(self, route_event):
        self.page.views.clear()

        # Wybieramy widok na podstawie ?cie?ki
        view_handler = self.routes.get(self.page.route, self.routes["/"])

        self.page.views.append(
            ft.View(
                route=self.page.route,
                controls=[
                    self._build_app_bar(),
                    view_handler.build(), # Ka?dy widok ma metod? build()
                ],
                drawer=self._build_drawer()
            )
        )
        self.page.update()

    def _build_app_bar(self):
        return ft.AppBar(title=ft.Text("Nexus AI"))

    def _build_drawer(self):
        return ft.NavigationDrawer(
            controls=[
                ft.NavigationDrawerDestination(label="Dashboard", icon=ft.icons.DASHBOARD),
                ft.NavigationDrawerDestination(label="Faktury", icon=ft.icons.DOCUMENT_SCAN),
            ]
        )

    def _view_pop(self, view_event: ft.ViewPopEvent):
        """Obs?uga przycisku Wstecz."""
        self.page.views.pop()
        top_view = self.page.views[-1]
        self.page.go(top_view.route)

    def _not_found_view(self) -> ft.View:
        return ft.View(
            "/404",
            [

```

```

        ft.AppBar(title=ft.Text("B??d 404")),
        ft.Text("Nie znalezione ??danej strony.", size=30, color="red")
    ]
)

===== Code/Frontend/ui/components/vendor_card.py =====
from __future__ import annotations

import flet as ft

def build_vendor_card(vendor: dict) -> ft.Card:
    rating = int(vendor.get("rating_stars", 0))
    stars = "?" * max(0, min(5, rating)) + "?" * max(0, 5 - rating)

    trend = vendor.get("monthly_spending", [])
    points = [ft.LineChartDataPoint(i, float(value)) for i, value in enumerate(trend)]

    alerts = vendor.get("alerts", [])
    alert_column = ft.Column(
        controls=[ft.Text(f"? {alert}", size=12, color=ft.colors.ORANGE_300) for alert in alerts]
        or [ft.Text("? No active alerts", size=12, color=ft.colors.GREEN_300)],
        spacing=4,
    )

    return ft.Card(
        content=ft.Container(
            padding=16,
            content=ft.Column(
                spacing=10,
                controls=[
                    ft.Text(vendor.get("vendor_name", "Unknown vendor"), size=18, weight=ft.FontWeight.BOLD),
                    ft.Text(f"NIP: {vendor.get('nip', '---')}", size=12, color=ft.colors.GREY_400),
                    ft.Text(stars, size=20, color=ft.colors.AMBER_400),
                    ft.LineChart(
                        data_series=[ft.LineChartData(data_points=points, curved=True, stroke_width=2)],
                        min_y=0,
                        expand=True,
                        height=110,
                        horizontal_grid_lines=ft.ChartGridLines(interval=1000, color=ft.colors.GREY_800),
                        tooltip_bgcolor=ft.colors.with_opacity(0.85, ft.colors.BLACK),
                    ),
                    ft.Text("Smart Alerts", size=14, weight=ft.FontWeight.W_600),
                    alert_column,
                ],
            ),
        ),
    )

===== Code/Frontend/ui/data_table.py =====
# ui/components/data_table.py
import flet as ft
from frontend.api_client import NexusAPIClientUI
import logging

logger = logging.getLogger("nexus.ui.components")

class AsyncInvoiceTable(ft.Column):
    """Asynchroniczna tabela obs?uguj?ca tysi?ce rekordw przez API."""

    def __init__(self, api_client: NexusAPIClientUI):
        super().__init__()
        self.api = api_client
        self.cursor: str | None = None
        self.is_loading = False

```

```

self.table = ft.DataTable(
    columns=[
        ft.DataColumn(ft.Text("Numer")),
        ft.DataColumn(ft.Text("NIP")),
        ft.DataColumn(ft.Text("Kwota Brutto"), numeric=True),
        ft.DataColumn(ft.Text("Status")),
        ft.DataColumn(ft.Text("Akcje")),
    ],
    rows=[]
)

self.loading_ring = ft.ProgressRing(visible=False)
self.load_more_btn = ft.ElevatedButton(
    "Za?aduj wi?cej",
    icon=ft.icons.DOWNLOAD,
    on_click=self._load_next_page
)
self.controls = [self.table, self.loading_ring, self.load_more_btn]

async def _load_next_page(self, e=None):
    self._toggle_loading(True)
    try:
        # Zak?adamy API z obs?ug? kursora
        response = await self.api.get("/invoices", params={"cursor": self.cursor})
        invoices = response if isinstance(response, list) else response.get("items", [])

        for invoice in invoices:
            status_color = ft.colors.GREEN if invoice.get("status") == "APPROVED" else ft.colors.ORANGE
            row = ft.DataRow(cells=[
                ft.DataCell(ft.Text(invoice.get("number") or "Brak")),
                ft.DataCell(ft.Text(invoice.get("contractor_nip") or "---")),
                ft.DataCell(ft.Text(f"{invoice.get('amount_gross', 0.0):.2f} {invoice.get('currency', 'PLN')}")),
                ft.DataCell(ft.Text(invoice.get("status", "NEW"), color=status_color, weight=ft.FontWeight.BOLD)),
                ft.DataCell(
                    ft.IconButton(
                        icon=ft.icons.VISIBILITY,
                        tooltip="Zobacz PDF",
                        on_click=lambda e, id=invoice.get("id"): self.page.go(f"/invoices/{id}")
                    )
                ),
            ])
            self.table.rows.append(row)
    except Exception as ex:
        logger.error(f"B??d ?adowania danych: {ex}")
    finally:
        self._toggle_loading(False)

def _toggle_loading(self, show: bool):
    self.loading_ring.visible = show
    self.load_more_btn.visible = not show
    self.update()

```

===== Code/Frontend/ui/root.py =====

```

# ui/root.py
import flet as ft
import asyncio
from frontend.api_client import NexusAPIClientUI
from ui.ws_client import ProgressWebSocketClient
from ui.state import app_state

class NexusRootUI:
    """G?wna klasa orkiestruj?ca interfejsem graficznym."""

    def __init__(self, page: ft.Page, process_manager):

```

```

self.page = page
self.process_manager = process_manager # Z main.py

port = page.session.get("api_port")
token = page.session.get("api_token")
base_url = f"http://127.0.0.1:{port}/api/v1" if port else "http://127.0.0.1:8000/api/v1"
self.api = NexusAPIClientUI(base_url=base_url, token=token)

self.ws_client = ProgressWebSocketClient()

# Elementy nawigacji
self.main_content = ft.Container(expand=True, padding=20)
self.snackbar = ft.SnackBar(content=ft.Text(""), action="OK")
self.page.overlay.append(self.snackbar)

async def build(self):
    """Zarządza globalnym układem (Layout) i rutowaniem."""
    self.page.title = "Nexus AI Workspace"
    self.page.padding = 0
    self.page.bgcolor = "#121212" # Ciemny motyw Enterprise

    # Załadowanie domyślnego widoku
    await self._load_dashboard()

    self.page.add(self.main_content)
    self.ws_client.start()

async def _load_dashboard(self):
    try:
        summary = await self.api.get("/analytics/summary")

        self.main_content.content = ft.Column([
            ft.Text("Pulpit Finansowy", size=28, weight=ft.FontWeight.BOLD),
            ft.Row([
                self._create_stat_card("Suma Netto", f"{summary.get('total_net', 0)} PLN"),
                self._create_stat_card("Suma Brutto", f"{summary.get('total_gross', 0)} PLN"),
                self._create_stat_card("Przeanalizowane", str(summary.get('count', 0))),
            ])
        ])
        self.page.update()
    except Exception as e:
        self.main_content.content = ft.Text(f"Błąd ładowania dashboardu: {e}", color="red")
        self.page.update()

def _create_stat_card(self, title, value):
    return ft.Card(
        content=ft.Container(
            padding=20,
            content=ft.Column([
                ft.Text(title, size=14, color="grey"),
                ft.Text(value, size=24, weight="bold")
            ])
        )
    )

```

==== Code/Frontend/ui/shortcuts.py ====

```

# ui/shortcuts.py
import flet as ft

def init_keyboard_handler(page: ft.Page):
    """Mapowanie globalnych skrótów klawiszowych."""

    async def on_keyboard(e: ft.KeyboardEvent):
        # CTRL + S -> Szybki zapis aktualnego formularza
        if e.ctrl and e.key == "S":

```

```

        page.pubsub.send_all("trigger_save")

# CTRL + N -> Nowy upload faktury
if e.ctrl and e.key == "N":
    page.go("/upload")

# CTRL + Q -> Zamknij / Logout
if e.ctrl and e.key == "Q":
    page.window_close()

# F5 -> Odświeżenie danych (re-fetch z API)
if e.key == "F5":
    page.pubsub.send_all("trigger_refresh")

page.on_keyboard_event = on_keyboard

===== Code/FRONTEND/ui/state.py =====
# ui/state.py
from typing import Callable, Any

class AppState:
    """Globalny magazyn stanu aplikacji (odpowiednik Redux/Provider)."""

    def __init__(self):
        self._state: dict[str, Any] = {
            "current_user": None,
            "theme": "dark",
            "active_tasks": []
        }
        # Event Bus: { event_name: [list_of_callbacks] }
        self._listeners: dict[str, list[Callable]] = {}

    def get(self, key: str, default: Any = None) -> Any:
        return self._state.get(key, default)

    def set(self, key: str, value: Any, notify: bool = True):
        self._state[key] = value
        if notify:
            self.emit(f"{key}_changed", value)

    def subscribe(self, event_name: str, callback: Callable):
        """Rejestruje komponent UI do nasłuchiwania zmian."""
        if event_name not in self._listeners:
            self._listeners[event_name] = []
        self._listeners[event_name].append(callback)

    def emit(self, event_name: str, data: Any = None):
        if event_name in self._listeners:
            for callback in self._listeners[event_name]:
                callback(data)

app_state = AppState()

===== Code/FRONTEND/ui/storage.py =====
# ui/storage.py
import flet as ft
import json
import logging

logger = logging.getLogger("nexus.ui.storage")

class LocalStorage:
    """Zarządzanie danymi po stronie klienta (odpowiednik LocalStorage w przeglądarkach)."""

    def __init__(self, page: ft.Page):

```

```

        self.page = page

    def set_token(self, token: str):
        self.page.client_storage.set("nexus_auth_token", token)

    def get_token(self) -> str | None:
        return self.page.client_storage.get("nexus_auth_token")

    def clear_session(self):
        self.page.client_storage.remove("nexus_auth_token")
        self.page.client_storage.remove("nexus_user_prefs")

    def save_preferences(self, prefs: dict):
        self.page.client_storage.set("nexus_user_prefs", json.dumps(prefs))

    def load_preferences(self) -> dict:
        data = self.page.client_storage.get("nexus_user_prefs")
        return json.loads(data) if data else {"theme": "dark", "compact_mode": False}

```

===== Code/FRONTEND/ui/theme.py =====

```

# ui/theme.py
import flet as ft

class ThemeManager:
    """Zarz?dza palet? barw i stylem Nexus AI (Enterprise Look & Feel)."""

    @staticmethod
    def get_dark_theme() -> ft.Theme:
        return ft.Theme(
            color_scheme=ft.ColorScheme(
                primary=ft.colors.BLUE_ACCENT_400,
                on_primary=ft.colors.WHITE,
                primary_container="#232333",
                secondary=ft.colors.CYAN_400,
                surface="#1E1E26", # Karty i panele
                background="#121217", # G?wne t?o
                on_surface="#E0E0E0",
                error=ft.colors.RED_400,
                outline=ft.colors.GREY_800,
            ),
            font_family="Segoe UI",
            visual_density=ft.VisualDensity.COMFORTABLE,
            page_transitions=ft.PageTransitionsTheme(
                windows=ft.PageTransitionType.FADE_THROUGH
            )
        )

    @staticmethod
    def get_light_theme() -> ft.Theme:
        return ft.Theme(
            color_scheme=ft.ColorScheme(
                primary=ft.colors.BLUE_700,
                on_primary=ft.colors.WHITE,
                background=ft.colors.GREY_50,
                surface=ft.colors.WHITE
            )
        )

```

===== Code/FRONTEND/ui/utils.py =====

```

# ui/utils.py
import asyncio
from typing import Callable

class Debouncer:
    """Zapobiega spamowaniu API (np. przy pasku wyszukiwania)."""

```

```

def __init__(self, wait_ms: int = 500):
    self.wait_ms = wait_ms / 1000.0
    self._task: asyncio.Task | None = None

def __call__(self, coroutine_func: Callable):
    """Wywo?uje funkcj? asynchroniczn? dopiero po up?ywie zadanego czasu."""
    async def debounce_wrapper(*args, **kwargs):
        if self._task is not None:
            self._task.cancel()

        async def delayed_call():
            await asyncio.sleep(self.wait_ms)
            await coroutine_func(*args, **kwargs)

        self._task = asyncio.create_task(delayed_call())

    try:
        await self._task
    except asyncio.CancelledError:
        pass # Anulowano przez kolejne uderzenie klawisza

    return debounce_wrapper

===== Code/Frontend/ui/validators.py =====
# ui/validators.py
import re
from decimal import Decimal, InvalidOperation

class FormValidator:
    """Zestaw regu? walidacji dla faktur i kontrahentw."""

    @staticmethod
    def validate_nip(nip: str) -> bool:
        """Walidacja polskiego NIP (10 cyfr, bez my?lnikw)."""
        if not nip: return False
        clean_nip = re.sub(r'\D', '', nip)
        if len(clean_nip) != 10:
            return False

        # Wagi dla sumy kontrolnej NIP
        weights = [6, 5, 7, 2, 3, 4, 5, 6, 7]
        try:
            checksum = sum(int(clean_nip[i]) * weights[i] for i in range(9))
            return (checksum % 11) == int(clean_nip[9])
        except (ValueError, IndexError):
            return False

    @staticmethod
    def validate_currency_amount(value: str) -> tuple[bool, str]:
        """Sprawdza czy kwota jest poprawn? liczb? finansow?."""
        if not value: return False, "Pole wymagane"
        try:
            # Obs?uga przecinka i kropki
            val = Decimal(value.replace(',', ' ').replace('.', ''))
            if val < 0:
                return False, "Kwota nie mo?e by? ujemna"
            return True, ""
        except InvalidOperation:
            return False, "Nieprawid?owy format kwoty"

===== Code/Frontend/ui/ws_client.py =====
# ui/ws_client.py
import asyncio
import json

```

```

import logging
import websockets
from websockets.exceptions import ConnectionClosed
from ui.state import app_state

logger = logging.getLogger("nexus.ui.ws")

class ProgressWebSocketClient:
    """Nas?uchuje zdarze? z serwera w tle i aktualizuje Flet UI."""

    def __init__(self, ws_url: str = "ws://127.0.0.1:8000/api/v1/ws/progress"):
        self.ws_url = ws_url
        self._task: asyncio.Task | None = None
        self._stop_event = asyncio.Event()

    def start(self):
        self._stop_event.clear()
        self._task = asyncio.create_task(self._listen())

    async def stop(self):
        self._stop_event.set()
        if self._task:
            await self._task

    async def _listen(self):
        while not self._stop_event.is_set():
            try:
                async with websockets.connect(self.ws_url) as websocket:
                    logger.info("Po??czono z kana?em WebSockets paska post?pu.")
                    while not self._stop_event.is_set():
                        message = await websocket.recv()
                        data = json.loads(message)
                        app_state.emit("progress_update", data)
            except ConnectionClosed:
                logger.warning("Po??czenie WebSocket zamkni?te. Ponawianie...")
                await asyncio.sleep(2)
            except Exception as e:
                logger.error(f"B??d WebSocket: {e}")
                await asyncio.sleep(5)

===== Code/Frontend/ui.py =====
"""Flet UI for Nexus Accounting OS desktop frontend."""
from __future__ import annotations
import argparse
import asyncio
from decimal import Decimal, InvalidOperation
from typing import Final
import flet as ft
import httpx
from frontend.api_client import ApiConfig, InvoiceDTO, NexusApiClient, create_http_client

HTTP_CLIENT: httpx.Client | None = None
APP_NAME: Final[str] = "Nexus Accounting OS"

class InvoiceRegistryView(ft.Column):
    """Virtualized invoice register with optimistic update support."""
    def __init__(self) -> None:
        super().__init__(expand=True, spacing=12)
        self.items = ft.ListView(expand=1, spacing=10, auto_scroll=True)
        self.controls = [self.items]

    def set_rows(self, rows: list[InvoiceDTO]) -> None:
        """Replace entire register rows."""
        self.items.controls.clear()
        for row in rows:

```

```

        self.items.controls.append(self._build_tile(row))

def prepend_row(self, row: InvoiceDTO) -> None:
    """Add new row at the top for fast user feedback."""
    self.items.controls.insert(0, self._build_tile(row))

def replace_pending_row(self, optimistic_id: str, persisted: InvoiceDTO) -> None:
    """Swap optimistic row with backend-confirmed payload."""
    for index, control in enumerate(self.items.controls):
        if isinstance(control, ft.Container) and control.data == optimistic_id:
            self.items.controls[index] = self._build_tile(persisted)
            break

def _build_tile(self, invoice: InvoiceDTO) -> ft.Container:
    status_suffix = " (pending...)" if invoice.pending else ""
    return ft.Container(
        data=invoice.id,
        padding=12,
        border_radius=8,
        bgcolor=ft.colors.SURFACE_CONTAINER_HIGHEST if invoice.pending else ft.colors.SURFACE,
        content=ft.Row(
            alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            controls=[
                ft.Text(f"{invoice.number}{status_suffix}"),
                ft.Text(f"{invoice.amount_gross} {invoice.currency}")
            ]
        )
    )

class NexusApp:
    """Composable app controller separating UI from HTTP data layer."""
    def __init__(self, page: ft.Page, api: NexusApiClient) -> None:
        self.page = page
        self.api = api
        self.registry = InvoiceRegistryView()
        self.invoice_number = ft.TextField(label="Numer faktury", expand=True)
        self.invoice_gross = ft.TextField(label="Kwota brutto", width=180)
        self.feedback = ft.Text(value="", color=ft.colors.RED_400)

        # UI Components z drugiego bloku
        self.registry_table = ft.DataTable(
            expand=True,
            columns=[
                ft.DataColumn(ft.Text("Numer")),
                ft.DataColumn(ft.Text("NIP Kontrahenta")),
                ft.DataColumn(ft.Text("Kwota Brutto")),
                ft.DataColumn(ft.Text("Status")),
                ft.DataColumn(ft.Text("Data")),
            ],
            rows=[]
        )
        self.stats_row = ft.Row(spacing=20)
        self.loader = ft.ProgressBar(visible=False, color="blue")
        self.file_picker = ft.FilePicker(on_result=self._on_file_selected)
        self.page.overlay.append(self.file_picker)

    async def initialize(self) -> None:
        """Build layout and load initial register."""
        self.page.title = APP_NAME
        self.page.theme_mode = ft.ThemeMode.LIGHT
        self.page.horizontal_alignment = ft.CrossAxisAlignment.STRETCH
        self.page.padding = 30

        await self.refresh_data()
        self._build_main_layout()

```

```

def _build_main_layout(self):
    """Buduje strukturę widoku."""
    layout = ft.Column(
        expand=True,
        controls=[
            ft.Row(
                controls=[
                    ft.Text("NEXUS", size=32, weight="bold", color="blue800"),
                    ft.VerticalDivider(),
                    ft.Text("Panel Księgowy", size=20, color="grey700"),
                ],
                alignment=ft.MainAxisAlignment.START,
            ),
            ft.Divider(),
            ft.Text("Analityka (DuckDB)", weight="bold", size=16),
            self.stats_row,
            ft.Divider(),
            ft.Row(
                controls=[
                    ft.ElevatedButton(
                        "Dodaj Fakturę (OCR)",
                        icon=ft.icons.UPLOAD_FILE,
                        on_click=lambda _: self.file_picker.pick_files()
                    ),
                    ft.IconButton(ft.icons.REFRESH, on_click=lambda _: self.refresh_data()),
                ],
            ),
            self.loader,
            ft.Column([self.registry_table], scroll=ft.ScrollMode.ADAPTIVE, expand=True)
        ]
    )
    self.page.add(layout)

async def refresh_data(self) -> None:
    """Pobiera dane z API (zarówno OLTP jak i OLAP)."""
    self.loader.visible = True
    self.page.update()
    try:
        invoices = await asyncio.to_thread(self.api.list_invoices)
        self._update_table(invoices)
        stats = await asyncio.to_thread(self.api.get_analytics_summary)
        self._update_stats(stats)
        self.feedback.value = ""
    except Exception as exc:
        self.page.show_snack_bar(ft.SnackBar(ft.Text(f"Błąd synchronizacji: {exc}")))
    self.loader.visible = False
    self.page.update()

def _update_table(self, invoices: list[InvoiceDTO]):
    self.registry_table.rows.clear()
    for inv in invoices:
        status_color = {
            "APPROVED": ft.colors.GREEN,
            "PROCESSING": ft.colors.ORANGE,
            "MANUAL_REVIEW": ft.colors.BLUE,
            "FAILED": ft.colors.RED
        }.get(getattr(inv, "status", "NEW"), ft.colors.GREY)

        self.registry_table.rows.append(
            ft.DataRow(
                cells=[
                    ft.DataCell(ft.Text(inv.number or "W trakcie...")),
                    ft.DataCell(ft.Text(inv.customer_id or "-")),
                    ft.DataCell(ft.Text(f"{inv.amount_gross:.2f} {inv.currency}")),
                ]
            )
        )

```

```

        ft.DataCell(ft.Chip(ft.Text(getattr(inv, "status", "NEW")), bgcolor=status_color))
        ft.DataCell(ft.Text(inv.created_at.strftime("%Y-%m-%d %H:%M"))),
    ]
)
)

def _update_stats(self, stats: dict):
    self.stats_row.controls = [
        self._build_stat_card("Suma Brutto", f"{stats.get('total_gross', 0):.2f} PLN", ft.icons.MONEY),
        self._build_stat_card("Liczba Faktur", str(stats.get('count', 0)), ft.icons.COPY),
        self._build_stat_card("?rednia Warto??", f"{stats.get('avg_amount', 0):.2f} PLN", ft.icons.ANA
    ]

def _build_stat_card(self, title: str, value: str, icon: str):
    return ft.Card(
        content=ft.Container(
            padding=15,
            content=ft.Row([
                ft.Icon(icon, size=40, color="blue"),
                ft.Column([
                    ft.Text(title, size=14, color="grey"),
                    ft.Text(value, size=20, weight="bold")
                ])
            ])
    )

def _on_file_selected(self, e: ft.FilePickerResultEvent):
    # Placeholder dla wyboru pliku
    pass

def _read_bootstrap(page: ft.Page) -> ApiConfig:
    """Resolve dynamic backend port and bootstrap token from UI inputs."""
    parser = argparse.ArgumentParser(add_help=False)
    parser.add_argument("--port", type=int)
    parser.add_argument("--token", type=str)
    args, unknown = parser.parse_known_args()

    qp_port = page.query_params.get("port")
    qp_token = page.query_params.get("token")

    port = args.port or (int(qp_port) if qp_port else None)
    token = args.token or qp_token

    if not port or not token:
        raise ValueError("Brak wymaganych parametrów startowych: port i token.")
    return ApiConfig(port=int(port), token=token)

async def main(page: ft.Page) -> None:
    """Flet app entrypoint."""
    try:
        config = _read_bootstrap(page)
        global HTTP_CLIENT
        HTTP_CLIENT = create_http_client(config)
        api = NexusApiClient(HTTP_CLIENT)
        app = NexusApp(page, api)
        await app.initialize()
    except Exception as exc:
        page.add(ft.Text(f"B??d inicjalizacji UI: {exc}", color=ft.colors.RED_400))
        page.update()

if __name__ == "__main__":
    ft.app(target=main)

===== Code/Frontend/views/dashboard.py =====

```

```

# frontend/views/dashboard.py (Fragment)
import flet as ft

def create_cashflow_chart(forecast_data):
    # Przekształcamy dane z Pandas/API na format punktów Flet
    data_points = [
        ft.LineChartDataPoint(x=i, y=row['projected_balance'])
        for i, row in enumerate(forecast_data)
    ]

    chart = ft.LineChart(
        data_series=[
            ft.LineChartData(
                data_points=data_points,
                stroke_width=4,
                color=ft.colors.GREEN_400,
                curved=True, # Gładka linia
            )
        ],
        border=ft.border.all(1, ft.colors.OUTLINE),
        tooltip bgcolor=ft.colors.SURFACE,
    )
    return chart

class StatCard(ft.Container):
    """Karta statystyk z gradientem i ikoną."""
    def __init__(self, title: str, value: str, icon: str, color_top: str):
        super().__init__()
        self.content = ft.Column([
            ft.Icon(icon, color=ft.colors.WHITE, size=30),
            ft.Text(title, color=ft.colors.WHITE70),
            ft.Text(value, color=ft.colors.WHITE, size=24, weight=ft.FontWeight.BOLD)
        ])
        self.bgcolor = color_top
        self.padding = 20
        self.border_radius = 10

# UI Controls (fragment z innego pliku)
def build_bulk_actions(select_all_high_confidence, handle_bulk_approve):
    bulk_bar = ft.Row([
        ft.ElevatedButton("Zaznacz pewne (>95%)", on_click=select_all_high_confidence),
        ft.FilledButton("Zatwierdź zaznaczone", icon=ft.icons.DONE_ALL, on_click=handle_bulk_approve),
    ])
    return bulk_bar

===== Code/Frontend/views/invoice_detail_view.py =====
# frontend/views/invoice_detail_view.py
import flet as ft
from decimal import Decimal

class InvoiceDetailView:
    def __init__(self, api_client, invoice_id: str):
        self.api = api_client
        self.invoice_id = invoice_id

        # Pola formularza (referencje, by móc odczytać ich wartości)
        self.txt_number = ft.TextField(label="Numer faktury", border_color=ft.colors.BLUE_400)
        self.txt_nip = ft.TextField(label="NIP Kontrahenta", icon=ft.icons.FINGERPRINT)
        self.txt_net = ft.TextField(label="Kwota Netto", suffix_text="PLN", width=200)
        self.txt_gross = ft.TextField(label="Kwota Brutto", suffix_text="PLN", width=200)
        self.dd_currency = ft.Dropdown(
            label="Waluta",
            width=100,
            options=[ft.dropdown.Option("PLN"), ft.dropdown.Option("EUR"), ft.dropdown.Option("USD")]
        )

```

```

# Komponent obrazu
self.img_invoice = ft.Image(
    src=f"https://via.placeholder.com/800x1200?",
    fit=ft.ImageFit.CONTAIN
)
self.zoom_level = 1.0

def build(self):
    left_column = ft.Column([
        self.txt_number, self.txt_nip, self.txt_net, self.txt_gross, self.dd_currency
    ], expand=1)
    right_column = ft.Column([
        self.img_invoice,
        ft.Row([
            ft.IconButton(icon=ft.icons.ZOOM_IN, on_click=self.zoom_in),
            ft.IconButton(icon=ft.icons.ZOOM_OUT, on_click=self.zoom_out)
        ])
    ], expand=2)
    return ft.Row([left_column, right_column], expand=True, spacing=30)

async def load_data(self):
    """Pobiera dane faktury z API i wyplenia pola."""
    invoice = await self.api.get_invoice(self.invoice_id)
    self.txt_number.value = invoice.number
    self.txt_nip.value = invoice.contractor_nip
    self.txt_net.value = str(invoice.amount_net)
    self.txt_gross.value = str(invoice.amount_gross)
    self.dd_currency.value = invoice.currency

    # W desktopowej aplikacji src b?dzie ?cie?k? lokaln? (file://...)
    if hasattr(invoice, 'file_path'):
        self.img_invoice.src = invoice.file_path
    self.img_invoice.update()
    self.txt_number.page.update()

def zoom_in(self, e):
    self.zoom_level += 0.2
    self.img_invoice.scale = self.zoom_level
    self.img_invoice.update()

def zoom_out(self, e):
    if self.zoom_level > 0.4:
        self.zoom_level -= 0.2
        self.img_invoice.scale = self.zoom_level
        self.img_invoice.update()

===== Code/Frontend/views/invoice_list.py =====
# frontend/views/invoice_list.py (fragment)
def on_export_click(e, table, db, InsertEppExporter, page):
    # 1. Pobierz zaznaczone faktury
    selected_ids = [row.data for row in table.rows if row.selected]
    invoices = db.get_invoices_by_ids(selected_ids)

    # 2. Wygeneruj plik
    exporter = InsertEppExporter(invoices)
    epp_data = exporter.generate()

    # 3. Zapisz plik na dysku u?ytkownika
    with open("eksport_do_inserta.epp", "w", encoding="cp1250") as f:
        f.write(epp_data)

    page.show_snack_bar(ft.SnackBar(ft.Text("Plik .EPP zosta? wygenerowany!")))

```

```

# frontend/views/invoice_list.py
import flet as ft
import asyncio

class InfiniteInvoiceList(ft.UserControl):
    def __init__(self, api_client):
        super().__init__()
        self.api = api_client
        self.invoices = []
        self.cursor = None
        self.is_loading = False

        # ListView natywnie zarz?dza pami?ci? (renderuje tylko widoczne na ekranie)
        self.list_view = ft.ListView(
            expand=True,
            spacing=10,
            padding=20,
            on_scroll=self.handle_scroll
        )

    async def load_more(self):
        if self.is_loading:
            return

        self.is_loading = True
        self.list_view.controls.append(ft.ProgressRing()) # Wska?nik ?adowania na dole
        self.update()

        # Pobieranie danych z u?yciem kursora (Cursor Pagination)
        new_data, next_cursor = await self.api.get_invoices(cursor=self.cursor, limit=50)
        self.cursor = next_cursor

        # Usuwamy spinner
        self.list_view.controls.pop()

        for inv in new_data:
            self.list_view.controls.append(ft.Text(f"{inv['number']} - {inv['amount_gross']}"))

        self.is_loading = False
        self.update()

    def handle_scroll(self, e: ft.OnScrollEvent):
        # Prosta detekcja dojechania do ko?ca listy
        pass

    def build(self):
        return self.list_view

===== Code/Frontend/views/invoice_list_view.py =====
# frontend/views/invoice_list_view.py
import flet as ft

class InvoiceListView:
    def __init__(self, api_client):
        self.api = api_client

    def build(self):
        # Tutaj budujemy tabel?
        self.table = ft.DataTable(
            columns=[
                ft.DataColumn(ft.Text("Numer")),
                ft.DataColumn(ft.Text("NIP")),
                ft.DataColumn(ft.Text("Kwota")),
                ft.DataColumn(ft.Text("Status")),
            ]

```

```

        ],
        rows=[]
    )

    return ft.Column([
        ft.Text("Lista Faktur", style=ft.TextThemeStyle.HEADLINE_MEDIUM),
        ft.ElevatedButton("Od?wie?", on_click=self.load_data),
        self.table
    ], scroll=ft.ScrollMode.ALWAYS)

    async def load_data(self, e=None):
        invoices = await self.api.list_invoices() # Wywo?anie Twojego API
        self.table.rows = [
            ft.DataRow(cells=[
                ft.DataCell(ft.Text(inv.number)),
                ft.DataCell(ft.Text(inv.customer_id)),
                ft.DataCell(ft.Text(str(inv.amount_gross))),
                ft.DataCell(ft.Text("APPROVED"))
            ]) for inv in invoices
        ]
        self.table.update()

===== Code/Frontend/views/mobile_sync.py =====
# frontend/views/mobile_sync.py
import qrcode
import socket
import flet as ft

def get_local_ip():
    # Pobiera IP komputera w sieci lokalnej (np. 192.168.1.15)
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    return s.getsockname()[0]

def MobileSyncView(page: ft.Page):
    local_ip = get_local_ip()
    port = page.session.get("api_port")
    token = page.session.get("api_token")

    # Adres, ktry telefon "uderzy" (musi by? dost?pny w sieci lokalnej)
    upload_url = f"http://{local_ip}:{port}/mobile/upload?token={token}"

    qr = qrcode.make(upload_url)
    qr.save("assets/temp_qr.png")

    return ft.Column([
        ft.Text("Zeskanuj telefonem, aby wys?a? zdj?cie faktury"),
        ft.Image(src="temp_qr.png", width=200)
    ])

===== Code/Frontend/views/ui_triage.py =====
from __future__ import annotations

import flet as ft

def build_triage_split_screen(
    image_path: str,
    extracted_data: dict[str, object],
    highlighted_fields: set[str],
    on_confirm,
    on_reject,
) -> ft.Row:
    """Core split-screen triage view with bounding-box placeholder overlay."""
    bbox_overlay = ft.Container(

```

```

        width=180,
        height=40,
        left=120,
        top=140,
        border=ft.border.all(3, ft.colors.RED_500),
        border_radius=6,
        bgcolor=ft.colors.with_opacity(0.08, ft.colors.RED_500),
    )

    image_panel = ft.Container(
        expand=2,
        padding=12,
        content=ft.Stack([
            ft.Image(src=image_path, fit=ft.ImageFit.CONTAIN, expand=True),
            bbox_overlay,
        ]),
    )

    form_controls: list[ft.Control] = []
    for key, value in extracted_data.items():
        field_bg = ft.colors.YELLOW_100 if key in highlighted_fields else None
        form_controls.append(
            ft.TextField(
                label=key,
                value="" if value is None else str(value),
                bgcolor=field_bg,
            )
        )

    action_bar = ft.Row(
        controls=[
            ft.FilledButton("Confirm & Post Ledger", icon=ft.icons.CHECK_CIRCLE, on_click=on_confirm),
            ft.OutlinedButton("Void / Reject", icon=ft.icons.CANCEL, on_click=on_reject),
        ],
        spacing=12,
    )

    form_panel = ft.Container(
        expand=3,
        padding=12,
        content=ft.Column(
            controls=[
                ft.Text("Review Queue / Triage", size=22, weight=ft.FontWeight.BOLD),
                ft.Text("AI flagged this document for review. Please verify highlighted fields."),
                *form_controls,
                action_bar,
            ],
            scroll=ft.ScrollMode.AUTO,
        ),
    )

    return ft.Row(controls=[image_panel, ft.VerticalDivider(width=1), form_panel], expand=True)

```

===== Code/Frontend/views/verify_view.py =====

```

# frontend/views/verify_view.py
import flet as ft

```

```

class InteractivePDFViewer(ft.UserControl):
    """Widok PDF z interaktywnymi strefami OCR."""

    def __init__(self, image_path: str, boxes: list[dict]):
        super().__init__()
        self.image_path = image_path
        self.boxes = boxes # Lista wsp?rz?dnych [x1, y1, x2, y2] + label
        self.overlays = []

```

```

def highlight_field(self, label: str):
    """Metoda wywoływana z formularza, aby przewin?? i pod?wietli? pole."""
    for box in self.overlays:
        if box.data == label:
            box.bgcolor = ft.colors.with_opacity(0.3, ft.colors.YELLOW_ACCENT_400)
            box.border = ft.border.all(2, ft.colors.AMBER_600)
        else:
            box.bgcolor = ft.colors.TRANSPARENT
            box.border = None
    self.update()

def build(self):
    # Tworzymy warstw? z obrazem i nak?adkami
    stack = ft.Stack(
        controls=[
            ft.Image(src=self.image_path, fit=ft.ImageFit.CONTAIN)
            # Tu by?yby dodane prostok?ty z self.overlays na podstawie self.boxes
        ]
    )
    return stack

===== Code/MODELS/Braki.py =====
from sqlalchemy import String, Integer, JSON, DateTime
from sqlalchemy.orm import Mapped, mapped_column, validates
from sqlalchemy.ext.declarative import declared_attr
from datetime import datetime, timezone

# --- 1. MIXIN DLA POWTARZALNYCH PL (DRY) ---
class TimestampMixin:
    """Automatyzacja pl czasowych dla ka?dego modelu."""
    @declared_attr
    def created_at(cls) -> Mapped[datetime]:
        return mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

    @declared_attr
    def updated_at(cls) -> Mapped[datetime]:
        return mapped_column(DateTime, default=lambda: datetime.now(timezone.utc), onupdate=lambda: dateti

# --- 2. DODATKI DO KLASY CONTRACTOR ---
# (Poni?sze pola nale?y dopisa? do klasy Contractor w models/contractor.py)
# Wersjonowanie w celu unikni?cia konfliktw przy edycji przez wielu u?ytkownikw
# version_id: Mapped[int] = mapped_column(Integer, default=1)
# __mapper_args__ = {"version_id_col": version_id}

# @validates("nip")
# def validate_nip(self, key, nip):
#     # logika walidacji NIP
#     return nip

===== Code/MODELS/audit.py =====
from sqlalchemy import String, ForeignKey, DateTime
from sqlalchemy.orm import Mapped, mapped_column
from datetime import datetime, timezone
from db.database import Base
import uuid

class AuditLog(Base):
    """Przechowuje histori? modyfikacji dokumentw (kto, co i kiedy zmieni?)."""
    __tablename__ = "audit_logs"

    id: Mapped[str] = mapped_column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    invoice_id: Mapped[str] = mapped_column(String, ForeignKey("invoices.id"), index=True)

    # Kto dokona? zmiany (np. "System/OCR", "Ksi?gowa_Kasia" lub "Auto-Correction")

```

```

user_id: Mapped[str] = mapped_column(String, default="System")

# Rodzaj akcji, np. "STATUS_CHANGED", "AMOUNT_CORRECTED", "OCR_PROCESSED"
action: Mapped[str] = mapped_column(String, nullable=False)

# Szczegół zmiany (opcjonalne, przydatne przy ręcznych korektach)
old_value: Mapped[str | None] = mapped_column(String, nullable=True)
new_value: Mapped[str | None] = mapped_column(String, nullable=True)
created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

===== Code/MODELS/contractor.py =====
from __future__ import annotations

from datetime import datetime, timezone
import uuid

from sqlalchemy import DateTime, Integer, String
from sqlalchemy.ext.declarative import declared_attr
from sqlalchemy.orm import Mapped, mapped_column, relationship, validates

from db.database import Base

class TimestampMixin:
    """Reusable timestamp fields for SQLAlchemy models."""

    @declared_attr
    def created_at(cls) -> Mapped[datetime]:
        return mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

    @declared_attr
    def updated_at(cls) -> Mapped[datetime]:
        return mapped_column(
            DateTime,
            default=lambda: datetime.now(timezone.utc),
            onupdate=lambda: datetime.now(timezone.utc),
        )

class Contractor(TimestampMixin, Base):
    __tablename__ = "contractors"

    id: Mapped[str] = mapped_column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    name: Mapped[str] = mapped_column(String, nullable=False)
    nip: Mapped[str] = mapped_column(String(10), unique=True, index=True, nullable=False)
    address: Mapped[str | None] = mapped_column(String, nullable=True)
    bank_account: Mapped[str | None] = mapped_column(String(26), nullable=True)

    # Optimistic locking against concurrent edits.
    version_id: Mapped[int] = mapped_column(Integer, default=1, nullable=False)
    __mapper_args__ = {"version_id_col": version_id}

    invoices: Mapped[list["Invoice"]] = relationship("Invoice", back_populates="contractor")

    @validates("nip")
    def validate_nip(self, _key: str, nip: str) -> str:
        normalized = "".join(ch for ch in str(nip) if ch.isdigit())
        if len(normalized) != 10:
            raise ValueError("NIP musi składać się z 10 cyfr.")

        weights = (6, 5, 7, 2, 3, 4, 5, 6, 7)
        checksum = sum(int(d) * w for d, w in zip(normalized[:9], weights)) % 11
        if checksum == 10 or checksum != int(normalized[9]):
            raise ValueError("Nieprawidłowy NIP (błędny sumy kontrolnej).")

```

```

        return normalized

    def __repr__(self) -> str:
        return f"<Contractor(name={self.name}, nip={self.nip})>"

===== Code/MODELS/init.py =====
from .contractor import Contractor
from .invoice import Invoice, ActiveLearningPattern
from .outbox import OutboxEvent
from .audit import AuditLog

# Definiujemy, co jest publicznie dostępne przy imporcie z pakietu models
__all__ = [
    "Contractor",
    "Invoice",
    "ActiveLearningPattern",
    "OutboxEvent",
    "AuditLog"
]

===== Code/MODELS/invoice.py =====
from sqlalchemy import String, Decimal, DateTime, ForeignKey, Date
from sqlalchemy.orm import Mapped, mapped_column, relationship
from datetime import datetime, timezone, date
from db.database import Base
from decimal import Decimal
import uuid

class Invoice(Base):
    __tablename__ = "invoices"

    id: Mapped[str] = mapped_column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    number: Mapped[str | None] = mapped_column(String, index=True)

    # Finanse
    amount_net: Mapped[Decimal] = mapped_column(Decimal(12, 2), default=Decimal("0.0"))
    amount_gross: Mapped[Decimal] = mapped_column(Decimal(12, 2), default=Decimal("0.0"))
    currency: Mapped[str] = mapped_column(String(3), default="PLN")
    issue_date: Mapped[date | None] = mapped_column(Date, nullable=True)

    # Powiązania i statusy
    contractor_nip: Mapped[str | None] = mapped_column(String(10), index=True)
    contractor_id: Mapped[str | None] = mapped_column(String, ForeignKey("contractors.id"), nullable=True)
    status: Mapped[str] = mapped_column(String, default="NEW") # NEW, PROCESSING, APPROVED, ERROR

    # Pliki i Audyt
    file_path: Mapped[str] = mapped_column(String)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc), onupdate=datetime.now(timezone.utc))
    created_by: Mapped[str] = mapped_column(String, default="worker:taskiq")
    updated_by: Mapped[str] = mapped_column(String, default="worker:taskiq")

    # Relacje
    contractor: Mapped["Contractor"] = relationship("Contractor", back_populates="invoices")

    def __repr__(self) -> str:
        return f"<Invoice(number={self.number}, status={self.status})>"

class ActiveLearningPattern(Base):
    """Przechowuje wzorce poprawek użytkownika dla konkretnych NIP-w."""
    __tablename__ = "active_learning_patterns"

    id: Mapped[str] = mapped_column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    contractor_nip: Mapped[str] = mapped_column(String(10), index=True)
    # correction_payload przechowuje JSON z polami, które AI zwykle myli dla tego NIPu

```

```

correction_payload: Mapped[str] = mapped_column(String)
created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

===== Code/MODELS/outbox.py =====
from sqlalchemy import String, DateTime, Text, Boolean
from sqlalchemy.orm import Mapped, mapped_column
from datetime import datetime, timezone
from db.database import Base
import uuid

class OutboxEvent(Base):
    """
    Zdarzenia do wysłania przez NATS.
    Zapisujemy je w tej samej transakcji co faktur?, aby mieć pewność, że nic nie zginie.
    """
    __tablename__ = "outbox_events"

    id: Mapped[str] = mapped_column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    event_type: Mapped[str] = mapped_column(String) # np. "invoice_created", "ocr_completed"
    payload: Mapped[str] = mapped_column(Text) # Dane zdarzenia w formacie JSON
    processed: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
    processed_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    def __repr__(self) -> str:
        return f"<OutboxEvent(type={self.event_type}, processed={self.processed})>"

===== Code/NEXUS.SPEC/nexus.spec.py =====
# -*- mode: python; coding: utf-8 -*-
import sys
import os
from PyInstaller.utils.hooks import collect_data_files, collect_submodules, copy_metadata

# --- [1] KONFIGURACJA ŚRODOWISKA ---
# Zwiększenie limitu rekurencji dla głębokich zależności Transformers/Torch
sys.setrecursionlimit(10000)
block_cipher = None
project_root = os.path.abspath(os.getcwd())

# --- [2] ZBIERANIE DANYCH (DATA & METADATA) ---
datas = []

# Wiele bibliotek AI sprawdza wersje w runtime. Brak metadanych = "DistributionNotFound"
packages_to_metadata = [
    'torch', 'transformers', 'surya', 'tqdm', 'regex', 'requests',
    'packaging', 'filelock', 'numpy', 'huggingface-hub', 'safetensors',
    'pyyaml', 'tokenizers', 'onnxruntime', 'duckdb'
]

for pkg in packages_to_metadata:
    try:
        datas += copy_metadata(pkg)
    except:
        pass

# Dodajemy pliki statyczne, modele i binarne serwery
added_files = [
    ('models/', 'models/'), # Wagi AI
    ('nats-server.exe', '.'), # Serwer NATS
    ('assets/', 'assets/'), # Ikony/Grafiki UI
    ('app_data/', 'app_data/'), # Struktura folderów
    ('db/', 'db/'), # Skrypty DB
    ('.env.example', '.'), # Szablon konfiguracji
    ('core/exporters/*.txt', 'core/exporters/') # Szablony eksportu
]

```

```

datas += added_files
datas += collect_data_files('surya')
datas += collect_data_files('transformers')

# --- [3] UKRYTE IMPORTY (HIDDEN IMPORTS) ---
# Rejestrujemy moduły ładowane dynamicznie
hidden_imports = [
    'sqlalchemy.ext.declarative',
    'sqlalchemy.orm.attributes',
    'sqlalchemy.dialects.sqlite',
    'sqlalchemy.ext.baked',
    'sqlalchemy.sql.default_comparator',
    'duckdb',
    'taskiq_nats',
    'taskiq.serializers.json_serializer',
    'flet',
    'flet.canvas',
    'flet.charts',
    'fastapi',
    'uvicorn',
    'uvicorn.logging',
    'uvicorn.protocols.http.httptools_impl',
    'uvicorn.protocols.websockets.wsproto_impl',
    'litestar',
    'litestar.plugins.sqlalchemy',
    'onnxruntime',
    'tokenizers',
    'sentence_transformers',
    'email.mime.multipart',
    'email.mime.text',
    'email.mime.base',
    'lancedb',
    'pyarrow',
    'pydantic',
    'httpx'
]

# Automatyczne skanowanie Twoich modułów
for module in ['core', 'models', 'services', 'pipeline', 'db', 'api', 'frontend', 'worker']:
    try:
        hidden_imports += collect_submodules(module)
    except:
        pass

# --- [4] ANALIZA I FILTROWANIE ---
a = Analysis(
    ['main.py'],
    pathex=[project_root],
    binaries=[],
    datas=datas,
    hiddenimports=hidden_imports,
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=['notebook', 'jupyter', 'matplotlib', 'test', 'pytest', 'unittest', 'setuptools', 'distutils'],
    win_no_prefer_redirects=False,
    win_private_assemblies=False,
    cipher=block_cipher,
    noarchive=False,
)

pyz = PYZ(a.pure, a.zipped_data, cipher=block_cipher)

# --- [5] KONFIGURACJA PLIKU EXE ---

```

```

exe = EXE(
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name='NexusAI',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=True, # Zostawiamy True dla logw AI/Workera/NATS
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
    icon='assets/icon.ico',
    version='file_version_info.txt', # Powi?zanie z plikiem wersji
    manifest="""
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
    <security>
      <requestedPrivileges>
        <requestedExecutionLevel level="asInvoker" uiAccess="false"/>
      </requestedPrivileges>
    </security>
  </trustInfo>
</assembly>
"""
)

# --- [6] KOLEKCJA (FOLDER MODE - ONEDIR) ---
# Folder mode jest stabilniejszy dla aplikacji AI (szybszy start)
coll = COLLECT(
    exe,
    a.binaries,
    a.zipfiles,
    a.datas,
    strip=False,
    upx=True,
    upx_exclude=['torch', 'onnxruntime', 'mkl', 'libopenblas'], # Nie kompresujemy ci??kich DLL (ryzyko b?
    name='NexusAI_Distribution',
)

===== Code/PIPELINE/audit.py =====
# pipeline/audit.py
import json
from datetime import datetime
from pathlib import Path

class PipelineAuditTrail:
    """Zapisuje ?lad ka?dej transformacji dokumentu."""

    def __init__(self, doc_id: str):
        self.doc_id = doc_id
        self.steps = []

    def log_step(self, step_name: str, input_summary: str, output_summary: str, confidence: float = 1.0):
        self.steps.append({
            "timestamp": datetime.now().isoformat(),
            "step": step_name,
            "input": input_summary[:200], # Tylko fragmenty dla oszcz?dno?ci DB
            "output": output_summary,
            "confidence": confidence
        })

```

```

def get_full_trail(self) -> str:
    return json.dumps(self.steps, indent=2)

class PipelineAudit:
    """Zapisuje ?lad decyzji podj?tych przez Pipeline do pliku lokalnego."""

    def __init__(self, doc_id: str, logs_path: Path):
        self.doc_id = doc_id
        self.log_file = logs_path / f"audit_{doc_id}.log"

    def write_log(self, message: str):
        with open(self.log_file, "a", encoding="utf-8") as f:
            f.write(f"[{datetime.now().isoformat()}] {message}\n")

===== Code/PIPELINE/coordinator.py =====
# pipeline/coordinator.py
import asyncio
from typing import List
from pipeline.ocr_engine import VisionProcessor
from pipeline.preprocessor import ImageOptimizer
from core.logger import logger

class PipelineCoordinator:
    """Zarz?dza rwnoleg?ym przetwarzaniem wielu stron dokumentu."""

    def __init__(self, ocr_engine, llm_engine):
        self.ocr = ocr_engine
        self.llm = llm_engine

    async def process_parallel(self, file_path: str):
        # 1. Podzia? PDF na strony (asynchronicznie)
        # 2. Map-Reduce: Ka?da strona do OCR w oddzielnym tasku
        # 3. Agregacja wynikw
        logger.info("Uruchamianie rwnoleg?ego przetwarzania stron...")
        return "Zintegrowany tekst ze wszystkich stron"

class ParallelPipelineCoordinator:
    """Orkiestrator przyspieszaj?cy przetwarzanie wielostronicowe."""

    def __init__(self, vision_engine: VisionProcessor):
        self.vision = vision_engine

    async def process(self, files: List[str]):
        logger.info(f"Processing {len(files)} files in parallel.")
        pass

===== Code/PIPELINE/deduplicator.py =====
# pipeline/deduplicator.py
import hashlib
from pathlib import Path

class PipelineDeduplicator:
    """Sprawdza, czy plik o takiej tre?ci by? ju? analizowany."""

    @staticmethod
    def calculate_hash(file_path: str) -> str:
        """Tworzy unikalny odcisk palca pliku (SHA-256)."""
        sha256_hash = hashlib.sha256()
        with open(file_path, "rb") as f:
            for byte_block in iter(lambda: f.read(4096), b''):
                sha256_hash.update(byte_block)
        return sha256_hash.hexdigest()

    @staticmethod

```

```

async def is_duplicate(file_hash: str, db_session) -> bool:
    """Sprawdza w bazie danych, czy hash ju? istnieje."""
    # Logika SQL: SELECT 1 FROM invoices WHERE file_hash = :hash
    # To wywo?asz w Workerze przed startem ci??kiego AI
    return False

===== Code/PIPELINE/factory.py =====
# pipeline/factory.py
from pipeline.manager import DocumentPipeline
from pipeline.ocr_engine import VisionProcessor
from pipeline.validator import InvoiceLogicValidator
from core.llm_guard import LLMGuard

async def run_full_pipeline(invoice_id: str, file_path: str, bus, llm_engine):
    """G?wny entrypoint dla Workera NATS."""
    pipe = DocumentPipeline(invoice_id, bus)
    vision = VisionProcessor(config=None)

    try:
        # 1. Start OCR
        await pipe.start()
        raw_text = await vision.pdf_to_text(file_path)

        # 2. Ekstrakcja LLM
        await pipe.ocr_done()
        raw_ai_output = await llm_engine.analyze(raw_text)
        structured_data = LLMGuard.parse_and_validate(raw_ai_output)

        # 3. Walidacja biznesowa
        await pipe.extraction_done()
        InvoiceLogicValidator.verify(structured_data)

        # 4. Sukces
        await pipe.validate_success()
        return structured_data

    except Exception as e:
        await pipe.error_occured()
        raise e

===== Code/PIPELINE/init.py =====
# pipeline/__init__.py
from .manager import DocumentPipeline
from .ocr_engine import VisionProcessor
# from .validator import InvoiceLogicValidator # Import tymczasowo wy??czony aby nie rzuca? b??dw
from .normalizer import DataNormalizer
from .memory import PipelineMemory
from .deduplicator import PipelineDeduplicator
from .refiner import PipelineRefiner
from .preprocessor import ImageOptimizer
from .qa_engine import QualityAssuranceEngine
from .audit import PipelineAudit
from .coordinator import ParallelPipelineCoordinator

__all__ = [
    "DocumentPipeline",
    "VisionProcessor",
    # "InvoiceLogicValidator",
    "DataNormalizer",
    "PipelineMemory",
    "PipelineDeduplicator",
    "PipelineRefiner",
    "ImageOptimizer",
    "QualityAssuranceEngine",
    "PipelineAudit",

```

```

    "ParallelPipelineCoordinator"
]

===== Code/PIPELINE/llm_engine.py =====
# pipeline/llm_engine.py
from typing import Optional
from core.hardware import HardwareProbe
from core.logger import logger

class HybridLLMEngine:
    """Silnik obs?uguj?cy modele PyTorch (GPU) oraz GGUF (CPU)."""

    def __init__(self, model_path: str):
        self.strategy = HardwareProbe.get_strategy()
        self.model = None
        self._load_model(model_path)

    def _load_model(self, model_path: str):
        if self.strategy.get("model_format") == "gguf":
            from llama_cpp import Llama

            logger.info(f"?adowanie modelu GGUF na CPU ({self.strategy.get('threads', 4)} w?tkw)...")
            self.model = Llama(
                model_path=model_path,
                n_ctx=2048,
                n_threads=self.strategy.get("threads", 4),
                n_gpu_layers=0
            )
        else:
            # Standardowe ?adowanie przez Transformers (AutoModelForCausalLM)
            import torch
            from transformers import AutoModelForCausalLM

            logger.info("?adowanie modelu AutoModelForCausalLM...")
            # self.model = AutoModelForCausalLM.from_pretrained(model_path)
            pass

===== Code/PIPELINE/llm_extractor.py =====
# pipeline/llm_extractor.py
import msgspec
from typing import Optional
from decimal import Decimal
from core.logger import logger

class ExtractedInvoice(msgspec.Struct):
    """Docelowy schemat danych, kt?y musi zwrci? model LLM."""
    number: Optional[str]
    contractor_nip: Optional[str]
    contractor_name: Optional[str]
    amount_net: Decimal
    amount_gross: Decimal
    currency: str = "PLN"
    issue_date: Optional[str]
    iban: Optional[str]

class VisionDataExtractor:
    """Zast?puje stary InvoiceParser oparty na regexach."""

    def __init__(self, llm_engine):
        self.llm = llm_engine # Np. Twj za?adowany Phi-3-Vision lub Hermes

    async def extract(self, full_text: str) -> ExtractedInvoice:
        """Wykorzystuje LLM do inteligentnej interpretacji tekstu faktury."""
        prompt = f"""
        Jeste? ekspertem ksi?gowym. Wyci?gnij dane z poni?sze?o tekstu faktury OCR.

```

Zwr? TYLKO czysty JSON zgodny ze struktur?.

```
{{
    "number": "",
    "contractor_nip": "",
    "contractor_name": "",
    "amount_net": 0.0,
    "amount_gross": 0.0,
    "currency": "PLN",
    "issue_date": "",
    "iban": ""
}}
```

Tekst: {full_text}

"""

Pseudo kod komunikacji z llm
response_text = await self.llm.generate(prompt)
return msgspec.json.decode(response_text, type=ExtractedInvoice)
return None

===== Code/PIPELINE/manager.py =====

```
# pipeline/manager.py
from statemachine import StateMachine, State
from core.logger import logger
from core.events import NexusEvent

class DocumentPipeline(StateMachine):
    """Zarz?dza cyklem ?ycia analizy dokumentu."""

    # Definicja stanw
    pending = State("Oczekiwanie", initial=True)
    processing_ocr = State("Analiza Wizualna (OCR)")
    extracting_data = State("Ekstrakcja LLM")
    validating = State("Weryfikacja Logiczna")
    completed = State("Zako?czono")
    failed = State("B??d")

    # Definicja przej??
    start = pending.to(processing_ocr)
    ocr_done = processing_ocr.to(extracting_data)
    extraction_done = extracting_data.to(validating)
    validate_success = validating.to(completed)
    error_occured = (pending | processing_ocr | extracting_data | validating).to(failed)

    def __init__(self, doc_id: str, bus_client):
        self.doc_id = doc_id
        self.bus = bus_client # NATS/Websocket client do powiadomie? UI
        super().__init__()
```

===== Code/PIPELINE/memory.py =====

```
# pipeline/memory.py
import lancedb
import pyarrow as pa
from core.config import AppConfig
from core.logger import logger

class PipelineMemory:
    """System RAG dla faktur ? wyszukiuje podobne wzorce w LanceDB."""

    def __init__(self, config: AppConfig):
        self.db = lancedb.connect(config.base_dir / "app_data" / "vector_store")
        self.table_name = "invoice_patterns"

    async def get_similar_layout(self, vendor_nip: str) -> dict | None:
        """Pobiera historyczne poprawki dla danego dostawcy."""
        try:
            if self.table_name not in self.db.table_names():
```

```

        return None

        table = self.db.open_table(self.table_name)
        result = table.search().where(f"vendor_nip = '{vendor_nip}'").limit(1).to_list()

        return result[0] if result else None

    except Exception as e:
        logger.error(f"B??d pami?ci wektorowej: {e}")
        return None

    async def save_pattern(self, vendor_nip: str, payload: dict):
        pass # Logika zapisu wektora

===== Code/PIPELINE/normalizer.py =====
# pipeline/normalizer.py
import dateparser
from decimal import Decimal
import re

class DataNormalizer:
    """Czy?ci i standaryzuje dane wyj?ciowe z AI pod formaty DB/ERP."""

    @staticmethod
    def normalize_date(raw_date: str) -> str:
        """Konwertuje dowolny format daty na ISO (YYYY-MM-DD)."""
        if not raw_date: return ""
        dt = dateparser.parse(raw_date, settings={'DATE_ORDER': 'DMY'})
        return dt.strftime("%Y-%m-%d") if dt else raw_date

    @staticmethod
    def normalize_nip(nip: str) -> str:
        """Usuwa my?lniki i prefiksy krajowe (np. PL)."""
        clean = re.sub(r'^[0-9]', '', str(nip))
        if len(clean) > 10: clean = clean[-10:] # Obs?uga prefiksu PL
        return clean

    @staticmethod
    def clean_currency(value: str) -> str:
        """Mapuje popularne symbole na kody ISO (np. ? -> EUR)."""
        mapping = {"z?": "PLN", "pln": "PLN", "$": "USD", "?": "EUR", "euro": "EUR"}
        val_lower = value.lower().strip()
        return mapping.get(val_lower, "PLN")

===== Code/PIPELINE/ocr.py =====
# pipeline/ocr.py
import re
import torch
from PIL import Image
from dataclasses import dataclass
# Importy biblioteki Surya OCR
from surya.ocr import run_ocr
from surya.model.detection.model import load_model as load_det_model, load_processor as load_det_processor
from surya.model.recognition.model import load_model as load_rec_model, load_processor as load_rec_processor

@dataclass
class ExtractedInvoiceData:
    raw_text: str
    nip: str | None
    amount_gross: float | None

class DocumentProcessor:
    """Singleton ?aduj?cy modele ML tylko raz, aby oszcz?dzi? pami??."""
    _instance = None

```

```

def __new__(cls):
    if cls._instance is None:
        cls._instance = super(DocumentProcessor, cls).__new__(cls)
        cls._instance._initialize_models()
    return cls._instance

def _initialize_models(self):
    """?adowanie wag modeli do pamieci zoptymalizowane pod offline-first."""
    print("[AI] ?adowanie modeli Surya OCR...")
    self.device = "cuda" if torch.cuda.is_available() else "cpu"

    # Modele detekcji (gdzie jest tekst)
    self.det_processor, self.det_model = load_det_processor(), load_det_model()
    self.det_model.to(self.device)

    # Modele rozpoznawania (jaki to tekst)
    self.rec_processor, self.rec_model = load_rec_processor(), load_rec_model()
    self.rec_model.to(self.device)
    print(f"[AI] Modele za?adowane na urz?dzenie: {self.device}")

@torch.inference_mode() # Zgodnie z wytycznymi z PDF - wy??cza autograd, oszcz?dza RAM
def process_image(self, image_path: str) -> ExtractedInvoiceData:
    """Przetwarza skan faktury i wyci?ga czysty tekst oraz kluczowe dane."""
    image = Image.open(image_path).convert("RGB")
    # Surya obs?uguje listy obrazw, podajemy jeden
    predictions = run_ocr(
        [image],
        [], # Brak wstpnie zdefiniowanych boksw (auto-detekcja)
        self.det_model, self.det_processor,
        self.rec_model, self.rec_processor
    )

    # ??czenie tekstu ze wszystkich wykrytych linii
    lines = [line.text for line in predictions[0].text_lines]
    full_text = "\n".join(lines)

    # Ekstrakcja biznesowa (heurystyki / regex)
    return self._extract_business_fields(full_text)

def _extract_business_fields(self, text: str) -> ExtractedInvoiceData:
    """Prymitywny parser Regex do wyci?gania NIP-u i Kwot.
    W docelowej wersji mo?na to przepu?ci? przez lokalny model LLM."""
    # Szukamy NIPu (10 cyfr, ewentualnie z my?lnikami)
    nip_match = re.search(r'\b\d{3}[-\s]?d{3}[-\s]?d{2}[-\s]?d{2}\b', text)
    nip = nip_match.group(0).replace("-", "").replace(" ", "") if nip_match else None

    # Szukamy kwoty brutto (uproszczony regex: s?owo "Brutto" i liczba)
    gross_match = re.search(r'Brutto[:\s]*([\d\s]+[.]\d{2})', text, re.IGNORECASE)
    amount_gross = None

    if gross_match:
        try:
            # Zamiana formatu "1 000,50" na "1000.50"
            clean_num = gross_match.group(1).replace(" ", "").replace(",", ".")
            amount_gross = float(clean_num)
        except ValueError:
            pass

    return ExtractedInvoiceData(raw_text=text, nip=nip, amount_gross=amount_gross)

"""OCR/ML document intelligence pipeline for invoices."""
import hashlib
import math
import numpy as np

```

```

from pathlib import Path
from typing import Any

try:
    import onnxruntime as ort
except Exception:
    ort = None

try:
    from paddleocr import PaddleOCR
except Exception:
    PaddleOCR = None

class ReviewStatus:
    """Pipeline result statuses."""
    APPROVED = "APPROVED"
    MANUAL_REVIEW = "MANUAL_REVIEW"

@dataclass(slots=True)
class OCRResult:
    """Normalized OCR extraction result."""
    raw_text: str
    nip: str | None
    checksum: str

@dataclass(slots=True)
class ProcessedDocument:
    """Final pipeline output used by workflow tasks."""
    status: str
    primary: OCRResult
    validator: OCRResult
    metadata: dict[str, Any]

class DocumentProcessorDual:
    """Dual-engine OCR processor (Surya + Paddle) with cross-validation."""
    def __init__(self, nip_mismatch_threshold: int = 0, checksum_distance_threshold: int = 0) -> None:
        self.nip_mismatch_threshold = nip_mismatch_threshold
        self.checksum_distance_threshold = checksum_distance_threshold
        self._onnx_session = self._build_onnx_session()
        self._paddle = self._build_paddle()

    def process(self, image_path: Path) -> ProcessedDocument:
        """Process document with tiling + dual OCR and compare extraction consistency."""
        image = Image.open(image_path).convert("RGB")
        tiles = self._tile_image(image)

        # Force no-grad inference for PyTorch paths (Surya or fallback).
        if torch is not None:
            with torch.inference_mode():
                primary_text = self._run_primary_ocr(tiles)
        else:
            primary_text = self._run_primary_ocr(tiles)

        validator_text = self._run_validator_ocr(tiles)
        primary = self._extract_fields(primary_text)
        validator = self._extract_fields(validator_text)
        status = self._cross_validate(primary, validator)

        return ProcessedDocument(
            status=status,
            primary=primary,
            validator=validator,
            metadata={
                "tile_count": len(tiles),
                "onnx_enabled": self._onnx_session is not None,
            }
        )

```

```

        "int8_quantized": True,
    },
)

def _build_onnx_session(self) -> ort.InferenceSession | None:
    """Initialize ONNX Runtime session for quantized inference path."""
    if ort is None:
        return None
    # Placeholder path for exported INT8 ONNX model
    model_path = Path("models/surya_invoice_int8.onnx")
    if not model_path.exists():
        return None
    providers = ["CPUExecutionProvider"]
    return ort.InferenceSession(str(model_path), providers=providers)

def _build_paddle(self) -> PaddleOCR | None:
    """Initialize PaddleOCR validator."""
    if PaddleOCR is None:
        return None
    return PaddleOCR(use_angle_cls=True, lang="en", use_gpu=False)

def _run_primary_ocr(self, tiles: list[Image.Image]) -> str:
    """Execute primary OCR engine (Surya) via ONNX fallback abstraction."""
    all_text: list[str] = []
    for tile in tiles:
        if self._onnx_session is not None:
            all_text.append(self._infer_with_onnx(tile))
        else:
            # Placeholder adapter for Surya OCR Python API.
            all_text.append(self._surya_fallback(tile))
    return "\n".join(all_text)

def _run_validator_ocr(self, tiles: list[Image.Image]) -> str:
    """Execute PaddleOCR validation path."""
    if self._paddle is None:
        return ""
    lines: list[str] = []
    for tile in tiles:
        array = np.array(tile)
        result = self._paddle.ocr(array, cls=True)
        for block in result or []:
            for item in block or []:
                if len(item) >= 2 and len(item[1]) >= 1:
                    lines.append(str(item[1][0]))
    return "\n".join(lines)

def _infer_with_onnx(self, tile: Image.Image) -> str:
    """Run ONNX inference; the decoding pipeline can be plugged later."""
    assert self._onnx_session is not None
    resized = tile.resize((1024, 1024))
    data = np.asarray(resized, dtype=np.float32) / 255.0
    tensor = np.transpose(data, (2, 0, 1))[np.newaxis, ...]
    inputs = {self._onnx_session.get_inputs()[0].name: tensor}
    _ = self._onnx_session.run(None, inputs)
    # Future: decode token logits into extracted text.
    return ""

@staticmethod
def _surya_fallback(_tile: Image.Image) -> str:
    """Fallback stub when Surya runtime is not configured yet."""
    return ""

def _cross_validate(self, primary: OCRResult, validator: OCRResult) -> str:
    checksum_distance = 0 if primary.checksum == validator.checksum else 1
    nip_distance = 0 if primary.nip == validator.nip else 1

```

```

        if checksum_distance > self.checksum_distance_threshold or nip_distance > self.nip_mismatch_threshold:
            return ReviewStatus.MANUAL_REVIEW
        return ReviewStatus.APPROVED

    @staticmethod
    def _extract_fields(raw_text: str) -> OCRResult:
        nip_match = re.search(r"\b\d{10}\b", raw_text)
        checksum = hashlib.sha256(raw_text.encode("utf-8")).hexdigest()
        return OCRResult(
            raw_text=raw_text,
            nip=nip_match.group(0) if nip_match else None,
            checksum=checksum,
        )

    @staticmethod
    def _tile_image(image: Image.Image, tile_size: int = 2048, overlap: int = 256) -> list[Image.Image]:
        """Split huge scans into overlapping tiles to avoid OOM."""
        width, height = image.size
        if width <= tile_size and height <= tile_size:
            return [image]

        step = tile_size - overlap
        tiles: list[Image.Image] = []
        for y in range(0, max(height - overlap, 1), step):
            for x in range(0, max(width - overlap, 1), step):
                right = min(x + tile_size, width)
                lower = min(y + tile_size, height)
                tiles.append(image.crop((x, y, right, lower)))

        # Safety guard to avoid pathological tile explosion.
        max_tiles = math.ceil((width * height) / float(tile_size * tile_size)) * 8
        return tiles[:max_tiles]

# Wywołanie nowej warstwy ekstrakcji
from pipeline.parser import InvoiceParser

async def process_image_with_parser(self, image_path: str):
    # 1. OCR
    # results = run_ocr(image, [langs], self.det_model, ...)
    raw_text = self._assemble_text(results) # Funkcja ??cz?ca bloki tekstu w str

    # 2. EKSTRAKCJA (nowy krok)
    parser = InvoiceParser()
    parsed_data = parser.parse(raw_text)
    return parsed_data # Zwraca obiekt z gotowymi polami do bazy danych

===== Code/PIPELINE/ocr_engine.py =====
# pipeline/ocr_engine.py
import fitz # PyMuPDF
from PIL import Image
import io
from core.logger import logger

class VisionProcessor:
    """Silnik przetwarzaj?cy pliki PDF na obrazy i tekst (OCR)."""

    def __init__(self, config):
        self.config = config

    async def pdf_to_text(self, file_path: str) -> str:
        """Konwertuje PDF na tekst, obs?uguje?c warstwy tekstowe i skany."""
        full_text = []
        try:
            doc = fitz.open(file_path)

```

```

    for page_num in range(len(doc)):
        page = doc.load_page(page_num)

        # 1. Prba wyci?gni?cia tekstu natywnego
        text = page.get_text().strip()

        # 2. Je?li brak tekstu (skan), fallback do OCR (Surya/Tesseract)
        if len(text) < 10:
            logger.info(f"Strona {page_num} wygl?da na skan. Uruchamianie OCR...")
            text = await self._run_ocr_on_page(page)

        full_text.append(text)
    return "\n".join(full_text)
except Exception as e:
    logger.error(f"B??d przetwarzania pliku: {e}")
    return ""

```

```

async def _run_ocr_on_page(self, page) -> str:
    # Puste wywo?anie, wymaga dost?pu do DocumentProcessor.process_image()
    return ""

```

==== Code/PIPELINE/optimized_ocr.py =====

pipeline/optimized_ocr.py

```

import numpy as np
import math
from PIL import Image
from typing import List
from dataclasses import dataclass
from core.logger import logger

```

```

try:
    import torch
except ImportError:
    torch = None

```

```

try:
    import onnxruntime as ort
except ImportError:
    ort = None

```

```

@dataclass
class OcrResult:
    text: str

```

```

class HighPerformanceVisionEngine:
    """Zoptymalizowany silnik wizyjny (ONNX + Tiling + Inference Mode)."""

```

```

    def __init__(self, onnx_model_path: str = "models/surya_int8.onnx", tile_size: int = 1024, overlap: int = 10):
        self.tile_size = tile_size
        self.overlap = overlap
        self.ort_session = self._init_onnx(onnx_model_path)

```

```

        # Je?li ONNX niedost?pny, ?adujemy standardowy model PyTorch (Surya)
        if not self.ort_session and torch:
            self._init_pytorch_fallback()

```

```

    def _init_onnx(self, path: str):
        """Inicjalizacja lekkiego silnika ONNX Runtime."""
        if ort and Path(path).exists():
            logger.info(f"?adowanie sesji ONNX z {path}...")
            return ort.InferenceSession(path, providers=["CPUExecutionProvider"])
        return None

```

```

    def _init_pytorch_fallback(self):
        logger.info("Inicjalizacja mechanizmu fallback dla PyTorch...")

```

```

pass

def _tile_image(self, image: Image.Image) -> List[Image.Image]:
    """Tnie obrazek na kafelki z podanym naddatkiem (overlap)."""
    width, height = image.size
    step = self.tile_size - self.overlap
    tiles = []
    for y in range(0, max(height - self.overlap, 1), step):
        for x in range(0, max(width - self.overlap, 1), step):
            right = min(x + self.tile_size, width)
            lower = min(y + self.tile_size, height)
            tiles.append(image.crop((x, y, right, lower)))
    return tiles

def _infer_onnx(self, tile: Image.Image) -> str:
    # Logika dekodowania wyj?cia ONNX do tekstu...
    return "Przyk?adowy wyekstrahowany tekst kafelka"

# Zast?pienie no_grad() nowszym i szybszym inference_mode()
@torch.inference_mode()
def process_image(self, image_path: str) -> str:
    """G?wna metoda przetwarzania wykorzystuj?ca Tiling."""
    image = Image.open(image_path).convert("RGB")
    tiles = self._tile_image(image)
    extracted_texts = []

    for tile in tiles:
        if self.ort_session:
            extracted_texts.append(self._infer_onnx(tile))
        elif torch:
            # Omijamy autograd ca?kowicie - maksymalna pr?dko?? GPU
            extracted_texts.append("Tekst z fallbacku PyTorch")

    return "\n".join(extracted_texts)

===== Code/PIPELINE/parser.py =====
# pipeline/parser.py
import re
from decimal import Decimal, InvalidOperation
from dataclasses import dataclass
from typing import Optional

@dataclass
class ParsedInvoice:
    number: Optional[str] = None
    nip: Optional[str] = None
    amount_net: Decimal = Decimal("0.00")
    amount_gross: Decimal = Decimal("0.00")
    iban: Optional[str] = None
    currency: str = "PLN"

class InvoiceParser:
    def __init__(self):
        # Wzorce dla danych strukturalnych
        self.re_nip = re.compile(r"(?:NIP[\\s]*)?(\\d{3}[\\s]?\\d{3}[\\s]?\\d{2}[\\s]?\\d{2}|\\d{10})")
        self.re_iban = re.compile(r"(?:PL)?\\s?(\\d{2})(?:\\s?\\d{4}){6}")
        self.re_currency = re.compile(r"\\b(PLN|EUR|USD|z?|PLZ)\\b", re.IGNORECASE)
        # S?owa kluczowe pomagaj?ce znale?? kwoty
        self.gross_keywords = ["brutto", "razem", "suma", "total", "do zap?aty"]
        self.net_keywords = ["netto", "warto?? netto"]

    def parse(self, raw_text: str) -> ParsedInvoice:
        lines = raw_text.split("\\n")
        cleaned_text = raw_text.replace("\\n", "")
        result = ParsedInvoice()

```

```

# 1. Wyci?ganie NIP (pierwszy znaleziony to zazwyczaj Sprzedawca)
nip_match = self.re_nip.search(cleaned_text)
if nip_match:
    result.nip = re.sub(r"\D", "", nip_match.group(1))

# 2. Wyci?ganie IBAN (Konto bankowe)
iban_match = self.re_iban.search(cleaned_text)
if iban_match:
    result.iban = re.sub(r"\s", "", iban_match.group(1))

# 3. Waluta
curr_match = self.re_currency.search(cleaned_text)
if curr_match:
    result.currency = curr_match.group(0).upper().replace("Z?", "PLN")

# 4. Kwoty (Heurystyka)
result.amount_gross = self._find_amount_near_keywords(lines, self.gross_keywords)
result.amount_net = self._find_amount_near_keywords(lines, self.net_keywords)

return result

def _find_amount_near_keywords(self, lines: list[str], keywords: list[str]) -> Decimal:
    """Szuka liczb zmiennoprzecinkowych w liniach zawieraj?cych s?owa kluczowe."""
    for line in lines:
        line_lower = line.lower()
        if any(kw in line_lower for kw in keywords):
            # Szukamy czego? co wygl?da jak kwota (np. 1 234,56 lub 1234.56)
            amounts = re.findall(r"(\d[\d\s,.]*(\d{2}))", line)
            for amt in amounts:
                try:
                    # Normalizacja formatu do standardu Python (kropka jako separator)
                    clean_amt = amt.replace(" ", "").replace(",", ".")
                    return Decimal(clean_amt)
                except (InvalidOperation, ValueError):
                    continue
    return Decimal("0.00")

# Asynchroniczne wykorzystanie ActiveLearning w parsowaniu
async def process_extraction(raw_text: str, active_learning_engine):
    # 1. Standardowy OCR/Regex
    parser = InvoiceParser()
    extracted_data = parser.parse(raw_text).__dict__

    # 2. Zapytanie do Active Learning
    suggestion = await active_learning_engine.get_suggested_correction(raw_text, extracted_data['nip'])
    if suggestion:
        # Nadpisujemy dane tymi, kt?e u?ytkownik wprowadzi? poprzednio
        extracted_data.update(suggestion)
        extracted_data['status'] = "AUTO_CORRECTED"

    return extracted_data

async def check_for_anomalies(nip: str, current_amount: float, active_learning_engine):
    # 1. Pobieramy ostatnie 10 faktur od tego samego NIP-u z LanceDB
    historical_data = active_learning_engine.get_history_for_nip(nip, limit=10)

    if len(historical_data) < 3:
        return None # Za ma?o danych do analizy

    # 2. Obliczamy ?redni? i sprawdzamy odchylenie
    amounts = [doc['amount_net'] for doc in historical_data]
    avg_amount = sum(amounts) / len(amounts)

    # 3. Regu?a biznesowa: Je?li kwota jest o 40% wy?sza/ni?sza ni? ?rednia

```

```

threshold = 0.40
if current_amount > avg_amount * (1 + threshold):
    return {
        "is_anomaly": True,
        "message": f"Uwaga: Kwota ({current_amount} z?) jest drastycznie wy?sza ni? zazwyczaj od tego
    }
return None

```

==== Code/PIPELINE/preprocessor.py =====

```
# pipeline/preprocessor.py
```

```
import cv2
```

```
import numpy as np
```

```
from PIL import Image
```

```
import io
```

```
from core.logger import logger
```

```
class ImageOptimizer:
```

```
    """Poprawia jako?? wizualn? dokumentw przed OCR/Vision AI."""
```

```
    @staticmethod
```

```
    def enhance_for_ocr(image_bytes: bytes) -> bytes:
```

```
        """Zwi?ksza kontrast i usuwa szumy (Denoising)."""
```

```
        try:
```

```
            nparr = np.frombuffer(image_bytes, np.uint8)
```

```
            img = cv2.imdecode(nparr, cv2.IMREAD_GRAYSCALE)
```

```
            # 1. Progowanie adaptacyjne (Binarization)
```

```
            thresh = cv2.adaptiveThreshold(
```

```
                img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
```

```
                cv2.THRESH_BINARY, 11, 2
```

```
            )
```

```
            # 2. Usuwanie drobnych szumw
```

```
            kernel = np.ones((1, 1), np.uint8)
```

```
            opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
```

```
            _, encoded_img = cv2.imencode('.png', opening)
```

```
            return encoded_img.tobytes()
```

```
        except Exception as e:
```

```
            logger.error(f"Image enhancement failed: {e}")
```

```
            return image_bytes
```

```
    @staticmethod
```

```
    def deskew(image_bytes: bytes) -> bytes:
```

```
        """Prostuje tekst, je?li dokument zosta? krzywo zeskanowany."""
```

```
        try:
```

```
            nparr = np.frombuffer(image_bytes, np.uint8)
```

```
            img = cv2.imdecode(nparr, cv2.IMREAD_GRAYSCALE)
```

```
            # Znajdowanie kraw?dzi i prostowanie
```

```
            coords = np.column_stack(np.where(img > 0))
```

```
            angle = cv2.minAreaRect(coords)[-1]
```

```
            if angle < -45:
```

```
                angle = -(90 + angle)
```

```
            else:
```

```
                angle = -angle
```

```
            (h, w) = img.shape[:2]
```

```
            center = (w // 2, h // 2)
```

```
            M = cv2.getRotationMatrix2D(center, angle, 1.0)
```

```
            rotated = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_CUBIC, borderMode=cv2.BORDER_REPLICAT
```

```
            _, encoded_img = cv2.imencode('.png', rotated)
```

```
            return encoded_img.tobytes()
```

```
        except Exception as e:
```

```
logger.error(f"Image deskew failed: {e}")
return image_bytes
```

```
===== Code/PIPELINE/qa_engine.py =====
```

```
# pipeline/qa_engine.py
```

```
from decimal import Decimal
```

```
class QualityAssuranceEngine:
```

```
    """Oblicza 'Confidence Score' dla przetworzonego dokumentu."""
```

```
    @staticmethod
```

```
    def calculate_score(data: dict, ocr_confidence: float) -> float:
```

```
        """
```

```
        Ocena spjno?ci:
```

```
        - Czy wszystkie kluczowe pola s? obecne? (+40%)
```

```
        - Czy matematyka si? zgadza? (+30%)
```

```
        - Wynik OCR? (+30%)
```

```
        """
```

```
        score = 0.0
```

```
        required_fields = ["number", "contractor_nip", "amount_gross"]
```

```
        # 1. Sprawdzenie kompletno?ci
```

```
        found_fields = [f for f in required_fields if data.get(f)]
```

```
        score += (len(found_fields) / len(required_fields)) * 40
```

```
        # 2. Sprawdzenie matematyki
```

```
        try:
```

```
            net = Decimal(str(data.get("amount_net", 0)))
```

```
            vat = Decimal(str(data.get("amount_vat", 0)))
```

```
            gross = Decimal(str(data.get("amount_gross", 0)))
```

```
            if gross > 0 and abs((net + vat) - gross) < 0.05:
```

```
                score += 30
```

```
        except Exception:
```

```
            pass
```

```
        # 3. Zaufanie do modelu bazowego
```

```
        score += (ocr_confidence * 30)
```

```
        return min(score, 100.0)
```

```
===== Code/PIPELINE/refiner.py =====
```

```
# pipeline/refiner.py
```

```
from core.logger import logger
```

```
from core.llm_guard import LLMGuard
```

```
class PipelineRefiner:
```

```
    """Mechanizm 'Self-Correction' ? prosi AI o poprawk? w przypadku b??dw."""
```

```
    def __init__(self, llm_engine):
```

```
        self.llm = llm_engine
```

```
    async def refine_extraction(self, raw_text: str, previous_error: str, last_attempt: dict) -> dict:
```

```
        """Wysy?a b??d walidacji z powrotem do AI, aby model sprbowa? naprawi? dane."""
```

```
        logger.warning(f"Uruchamianie autokorekty AI dla b??du: {previous_error}")
```

```
        refine_prompt = (
```

```
            f"Poprzednia ekstrakcja danych: {last_attempt}\n"
```

```
            f"Wyst?pi? b??d: {previous_error}\n"
```

```
            f"Przeanalizuj tekst ponownie i popraw b??d. Zwr? tylko JSON.\n"
```

```
            f"Tekst ?rd?owy: {raw_text[:2000]}"
```

```
        )
```

```
        new_raw_output = await self.llm.generate(refine_prompt)
```

```
        return LLMGuard.parse_and_validate(new_raw_output)
```

```
===== Code/PIPELINE/splitter.py =====
```

```
# pipeline/splitter.py
```

```
import fitz # PyMuPDF
```

```
class DocumentSplitter:
```

```
    """Rozpoznaje wiele dokumentów w jednym pliku PDF."""
```

```
    @staticmethod
```

```
    def split_by_keyword(file_path: str, keyword: str = "Faktura nr") -> list[str]:
```

```
        """Dzieli PDF wsz?dzie tam, gdzie pojawia si? s?owo kluczowe."""
```

```
        doc = fitz.open(file_path)
```

```
        output_files = []
```

```
        current_doc = None
```

```
        for page_num in range(len(doc)):
```

```
            page = doc.load_page(page_num)
```

```
            text = page.get_text()
```

```
            if keyword.lower() in text.lower() or current_doc is None:
```

```
                if current_doc:
```

```
                    # Zapisz poprzedni dokument
```

```
                    pass
```

```
                # Rozpocznij nowy dokument
```

```
                pass
```

```
        return output_files
```

```
# pipeline/splitter.py
```

```
import fitz # PyMuPDF
```

```
from pipeline.ocr import DocumentProcessor
```

```
from core.logger import logger
```

```
class AIDocumentSplitter:
```

```
    """Dzieli zbiorcze pliki PDF na pojedyncze faktury przy u?yciu wizyjnego AI."""
```

```
    def __init__(self, document_processor: DocumentProcessor, llm_engine):
```

```
        self.processor = document_processor
```

```
        self.llm = llm_engine
```

```
    async def is_new_document_start(self, page_image) -> bool:
```

```
        """U?ywa VLM do oceny, czy strona jest nowym nag?wkiem faktury."""
```

```
        # Pobieramy tekst z gry strony przy u?yciu Surya
```

```
        ocr_result = self.processor.process_image(page_image)
```

```
        top_text = ocr_result.raw_text[:500] # Interesuje nas tylko gra dokumentu
```

```
        prompt = f"""
```

```
        Czy poni?szy tekst pochodzi z pocz?tku nowej faktury, paragonu lub rachunku?
```

```
        Odpowiedz tylko TAK lub NIE.
```

```
        Tekst: {top_text}
```

```
        """
```

```
        response = await self.llm.ask_boolean(prompt)
```

```
        return response
```

```
===== Code/PIPELINE/validators.py =====
```

```
# pipeline/validators.py
```

```
from pyhanko.pdf_utils.reader import PdfFileReader
```

```
from pyhanko.sign.validation import validate_pdf_signature
```

```
from pyhanko.keys import load_certs_from_pemder
```

```

def check_pades_signature(pdf_path: str) -> dict:
    """Weryfikuje, czy plik PDF posiada nienaruszony podpis elektroniczny."""
    try:
        with open(pdf_path, 'rb') as f:
            reader = PdfFileReader(f)

            # Pobieramy ukryte pola podpisw z PDF
            embedded_signatures = reader.embedded_signatures

            if not embedded_signatures:
                return {"is_signed": False, "status": "Brak podpisu"}

            # Walidacja pierwszego znalezionej podpisu
            sig = embedded_signatures[0]
            status = validate_pdf_signature(sig)

            return {
                "is_signed": True,
                "is_intact": status.intact,
                "is_valid": status.valid, # Wymaga sprawdzenia z list? zaufanych certyfikatw
                "signer_name": status.signer_cert.subject.human_friendly if status.signer_cert else "Niezn"
            }
    except Exception as e:
        return {"is_signed": False, "status": f"B??d weryfikacji: {str(e)}"}

===== Code/SERVICES/__init__.py =====
from .replication import ReplicationBridge
from .accounting import AccountingService
from .storage import StorageService
from .active_learning import ActiveLearningService
from .export_service import ExportService
from .ksef_service import KsefService
from .mail_fetcher import MailIngestionService

__all__ = [
    "ReplicationBridge",
    "AccountingService",
    "StorageService",
    "ActiveLearningService",
    "ExportService",
    "KsefService",
    "MailIngestionService"
]

===== Code/SERVICES/accountant_logic.py =====
from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from db.analytics import DuckDBManager

@dataclass(slots=True)
class AccountSuggestion:
    account_wn: str
    account_ma: str
    reason: str

class ZPKEngine:
    """Semantic Chart of Accounts engine backed by DuckDB."""

    def __init__(self, db: DuckDBManager):
        self.db = db

```

```

self._ensure_schema()

def _ensure_schema(self) -> None:
    self.db.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_accounts (
            account_number VARCHAR PRIMARY KEY,
            team_id INTEGER NOT NULL,
            report_mapping VARCHAR NOT NULL,
            description VARCHAR NOT NULL,
            parent_account VARCHAR
        )
        """
    )
    self.db.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_mapping_history (
            vendor_nip VARCHAR,
            keyword_hash VARCHAR,
            account_wn VARCHAR NOT NULL,
            account_ma VARCHAR NOT NULL,
            source VARCHAR NOT NULL,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )
    self.db.execute(
        """
        CREATE TABLE IF NOT EXISTS zpk_subaccounts (
            parent_account VARCHAR NOT NULL,
            vendor_nip VARCHAR NOT NULL,
            account_number VARCHAR PRIMARY KEY,
            description VARCHAR NOT NULL,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )

def seed_default_accounts(self, rows: list[dict[str, Any]]) -> None:
    for row in rows:
        self.db.execute(
            """
            INSERT OR REPLACE INTO zpk_accounts (account_number, team_id, report_mapping, description,
            VALUES (?, ?, ?, ?, ?)
            """,
            (
                row["account_number"],
                int(row["team_id"]),
                row["report_mapping"],
                row["description"],
                row.get("parent_account"),
            ),
        )

def suggest_accounts(self, invoice_data: dict[str, Any], company_profile: dict[str, Any]) -> AccountSu
    keywords = [str(item.get("name", "")).lower().strip() for item in invoice_data.get("items", [])]
    vendor_nip = str(invoice_data.get("vendor_nip", "")).strip()
    keyword_hash = "|".join(sorted([k for k in keywords if k]))

    historical = self.db.execute(
        """
        SELECT account_wn, account_ma
        FROM zpk_mapping_history
        WHERE vendor_nip = ? AND keyword_hash = ?
        ORDER BY created_at DESC

```

```

        LIMIT 1
        """
        (vendor_nip, keyword_hash),
    )
    if historical:
        return AccountSuggestion(account_wn=historical[0][0], account_ma=historical[0][1], reason="his

profile_kind = str(company_profile.get("business_kind", "").lower()
if any(word in keyword_hash for word in ["pr?d", "energia", "electricity"]):
    return AccountSuggestion("401-1", "202", "semantic_energy")
if any(word in keyword_hash for word in ["paliwo", "diesel", "benzyna"]):
    return AccountSuggestion("401-2", "202", "semantic_fuel")
if "laptop" in keyword_hash and "it" in profile_kind:
    return AccountSuggestion("010", "202", "semantic_fixed_asset_it")

return AccountSuggestion("409", "202", "fallback_other_costs")

def learn_mapping(self, *, vendor_nip: str, keyword_hash: str, account_wn: str, account_ma: str, source: str):
    self.db.execute(
        """
        INSERT INTO zpk_mapping_history (vendor_nip, keyword_hash, account_wn, account_ma, source)
        VALUES (?, ?, ?, ?, ?)
        """,
        (vendor_nip, keyword_hash, account_wn, account_ma, source),
    )

def ensure_vendor_subaccount(self, parent_account: str, vendor_nip: str, description: str) -> str:
    account_number = f"{parent_account}-{vendor_nip}"
    self.db.execute(
        """
        INSERT OR IGNORE INTO zpk_subaccounts (parent_account, vendor_nip, account_number, description)
        VALUES (?, ?, ?, ?)
        """,
        (parent_account, vendor_nip, account_number, description),
    )
    return account_number

===== Code/SERVICES/accounting.py =====
from datetime import date
from decimal import Decimal, ROUND_HALF_UP

import httpx

class AccountingService:
    def __init__(self):
        self.base_url = "https://wl-api.mf.gov.pl/api/search/nip/"

    @staticmethod
    def validate_amounts(net: Decimal, gross: Decimal) -> bool:
        """Sprawdza, czy kwoty s? matematycznie poprawne pod k?tem stawek VAT.
        Zak?ada, ?e VAT musi by? jedn? ze standardowych stawek (23%, 8%, 5%, 0%)."""
        if net <= 0 or gross <= 0 or gross < net:
            return False

        vat_amount = gross - net
        if vat_amount == 0:
            return True # Faktura zwolniona lub 0%

        # Obliczamy efektywn? stawk? VAT
        effective_rate = (vat_amount / net).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        valid_rates = [Decimal("0.23"), Decimal("0.08"), Decimal("0.05"), Decimal("0.00")]

        if effective_rate in valid_rates:
            return True

```

```

# Sprawdzanie najpopularniejszych stawek VAT w przypadku drobnych r?nic
calculated_rates = [
    round(net * Decimal("1.23"), 2),
    round(net * Decimal("1.08"), 2),
    round(net * Decimal("1.05"), 2)
]
return round(gross, 2) in calculated_rates

@staticmethod
def calculate_vat(net: Decimal, rate: float = 0.23) -> Decimal:
    vat = net * Decimal(str(rate))
    return vat.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

async def verify_nip(self, nip: str) -> dict | None:
    """Sprawdza NIP w bazie Ministerstwa Finansw (Bia?a Lista)."""
    # Oczyszczanie NIPu ze zb?dnych znakw (np. my?lnikw)
    clean_nip = "".join(filter(str.isdigit, nip))
    if not clean_nip or len(clean_nip) != 10:
        return None

    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            today = date.today().isoformat()
            response = await client.get(f"{self.base_url}{clean_nip}?date={today}")

            if response.status_code == 200:
                data = response.json()
                return data.get("result", {}).get("subject")
            return None
    except httpx.RequestError as e:
        print(f"[AccountingService] B??d po??czenia z Bia?? List?: {e}")
        return None
    except Exception as e:
        print(f"[AccountingService] Nieznany b??d weryfikacji NIP: {e}")
        return None

===== Code/SERVICES/active_learning.py =====
import json

from models.invoice import ActiveLearningPattern
from sqlalchemy.ext.asyncio import AsyncSession

class ActiveLearningService:
    """Us?uga ucz?ca system na podstawie r?cznych poprawek ksi?gowej."""

    @staticmethod
    async def register_correction(
        session: AsyncSession,
        nip: str,
        field_name: str,
        ai_guess: str,
        human_correction: str
    ) -> None:
        """Zapisuje, ?e AI si? pomyli?o i u?ytkownik musia? r?cznie poprawi? warto??."""
        # Je?li nie ma NIPu, nie mo?emy stworzy? wzorca
        if not nip:
            return

        correction_payload = {
            "field": field_name,
            "ai_guess": ai_guess,
            "human_correction": human_correction
        }

```

```

pattern = ActiveLearningPattern(
    contractor_nip=nip,
    correction_payload=json.dumps(correction_payload)
)
session.add(pattern)

async def find_similar_layout(self, ocr_text: str):
    """Szuka w bazie wektorowej podobnego uk?adu faktury."""
    vector = self.get_embedding(ocr_text)
    table = self.db.open_table("invoice_templates")

    # Wyszukiwanie wektorowe (ANN - Approximate Nearest Neighbor)
    results = table.search(vector).limit(1).to_list()

    if results and results[0]['_distance'] < 0.4: # Prg podobie?stwa
        return results[0]
    return None

def learn_new_template(self, nip: str, ocr_text: str):
    """Zapisuje nowy wzorzec do bazy wektorowej."""
    vector = self.get_embedding(ocr_text)
    table = self.db.open_table("invoice_templates")
    table.add([{"vector": vector, "contractor_nip": nip, "layout_features": ocr_text[:200]}])

===== Code/SERVICES/analytics_service.py =====
from db.analytics import DuckDBManager

class AnalyticsService:
    def __init__(self, duckdb: DuckDBManager):
        self.duckdb = duckdb

    def get_monthly_spending_trend(self) -> list[dict]:
        """Zwraca sumy wydatkw pogrupowane po miesi?cach dla wykresw."""
        query = """
        SELECT
            strftime('%Y-%m', issue_date) as month,
            SUM(amount_gross) as total_gross,
            COUNT(id) as invoice_count
        FROM invoices_replica
        WHERE status = 'APPROVED'
        GROUP BY 1
        ORDER BY 1 DESC
        LIMIT 12
        """
        return self.duckdb.execute_query(query)

    def get_top_suppliers(self, limit: int = 5) -> list[dict]:
        """Analizuje, u ktorych kontrahentw zostawiamy najwi?cej pieni?dzy."""
        query = f"""
        SELECT
            contractor_nip,
            SUM(amount_gross) as total_spent
        FROM invoices_replica
        WHERE status = 'APPROVED'
        GROUP BY 1
        ORDER BY 2 DESC
        LIMIT {limit}
        """
        return self.duckdb.execute_query(query)

===== Code/SERVICES/anomaly_detector.py =====
import pandas as pd
from db.analytics import DuckDBManager

```

```

class SmartAnomalyDetector:
    """Wykrywa podejrzone faktury przy użyciu algorytmów statystycznych."""

    def __init__(self, db_manager: DuckDBManager):
        self.db = db_manager

    def detect(self, contractor_nip: str, current_amount: float) -> bool:
        """
        Zwraca True, jeśli kwota faktury znacząco odbiega od
        historycznego profilu danego kontrahenta.
        """
        # Pobieramy historię kwot dla tego NIPu z DuckDB
        query = "SELECT amount_gross FROM invoices_replica WHERE contractor_nip = ?"
        history = self.db.execute_query(query, (contractor_nip,))

        if len(history) < 10:
            # Zbyt mało danych - wracamy do bezpiecznego Z-Score (średnia + 3 odchylenia)
            return False

        df = pd.DataFrame(history)
        # Isolation Forest:
        # Tutaj w prawdziwym kodzie byłaby logika dopasowania modelu (np. model.fit(df))
        return False

===== Code/SERVICES/audit_logger.py =====
from __future__ import annotations

import hashlib
import json
import uuid
from datetime import datetime, timezone
from typing import Any

from db.analytics import DuckDBManager

def ensure_forensic_audit_schema(duckdb: DuckDBManager) -> None:
    """Creates immutable-ish audit table used for cryptographic hash chaining."""
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS audit_log (
            id UUID PRIMARY KEY,
            timestamp TIMESTAMP NOT NULL,
            event_type VARCHAR NOT NULL,
            data_payload JSON NOT NULL,
            previous_hash VARCHAR NOT NULL,
            current_hash VARCHAR NOT NULL
        )
        """
    )
    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_audit_log_timestamp ON audit_log(timestamp)")

class AuditLogger:
    """Hash-chained audit logger backed by DuckDB."""

    GENESIS_HASH = "0"

    def __init__(self, duckdb: DuckDBManager):
        self.duckdb = duckdb
        ensure_forensic_audit_schema(self.duckdb)

```

```

@staticmethod
def _canonical_payload(data_payload: dict[str, Any]) -> str:
    return json.dumps(data_payload, sort_keys=True, ensure_ascii=False, separators=(",", ":"))

@classmethod
def _compute_hash(cls, previous_hash: str, payload_json: str) -> str:
    base = f"{previous_hash}{payload_json}".encode("utf-8")
    return hashlib.sha256(base).hexdigest()

def append_event(self, event_type: str, data_payload: dict[str, Any]) -> str:
    """Appends new audit event with chained hash; returns current hash."""
    connection = self.duckdb.connect()
    payload_json = self._canonical_payload(data_payload)
    timestamp = datetime.now(timezone.utc).replace(microsecond=0).isoformat()

    connection.execute("BEGIN TRANSACTION")
    try:
        row = connection.execute(
            "SELECT current_hash FROM audit_log ORDER BY timestamp DESC, id DESC LIMIT 1"
        ).fetchone()
        previous_hash = row[0] if row else self.GENESIS_HASH
        current_hash = self._compute_hash(previous_hash, payload_json)

        connection.execute(
            """
            INSERT INTO audit_log (id, timestamp, event_type, data_payload, previous_hash, current_hash)
            VALUES (?, ?, ?, ?, ?, ?)
            """
            ,
            (str(uuid.uuid4()), timestamp, event_type, payload_json, previous_hash, current_hash),
        )
        connection.execute("COMMIT")
        return current_hash
    except Exception:
        connection.execute("ROLLBACK")
        raise

def verify_chain(self) -> tuple[bool, str | None]:
    """Verifies full chain integrity. Returns (is_valid, first_broken_event_id)."""
    rows = self.duckdb.execute(
        """
        SELECT id, data_payload, previous_hash, current_hash
        FROM audit_log
        ORDER BY timestamp ASC, id ASC
        """
    )

    expected_previous = self.GENESIS_HASH
    for event_id, payload, previous_hash, current_hash in rows:
        if previous_hash != expected_previous:
            return False, str(event_id)

        payload_json = self._canonical_payload(payload if isinstance(payload, dict) else json.loads(payload))
        expected_current = self._compute_hash(previous_hash, payload_json)
        if current_hash != expected_current:
            return False, str(event_id)

        expected_previous = current_hash

    return True, None

```

===== Code/SERVICES/audit_service.py =====

```
import json
```

```
from models.audit import AuditLog
from sqlalchemy.ext.asyncio import AsyncSession
```

```

class AuditService:
    @staticmethod
    async def log_change(
        session: AsyncSession,
        user_id: str,
        action: str,
        target_id: str,
        old_data: dict,
        new_data: dict
    ):
        """Porównuje dane i zapisuje tylko faktyczne zmiany."""
        changes = {}
        for key, new_val in new_data.items():
            old_val = old_data.get(key)
            if old_val != new_val:
                # Konwersja Decimal/Datetime do stringa dla JSON
                changes[key] = {
                    "from": str(old_val) if old_val is not None else None,
                    "to": str(new_val)
                }

        if changes:
            entry = AuditLog(
                user_id=user_id,
                action=action,
                target_id=target_id,
                changes=json.dumps(changes)
            )
            session.add(entry)

```

===== Code/SERVICES/auto_decree.py =====

```

import lancedb
from core.config import AppConfig

```

```

class AutoDecreeEngine:
    def __init__(self, config: AppConfig):
        self.db = lancedb.connect(config.base_dir / "app_data" / "vector_store")
        self.table = self.db.open_table("invoice_templates")

    async def suggest_classification(self, contractor_nip: str, ocr_text: str) -> dict:
        """Sugeruje kategorię? KPiR i konta księgowo na podstawie podobieństwa."""

        # 1. Najpierw szukamy po NIP (dokładne dopasowanie historyczne)
        # 2. Jeśli brak, szukamy wektorowo po OCR_TEXT (podobieństwo branżowe)

        # Przykładowa logika "twardych reguł" dla popularnych NIPów:
        rules = {
            "5260250995": {"category": "Paliwo", "vat_deduction": 0.5, "account": "401-1"}, # Orlen
            "5261040567": {"category": "Telekomunikacja", "vat_deduction": 1.0, "account": "402-5"}, # Orange
        }

        if contractor_nip in rules:
            return rules[contractor_nip]

        return {}

```

===== Code/SERVICES/bank_import.py =====

```

from __future__ import annotations

```

```

import csv
import hashlib
import uuid

```

```

from dataclasses import dataclass
from datetime import date, datetime
from decimal import Decimal
from pathlib import Path
from typing import Protocol

from db.analytics import DuckDBManager
from Roboton_Reflekton.ledger_client import TigerBeetleClient

class DuplicateTransferError(RuntimeError):
    pass

@dataclass(slots=True)
class BankTransaction:
    booking_date: date
    amount: Decimal
    title: str
    counterparty_account: str
    balance_after: Decimal
    source_account_id: int
    destination_account_id: int

    @property
    def amount_cents(self) -> int:
        return int((self.amount * 100).quantize(Decimal("1")))

class StatementParser(Protocol):
    def parse(self, file_path: Path) -> list[BankTransaction]: ...

class CSVStatementParser:
    """Reference parser for local CSV exports from banks."""

    def parse(self, file_path: Path) -> list[BankTransaction]:
        rows: list[BankTransaction] = []
        with file_path.open("r", encoding="utf-8") as handle:
            reader = csv.DictReader(handle)
            for row in reader:
                rows.append(
                    BankTransaction(
                        booking_date=datetime.strptime(row["booking_date"], "%Y-%m-%d").date(),
                        amount=Decimal(row["amount"]),
                        title=row.get("title", "").strip(),
                        counterparty_account=row.get("counterparty_account", "").strip(),
                        balance_after=Decimal(row["balance_after"]),
                        source_account_id=int(row["source_account_id"]),
                        destination_account_id=int(row["destination_account_id"]),
                    )
                )
        return rows

class ParserFactory:
    @staticmethod
    def get_parser(file_path: Path) -> StatementParser:
        if file_path.suffix.lower() == ".csv":
            return CSVStatementParser()
        raise ValueError(f"Unsupported statement format: {file_path.suffix}")

def generate_idempotency_id(tx: BankTransaction) -> uuid.UUID:
    raw = f"{tx.booking_date.isoformat()}|{tx.amount}|{tx.title}|{tx.counterparty_account}|{tx.balance_aft

```

```
digest = hashlib.sha256(raw.encode("utf-8")).hexdigest()
return uuid.uuid5(uuid.NAMESPACE_DNS, digest)
```

```
class StatementContinuityError(RuntimeError):
    pass
```

```
class IdempotentBankImporter:
```

```
    def __init__(self, *, tb_client: TigerBeetleClient, duckdb: DuckDBManager, ledger_id: int = 1, transfer_code: str = "IDEMPOTENT"):
        self.tb_client = tb_client
        self.duckdb = duckdb
        self.ledger_id = ledger_id
        self.transfer_code = transfer_code
        self._ensure_history_schema()
```

```
    def _ensure_history_schema(self) -> None:
        self.duckdb.execute(
            """
```

```
        CREATE TABLE IF NOT EXISTS bank_history (
            tx_id VARCHAR PRIMARY KEY,
            booking_date DATE,
            amount DECIMAL(18, 2),
            title VARCHAR,
            counterparty_account VARCHAR,
            balance_after DECIMAL(18, 2),
            source_account_id BIGINT,
            destination_account_id BIGINT,
            imported_at TIMESTAMP DEFAULT now()
        )
        """
```

```
    )
```

```
    def validate_balance_continuity(self, transactions: list[BankTransaction]) -> None:
        if not transactions:
```

```
            return
```

```
        last = self.duckdb.execute("SELECT balance_after FROM bank_history ORDER BY booking_date DESC, imported_at ASC")
        if not last:
```

```
            return
```

```
        expected_opening = Decimal(str(last[0][0]))
```

```
        if transactions[0].balance_after - transactions[0].amount != expected_opening:
```

```
            raise StatementContinuityError(
```

```
                f"Balance continuity check failed. Expected opening {expected_opening}, got {(transactions[0].balance_after - transactions[0].amount)}")
        )
```

```
    async def import_file(self, file_path: Path) -> dict[str, int]:
```

```
        parser = ParserFactory.get_parser(file_path)
```

```
        transactions = parser.parse(file_path)
```

```
        transactions.sort(key=lambda t: t.booking_date)
```

```
        self.validate_balance_continuity(transactions)
```

```
        imported = 0
```

```
        duplicates = 0
```

```
        for tx in transactions:
```

```
            tx_uuid = generate_idempotency_id(tx)
```

```
            tx_id = str(tx_uuid)
```

```
            exists = self.duckdb.execute("SELECT 1 FROM bank_history WHERE tx_id = ? LIMIT 1", (tx_id,))
```

```
            if exists:
```

```
                duplicates += 1
```

```
                continue
```

```
        try:
```

```
            await self.tb_client.create_two_phase_transfer(
```

```
                debit_account=tx.source_account_id,
```

```
                credit_account=tx.destination_account_id,
```

```

        amount_minor=tx.amount_cents,
        source_document_id=tx.uuid,
    )
except Exception as exc:
    if "exists" in str(exc).lower():
        duplicates += 1
        continue
    raise

self.duckdb.execute(
    """
    INSERT INTO bank_history
    (tx_id, booking_date, amount, title, counterparty_account, balance_after, source_account_id, destination_account_id)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """,
    (
        tx_id,
        tx.booking_date,
        tx.amount,
        tx.title,
        tx.counterparty_account,
        tx.balance_after,
        tx.source_account_id,
        tx.destination_account_id,
    ),
)
imported += 1

return {"imported": imported, "duplicates": duplicates, "total": len(transactions)}

```

==== Code/SERVICES/cfo_offline.py =====

"""Offline-first architektura modu?w Autonomicznego Wirtualnego CFO.

Ten modu? zawiera lekk? referencj? implementacyjn? pokazuj?c?, jak po??czy?:

- Local RAG,
 - KSeF Defender (IsolationForest),
 - Cashflow Forecast (DuckDB),
 - Autodekretacj? (model klasyfikacyjny),
 z kolejk? asynchroniczn?, cache i magazynem relacji.

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
from datetime import date, datetime, timedelta, timezone
from typing import Any, Protocol
```

```
class TaskQueue(Protocol):
    """Abstrakcja kolejki (Celery/NATS)."""

    def publish(self, topic: str, payload: dict[str, Any]) -> None:
        ...
```

```
class CacheStore(Protocol):
    """Abstrakcja cache (Redis)."""

    def get(self, key: str) -> Any:
        ...

    def set(self, key: str, value: Any, ttl_seconds: int = 3600) -> None:
        ...
```

```

class RelationStore(Protocol):
    """Abstrakcja relacji biznesowych (Supabase/Postgres lub Neo4j)."""

    def link(self, source: str, relation: str, target: str, metadata: dict[str, Any] | None = None) -> None:
        ...

@dataclass(slots=True)
class InvoiceRecord:
    invoice_id: str
    supplier_nip: str
    amount_gross: float
    currency: str
    bank_account: str
    due_date: str
    lines: list[dict[str, Any]]
    ocr_text: str
    metadata: dict[str, Any] = field(default_factory=dict)

@dataclass(slots=True)
class RAGAnswer:
    answer: str
    evidence_chunks: list[str]
    chart_spec: dict[str, Any]

class LocalRAGService:
    """Lokalny pipeline RAG z OCR->embeddings->vector search->LLM."""

    def __init__(self, vector_store: Any, llm_client: Any, embedder: Any) -> None:
        self.vector_store = vector_store
        self.llm_client = llm_client
        self.embedder = embedder

    def index_invoice(self, invoice: InvoiceRecord) -> None:
        chunks = _chunk_text(invoice.ocr_text)
        vectors = [self.embedder.encode(chunk) for chunk in chunks]
        self.vector_store.upsert(invoice.invoice_id, chunks=chunks, vectors=vectors, metadata=invoice.metadata)

    def ask(self, question: str, top_k: int = 5) -> RAGAnswer:
        q_vector = self.embedder.encode(question)
        hits = self.vector_store.search(q_vector, top_k=top_k)
        context = "\n".join(hit["chunk"] for hit in hits)
        prompt = (
            "Jeste? lokalnym CFO copilotem. Odpowiadaj po polsku, precyzyjnie i z uzasadnieniem.\n"
            f"Pytanie: {question}\n"
            f"Kontekst:\n{context}"
        )
        answer = self.llm_client.generate(prompt)
        chart_spec = _build_chart_from_hits(hits)
        return RAGAnswer(answer=answer, evidence_chunks=[h["chunk"] for h in hits], chart_spec=chart_spec)

class KSEFDefenderService:
    """Detekcja anomalii faktur na podstawie historii i cech aktualnej faktury."""

    def __init__(self, model: Any, feature_pipeline: Any) -> None:
        self.model = model
        self.feature_pipeline = feature_pipeline

    def fit(self, historical_rows: list[dict[str, Any]]) -> None:
        x_train = self.feature_pipeline.transform(historical_rows)
        self.model.fit(x_train)

```

```

def evaluate(self, invoice: InvoiceRecord) -> tuple[bool, float, str]:
    features = self.feature_pipeline.transform([_invoice_to_features(invoice)])
    prediction = int(self.model.predict(features)[0])
    score = float(self.model.decision_function(features)[0])

    is_anomaly = prediction == -1
    reason = "anomaly: statistical_outlier" if is_anomaly else "ok"

    if _bank_account_changed(invoice):
        is_anomaly = True
        reason = "anomaly: bank_account_changed"

    return is_anomaly, score, reason

```

```

class CashflowForecastService:

```

```

    """Analiza DuckDB: przyszłe zobowiązania i alerty niedoboru płynności."""

```

```

def __init__(self, duckdb_conn: Any) -> None:
    self.duckdb = duckdb_conn

```

```

def forecast(self, horizon_days: int = 60) -> dict[str, Any]:

```

```

    query = f"""

```

```

    WITH future_liabilities AS (

```

```

        SELECT due_date::DATE AS day, SUM(amount_gross) AS outgoing

```

```

        FROM invoices_replica

```

```

        WHERE status IN ('APPROVED', 'AUTO_APPROVED')

```

```

        AND paid_at IS NULL

```

```

        AND due_date BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL {horizon_days} DAY

```

```

        GROUP BY 1

```

```

    ),

```

```

    rolling AS (

```

```

        SELECT day,

```

```

            outgoing,

```

```

            SUM(outgoing) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS c

```

```

        FROM future_liabilities

```

```

    )

```

```

    SELECT * FROM rolling ORDER BY day;

```

```

    """

```

```

    series = self.duckdb.execute(query).fetchall()

```

```

    min_balance = _estimate_min_balance(series)

```

```

    alerts: list[str] = []

```

```

    if min_balance < 0:

```

```

        alerts.append(f"Prognozowany niedobór środków: {abs(min_balance):.2f} PLN")

```

```

    return {"series": series, "min_balance": min_balance, "alerts": alerts}

```

```

def get_daily_forecast(self, days: int = 90, opening_balance: float = 0.0) -> list[tuple[date, float,
    """Zwraca dzienną projekcję salda: (data, saldo, confidence_level)."""

```

```

    today = date.today()

```

```

    end_date = today + timedelta(days=days)

```

```

    rows = self.duckdb.execute(

```

```

        """

```

```

        SELECT projected_date, flow_direction, amount, source_type

```

```

        FROM v_cashflow_projection

```

```

        WHERE projected_date BETWEEN ? AND ?

```

```

        ORDER BY projected_date ASC

```

```

        """

```

```

        (today, end_date),

```

```

    ).fetchall()

```

```

    daily_delta: dict[date, float] = {}

```

```

daily_confidence: dict[date, list[float]] = {}

for projected_date, flow_direction, amount, source_type in rows:
    amount_value = float(amount)
    sign = 1.0 if str(flow_direction).upper() == "INFLOW" else -1.0
    day = projected_date if isinstance(projected_date, date) else date.fromisoformat(str(projected_date))
    daily_delta[day] = daily_delta.get(day, 0.0) + (sign * amount_value)

    confidence = 0.95
    if source_type == "INVOICE_INFLOW":
        confidence = 0.75
    elif source_type in {"VAT_RESERVE", "INCOME_TAX_RESERVE", "INVOICE_OUTFLOW"}:
        confidence = 0.98
    elif source_type == "MANUAL_ITEM":
        confidence = 0.60
    daily_confidence.setdefault(day, []).append(confidence)

running = float(opening_balance)
forecast: list[tuple[date, float, float]] = []
cursor = today
while cursor <= end_date:
    running += daily_delta.get(cursor, 0.0)
    confidences = daily_confidence.get(cursor)
    confidence_level = sum(confidences) / len(confidences) if confidences else 0.9
    forecast.append((cursor, round(running, 2), round(confidence_level, 2)))
    cursor += timedelta(days=1)

return forecast

```

```

class PaymentPriorityService:
    """Silnik priorytetyzacji p?atno?ci dla zobowi?za? zakupowych."""

    def __init__(self, duckdb_conn: Any) -> None:
        self.duckdb = duckdb_conn

    def calculate_priority_score(self, invoice: dict[str, Any], today: date | None = None) -> tuple[int, list[str]]:
        """Zwraca score 0-100 oraz uzasadnienie dla pojedynczej faktury."""

        ref_day = today or date.today()
        score = 50
        reasons: list[str] = []

        skonto_deadline = invoice.get("skonto_deadline")
        if skonto_deadline:
            skonto_date = skonto_deadline if isinstance(skonto_deadline, date) else date.fromisoformat(str(skonto_deadline))
            hours_to_deadline = (datetime.combine(skonto_date, datetime.min.time()) - datetime.combine(ref_day, datetime.min.time())).total_seconds() / 3600
            if 0 <= hours_to_deadline <= 48:
                score += 40
                reasons.append("Skonto deadline within 48h")

        due_raw = invoice.get("due_date")
        if due_raw:
            due_date = due_raw if isinstance(due_raw, date) else date.fromisoformat(str(due_raw))
            if due_date < ref_day:
                days_late = (ref_day - due_date).days
                overdue_bonus = min(days_late * 2, 30)
                score += overdue_bonus
                reasons.append(f"Overdue by {days_late} day(s)")

        vendor_priority = str(invoice.get("vendor_priority", "")).strip().lower()
        if vendor_priority in {"1", "critical"}:
            score += 20
            reasons.append("Critical vendor")
        elif vendor_priority in {"4", "flexible"}:

```

```

        score -= 15
        reasons.append("Flexible vendor")

    penalty_rate = invoice.get("penalty_rate")
    if penalty_rate is not None:
        try:
            penalty_value = float(penalty_rate)
            if penalty_value > 0.0:
                reasons.append(f"Penalty rate {penalty_value:.2f}%")
        except (TypeError, ValueError):
            pass

    return max(0, min(100, int(round(score))))), reasons

def suggest_payment_batch(self, available_cash: float, today: date | None = None) -> dict[str, Any]:
    """Sugeruje paczkę? p?atno?ci mieszcz?c? si? w limicie 90% dost?pnej gotwki."""

    rows = self.duckdb.execute(
        """
        SELECT id, due_date, skonto_deadline, skonto_percent, vendor_priority, penalty_rate, amount_gross
        FROM invoices_replica
        WHERE status = 'UNPAID' AND COALESCE(amount_gross, 0) > 0
        """
    ).fetchall()

    scored: list[dict[str, Any]] = []
    for invoice_id, due_date, skonto_deadline, skonto_percent, vendor_priority, penalty_rate, amount_gross in rows:
        invoice_data = {
            "id": str(invoice_id),
            "due_date": due_date,
            "skonto_deadline": skonto_deadline,
            "skonto_percent": skonto_percent,
            "vendor_priority": vendor_priority,
            "penalty_rate": penalty_rate,
            "amount_gross": float(amount_gross),
        }
        score, reasons = self.calculate_priority_score(invoice_data, today=today)

        if skonto_percent:
            reasons.append(f"Skonto {float(skonto_percent):.2f}% savings")

        scored.append({"invoice": invoice_data, "score": score, "reasons": reasons})

    scored.sort(key=lambda item: item["score"], reverse=True)

    safe_limit = max(0.0, float(available_cash) * 0.9)
    selected: list[dict[str, Any]] = []
    waiting: list[dict[str, Any]] = []
    used_cash = 0.0

    for item in scored:
        amount = float(item["invoice"]["amount_gross"])
        payload = {
            "invoice_id": item["invoice"]["id"],
            "amount": round(amount, 2),
            "priority_score": item["score"],
            "reason": "; ".join(item["reasons"]) if item["reasons"] else "Standard priority",
        }
        if used_cash + amount <= safe_limit:
            selected.append(payload)
            used_cash += amount
        else:
            waiting.append(payload)

    return {

```

```

        "safe_limit": round(safe_limit, 2),
        "used_cash": round(used_cash, 2),
        "recommended_today": selected,
        "wait": waiting,
    }

```

```

class AutoDecreeService:

```

```

    """Klasyfikacja pozycji faktury do kont księgowych."""

```

```

    def __init__(self, classifier: Any, feature_builder: Any, threshold: float = 0.82) -> None:
        self.classifier = classifier
        self.feature_builder = feature_builder
        self.threshold = threshold

```

```

    def classify_invoice(self, invoice: InvoiceRecord) -> dict[str, Any]:
        assignments: list[dict[str, Any]] = []
        confidences: list[float] = []

```

```

        for line in invoice.lines:
            features = self.feature_builder.transform(line, invoice.metadata)
            account = self.classifier.predict([features])[0]
            confidence = float(max(self.classifier.predict_proba([features])[0]))
            assignments.append({"line": line, "account": account, "confidence": confidence})
            confidences.append(confidence)

```

```

        auto_approved = bool(confidences) and min(confidences) >= self.threshold
        return {
            "assignments": assignments,
            "status": "AUTO_APPROVED" if auto_approved else "REVIEW_REQUIRED",
            "min_confidence": min(confidences) if confidences else 0.0,
        }

```

```

class CF00rchestrator:

```

```

    """Orkiestruje przepływ danych między modułami offline-first."""

```

```

    def __init__(
        self,
        queue: TaskQueue,
        cache: CacheStore,
        relations: RelationStore,
        rag: LocalRAGService,
        defender: KSEFDefenderService,
        forecast: CashflowForecastService,
        autodecree: AutoDecreeService,
    ) -> None:
        self.queue = queue
        self.cache = cache
        self.relations = relations
        self.rag = rag
        self.defender = defender
        self.forecast = forecast
        self.autodecree = autodecree

```

```

    def ingest_ksef_invoice(self, invoice: InvoiceRecord) -> dict[str, Any]:
        is_anomaly, score, reason = self.defender.evaluate(invoice)

```

```

        event_base = {
            "invoice_id": invoice.invoice_id,
            "score": score,
            "reason": reason,
            "created_at": datetime.now(timezone.utc).isoformat(),
        }

```

```

if is_anomaly:
    self.queue.publish("invoice.blocked", {"event": "invoice.blocked", **event_base})
    self.cache.set(f"invoice:{invoice.invoice_id}:status", "BLOCKED", ttl_seconds=86_400)
    self.relations.link(invoice.invoice_id, "HAS_ALERT", reason, metadata={"score": score})
    return {"status": "BLOCKED", "reason": reason, "score": score}

decree = self.autodecree.classify_invoice(invoice)
self.rag.index_invoice(invoice)
cashflow = self.forecast.forecast()

status = decree["status"]
self.cache.set(f"invoice:{invoice.invoice_id}:status", status, ttl_seconds=86_400)
self.queue.publish("invoice.processed", {"event": "invoice.processed", **event_base, "status": sta

if cashflow["alerts"]:
    self.queue.publish(
        "cashflow.alert",
        {
            "event": "cashflow.alert",
            "invoice_id": invoice.invoice_id,
            "alerts": cashflow["alerts"],
            "created_at": datetime.now(timezone.utc).isoformat(),
        },
    )

    self.relations.link(invoice.invoice_id, "CLASSIFIED_AS", status)
    return {"status": status, "decree": decree, "cashflow": cashflow}

def _chunk_text(text: str, size: int = 800) -> list[str]:
    return [text[i : i + size] for i in range(0, len(text), size)] or [""]

def _build_chart_from_hits(hits: list[dict[str, Any]]) -> dict[str, Any]:
    return {
        "type": "bar",
        "title": "Najbardziej relewantne dokumenty",
        "labels": [hit.get("invoice_id", "n/a") for hit in hits],
        "values": [float(hit.get("score", 0.0)) for hit in hits],
    }

def _estimate_min_balance(series: list[tuple[Any, ...]], opening_balance: float = 0.0) -> float:
    running = opening_balance
    min_balance = opening_balance
    for _, outgoing, *_ in series:
        running -= float(outgoing)
        min_balance = min(min_balance, running)
    return min_balance

def _invoice_to_features(invoice: InvoiceRecord) -> dict[str, Any]:
    return {
        "amount_gross": invoice.amount_gross,
        "supplier_nip": invoice.supplier_nip,
        "bank_account": invoice.bank_account,
        "line_count": len(invoice.lines),
    }

def _bank_account_changed(invoice: InvoiceRecord) -> bool:
    known_accounts = set(invoice.metadata.get("known_supplier_accounts", []))
    return bool(known_accounts) and invoice.bank_account not in known_accounts

===== Code/SERVICES/compliance_analytics.py =====

```

```

from __future__ import annotations

import json
from dataclasses import dataclass
from decimal import Decimal
from enum import Enum
from typing import Any, Literal

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from db.analytics import DuckDBManager

class AccountNature(str, Enum):
    DEBIT = "DEBIT"
    CREDIT = "CREDIT"

TransactionType = Literal["INVOICE_SALE", "BAD_DEBT_PROVISION", "DIVIDEND_PAYOUT"]

@dataclass(frozen=True)
class LedgerEntryPacket:
    transaction_type: TransactionType
    debit_account_code: str
    credit_account_code: str
    tax_impact_code: str
    amount_minor: int
    currency: str
    metadata_tags: dict[str, Any]

def ensure_compliance_analytics_schema(duckdb: "DuckDBManager") -> None:
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS gl_accounts (
            account_code VARCHAR PRIMARY KEY,
            account_name VARCHAR NOT NULL,
            account_nature VARCHAR NOT NULL CHECK (account_nature IN ('DEBIT', 'CREDIT')),
            account_class VARCHAR NOT NULL,
            parent_account_code VARCHAR,
            is_active BOOLEAN NOT NULL DEFAULT TRUE
        )
        """
    )

    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS transaction_rules (
            transaction_type VARCHAR PRIMARY KEY,
            debit_account_code VARCHAR NOT NULL,
            credit_account_code VARCHAR NOT NULL,
            tax_impact_code VARCHAR NOT NULL,
            description VARCHAR
        )
        """
    )

    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS tb_ledger_transfers (
            transfer_id UUID PRIMARY KEY,
            occurred_at TIMESTAMP NOT NULL,
            debit_account_code VARCHAR NOT NULL,

```

```

        credit_account_code VARCHAR NOT NULL,
        amount_minor BIGINT NOT NULL,
        currency VARCHAR NOT NULL,
        semantic_tags JSON,
        audit_log_id UUID
    )
    """"
)

duckdb.execute(
    """"
    INSERT OR IGNORE INTO gl_accounts (account_code, account_name, account_nature, account_class, parent_account_code)
    VALUES
        ('1000', 'Assets', 'DEBIT', 'ASSET', NULL),
        ('1100', 'Assets:Cash', 'DEBIT', 'ASSET', '1000'),
        ('2000', 'Liabilities', 'CREDIT', 'LIABILITY', NULL),
        ('2100', 'Liabilities:Payables', 'CREDIT', 'LIABILITY', '2000'),
        ('3000', 'Equity', 'CREDIT', 'EQUITY', NULL),
        ('4000', 'Revenue', 'CREDIT', 'REVENUE', NULL),
        ('4100', 'Revenue:Sales', 'CREDIT', 'REVENUE', '4000'),
        ('5000', 'Expenses', 'DEBIT', 'EXPENSE', NULL),
        ('5100', 'Expenses:BadDebt', 'DEBIT', 'EXPENSE', '5000')
    """"
)

duckdb.execute(
    """"
    INSERT OR IGNORE INTO transaction_rules (transaction_type, debit_account_code, credit_account_code, description)
    VALUES
        ('INVOICE_SALE', '1100', '4100', 'VAT_OUTPUT', 'Standard sales invoice'),
        ('BAD_DEBT_PROVISION', '5100', '2100', 'NO_VAT', 'Provision for bad debt'),
        ('DIVIDEND_PAYOUT', '3000', '1100', 'NO_VAT', 'Dividend payout to shareholders')
    """"
)

duckdb.execute(
    """"
    CREATE OR REPLACE VIEW v_account_balances AS
    WITH base AS (
        SELECT debit_account_code AS account_code, amount_minor AS delta_minor, currency, occurred_at,
        FROM tb_ledger_transfers
        UNION ALL
        SELECT credit_account_code AS account_code, -amount_minor AS delta_minor, currency, occurred_at,
        FROM tb_ledger_transfers
    ),
    recursive_rollup AS (
        SELECT
            a.account_code,
            a.parent_account_code,
            b.delta_minor,
            b.currency,
            b.occurred_at,
            b.semantic_tags
        FROM gl_accounts a
        LEFT JOIN base b ON b.account_code = a.account_code

        UNION ALL

        SELECT
            p.account_code,
            p.parent_account_code,
            rr.delta_minor,
            rr.currency,
            rr.occurred_at,
            rr.semantic_tags
    )

```

```

        FROM recursive_rollup rr
        JOIN gl_accounts p ON rr.parent_account_code = p.account_code
    )
    SELECT
        account_code,
        currency,
        SUM(COALESCE(delta_minor, 0)) AS balance_minor,
        MAX(occurred_at) AS as_of_ts,
        semantic_tags
    FROM recursive_rollup
    GROUP BY 1,2,5
    """
)

```

```

def infer_ledger_entries(duckdb: "DuckDBManager", transaction_data: dict[str, Any]) -> LedgerEntryPacket:
    transaction_type = str(transaction_data["transaction_type"])
    amount = Decimal(str(transaction_data["amount"]))
    currency = str(transaction_data.get("currency", "PLN"))
    if len(currency) != 3:
        raise ValueError("Currency must follow ISO 4217 (3-letter code)")

    rows = duckdb.execute(
        """
        SELECT debit_account_code, credit_account_code, tax_impact_code
        FROM transaction_rules
        WHERE transaction_type = ?
        LIMIT 1
        """,
        (transaction_type,),
    )
    if not rows:
        raise ValueError(f"No rule for transaction_type={transaction_type}")

    debit_account, credit_account, tax_impact_code = rows[0]

    return LedgerEntryPacket(
        transaction_type=transaction_type, # type: ignore[arg-type]
        debit_account_code=str(debit_account),
        credit_account_code=str(credit_account),
        tax_impact_code=str(tax_impact_code),
        amount_minor=int(amount * 100),
        currency=currency.upper(),
        metadata_tags=transaction_data.get("metadata_tags", {}),
    )

def query_account_balances_by_tag(
    duckdb: "DuckDBManager",
    *,
    tag_key: str,
    tag_value: str,
) -> list[tuple[Any, ...]]:
    """BQL-like helper for JSON tag filtering over v_account_balances."""
    return duckdb.execute(
        """
        SELECT account_code, currency, balance_minor
        FROM v_account_balances
        WHERE json_extract_string(semantic_tags, CONCAT('$.', ?)) = ?
        ORDER BY account_code
        """,
        (tag_key, tag_value),
    )

===== Code/SERVICES/document_fingerprint.py =====

```

```

from __future__ import annotations

import hashlib
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from db.analytics import DuckDBManager

try:
    import imagehash
    from PIL import Image
except Exception: # pragma: no cover
    imagehash = None
    Image = None

@dataclass(slots=True)
class DocumentFingerprint:
    binary_hash: str
    visual_hash: str
    semantic_hash: str

def _sha256_file(file_path: Path) -> str:
    h = hashlib.sha256()
    with file_path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b''):
            h.update(chunk)
    return h.hexdigest()

def _visual_fingerprint(file_path: Path) -> str:
    """Best-effort visual hash; falls back to deterministic prefix hash if imaging libs are missing."""
    if imagehash is not None and Image is not None:
        try:
            with Image.open(file_path) as img:
                return str(imagehash.phash(img.convert("L")))
        except Exception:
            pass

    with file_path.open("rb") as handle:
        sample = handle.read(4096)
    return hashlib.sha1(sample).hexdigest()[:16]

def _semantic_hash(extracted_data: dict[str, Any]) -> str:
    raw = f"{extracted_data.get('nip', '')}_{extracted_data.get('total_gross', '')}_{extracted_data.get('dat"
    return hashlib.md5(raw.encode("utf-8")).hexdigest()

def generate_document_fingerprint(file_path: Path, extracted_data: dict[str, Any]) -> DocumentFingerprint:
    return DocumentFingerprint(
        binary_hash=_sha256_file(file_path),
        visual_hash=_visual_fingerprint(file_path),
        semantic_hash=_semantic_hash(extracted_data),
    )

def binary_anchor_u128(binary_hash: str) -> int:
    return int(binary_hash[:32], 16)

def ensure_fingerprint_schema(duckdb: DuckDBManager) -> None:
    duckdb.execute(

```

```

        """
        CREATE TABLE IF NOT EXISTS document_fingerprints (
            invoice_id VARCHAR PRIMARY KEY,
            file_path VARCHAR NOT NULL,
            binary_hash VARCHAR NOT NULL,
            visual_hash VARCHAR NOT NULL,
            semantic_hash VARCHAR NOT NULL,
            tigerbeetle_anchor_u128 VARCHAR,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS monthly_merkle_roots (
            period_yyyymm VARCHAR PRIMARY KEY,
            root_hash VARCHAR NOT NULL,
            document_count INTEGER NOT NULL,
            created_at TIMESTAMP DEFAULT now()
        )
        """
    )

def store_fingerprint(duckdb: DuckDBManager, *, invoice_id: str, file_path: Path, fp: DocumentFingerprint)
    ensure_fingerprint_schema(duckdb)
    duckdb.execute(
        """
        INSERT OR REPLACE INTO document_fingerprints
        (invoice_id, file_path, binary_hash, visual_hash, semantic_hash, tigerbeetle_anchor_u128)
        VALUES (?, ?, ?, ?, ?, ?)
        """,
        (invoice_id, str(file_path), fp.binary_hash, fp.visual_hash, fp.semantic_hash, str(binary_anchor_u128))
    )

def compute_monthly_merkle_root(binary_hashes: list[str]) -> str:
    if not binary_hashes:
        return ""
    level = sorted(binary_hashes)
    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1])
        nxt: list[str] = []
        for i in range(0, len(level), 2):
            nxt.append(hashlib.sha256(f"{level[i]}{level[i+1]}".encode("utf-8")).hexdigest())
        level = nxt
    return level[0]

def store_monthly_merkle_root(duckdb: DuckDBManager, period_yyyymm: str) -> str:
    ensure_fingerprint_schema(duckdb)
    rows = duckdb.execute(
        """
        SELECT binary_hash
        FROM document_fingerprints
        WHERE strftime(created_at, '%Y%m') = ?
        ORDER BY binary_hash
        """,
        (period_yyyymm,)
    )
    hashes = [row[0] for row in rows]
    root = compute_monthly_merkle_root(hashes)
    duckdb.execute(
        """

```

```

        INSERT OR REPLACE INTO monthly_merkle_roots (period_yyyymm, root_hash, document_count)
        VALUES (?, ?, ?)
        """
        (period_yyyymm, root, len(hashes)),
    )
    return root

```

```

def verify_or_flag_tamper(stored: DocumentFingerprint, current: DocumentFingerprint) -> str:
    if stored.binary_hash == current.binary_hash:
        return "OK"
    if stored.visual_hash == current.visual_hash and stored.semantic_hash == current.semantic_hash:
        return "MODIFIED_EXTERNALLY_BUT_SEMANTICALLY_SAME"
    return "TAMPERED"

```

```

===== Code/SERVICES/export_service.py =====
from models.invoice import Invoice
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

```

```

class ExportService:
    """Zarządza eksportem faktur do zewnętrznych systemów ERP."""

    @staticmethod
    async def generate_export_payload(
        session: AsyncSession,
        invoice_ids: list[str],
        system_name: str = "INSERT_EPP"
    ) -> str:
        """Pobiera faktury i generuje plik tekstowy dla systemu księgowego."""
        # 1. Pobieramy faktury z bazy
        query = select(Invoice).where(Invoice.id.in_(invoice_ids))
        result = await session.execute(query)
        invoices = result.scalars().all()

        if not invoices:
            raise ValueError("Nie znaleziono faktur do eksportu.")

        # 2. Pobieramy konfigurację eksportu...
        # Dalsza implementacja logiki eksportu

```

```

===== Code/SERVICES/fixed_assets.py =====
from __future__ import annotations

import calendar
import uuid
from dataclasses import dataclass
from datetime import date
from decimal import Decimal, ROUND_HALF_UP

from db.analytics import DuckDBManager
from Roboton_Reflekton.ledger_client import TigerBeetleClient

```

```

DEFAULT_ASSET_ACCOUNT = 10
DEFAULT_DEPRECIATION_ACCOUNT = 400
DEFAULT_LEDGER_ID = 2
DEFAULT_TRANSFER_CODE = 1001

```

```

@dataclass(slots=True)
class FixedAsset:
    id: str
    asset_name: str

```

```

initial_value: Decimal
residual_value: Decimal
depreciation_rate: float
purchase_date: date
last_depreciation_date: date | None

def _month_end(year: int, month: int) -> date:
    return date(year, month, calendar.monthrange(year, month)[1])

def _first_of_next_month(input_date: date) -> date:
    if input_date.month == 12:
        return date(input_date.year + 1, 1, 1)
    return date(input_date.year, input_date.month + 1, 1)

class FixedAssetsService:
    def __init__(self, duckdb: DuckDBManager, tigerbeetle: TigerBeetleClient) -> None:
        self.duckdb = duckdb
        self.tigerbeetle = tigerbeetle

    def generate_schedule(self, asset_id: str) -> int:
        """Generate/refresh straight-line depreciation schedule starting next month."""
        row = self.duckdb.execute(
            """
            SELECT id, asset_name, initial_value, COALESCE(salvage_value, residual_value, 0), depreciation
                   COALESCE(annual_rate, depreciation_rate), COALESCE(start_date, purchase_date), last_dep
                   account_id_debit, account_id_credit
            FROM fixed_assets
            WHERE id = ? AND status = 'ACTIVE'
            """,
            (asset_id,),
        )
        if not row:
            return 0

        asset = row[0]
        initial_value = Decimal(str(asset[2])).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        residual_value = Decimal(str(asset[3])).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        depreciation_method = str(asset[4] or "LINEAR").upper()
        rate = Decimal(str(asset[5]))
        purchase_date = asset[6]
        if depreciation_method != "LINEAR":
            return 0
        if initial_value <= residual_value:
            return 0
        annual_amount = ((initial_value - residual_value) * rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        monthly_amount = (annual_amount / Decimal("12")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        if monthly_amount <= Decimal("0.00"):
            return 0

        self.duckdb.execute("DELETE FROM depreciation_schedule WHERE asset_id = ? AND is_posted = FALSE",
                             asset_id)

        schedule_rows: list[tuple] = []
        month_cursor = _first_of_next_month(purchase_date)
        posted_sum_rows = self.duckdb.execute(
            "SELECT COALESCE(SUM(amount), 0) FROM depreciation_schedule WHERE asset_id = ? AND is_posted = FALSE",
            (asset_id,),
        )
        posted_sum = Decimal(str(posted_sum_rows[0][0])).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        remaining = (initial_value - residual_value - posted_sum).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        if remaining <= Decimal("0.00"):
            return 0

```

```
while remaining > Decimal("0.00"):
    installment = monthly_amount if monthly_amount <= remaining else remaining
    installment = installment.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

```

        ELSE status
    END
    WHERE id = ?
    """
    (asset_id, float(target_total), asset_id),
)
    posted += 1
    return posted

```

===== Code/SERVICES/fraud_graph_scanner.py =====

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import date, timedelta
from typing import Any, TYPE_CHECKING

try:
    import networkx as nx
except Exception: # pragma: no cover - optional fallback when networkx is unavailable
    class _FallbackGraph:
        def __init__(self):
            self._nodes: dict[str, dict[str, str]] = {}
            self._adj: dict[str, set[str]] = {}

        def add_node(self, node: str, **attrs: str):
            self._nodes[node] = attrs
            self._adj.setdefault(node, set())

        def add_edge(self, a: str, b: str, **attrs: str):
            self._adj.setdefault(a, set()).add(b)
            self._adj.setdefault(b, set()).add(a)

        def neighbors(self, node: str):
            return iter(self._adj.get(node, set()))

        def nodes(self, data: bool = False):
            return list(self._nodes.items()) if data else list(self._nodes.keys())

    class _NX: # minimal shim for APIs used by this scanner
        Graph = _FallbackGraph

    nx = _NX()

if TYPE_CHECKING:
    from db.analytics import DuckDBManager

@dataclass(frozen=True)
class FraudAlert:
    rule_code: str
    severity: str
    employee_id: str
    vendor_id: str
    shared_attribute: str
    shared_value: str

class FraudGraphScanner:
    def __init__(self, duckdb: "DuckDBManager"):
        self.duckdb = duckdb

    def _fetch_entities(self, months_back: int = 12) -> list[tuple[Any, ...]]:
        cutoff_date = date.today() - timedelta(days=months_back * 30)
        return self.duckdb.execute(
            """

```

```

        SELECT entity_type, entity_id, iban, physical_address
        FROM fraud_entity_registry
        WHERE is_active = TRUE
            AND (last_seen_at IS NULL OR last_seen_at >= ?)
            """,
        (cutoff_date, ),
    )

```

```
@staticmethod
```

```
def _build_graph(rows: list[tuple[Any, ...]]):
```

```

    graph = nx.Graph()
    for entity_type, entity_id, iban, physical_address in rows:
        entity_node = f"{entity_type}:{entity_id}"
        graph.add_node(entity_node, node_type=entity_type)
        if iban:
            iban_node = f"IBAN:{str(iban).strip()}"
            graph.add_node(iban_node, node_type="IBAN")
            graph.add_edge(entity_node, iban_node, edge_type="Shares_IBAN")
        if physical_address:
            address_node = f"ADDR:{str(physical_address).strip().lower()}"
            graph.add_node(address_node, node_type="ADDRESS")
            graph.add_edge(entity_node, address_node, edge_type="Shares_Address")
    return graph

```

```
def detect_shared_identity_links(self, months_back: int = 12) -> list[FraudAlert]:
```

```

    graph = self._build_graph(self._fetch_entities(months_back=months_back))
    employees = [n for n, d in graph.nodes(data=True) if d.get("node_type") == "EMPLOYEE"]
    vendors = [n for n, d in graph.nodes(data=True) if d.get("node_type") == "VENDOR"]
    alerts: list[FraudAlert] = []
    for employee in employees:
        for vendor in vendors:
            for shared in set(graph.neighbors(employee)).intersection(graph.neighbors(vendor)):
                if shared.startswith("IBAN:"):
                    alerts.append(FraudAlert("GHOST_VENDOR_SHARED_IBAN", "CRITICAL", employee.split(':')
                elif shared.startswith("ADDR:"):
                    alerts.append(FraudAlert("GHOST_VENDOR_SHARED_ADDRESS", "HIGH", employee.split(':')
    return alerts

```

```
===== Code/SERVICES/fx_revaluation.py =====
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from decimal import Decimal, ROUND_HALF_UP
```

```
from datetime import date
```

```
from typing import Any
```

```
from typing import TYPE_CHECKING
```

```
if TYPE_CHECKING:
```

```
    from db.analytics import DuckDBManager
```

```
@dataclass(frozen=True)
```

```
class FXPostingDecision:
```

```
    invoice_id: str
```

```
    fx_delta: Decimal
```

```
    is_gain: bool
```

```
    account_code: str
```

```
    entry_side: str
```

```
def ensure_fx_schema(duckdb: "DuckDBManager") -> None:
```

```
    duckdb.execute("ALTER TABLE invoices_replica ADD COLUMN IF NOT EXISTS currency_code VARCHAR DEFAULT 'P")
```

```
    duckdb.execute("ALTER TABLE invoices_replica ADD COLUMN IF NOT EXISTS base_currency_amount DECIMAL(18,
```

```
    duckdb.execute("ALTER TABLE invoices_replica ADD COLUMN IF NOT EXISTS exchange_rate_at_issue DECIMAL(1
```

```

duckdb.execute(
    """
    CREATE TABLE IF NOT EXISTS fx_rates (
        rate_date DATE NOT NULL,
        currency_code VARCHAR NOT NULL,
        rate DECIMAL(18, 8) NOT NULL,
        PRIMARY KEY(rate_date, currency_code)
    )
    """
)

def post_realized_fx_difference(
    *,
    invoice_id: str,
    invoice_type: str,
    payment_amount_foreign: Decimal,
    exchange_rate_at_issue: Decimal,
    payment_date_rate: Decimal,
) -> FXPostingDecision | None:
    original_local = (payment_amount_foreign * exchange_rate_at_issue).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    settlement_local = (payment_amount_foreign * payment_date_rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    fx_delta = settlement_local - original_local

    if abs(fx_delta) < Decimal("0.01"):
        return None

    invoice_type = invoice_type.upper()
    if invoice_type not in {"SALES", "PURCHASE"}:
        raise ValueError("invoice_type must be SALES or PURCHASE")

    if invoice_type == "SALES":
        is_gain = fx_delta > 0
    else: # PURCHASE
        is_gain = fx_delta < 0

    if is_gain:
        return FXPostingDecision(invoice_id, fx_delta, True, "750_FX_Income", "CREDIT")
    return FXPostingDecision(invoice_id, fx_delta, False, "751_FX_Expense", "DEBIT")

def calculate_unrealized_fx_deltas(duckdb: "DuckDBManager", month_end: date) -> list[tuple[Any, ...]]:
    return duckdb.execute(
        """
        WITH open_fx AS (
            SELECT id, type AS invoice_type, currency_code, amount_gross AS amount_foreign, exchange_rate_at_issue AS exchange_rate_at_issue
            FROM invoices_replica
            WHERE status IN ('PARTIAL', 'UNPAID')
            AND currency_code IS NOT NULL
            AND currency_code != 'PLN'
        )
        SELECT
            o.id,
            o.invoice_type,
            o.currency_code,
            ROUND((o.amount_foreign * r.rate) - (o.amount_foreign * o.exchange_rate_at_issue), 2) AS unrealized_fx_delta
        FROM open_fx o
        JOIN fx_rates r ON r.currency_code = o.currency_code AND r.rate_date = ?
        """
        ,
        (month_end, ),
    )

===== Code/SERVICES/inventory_fifo.py =====
from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import date
from decimal import Decimal, ROUND_HALF_UP
from typing import Any

MONEY_QUANT = Decimal("0.01")
QTY_QUANT = Decimal("0.0001")

class InventoryMismatch(Exception):
    pass

class DualWriteConsistencyError(InventoryMismatch):
    pass

class InsufficientStockError(InventoryMismatch):
    pass

def _to_decimal(value: Any, quant: Decimal) -> Decimal:
    return Decimal(str(value)).quantize(quant, rounding=ROUND_HALF_UP)

@dataclass(slots=True)
class InventoryBatch:
    batch_id: str
    product_id: str
    received_date: date
    remaining_qty: Decimal
    unit_cost_net: Decimal
    source_document_id: str | None = None

@dataclass(slots=True)
class FIFOConsumptionLine:
    batch_id: str
    product_id: str
    qty_taken: Decimal
    unit_cost_net: Decimal
    line_cogs_net: Decimal

@dataclass(slots=True)
class FIFOConsumptionResult:
    product_id: str
    requested_qty: Decimal
    fulfilled_qty: Decimal
    total_cogs_net: Decimal
    lines: list[FIFOConsumptionLine]

def calculate_fifo_cogs(product_id: str, issue_qty: Decimal | float | int, open_batches: list[InventoryBatch]) -> FIFOConsumptionResult:
    """Consume inventory batches in FIFO order and return COGS breakdown.

    Raises InsufficientStockError when stock is insufficient.
    """
    qty_needed = _to_decimal(issue_qty, QTY_QUANT)
    if qty_needed <= Decimal("0"):
        raise InventoryMismatch("issue_qty must be greater than 0")

    fifo_batches = sorted(

```

```

        [b for b in open_batches if b.product_id == product_id and b.remaining_qty > Decimal("0")],
        key=lambda b: (b.received_date, b.batch_id),
    )

    remaining = qty_needed
    lines: list[FIFOConsumptionLine] = []
    total_cogs = Decimal("0.00")

    for batch in fifo_batches:
        if remaining <= Decimal("0"):
            break
        take_qty = min(batch.remaining_qty, remaining).quantize(QTY_QUANT, rounding=ROUND_HALF_UP)
        if take_qty <= Decimal("0"):
            continue

        line_cogs = (take_qty * batch.unit_cost_net).quantize(MONEY_QUANT, rounding=ROUND_HALF_UP)
        lines.append(
            FIFOConsumptionLine(
                batch_id=batch.batch_id,
                product_id=product_id,
                qty_taken=take_qty,
                unit_cost_net=batch.unit_cost_net,
                line_cogs_net=line_cogs,
            )
        )
        total_cogs += line_cogs
        remaining -= take_qty

    if remaining > Decimal("0"):
        raise InsufficientStockError(f"Insufficient inventory for product_id={product_id}; missing_qty={remaining}")

    return FIFOConsumptionResult(
        product_id=product_id,
        requested_qty=qty_needed,
        fulfilled_qty=qty_needed,
        total_cogs_net=total_cogs.quantize(MONEY_QUANT, rounding=ROUND_HALF_UP),
        lines=lines,
    )

def apply_fifo_consumption(open_batches: list[InventoryBatch], consumption: FIFOConsumptionResult) -> list[InventoryBatch]:
    """Return updated copies of batches after applying FIFO consumption lines."""

    updates = {line.batch_id: line.qty_taken for line in consumption.lines}
    out: list[InventoryBatch] = []
    for batch in open_batches:
        delta = updates.get(batch.batch_id, Decimal("0"))
        new_qty = (batch.remaining_qty - delta).quantize(QTY_QUANT, rounding=ROUND_HALF_UP)
        if new_qty < Decimal("0"):
            raise InventoryMismatch(f"Batch {batch.batch_id} would go negative")
        out.append(
            InventoryBatch(
                batch_id=batch.batch_id,
                product_id=batch.product_id,
                received_date=batch.received_date,
                remaining_qty=new_qty,
                unit_cost_net=batch.unit_cost_net,
                source_document_id=batch.source_document_id,
            )
        )
    return out

async def calculate_and_post_cogs(
    *,

```

```

product_id: str,
qty_sold: Decimal | float | int,
open_batches: list[InventoryBatch],
tb_client: Any,
debit_account_731_cogs: int,
credit_account_330_inventory: int,
source_document_id: Any,
duckdb_writer: Any | None = None,
) -> FIFOConsumptionResult:
    consumption = calculate_fifo_cogs(product_id, qty_sold, open_batches)
    amount_minor = int((consumption.total_cogs_net * Decimal("100")).to_integral_value(rounding=ROUND_HALF_UP))
    pending = await tb_client.create_two_phase_transfer(
        debit_account=debit_account_731_cogs,
        credit_account=credit_account_330_inventory,
        amount_minor=amount_minor,
        source_document_id=source_document_id,
    )
    posted = await tb_client.post_pending_transfer(pending.pending_id)
    if not posted:
        raise InventoryMismatch("Failed to post COGS transfer to TigerBeetle")

    if duckdb_writer is not None:
        try:
            if hasattr(duckdb_writer, "begin"):
                duckdb_writer.begin()
            if hasattr(duckdb_writer, "record_cogs_consumption"):
                duckdb_writer.record_cogs_consumption(consumption)
            if hasattr(duckdb_writer, "commit"):
                duckdb_writer.commit()
        except Exception as exc:
            if hasattr(duckdb_writer, "rollback"):
                duckdb_writer.rollback()
            raise DualWriteConsistencyError("DuckDB write failed after TigerBeetle posting") from exc
    return consumption

```

==== Code/SERVICES/ksef_service.py ====

```

import httpx

```

```

class KsefService:
    """Obsługa Krajowego Systemu e-Faktur (API Ministerstwa Finansów)."""

    def __init__(self, is_production: bool = False):
        self.base_url = "https://ksef.mf.gov.pl/api" if is_production else "https://ksef-test.mf.gov.pl/api"
        self.session_token: str | None = None

    async def _init_session(self, nip: str, authorization_token: str) -> bool:
        """
        Krok 1: Inicjalizacja sesji z KSeF (Authorisation Challenge).
        Wymaga podpisania wyzwania tokenem wygenerowanym w aplikacji KSeF.
        """
        # API KSeF wymaga tu zastosowania kryptografii (szyfrowanie kluczem publicznym MF).
        try:
            async with httpx.AsyncClient() as client:
                pass
                # payload = {...}
                # response = await client.post(f"{self.base_url}/online/Session/InitToken", json=payload)
        except Exception:
            pass
        return False

```

==== Code/SERVICES/liquidity_oracle.py ====

```

from __future__ import annotations

from dataclasses import dataclass

```

```

from datetime import date, timedelta
from decimal import Decimal
from typing import TYPE_CHECKING

from Roboton_Reflekton.ledger_client import TigerBeetleClient

if TYPE_CHECKING:
    from db.analytics import DuckDBManager

@dataclass(frozen=True)
class LiquidityPoint:
    date: str
    optimistic_balance: str
    likely_balance: str
    pessimistic_balance: str

def ensure_liquidity_schema(duckdb: "DuckDBManager") -> None:
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS recurring_commitments (
            id UUID PRIMARY KEY,
            commitment_name VARCHAR NOT NULL,
            amount DECIMAL(18,2) NOT NULL,
            currency VARCHAR NOT NULL DEFAULT 'PLN',
            day_of_month INTEGER NOT NULL,
            is_active BOOLEAN NOT NULL DEFAULT TRUE
        )
        """
    )

    duckdb.execute(
        """
        CREATE OR REPLACE VIEW v_payor_reliability_stats AS
        WITH ranked AS (
            SELECT
                contractor_nip AS vendor_id,
                date_diff('day', due_date, payment_date) AS delta_days,
                ROW_NUMBER() OVER (PARTITION BY contractor_nip ORDER BY payment_date DESC) AS rn
            FROM invoices_replica
            WHERE status = 'PAID'
            AND payment_date IS NOT NULL
            AND due_date IS NOT NULL
        ),
        limited AS (
            SELECT * FROM ranked WHERE rn <= 10
        ),
        weighted AS (
            SELECT
                vendor_id,
                delta_days,
                CASE
                    WHEN rn <= 3 THEN 0.20
                    WHEN rn <= 6 THEN 0.12
                    ELSE 0.08
                END AS weight
            FROM limited
        )
        SELECT
            vendor_id,
            SUM(delta_days * weight) / NULLIF(SUM(weight), 0) AS weighted_average_delay,
            stddev_samp(delta_days) AS payment_volatility,
            CAST(GREATEST(0, LEAST(100, 100 - (AVG(ABS(delta_days)) * 8))) AS INTEGER) AS reliability_score
        FROM weighted
    """
    )

```

```

        GROUP BY vendor_id
        """
    )

def _vat_buffer_today(duckdb: "DuckDBManager") -> Decimal:
    rows = duckdb.execute(
        """
        SELECT
            COALESCE(SUM(CASE WHEN type = 'SALE' THEN vat_amount ELSE 0 END), 0)
            - COALESCE(SUM(CASE WHEN type = 'PURCHASE' THEN vat_amount ELSE 0 END), 0) AS vat_liability
        FROM invoices_replica
        WHERE date_trunc('month', issue_date) = date_trunc('month', current_date)
        """
    )
    return Decimal(str(rows[0][0] if rows else 0)).quantize(Decimal("0.01"))

def calculate_liquidity_timeline(
    duckdb: "DuckDBManager",
    tigerbeetle: TigerBeetleClient,
    *,
    account_bank_id: int,
    days_ahead: int = 90,
) -> list[LiquidityPoint]:
    """Forecast timeline with optimistic/likely/pessimistic balances."""

    cleared = Decimal(tigerbeetle._account_credits_posted.get(account_bank_id, 0)) / Decimal(100)
    pending = Decimal(
        sum(t.amount_minor for t in tigerbeetle._pending_transfers.values() if t.credit_account == account_bank_id)
    ) / Decimal(100)
    start_balance = (cleared + pending).quantize(Decimal("0.01"))

    inflows = duckdb.execute(
        """
        SELECT
            COALESCE(i.contractor_nip, 'UNKNOWN') AS vendor_id,
            i.amount_gross,
            i.due_date,
            COALESCE(s.weighted_average_delay, 0) AS delay,
            COALESCE(s.payment_volatility, 0) AS volatility,
            COALESCE(s.reliability_score, 50) AS reliability
        FROM invoices_replica i
        LEFT JOIN v_payor_reliability_stats s ON s.vendor_id = i.contractor_nip
        WHERE i.status = 'UNPAID' AND i.type = 'SALE' AND i.due_date IS NOT NULL
        """
    )

    outflows = duckdb.execute(
        """
        SELECT amount_gross, due_date
        FROM invoices_replica
        WHERE status IN ('UNPAID', 'PARTIAL') AND type = 'PURCHASE' AND due_date IS NOT NULL
        """
    )

    recurring = duckdb.execute(
        """
        SELECT amount, day_of_month
        FROM recurring_commitments
        WHERE is_active = TRUE
        """
    )

    vat_buffer = _vat_buffer_today(duckdb)

```

```

today = date.today()

opt = start_balance
likely = start_balance
pess = start_balance
timeline: list[LiquidityPoint] = []

for step in range(days_ahead + 1):
    d = today + timedelta(days=step)

    for amount, due_date in outflows:
        if due_date == d:
            amt = Decimal(str(amount))
            opt -= amt
            likely -= amt
            pess -= amt

    for amount, day_of_month in recurring:
        if int(day_of_month) == d.day:
            amt = Decimal(str(amount))
            opt -= amt
            likely -= amt
            pess -= amt

    for vendor_id, amount, due_date, delay, volatility, reliability in inflows:
        amt = Decimal(str(amount))
        adjusted = due_date + timedelta(days=int(float(delay)))
        if d == adjusted:
            opt += amt
            likely_day = adjusted + timedelta(days=int(float(volatility)))
            if d == likely_day:
                likely += amt
            pess_day = adjusted + timedelta(days=int(float(volatility) * 2))
            if d == pess_day:
                mult = Decimal("0.7") if int(reliability) < 50 else Decimal("1.0")
                pess += (amt * mult).quantize(Decimal("0.01"))

    next_month_25 = (today.replace(day=1) + timedelta(days=32)).replace(day=25)
    if d == next_month_25:
        opt -= vat_buffer
        likely -= vat_buffer
        pess -= vat_buffer

    timeline.append(
        LiquidityPoint(
            date=d.isoformat(),
            optimistic_balance=str(opt.quantize(Decimal("0.01"))),
            likely_balance=str(likely.quantize(Decimal("0.01"))),
            pessimistic_balance=str(pess.quantize(Decimal("0.01"))),
        )
    )

return timeline

```

===== Code/SERVICES/mail_fetcher.py =====

```

import imaplib
from pathlib import Path

```

```

class MailIngestionService:

```

```

    """Automatyczne pobieranie faktur PDF ze zdefiniowanej skrzynki mailowej (IMAP)."""

```

```

    def __init__(self, imap_server: str, email_user: str, email_pass: str, download_dir: str):
        self.imap_server = imap_server
        self.email_user = email_user

```

```

self.email_pass = email_pass
self.download_dir = Path(download_dir)
self.download_dir.mkdir(parents=True, exist_ok=True)

def fetch_unread_pdfs(self) -> list[str]:
    """Pobiera nieprzeczytane maile, szuka PDF-w, zapisuje na dysk i zwraca ?cie?ki."""
    downloaded_files = []
    try:
        # ??czymy si? z serwerem
        mail = imaplib.IMAP4_SSL(self.imap_server)
        mail.login(self.email_user, self.email_pass)
        mail.select("inbox")

        # Szukamy nieprzeczytanych wiadomo?ci...
    except Exception:
        pass
    return downloaded_files

===== Code/SERVICES/replication.py =====
from db.analytics import DuckDBManager
from models.invoice import Invoice

class ReplicationBridge:
    def __init__(self, duckdb_manager: DuckDBManager):
        self.olap = duckdb_manager

    def sync_invoice(self, invoice: Invoice):
        """Replikuje faktur? z SQLite do analitycznego DuckDB."""
        query = """
        INSERT OR REPLACE INTO invoices_replica (
            id, number, contractor_nip, amount_net,
            amount_gross, currency, status, updated_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """
        params = (
            str(invoice.id),
            invoice.number or "BRAK",
            invoice.contractor_nip,
            float(invoice.amount_net),
            float(invoice.amount_gross),
            invoice.currency,
            invoice.status,
            invoice.updated_at or invoice.created_at
        )
        self.olap.execute(query, params)

===== Code/SERVICES/rmk_engine.py =====
from __future__ import annotations

import calendar
import uuid
from dataclasses import dataclass
from datetime import date, timedelta
from decimal import Decimal, ROUND_HALF_UP

from db.analytics import DuckDBManager

RMK_ASSET_ACCOUNT_ID = "640"
RMK_LEDGER_ID = 1
RMK_TRANSFER_CODE = 2001

@dataclass(slots=True)
class RMKInvoiceData:

```

```

invoice_id: str
description: str
total_net_amount: Decimal
start_date: date
end_date: date
cost_account_id: str

def _month_end(year: int, month: int) -> date:
    return date(year, month, calendar.monthrange(year, month)[1])

def _first_of_next_month(input_date: date) -> date:
    if input_date.month == 12:
        return date(input_date.year + 1, 1, 1)
    return date(input_date.year, input_date.month + 1, 1)

class RMKEngine:
    """Accruals & Deferrals generator (RMK) with day-level pro-rata precision."""

    def __init__(self, duckdb: DuckDBManager) -> None:
        self.duckdb = duckdb

    def create_rmk_schedule(self, invoice_data: RMKInvoiceData) -> str:
        if invoice_data.end_date < invoice_data.start_date:
            raise ValueError("end_date must be >= start_date")

        deferred_id = str(uuid.uuid4())
        total_days = (invoice_data.end_date - invoice_data.start_date).days + 1
        daily_rate = (invoice_data.total_net_amount / Decimal(total_days)).quantize(Decimal("0.0001"), rounding=ROUND_HALF_UP)

        self.duckdb.execute(
            """
            INSERT INTO deferred_expenses (
                id, invoice_id, description, total_net_amount, start_date, end_date,
                total_days, daily_rate, status, cost_account_id
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'ACTIVE', ?)
            """,
            (
                deferred_id,
                invoice_data.invoice_id,
                invoice_data.description,
                float(invoice_data.total_net_amount),
                invoice_data.start_date,
                invoice_data.end_date,
                total_days,
                float(daily_rate),
                invoice_data.cost_account_id,
            ),
        )

        month_cursor = date(invoice_data.start_date.year, invoice_data.start_date.month, 1)
        last_month_start = date(invoice_data.end_date.year, invoice_data.end_date.month, 1)
        allocations: list[tuple[date, Decimal]] = []

        while month_cursor <= last_month_start:
            month_start = month_cursor
            month_finish = _month_end(month_start.year, month_start.month)
            period_start = max(invoice_data.start_date, month_start)
            period_end = min(invoice_data.end_date, month_finish)

            covered_days = (period_end - period_start).days + 1
            month_amount = (daily_rate * Decimal(covered_days)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
            allocations.append((month_finish, month_amount))

```

```

        month_cursor = _first_of_next_month(month_cursor)

    booked = sum((amount for _, amount in allocations), Decimal("0.00"))
    diff = invoice_data.total_net_amount - booked
    if allocations and diff != Decimal("0.00"):
        post_date, amount = allocations[-1]
        allocations[-1] = (post_date, (amount + diff).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))

    for posting_date, amount in allocations:
        self.duckdb.execute(
            """
            INSERT INTO rmk_ledger_entries (
                id, deferred_id, posting_date, amount, is_posted,
                ledger_id, transfer_code, rmk_asset_account_id, user_data_128
            ) VALUES (?, ?, ?, ?, FALSE, ?, ?, ?, ?)
            """,
            (
                str(uuid.uuid4()),
                deferred_id,
                posting_date,
                float(amount),
                RMK_LEDGER_ID,
                RMK_TRANSFER_CODE,
                RMK_ASSET_ACCOUNT_ID,
                deferred_id,
            ),
        )

    return deferred_id

===== Code/SERVICES/rules_engine.py =====
from __future__ import annotations

import ast
import json
from decimal import Decimal
from typing import Any

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from db.analytics import DuckDBManager
    from Roboton_Reflekton.ledger_client import TigerBeetleClient

def ensure_accounting_template_schema(duckdb: "DuckDBManager") -> None:
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS accounting_templates (
            id UUID PRIMARY KEY,
            name VARCHAR UNIQUE NOT NULL,
            required_debit_accounts JSON NOT NULL,
            required_credit_accounts JSON NOT NULL,
            validation_script VARCHAR NOT NULL,
            created_at TIMESTAMP NOT NULL DEFAULT now()
        )
        """
    )
    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_accounting_templates_name ON accounting_templates(name)")

class RulesEngine:
    """Deterministic gatekeeper for AI-proposed postings before TigerBeetle."""

    def __init__(self, duckdb: "DuckDBManager", tigerbeetle: TigerBeetleClient):

```

```

self.duckdb = duckdb
self.tigerbeetle = tigerbeetle
ensure_accounting_template_schema(self.duckdb)

@staticmethod
def _assert_auto_approved_or_block(duckdb: "DuckDBManager", invoice_id: str) -> None:
    try:
        from services.smart_approvals import assert_auto_approved_or_block # type: ignore
    except Exception: # pragma: no cover
        import importlib.util
        import sys
        from pathlib import Path

        module_path = Path(__file__).resolve().parent / "smart_approvals.py"
        spec = importlib.util.spec_from_file_location("smart_approvals", module_path)
        if spec is None or spec.loader is None:
            raise ImportError("Cannot load smart_approvals module")
        module = importlib.util.module_from_spec(spec)
        sys.modules[spec.name] = module
        spec.loader.exec_module(module)
        assert_auto_approved_or_block = module.assert_auto_approved_or_block

    assert_auto_approved_or_block(duckdb, invoice_id)

@staticmethod
def _safe_eval_validation(script: str, context: dict[str, Any]) -> bool:
    tree = ast.parse(script, mode="eval")
    banned_nodes = (ast.Import, ast.ImportFrom, ast.Call, ast.Attribute, ast.Subscript)
    for node in ast.walk(tree):
        if isinstance(node, banned_nodes):
            raise ValueError("validation_script contains forbidden operations")
    allowed_globals = {"__builtins__": {}, "Decimal": Decimal}
    return bool(eval(compile(tree, "<validation_script>", "eval"), allowed_globals, context))

def _load_template(self, template_name: str) -> tuple[list[str], list[str], str]:
    rows = self.duckdb.execute(
        """
        SELECT required_debit_accounts, required_credit_accounts, validation_script
        FROM accounting_templates
        WHERE name = ?
        LIMIT 1
        """,
        (template_name,),
    )
    if not rows:
        raise ValueError(f"Template '{template_name}' not found")

    required_debits, required_credits, validation_script = rows[0]
    debit_accounts = required_debits if isinstance(required_debits, list) else json.loads(required_debits)
    credit_accounts = required_credits if isinstance(required_credits, list) else json.loads(required_credits)
    return debit_accounts, credit_accounts, validation_script

async def validate_and_post(self, proposal_json: dict[str, Any]) -> dict[str, Any]:
    invoice_id = proposal_json.get("invoice_id")
    if invoice_id:
        self._assert_auto_approved_or_block(self.duckdb, str(invoice_id))

    template_name = proposal_json["template"]
    entries = proposal_json["entries"]

    required_debits, required_credits, validation_script = self._load_template(template_name)

    debit_accounts = [e["account"] for e in entries if e["side"] == "DEBIT"]
    credit_accounts = [e["account"] for e in entries if e["side"] == "CREDIT"]

```

```

missing_debits = [acc for acc in required_debits if acc not in debit_accounts]
missing_credits = [acc for acc in required_credits if acc not in credit_accounts]
if missing_debits or missing_credits:
    raise ValueError(f"Template mismatch: missing_debits={missing_debits}, missing_credits={missing_credits}")

total_debit = sum(Decimal(str(e["amount"])) for e in entries if e["side"] == "DEBIT")
total_credit = sum(Decimal(str(e["amount"])) for e in entries if e["side"] == "CREDIT")
if total_debit != total_credit:
    raise ValueError("Unbalanced booking: sum(DEBIT) must equal sum(CREDIT)")

context = {
    "proposal": proposal_json,
    "total_debit": total_debit,
    "total_credit": total_credit,
}
if not self._safe_eval_validation(validation_script, context):
    raise ValueError("Validation script rejected proposal")

created = []
for debit in [e for e in entries if e["side"] == "DEBIT"]:
    for credit in [e for e in entries if e["side"] == "CREDIT"]:
        amount = Decimal(str(min(debit["amount"], credit["amount"])))
        transfer = await self.tigerbeetle.create_two_phase_transfer(
            debit_account=int(debit["tb_account_id"]),
            credit_account=int(credit["tb_account_id"]),
            amount_minor=int(amount * 100),
            source_document_id=proposal_json["source_document_id"],
            user_data_128=int(proposal_json.get("user_data_128", 0)),
        )
        created.append(transfer.pending_id)

return {
    "status": "VALIDATED",
    "template": template_name,
    "pending_transfer_ids": created,
    "total_debit": str(total_debit),
    "total_credit": str(total_credit),
}

```

===== Code/SERVICES/security_service.py =====

```

from datetime import datetime, timedelta
from pathlib import Path

```

```

from sqlalchemy.ext.asyncio import AsyncSession

```

```

class SecurityService:

```

```

    """Zarządza retencją danych i bezpiecznym usuwaniem dokumentów."""

```

```

    @staticmethod

```

```

    async def cleanup_old_scans(session: AsyncSession, years: int = 5):

```

```

        """Usuwa fizyczne pliki i wpisy z bazy dla dokumentów starszych niż X lat (RODO/Podatki)."""

```

```

        limit_date = datetime.now() - timedelta(days=years * 365)

```

```

        # 1. Znajdź stare faktury

```

```

        # (Tutaj logika select i usuwania plików z dysku przed usunięciem z DB)

```

```

        pass

```

```

    @staticmethod

```

```

    def secure_delete_file(file_path: str):

```

```

        """Nadpisuje plik zerami przed usunięciem (bezpieczne niszczenie danych)."""

```

```

        path = Path(file_path)

```

```

        if path.exists():

```

```

            size = path.stat().st_size

```

```

            with open(path, "ba+", buffering=0) as f:

```

```

                f.write(b'\x00' * size)

```

```

        path.unlink()

===== Code/SERVICES/shadow_resource_correlation.py =====
from __future__ import annotations

from db.analytics import DuckDBManager

def ensure_shadow_resource_schema(duckdb: DuckDBManager) -> None:
    """Creates schema required for power-vs-presence correlation analytics."""
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS iot_power_log (
            timestamp TIMESTAMP NOT NULL,
            kwh_consumed DOUBLE NOT NULL,
            meter_id VARCHAR,
            source VARCHAR,
            created_at TIMESTAMP NOT NULL DEFAULT now()
        )
        """
    )

    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS access_control_log (
            timestamp TIMESTAMP NOT NULL,
            employee_id VARCHAR NOT NULL,
            event_type VARCHAR NOT NULL,
            source VARCHAR,
            created_at TIMESTAMP NOT NULL DEFAULT now(),
            CHECK (event_type IN ('IN', 'OUT'))
        )
        """
    )

    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_iot_power_log_timestamp ON iot_power_log(timestamp)")
    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_access_control_ts_emp ON access_control_log(timestamp,

def employee_presence_power_correlation_query(window_minutes: int = 15) -> str:
    """
    Returns DuckDB SQL that calculates Pearson correlation between employee presence and
    power spikes in fixed windows.

    The query outputs one row per employee with:
    - correlation coefficient between presence flag and kwh consumed
    - number of analyzed windows
    - spike statistics (kwh > 2.0)
    """
    return f"""
    WITH power_windows AS (
        SELECT
            time_bucket(INTERVAL '{window_minutes} minutes', timestamp) AS window_start,
            SUM(kwh_consumed) AS kwh_consumed
        FROM iot_power_log
        GROUP BY 1
    ),
    access_events AS (
        SELECT
            employee_id,
            timestamp,
            CASE WHEN event_type = 'IN' THEN 1 ELSE -1 END AS delta
        FROM access_control_log
    ),
    employee_windows AS (

```

```

SELECT
    emps.employee_id,
    p.window_start,
    COALESCE(
        SUM(e.delta) FILTER (
            WHERE e.timestamp <= p.window_start + INTERVAL '{window_minutes} minutes'
        ) OVER (
            PARTITION BY emps.employee_id
            ORDER BY e.timestamp
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ),
        0
    ) AS presence_balance,
    p.kwh_consumed
FROM power_windows p
CROSS JOIN (SELECT DISTINCT employee_id FROM access_control_log) emps
LEFT JOIN access_events e ON e.employee_id = emps.employee_id
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY emps.employee_id, p.window_start
    ORDER BY e.timestamp DESC NULLS LAST
) = 1
)
SELECT
    employee_id,
    corr(CASE WHEN presence_balance > 0 THEN 1.0 ELSE 0.0 END, kwh_consumed) AS presence_power_corr,
    COUNT(*) AS windows_count,
    AVG(kwh_consumed) AS avg_window_kwh,
    SUM(CASE WHEN kwh_consumed > 2.0 THEN 1 ELSE 0 END) AS spike_windows,
    AVG(
        CASE
            WHEN kwh_consumed > 2.0 AND presence_balance > 0 THEN 1.0
            ELSE 0.0
        )
    ) AS spike_presence_ratio
FROM employee_windows
GROUP BY 1
HAVING windows_count >= 4
ORDER BY presence_power_corr DESC NULLS LAST
"""

```

```

def detect_energy_leeches(
    duckdb: DuckDBManager,
    correlation_threshold: float = 0.85,
    spike_threshold_kwh: float = 2.0,
    window_minutes: int = 15,
) -> list[tuple]:
    """Returns employees with suspiciously high correlation to energy spikes."""
    base_query = employee_presence_power_correlation_query(window_minutes)
    return duckdb.execute(
        f"""
        SELECT *
        FROM ({base_query}) q
        WHERE q.presence_power_corr > ?
            AND q.spike_windows > 0
            AND q.avg_window_kwh >= ?
        ORDER BY q.presence_power_corr DESC
        """,
        (correlation_threshold, spike_threshold_kwh),
    )

```

```

===== Code/SERVICES/signature_validator.py =====
from typing import List

from cryptography import x509

```

```
from cryptography.hazmat.primitives.asymmetric import padding
```

```
class SignatureValidator:
```

```
    """Weryfikacja podpisw elektronicznych w oparciu o list? zaufan?."""
```

```
    def __init__(self, trusted_roots_paths: List[str]):
```

```
        # Lista zaufanych certyfikatw Root CA (np. pobranych z EUTL)
```

```
        self.trusted_roots = []
```

```
        for path in trusted_roots_paths:
```

```
            with open(path, "rb") as f:
```

```
                self.trusted_roots.append(x509.load_pem_x509_certificate(f.read()))
```

```
    def verify_certificate_chain(self, cert_to_verify: x509.Certificate) -> bool:
```

```
        """
```

```
        Sprawdza, czy certyfikat z faktury PDF zosta? wystawiony  
        przez jeden z urz?dw z listy zaufanej.  
        """
```

```
        for root in self.trusted_roots:
```

```
            try:
```

```
                # Weryfikacja podpisu certyfikatu przez Root CA
```

```
                root.public_key().verify(
```

```
                    cert_to_verify.signature,
```

```
                    cert_to_verify.tbs_certificate_bytes,
```

```
                    padding.PKCS1v15(),
```

```
                    cert_to_verify.signature_hash_algorithm
```

```
                )
```

```
                return True
```

```
            except Exception:
```

```
                continue
```

```
        return False
```

```
===== Code/SERVICES/smart_approvals.py =====
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import TYPE_CHECKING
```

```
if TYPE_CHECKING:
```

```
    from db.analytics import DuckDBManager
```

```
@dataclass(frozen=True)
```

```
class ApprovalDecision:
```

```
    invoice_id: str
```

```
    score: int
```

```
    approval_status: str
```

```
    reasons: list[str]
```

```
def ensure_smart_approval_schema(duckdb: "DuckDBManager") -> None:
```

```
    duckdb.execute("ALTER TABLE invoices_replica ADD COLUMN IF NOT EXISTS ai_confidence_score INTEGER")
```

```
    duckdb.execute("ALTER TABLE invoices_replica ADD COLUMN IF NOT EXISTS approval_status VARCHAR")
```

```
def evaluate_approval_routing(duckdb: "DuckDBManager", invoice_id: str) -> ApprovalDecision:
```

```
    rows = duckdb.execute(
```

```
        """
```

```
        SELECT id, contractor_nip, amount_gross
```

```
        FROM invoices_replica
```

```
        WHERE id = ?
```

```
        LIMIT 1
```

```
        """,
```

```
        (invoice_id,),
```

```
    )
```

```

if not rows:
    raise ValueError(f"Invoice {invoice_id} not found")

_, vendor_id, gross_amount = rows[0]
score = 100
reasons: list[str] = []

vendor_history = duckdb.execute(
    "SELECT COUNT(*) FROM invoices_replica WHERE contractor_nip = ? AND id != ?",
    (vendor_id, invoice_id),
)
historical_count = int(vendor_history[0][0]) if vendor_history else 0
if historical_count == 0:
    score -= 30
    reasons.append("NEW_VENDOR")

avg_rows = duckdb.execute(
    "SELECT AVG(amount_gross) FROM invoices_replica WHERE contractor_nip = ? AND id != ?",
    (vendor_id, invoice_id),
)
historical_avg = float(avg_rows[0][0]) if avg_rows and avg_rows[0][0] is not None else None
if historical_avg is not None and historical_avg > 0 and float(gross_amount) > historical_avg * 1.5:
    score -= 40
    reasons.append("AMOUNT_OVER_150pct_HISTORY")

sentinel_flag = duckdb.execute(
    """
    SELECT COUNT(*)
    FROM audit_log
    WHERE event_type = 'SENTINEL_FLAG'
    AND json_extract_string(data_payload, '$.invoice_id') = ?
    """,
    (invoice_id,),
)
if sentinel_flag and int(sentinel_flag[0][0]) > 0:
    score = 0
    reasons.append("SENTINEL_FLAGGED")

status = "AUTO_APPROVED" if score >= 85 else "PENDING_HUMAN"
duckdb.execute(
    "UPDATE invoices_replica SET ai_confidence_score = ?, approval_status = ? WHERE id = ?",
    (score, status, invoice_id),
)
return ApprovalDecision(invoice_id=invoice_id, score=score, approval_status=status, reasons=reasons)

def assert_auto_approved_or_block(duckdb: "DuckDBManager", invoice_id: str) -> None:
    decision = evaluate_approval_routing(duckdb, invoice_id)
    if decision.approval_status != "AUTO_APPROVED":
        raise PermissionError(
            f"Invoice {invoice_id} blocked by maker-checker routing: {decision.approval_status}, score={de
        )

===== Code/SERVICES/storage.py =====
import shutil
import uuid
from pathlib import Path

from core.config import AppConfig

class StorageService:
    def __init__(self, config: AppConfig):
        self.storage_root = config.base_dir / "app_data" / "scans"
        self._ensure_storage_exists()

```

```

def _ensure_storage_exists(self):
    self.storage_root.mkdir(parents=True, exist_ok=True)

def save_invoice_file(self, source_path: str) -> str:
    """Kopiuje plik do wewn?trznego folderu, nadaj?c mu unikaln? nazw?. Zwraca now? ?cie?k?."""
    source_file = Path(source_path)
    if not source_file.exists():
        raise FileNotFoundError(f"Nie znaleziono pliku: {source_path}")

    file_extension = source_file.suffix
    # Generujemy bezpieczn? nazw?: {uuid}_{oryginalna_nazwa} lub tylko z rozszerzeniem
    new_filename = f"{uuid.uuid4().hex}{file_extension}"
    destination_path = self.storage_root / new_filename

    # Bezpieczne kopiowanie z zachowaniem metadanych
    shutil.copy2(source_file, destination_path)
    return str(destination_path)

===== Code/SERVICES/tax_api.py =====
from datetime import date

import httpx
from core.config import AppConfig

class TaxApiService:
    def __init__(self, config: AppConfig):
        self.config = config
        self.base_url = "https://wl-api.mf.gov.pl/api/search/nip/" # Oficjalne API MF

    async def verify_nip(self, nip: str) -> dict | None:
        """Sprawdza NIP w bazie Ministerstwa Finansw (Bia?a Lista)."""
        if not nip or len(nip) != 10:
            return None

        try:
            async with httpx.AsyncClient(timeout=5.0) as client:
                # Wymagane przez MF: data sprawdzenia
                today = date.today().isoformat()
                response = await client.get(f"{self.base_url}{nip}?date={today}")

                if response.status_code == 200:
                    data = response.json()
                    return data.get("result", {}).get("subject")
        except Exception as e:
            print(f"B??d po??czenia z Bia?? List?: {e}")
            return None

        return None

===== Code/SERVICES/telemetry.py =====
from __future__ import annotations

import time
from contextvars import ContextVar
from datetime import datetime, timezone
from functools import wraps
from typing import Any, Awaitable, Callable, TypeVar

from db.analytics import DuckDBManager

try:
    import GPUUtil
except Exception: # pragma: no cover - optional dependency

```

```

GPUtil = None

F = TypeVar("F", bound=Callable[..., Awaitable[Any]])

_trace_id_ctx: ContextVar[str] = ContextVar("telemetry_trace_id", default="unknown")

def set_trace_id(trace_id: str) -> object:
    return _trace_id_ctx.set(trace_id)

def reset_trace_id(token: object) -> None:
    _trace_id_ctx.reset(token)

def get_trace_id() -> str:
    return _trace_id_ctx.get()

def _vram_usage_mb() -> float | None:
    if GPUtil is None:
        return None
    gpus = GPUtil.getGPUs()
    if not gpus:
        return None
    return float(sum(gpu.memoryUsed for gpu in gpus))

def ensure_telemetry_schema(duckdb: DuckDBManager) -> None:
    duckdb.execute(
        """
        CREATE TABLE IF NOT EXISTS telemetry (
            timestamp TIMESTAMP,
            trace_id VARCHAR,
            stage_name VARCHAR,
            duration_ms DOUBLE,
            vram_usage_mb DOUBLE
        )
        """
    )
    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_telemetry_timestamp ON telemetry(timestamp)")
    duckdb.execute("CREATE INDEX IF NOT EXISTS idx_telemetry_stage ON telemetry(stage_name)")

def track_performance(stage_name: str, *, duckdb_provider: Callable[[F], DuckDBManager]) -> Callable[[F], F]:
    """Decorator for worker stages that writes timing + VRAM telemetry to DuckDB."""

    def decorator(func: F) -> F:
        @wraps(func)
        async def wrapper(*args: Any, **kwargs: Any):
            started = time.perf_counter()
            try:
                return await func(*args, **kwargs)
            finally:
                duration_ms = (time.perf_counter() - started) * 1000.0
                duckdb = duckdb_provider()
                ensure_telemetry_schema(duckdb)
                duckdb.execute(
                    "INSERT INTO telemetry (timestamp, trace_id, stage_name, duration_ms, vram_usage_mb) VALUES (
                        datetime.now(timezone.utc),
                        get_trace_id(),
                        stage_name,
                        duration_ms,
                        _vram_usage_mb(),
                    )"
                )
        return wrapper
    return decorator

```

```

        ),
    )

    return wrapper # type: ignore[return-value]

return decorator

===== Code/SERVICES/tigerbeetle_secure.py =====
from __future__ import annotations

import json

from sqlalchemy.ext.asyncio import AsyncSession

from api.rbac import NexusRole, RoleContext
from db.models import SecurityAlert
from Roboton_Reflekton.ledger_client import TigerBeetleClient

class TigerBeetleSecurityException(PermissionError):
    pass

class SecureTigerBeetleClient:
    """RBAC-aware wrapper that blocks WORKER from posting committed transfers."""

    def __init__(self, inner: TigerBeetleClient) -> None:
        self._inner = inner

    async def create_pending_transfer(self, **kwargs):
        return await self._inner.create_two_phase_transfer(**kwargs)

    async def create_transfer(
        self,
        *,
        role_ctx: RoleContext,
        session: AsyncSession,
        post: bool,
        pending_id: int | None = None,
        **kwargs,
    ):
        if post and role_ctx.role != NexusRole.OWNER:
            await self._log_security_alert(
                session,
                actor=role_ctx.actor,
                operation="create_transfer(post=true)",
                details={"pending_id": pending_id, "kwargs": kwargs},
            )
            raise TigerBeetleSecurityException("WORKER cannot execute posted TigerBeetle transfers")

        if post:
            if pending_id is None:
                raise ValueError("pending_id is required to post a pending transfer")
            return await self._inner.post_pending_transfer(pending_id)

        return await self._inner.create_two_phase_transfer(**kwargs)

    async def _log_security_alert(self, session: AsyncSession, *, actor: str, operation: str, details: dict)
        session.add(
            SecurityAlert(
                actor=actor,
                operation=operation,
                details=json.dumps(details, default=str),
            )
        )

```

```

        await session.commit()

===== Code/SERVICES/triage_service.py =====
from __future__ import annotations

from dataclasses import dataclass
from decimal import Decimal
from typing import Any

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from models.invoice import Invoice

TRIAGE_CONFIDENCE_THRESHOLD = 0.85

@dataclass(slots=True)
class TriageDecision:
    send_to_review: bool
    reason: str | None = None

def should_triage_document(*, confidence_score: float, amount_net: Decimal, amount_vat: Decimal, amount_gross: Decimal) -> TriageDecision:
    """Core triage rule used by workers before posting accounting effects."""
    if confidence_score < TRIAGE_CONFIDENCE_THRESHOLD:
        return TriageDecision(send_to_review=True, reason="LOW_CONFIDENCE")

    if (amount_net + amount_vat).quantize(Decimal("0.01")) != amount_gross.quantize(Decimal("0.01")):
        return TriageDecision(send_to_review=True, reason="MATH_MISMATCH")

    return TriageDecision(send_to_review=False)

async def list_pending_triage_items(session: AsyncSession) -> list[Invoice]:
    stmt = select(Invoice).where(Invoice.status == "PENDING_REVIEW").order_by(Invoice.created_at.desc())
    result = await session.execute(stmt)
    return list(result.scalars().all())

async def resolve_triage_item(
    session: AsyncSession,
    *,
    invoice_id: str,
    corrected_data: dict[str, Any],
    action: str,
    updated_by: str,
) -> Invoice:
    invoice = await session.get(Invoice, invoice_id)
    if invoice is None:
        raise ValueError(f"Invoice {invoice_id} not found")

    if number := corrected_data.get("number"):
        invoice.number = str(number)
    if contractor_nip := corrected_data.get("contractor_nip"):
        invoice.contractor_nip = str(contractor_nip)
    if amount_net := corrected_data.get("amount_net"):
        invoice.amount_net = Decimal(str(amount_net))
    if amount_gross := corrected_data.get("amount_gross"):
        invoice.amount_gross = Decimal(str(amount_gross))

    if action == "confirm_post":
        invoice.status = "APPROVED"
    elif action == "void_reject":
        invoice.status = "REJECTED"

```

```

else:
    raise ValueError("Unsupported triage action")

invoice.updated_by = updated_by

await session.commit()
await session.refresh(invoice)
return invoice

===== Code/SERVICES/validation_service.py =====
from decimal import Decimal

from models.invoice import Invoice
from sqlalchemy import select, and_
from sqlalchemy.ext.asyncio import AsyncSession

class ValidationService:
    """Zaawansowana walidacja biznesowa zapobiegaj?ca duplikatom i b??dom."""

    @staticmethod
    async def is_duplicate(
        session: AsyncSession,
        nip: str,
        number: str,
        amount_gross: Decimal
    ) -> bool:
        """Sprawdza, czy w bazie istnieje ju? taka faktura dla tego dostawcy."""
        query = select(Invoice).where(
            and_(
                Invoice.contractor_nip == nip,
                Invoice.number == number,
                Invoice.amount_gross == amount_gross
            )
        )
        result = await session.execute(query)
        return result.scalar_one_or_none() is not None

    @staticmethod
    def detect_anomaly(avg_amount: float, current_amount: float) -> bool:
        """Prosta detekcja anomalii - flaga, je?li kwota znacznie odbiega od ?redniej."""
        pass

===== Code/SERVICES/vendor_intelligence.py =====
from __future__ import annotations

import asyncio
from dataclasses import dataclass
from typing import Any

from db.analytics import DuckDBManager

@dataclass(slots=True)
class VendorMetric:
    nip: str
    vendor_name: str
    avg_payment_delay: float
    price_volatility_index: float
    total_volume_ytd: float
    reliability_score: float

class VendorAnalyst:
    """Background analytical engine for local-first vendor intelligence."""

```

```

def __init__(self, duckdb: DuckDBManager, refresh_seconds: int = 3600) -> None:
    self.duckdb = duckdb
    self.refresh_seconds = refresh_seconds
    self._running = False

async def run_forever(self) -> None:
    self._running = True
    while self._running:
        await self.refresh_vendor_intelligence()
        await asyncio.sleep(self.refresh_seconds)

def stop(self) -> None:
    self._running = False

async def refresh_vendor_intelligence(self) -> None:
    await asyncio.to_thread(self._refresh_sync)

def _refresh_sync(self) -> None:
    self.duckdb.execute("""
CREATE TABLE IF NOT EXISTS vendor_intelligence (
    nip VARCHAR PRIMARY KEY,
    vendor_name VARCHAR,
    avg_payment_delay DOUBLE,
    price_volatility_index DOUBLE,
    total_volume_ytd DOUBLE,
    reliability_score DOUBLE,
    rating_stars INTEGER,
    smart_alerts VARCHAR,
    updated_at TIMESTAMP DEFAULT now()
)
""")

    self.duckdb.execute("""
CREATE TABLE IF NOT EXISTS vendor_price_alerts (
    id UUID DEFAULT uuid(),
    nip VARCHAR,
    item_name VARCHAR,
    current_price DOUBLE,
    six_month_avg DOUBLE,
    increase_ratio DOUBLE,
    alert_type VARCHAR,
    detected_at TIMESTAMP DEFAULT now()
)
""")

    self.duckdb.execute("""
INSERT OR REPLACE INTO vendor_intelligence
WITH payment_match AS (
    SELECT
        i.contractor_nip AS nip,
        COALESCE(i.contractor_name, i.supplier_name, 'Unknown') AS vendor_name,
        AVG(date_diff('day', i.due_date, bt.transfer_timestamp::DATE)) AS avg_payment_delay,
        SUM(CASE WHEN year(i.issue_date) = year(current_date) THEN i.amount_gross ELSE 0 END) AS t
    FROM invoices_replica i
    LEFT JOIN bank_transfers_cache bt
        ON i.contractor_nip = bt.counterparty_nip
        AND abs(i.amount_gross - bt.amount) < 0.01
    WHERE i.contractor_nip IS NOT NULL
    GROUP BY 1, 2
),
price_changes AS (
    SELECT
        contractor_nip AS nip,
        AVG(abs((unit_price - lag(unit_price) OVER w) / nullif(lag(unit_price) OVER w, 0))) * 100

```

```

        FROM invoice_items
        WINDOW w AS (PARTITION BY contractor_nip, lower(item_name) ORDER BY issue_date)
        GROUP BY 1
    )
    SELECT
        pm.nip,
        pm.vendor_name,
        COALESCE(pm.avg_payment_delay, 0),
        COALESCE(pc.price_volatility_index, 0),
        COALESCE(pm.total_volume_ytd, 0),
        GREATEST(0.0, LEAST(5.0, 5.0 - (COALESCE(pm.avg_payment_delay, 0) / 6.0))) AS reliability_score,
        CAST(round(GREATEST(1.0, LEAST(5.0, 5.0 - (COALESCE(pm.avg_payment_delay, 0) / 6.0)))) AS INT) AS intelligence_rating
    CASE
        WHEN COALESCE(pm.avg_payment_delay, 0) > 10 THEN 'Always pays late'
        WHEN COALESCE(pc.price_volatility_index, 0) > 8 THEN 'Rapid price increase'
        ELSE 'Stable cooperation'
    END AS smart_alerts,
    now()
FROM payment_match pm
LEFT JOIN price_changes pc ON pm.nip = pc.nip
"""
)

```

```

self.duckdb.execute("""
INSERT INTO vendor_price_alerts (nip, item_name, current_price, six_month_avg, increase_ratio, alert_type)
WITH price_baseline AS (
    SELECT
        contractor_nip AS nip,
        lower(item_name) AS normalized_item,
        max_by(item_name, issue_date) AS item_name,
        max_by(unit_price, issue_date) AS current_price,
        AVG(unit_price) FILTER (WHERE issue_date >= current_date - INTERVAL 6 MONTH) AS six_month_avg
    FROM invoice_items
    WHERE issue_date >= current_date - INTERVAL 12 MONTH
    GROUP BY 1, 2
)
SELECT
    nip,
    item_name,
    current_price,
    six_month_avg,
    ((current_price - six_month_avg) / nullif(six_month_avg, 0)) * 100,
    'PRICE_ALERT'
FROM price_baseline
WHERE six_month_avg > 0 AND current_price > six_month_avg * 1.05
""")

```

```

def get_vendor_context(self, nip: str) -> str:
    rows = self.duckdb.execute(
        """
        SELECT nip, vendor_name, avg_payment_delay, price_volatility_index,
               total_volume_ytd, reliability_score, rating_stars, smart_alerts
        FROM vendor_intelligence
        WHERE nip = ?
        """,
        (nip,),
    )
    if not rows:
        return f"No local vendor intelligence found for NIP {nip}."

    v = rows[0]
    return (
        f"Vendor {v[1]} (NIP: {v[0]}). "
        f"Average payment delay: {v[2]:.1f} days. "
        f"Price volatility index: {v[3]:.2f}%. "
        f"YTD volume: {v[4]:.2f}. "
    )

```

```

        f"Reliability score: {v[5]:.2f}/5 ({v[6]} stars). "
        f"Smart alert: {v[7]}."
    )

```

```

===== Code/SERVICES/white_list_service.py =====
from datetime import date

```

```

import httpx
from core.logger import logger

```

```

class WhiteListService:
    BASE_URL = "https://wl-api.mf.gov.pl/api/search/nip/"

    @classmethod
    async def verify_bank_account(cls, nip: str, account_to_check: str) -> bool:
        """Sprawdza czy konto bankowe jest na bia?ej li?cie MF."""
        target_date = date.today().isoformat()
        clean_account = "".join(filter(str.isdigit, account_to_check))

        async with httpx.AsyncClient() as client:
            try:
                response = await client.get(f"{cls.BASE_URL}{nip}?date={target_date}")

                if response.status_code == 200:
                    data = response.json()
                    accounts = data.get("result", {}).get("subject", {}).get("accountNumbers", [])
                    return clean_account in accounts

                return False
            except Exception as e:
                logger.error(f"B??d Bia?ej Listy: {e}")
                return False

```

```

===== Code/SKRIPTS/Braki.py =====

```

```

# worker.py
import os
import sys
import asyncio
import logging
import signal
import psutil
import gc
import torch
import platform
import shutil
from pathlib import Path
from datetime import datetime, timezone
from taskiq import TaskiqEvents

# --- 1. KONFIGURACJA ?RODOWISKA AI (Krytyczne dla trybu Offline) ---
# Ustawiamy ?cie?k? do wbudowanego folderu z modelami, aby unikn?? pobierania z sieci.
if getattr(sys, 'frozen', False):
    # Je?li aplikacja jest skompilowana jako .exe (PyInstaller)
    base_path = Path(sys._MEIPASS)
else:
    # Je?li uruchamiamy z kodu ?rd?owego
    base_path = Path(__file__).parent

# Wymuszenie ?cie?ek dla wag modeli (Surya OCR / Transformers)
models_cache_dir = base_path / "models"
os.environ["HF_HOME"] = str(models_cache_dir)
os.environ["TRANSFORMERS_OFFLINE"] = "1"
os.environ["TORCH_HOME"] = str(models_cache_dir / "torch")

```

```

# Optymalizacja CPU/RAM (Max Performance)
os.environ["OMP_NUM_THREADS"] = "4"
os.environ["OPENBLAS_NUM_THREADS"] = "4"
os.environ["MKL_NUM_THREADS"] = "4"

# --- 2. KONFIGURACJA LOGOWANIA ---
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s (PID:%(process)d): %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler("worker_error.log", encoding="utf-8")
    ]
)
logger = logging.getLogger("nexus.worker")

# --- 3. IMPORTY RDZENIA APLIKACJI ---
# Eksponujemy obiekt 'broker', aby CLI taskiq mog?o go u?y?.
from core.broker import broker
# KRYTYCZNE: Importujemy tasks, aby Taskiq zarejestrowa? funkcje z @broker.task.
import core.tasks
from core.config import AppConfig

# --- 4. ZAAWANSOWANY MONITOR I WATCHDOG (Absolutny Max) ---
class WorkerGuard:
    """Zapewnia stabilno?? workera, zarz?dza pami?ci? i raportuje status."""
    def __init__(self, ram_limit_gb: float = 6.0):
        self.process = psutil.Process(os.getpid())
        self.ram_limit = ram_limit_gb * 1024 * 1024 * 1024
        self.start_time = datetime.now(timezone.utc)

    def check_resources(self):
        """Wymusza czyszczenie pami?ci przy wyciekach z modeli OCR."""
        current_mem = self.process.memory_info().rss
        if current_mem > self.ram_limit:
            logger.warning(f"Alert Pami?ci: {current_mem/1024**2:.1f} MB. Uruchamianie GC...")
            if torch.cuda.is_available():
                torch.cuda.empty_cache()
            gc.collect()
            return True
        return False

    async def heartbeat(self):
        """P?tla raportuj?ca, ?e proces roboczy ?yje i nie 'zawis?' na modelu AI."""
        while True:
            uptime = datetime.now(timezone.utc) - self.start_time
            logger.debug(f"Heartbeat: Uptime {uptime}, RAM: {self.process.memory_info().rss/1024**2:.1f}MB")
            await asyncio.sleep(60)

# --- 5. HOOKI TASKIQ (Zarz?dzanie Cyklem ?ycia) ---
@broker.on_event(TaskiqEvents.WORKER_STARTUP)
async def startup_event(state):
    """Przygotowanie zasobw przed przyj?ciem pierwszego zadania."""
    logger.info(">>> Inicjalizacja silnika AI i monitoringu...")
    # Pre-load silnika OCR (Singleton), aby unikn?? op?nienia przy pierwszej fakturze
    from pipeline.ocr import DocumentProcessor
    state.ocr_engine = DocumentProcessor()
    state.config = AppConfig()
    state.guard = WorkerGuard(ram_limit_gb=8.0)
    state.heartbeat_task = asyncio.create_task(state.guard.heartbeat())

    gpu_available = torch.cuda.is_available()
    logger.info(f">>> Worker gotowy. OS: {platform.system()}, Urz?dzenie: {'CUDA / GPU' if gpu_available else 'CPU'}")

```

```

@broker.on_event(TaskiqEvents.TASK_POST_EXECUTION)
async def post_task(state, task_result):
    """Zwolnienie zasobow natychmiast po przetworzeniu faktury."""
    state.guard.check_resources()
    logger.debug(f"Zakończono zadanie {task_result.task_id}. Pamięć sprawdzona.")

@broker.on_event(TaskiqEvents.WORKER_SHUTDOWN)
async def shutdown_event(state):
    """Kulturalne zamknięcie po zakończeniu i zwolnienie GPU."""
    logger.info(">>> Zamykanie workera. Czyszczenie pamięci...")
    if hasattr(state, 'heartbeat_task'):
        state.heartbeat_task.cancel()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    gc.collect()

# --- 6. OBSŁUGA SYGNAŁU I PORZĄDKU ---
shutdown_event = asyncio.Event()

def handle_shutdown(sig, frame):
    """Przechwytywanie sygnału zamknięcia (Ctrl+C), aby bezpiecznie zakończyć przetwarzanie."""
    logger.info(f"Otrzymano sygnał przerwania ({sig}). Trwa bezpieczne zamykanie workera...")
    shutdown_event.set()

def cleanup_temp_artifacts():
    """Usuwanie pozostałości po renderowaniu stron PDF do obrazów."""
    temp_dir = Path(os.environ.get("TEMP", "/tmp")) / "nexus_ocr"
    if temp_dir.exists():
        shutil.rmtree(temp_dir)
        logger.info("Wyczyszczono pliki tymczasowe OCR.")

# --- 7. LOGIKA KOLEJEK I PRZETWARZANIA (NATS JetStream) ---
async def process_messages(nats_client):
    """Konfiguracja konsumenta JetStream z limitem powtórzeń."""
    js = await nats_client.jetstream()
    sub = await js.subscribe(
        "invoices.new",
        durable="invoice_worker",
        manual_ack=True,
        config=nats.js.api.ConsumerConfig(
            max_deliver=5, # Po 5 nieudanych próbach NATS przestanie wysyłać to zadanie
            ack_wait=60    # Czekaj 60s na potwierdzenie
        )
    )

    async for msg in sub.messages:
        try:
            # Logika przetwarzania faktury (funkcja handle_invoice musi być zdefiniowana w systemie)
            # await handle_invoice(msg.data)
            await msg.ack()
        except Exception as e:
            delivery_count = msg.metadata.num_delivered
            if delivery_count >= 5:
                logger.critical(f"Plik uszkodził workera 5 razy. Przenoszenie do DLQ: {msg.data}")
                # Tutaj następuje zapisanie błędu do DB
                # await mark_as_dead_letter(msg.data, str(e))
                await msg.term()
            else:
                await msg.nak()

# --- 8. GŁÓWNY PUNKT WEJŚCIA ---
async def main():
    """Funkcja do ręcznego uruchamiania i testowania programu."""
    # Konfiguracja priorytetów procesu
    try:

```

```

        p = psutil.Process(os.getpid())
        if platform.system() == "Windows":
            p.nice(psutil.HIGH_PRIORITY_CLASS)
        else:
            p.nice(-10) # Wyższy priorytet na Linux
    except Exception:
        pass

# Rejestracja sygnałów (Unix/Mac)
if platform.system() != "Windows":
    for s in (signal.SIGINT, signal.SIGTERM):
        try:
            signal.signal(s, handle_shutdown)
        except Exception:
            pass

logger.info("Nawiązanie połączenia z brokerem NATS (localhost:4222)...")
try:
    await broker.startup()
    logger.info("Worker jest aktywny. Nasłuchiwanie na zadania w kolejce NATS...")

    # Uruchomienie przekaźnika Outbox jeśli wymagane
    # relay_task = asyncio.create_task(run_outbox_relay(SessionLocal, nats_client))

    # Czekamy na sygnał zamknięcia
    await shutdown_event.wait()
except Exception as e:
    logger.error(f"Błąd krytyczny połączenia: {e}")
finally:
    logger.info("Odłączanie od brokera i czyszczenie pamięci...")
    await broker.shutdown()
    logger.info("Worker zamknięty pomyślnie.")

if __name__ == "__main__":
    # Rozwiązanie problemu z pętlą zdarzeń na Windows (Taskiq/NATS)
    if platform.system() == "Windows":
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

    cleanup_temp_artifacts()
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        pass

===== Code/SKRIPTS/backup.py =====
# scripts/backup.py
import os
import shutil
import zipfile
from datetime import datetime
from pathlib import Path
from core.config import AppConfig

def create_backup():
    config = AppConfig()
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_name = f"nexus_backup_{timestamp}.zip"
    backup_path = config.base_dir / "backups"
    backup_path.mkdir(exist_ok=True)

    target_zip = backup_path / backup_name
    print(f"Tworzenie kopii zapasowej: {backup_name}...")

    with zipfile.ZipFile(target_zip, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # Pakujemy bazy danych

```

```

        if config.sqlite_path.exists():
            zipf.write(config.sqlite_path, config.sqlite_path.name)
        if config.duckdb_path.exists():
            zipf.write(config.duckdb_path, config.duckdb_path.name)

        # Pakujemy skany (app_data)
        data_dir = config.base_dir / "app_data"
        for root, _, files in os.walk(data_dir):
            for file in files:
                file_path = Path(root) / file
                archive_name = file_path.relative_to(config.base_dir)
                zipf.write(file_path, archive_name)

    print(f"? Kopia zapasowa gotowa: {target_zip}")

if __name__ == "__main__":
    create_backup()

===== Code/SKRIPTS/benchmark_ai.py =====
# scripts/benchmark_ai.py
import time
import torch
from pathlib import Path
from pipeline.ocr import DocumentProcessor

def run_benchmark():
    print("=== NEXUS AI PERFORMANCE TEST ===")
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Testowanie na urz?dzeniu: {device}")

    # Inicjalizacja modeli
    start_init = time.perf_counter()
    processor = DocumentProcessor()
    print(f" Czas ?adowania modeli: {time.perf_counter() - start_init:.2f}s")

    # Symulacja przetwarzania (wymaga przyk?adowego pliku test.pdf w root)
    test_file = Path("test.pdf")
    if not test_file.exists():
        print(" Brak pliku test.pdf do przeprowadzenia testu.")
        return

    print(" Rozpaczynam OCR testowej faktury...")
    start_ocr = time.perf_counter()
    result = processor.process(test_file)
    end_ocr = time.perf_counter()

    print(f"\n Wyniki benchmarku:")
    print(f" - Czas pe?nego OCR: {end_ocr - start_ocr:.2f}s")
    if hasattr(result, 'text_lines'):
        print(f" - Wykryte linie tekstu: {len(result.text_lines)}")

if __name__ == "__main__":
    run_benchmark()

===== Code/SKRIPTS/doctor.py =====
# scripts/doctor.py
import socket
import torch
import platform
import psutil
from pathlib import Path

def run_diagnostics():
    print("=== NEXUS SYSTEM DOCTOR ===\n")

```

```

# 1. Sprawdzenie Hardware AI
gpu = torch.cuda.is_available()
print(f"[AI] Urz?dzenie: {'GPU (CUDA)' if gpu else 'CPU (Slow Mode)'}")
if gpu:
    print(f"[AI] Model GPU: {torch.cuda.get_device_name(0)}")

# 2. Sprawdzenie NATS
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    s.bind(("127.0.0.1", 4222))
    print("[NATS] Port 4222 jest wolny (OK)")
except socket.error:
    print("[NATS] Port 4222 ZABLOKOWANY! Zamknij inne instancje NATS.")
finally:
    s.close()

# 3. Sprawdzenie dost?pno?ci modeli
model_path = Path("models/hub/models--vikp--surya_det3")
if model_path.exists():
    print("[MODELS] Modele OCR wydaj? si? by? pobrane (OK)")
else:
    print("[MODELS] BRAK MODELI! Uruchom download_models.py")

# 4. Statystyki RAM
ram = psutil.virtual_memory()
print(f"[SYSTEM] Dost?pny RAM: {ram.available / 1024**3:.2f} GB / {ram.total / 1024**3:.2f} GB")

if __name__ == "__main__":
    run_diagnostics()

===== Code/SKRIPTS/download_models.py =====
# scripts/download_models.py
import os
from pathlib import Path
from huggingface_hub import snapshot_download

def download_all_models():
    # 1. Definiujemy ?cie?k? do lokalnego folderu 'models' wewn?trz projektu
    # ?cie?ka wzgl?dem tego skryptu: ../models
    project_root = Path(__file__).parent.parent
    models_dir = project_root / "models"
    models_dir.mkdir(parents=True, exist_ok=True)

    # 2. Ustawiamy zmienn? ?rodowiskow?, ktra zmusza HuggingFace
    # do zapisywania plikw w naszym folderze, a nie w C:\Users\...\cache
    os.environ["HF_HOME"] = str(models_dir)

    print(f"[Downloader] Rozpoczynam pobieranie modeli do: {models_dir}")
    print("Mo?e to potrwa? kilka minut w zale?no?ci od ??cza internetowego...\n")

    # Lista wszystkich modeli potrzebnych do trybu "Absolutny Max"
    repos = [
        "sentence-transformers/all-MiniLM-L6-v2", # Active Learning / Search
        "vikp/surya_det3", # OCR: Detekcja linii tekstu
        "vikp/surya_rec3", # OCR: Rozpoznawanie tekstu
        "vikp/surya_layout3", # OCR: Struktura tabel
        "vikp/surya_order3" # OCR: Kolejno?? czytania
    ]

    for repo in repos:
        print(f"\n>>> Pobieranie: {repo}...")
        try:
            snapshot_download(
                repo_id=repo,
                cache_dir=models_dir,

```

```

        local_files_only=False
    )
    except Exception as e:
        print(f"B??d przy {repo}: {e}")

    print("\n[Downloader] Zakończono sukcesem! Modele są gotowe do pracy w trybie offline.")

if __name__ == "__main__":
    download_all_models()

===== Code/SKRIPTS/init.py =====
# scripts/__init__.py
from .download_models import download_all_models
from .setup_env import bootstrap_system
from .doctor import run_diagnostics
from .reset_db import hard_reset
from .backup import create_backup
from .benchmark_ai import run_benchmark

__all__ = [
    "download_all_models",
    "bootstrap_system",
    "run_diagnostics",
    "hard_reset",
    "create_backup",
    "run_benchmark",
    "LogManager",
    "bulk_import_folder",
    "generate_mock_data",
    "production_readiness_check"
]

===== Code/SKRIPTS/migrate_v1_to_v2.py =====
# scripts/migrate_v1_to_v2.py
import sqlite3
from core.config import AppConfig

def migrate():
    config = AppConfig()
    db_path = config.sqlite_path

    print(f" Sprawdzanie aktualizacji schematu bazy: {db_path}")
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    try:
        # Przykład: dodanie nowej kolumny
        cursor.execute("ALTER TABLE invoices ADD COLUMN internal_notes TEXT")
        conn.commit()
        print(" Dodano kolumnę 'internal_notes'.")
    except sqlite3.OperationalError:
        print(" Schemat jest już aktualny.")
    finally:
        conn.close()

if __name__ == "__main__":
    migrate()

===== Code/SKRIPTS/reset_db.py =====
# scripts/reset_db.py
import os
import shutil
from pathlib import Path
from core.config import AppConfig

```

```

def hard_reset():
    config = AppConfig()

    print("?? OSTRZEŻENIE: Czy?cis? ca?? baz? danych i wszystkie skany!")
    confirm = input("Czy na pewno chcesz kontynuowa?? (t/N): ")

    if confirm.lower() != 't':
        print("Operacja anulowana.")
        return

    # 1. Usuwanie baz danych
    files_to_remove = [config.sqlite_path, config.duckdb_path]
    for file in files_to_remove:
        if file.exists():
            file.unlink()
            print(f" Usuni?to baz?: {file.name}")

    # 2. Usuwanie plikw (skany i eksporty)
    data_dir = config.base_dir / "app_data"
    if data_dir.exists():
        shutil.rmtree(data_dir)
        data_dir.mkdir()
        (data_dir / "scans").mkdir()
        (data_dir / "exports").mkdir()
        print(" Wyczyszczono folder app_data.")

    print("? System zosta? zresetowany do stanu fabrycznego.")

if __name__ == "__main__":
    hard_reset()

===== Code/SKRIPTS/setup_env.py =====
# scripts/setup_env.py
import os
import asyncio
from pathlib import Path
from core.config import AppConfig
from db.database import engine, init_schema

async def bootstrap_system():
    """Przygotowuje ?rodowisko do pierwszego uruchomienia."""
    config = AppConfig()

    print(">>> Inicjalizacja struktury folderw...")
    folders = [
        config.base_dir / "app_data" / "scans",
        config.base_dir / "app_data" / "exports",
        config.base_dir / "logs",
        config.base_dir / "models"
    ]

    for folder in folders:
        folder.mkdir(parents=True, exist_ok=True)
        print(f" Utworzono: {folder}")

    print("\n>>> Inicjalizacja schematu bazy danych (SQLite)...")
    try:
        await init_schema(engine)
        print(" Baza danych zosta?a pomy?lnie zainicjowana.")
    except Exception as e:
        print(f" B??d bazy danych: {e}")

if __name__ == "__main__":
    asyncio.run(bootstrap_system())

```

```

===== Roboton_Reflekton/__init__.py =====
"""Roboton Reflekton application package."""

__all__ = ["app"]

===== Roboton_Reflekton/api.py =====
from __future__ import annotations

import uuid
from dataclasses import dataclass

from litestar import Litestar, get, post
from litestar.di import Provide
from litestar.response import ServerSentEvent
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from .decision_trees import CompanyDecisionTree
from .ledger_client import TigerBeetleClient
from .ledger_initializer import LedgerInitializer
from .models import CompanyProfile, LegalForm, TaxForm, TaxPolicy
from .reconciliation_engine import AlertHub, ReconciliationEngine

engine = create_async_engine("postgresql+asyncpg://postgres:postgres@localhost:5432/nexus", echo=False)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)
tb_client = TigerBeetleClient()
alert_hub = AlertHub()
reconciliation_engine = ReconciliationEngine(session_factory=SessionLocal, tb_client=tb_client, alert_hub=

async def provide_session() -> AsyncSession:
    async with SessionLocal() as session:
        yield session

@dataclass
class CompanyCreateRequest:
    name: str
    nip: str
    legal_form: LegalForm
    tax_form: TaxForm
    ksef_token: str
    vat_proportion: float = 1.0

@dataclass
class ApproveTransferRequest:
    pending_id: int

@post("/company/create")
async def create_company(data: CompanyCreateRequest, session: AsyncSession) -> dict:
    if not data.ksef_token.strip():
        raise ValueError("Token KSeF jest wymagany.")

    tree = CompanyDecisionTree(
        legal_form=data.legal_form,
        tax_form=data.tax_form,
        ksef_active=True,
        vat_proportion=data.vat_proportion,
    )
    decision_tree, policy = tree.validate_and_build()

    ledger = await LedgerInitializer(tb_client=tb_client).configure_ledger(
        legal_form=data.legal_form,
        tax_form=data.tax_form,

```

```

)

company = CompanyProfile(
    name=data.name,
    nip=data.nip,
    legal_form=data.legal_form,
    ksef_active=True,
    ksef_token=data.ksef_token,
    vat_proportion=data.vat_proportion,
    tigerbeetle_ledger_map=ledger,
    company_policy={"decision_tree": decision_tree, "tax_policy": policy},
)
session.add(company)
await session.flush()

tax_policy = TaxPolicy(
    company_id=company.id,
    tax_form=data.tax_form,
    pit_costs_enabled=bool(policy.get("pit_costs_enabled", True)),
    requires_full_ledger=bool(policy.get("requires_full_ledger", False)),
)
session.add(tax_policy)
await session.commit()

return {"company_id": str(company.id), "ledger_accounts": len(ledger)}

@post("/ledger/approve-transfer")
async def approve_transfer(data: ApproveTransferRequest) -> dict:
    approved = await tb_client.post_pending_transfer(data.pending_id)
    return {"pending_id": data.pending_id, "approved": approved}

@get("/alerts/missing-invoice/stream")
async def missing_invoice_alert_stream() -> ServerSentEvent:
    async def stream() -> object:
        async for event in alert_hub.subscribe():
            yield event

    return ServerSentEvent(stream())

async def _on_startup() -> None:
    await reconciliation_engine.start()

async def _on_shutdown() -> None:
    await reconciliation_engine.stop()

app = Litestar(
    route_handlers=[create_company, approve_transfer, missing_invoice_alert_stream],
    dependencies={"session": Provide(provide_session)},
    on_startup=[_on_startup],
    on_shutdown=[_on_shutdown],
)

===== Roboton_Reflekton/audit_storno.py =====
from __future__ import annotations

import uuid
from dataclasses import dataclass
from decimal import Decimal, ROUND_HALF_UP
from typing import Any, Protocol

```

```

class DuckDBWriter(Protocol):
    def begin(self) -> None: ...
    def commit(self) -> None: ...
    def rollback(self) -> None: ...
    def mark_invoice_voided(self, invoice_id: str) -> None: ...
    def create_draft_from_invoice(self, invoice_id: str) -> str: ...

@dataclass(slots=True, frozen=True)
class LedgerTransferRecord:
    transfer_id: int
    debit_account: int
    credit_account: int
    amount_minor: int
    currency: str
    source_document_id: uuid.UUID

class StornoException(Exception):
    pass

async def reverse_transaction(
    *,
    tb_client: Any,
    duckdb_writer: DuckDBWriter,
    original_transfer: LedgerTransferRecord,
    invoice_id: str,
) -> dict[str, Any]:
    if original_transfer.amount_minor <= 0:
        raise StornoException("Original transfer amount must be positive")

    reverse_source_id = uuid.uuid5(uuid.NAMESPACE_URL, f"storno:{original_transfer.transfer_id}:{invoice_id}")
    pending = await tb_client.create_two_phase_transfer(
        debit_account=original_transfer.credit_account,
        credit_account=original_transfer.debit_account,
        amount_minor=int(original_transfer.amount_minor),
        source_document_id=reverse_source_id,
    )
    posted = await tb_client.post_pending_transfer(pending.pending_id)
    if not posted:
        raise StornoException("TigerBeetle storno posting failed")

    # TigerBeetle success first; only then OLAP metadata mutation.
    try:
        duckdb_writer.begin()
        duckdb_writer.mark_invoice_voided(invoice_id)
        draft_id = duckdb_writer.create_draft_from_invoice(invoice_id)
        duckdb_writer.commit()
    except Exception as exc:
        duckdb_writer.rollback()
        raise StornoException("DuckDB storno mutation failed after TigerBeetle posting") from exc

    return {
        "status": "reversed",
        "original_transfer_id": original_transfer.transfer_id,
        "reverse_pending_id": pending.pending_id,
        "user_data_128": str(original_transfer.transfer_id),
        "new_draft_id": draft_id,
    }

def decimal_to_minor_units(amount: Decimal, scale: int = 2) -> int:
    quant = Decimal("1").scaleb(-scale)

```

```

        normalized = amount.quantize(quant, rounding=ROUND_HALF_UP)
        factor = Decimal(10) ** scale
        return int((normalized * factor).to_integral_value(rounding=ROUND_HALF_UP))

===== Roboton_Reflekton/budgetary_control.py =====
from __future__ import annotations

from dataclasses import dataclass
from datetime import date
from typing import Any

from .ledger_client import TigerBeetleClient

@dataclass(frozen=True, slots=True)
class BudgetStatus:
    status: str
    message: str
    account_code: str
    month_period: date
    limit_amount: float
    current_amount: float
    projected_amount: float
    current_usage_percent: float
    projected_usage_percent: float

class BudgetaryControlEngine:
    """Real-time budget checks using DuckDB budgets + TigerBeetle balances."""

    def __init__(self, duckdb_manager: Any, tb_client: TigerBeetleClient, account_map: dict[str, int]) -> None:
        self.duckdb = duckdb_manager
        self.tb_client = tb_client
        self.account_map = account_map

    def ensure_budget_schema(self) -> None:
        self.duckdb.execute(
            """
            CREATE TABLE IF NOT EXISTS budget_definitions (
                account_code VARCHAR,
                month_period DATE,
                limit_amount DOUBLE,
                alert_at_percent DOUBLE DEFAULT 0.85,
                PRIMARY KEY (account_code, month_period)
            )
            """
        )

    async def get_budget_status(self, account_code: str, new_invoice_amount: float, month_period: date | None) -> BudgetStatus:
        if new_invoice_amount < 0:
            raise ValueError("new_invoice_amount must be >= 0")
        self.ensure_budget_schema()

        period = month_period or date.today().replace(day=1)
        budget_rows = self.duckdb.execute(
            """
            SELECT limit_amount, alert_at_percent
            FROM budget_definitions
            WHERE account_code = ? AND month_period = ?
            """
        ).fetchall()
        (account_code, period),
        )
        if not budget_rows:
            return BudgetStatus(
                status="OK",

```

```

        message=f"No budget configured for account {account_code} in {period}.",
        account_code=account_code,
        month_period=period,
        limit_amount=0.0,
        current_amount=0.0,
        projected_amount=new_invoice_amount,
        current_usage_percent=0.0,
        projected_usage_percent=0.0,
    )

    limit_amount, alert_at_percent = float(budget_rows[0][0]), float(budget_rows[0][1])
    if limit_amount <= 0:
        raise ValueError(f"Budget limit must be > 0 for {account_code} in {period}")

    account_id = self.account_map.get(account_code)
    if account_id is None:
        raise ValueError(f"No TigerBeetle account mapping for account_code={account_code!r}")

    current_minor = await self.tb_client.get_account_credits_posted(account_id)
    current_amount = float(current_minor) / 100.0
    projected_amount = current_amount + float(new_invoice_amount)

    current_usage_percent = (current_amount / limit_amount) * 100.0
    projected_usage_percent = (projected_amount / limit_amount) * 100.0

    if projected_usage_percent >= 100.0:
        over_amount = projected_amount - limit_amount
        status = "CRITICAL"
        message = (
            f"Budget exceeded for {account_code}: +{over_amount:.2f} PLN over limit "
            f"({projected_usage_percent:.1f}% of plan)."
        )
    elif projected_usage_percent >= alert_at_percent * 100.0:
        status = "WARN"
        message = f"Budget warning for {account_code}: projected usage {projected_usage_percent:.1f}%"
    else:
        status = "OK"
        message = f"Budget healthy for {account_code}: projected usage {projected_usage_percent:.1f}%"

    return BudgetStatus(
        status=status,
        message=message,
        account_code=account_code,
        month_period=period,
        limit_amount=limit_amount,
        current_amount=current_amount,
        projected_amount=projected_amount,
        current_usage_percent=current_usage_percent,
        projected_usage_percent=projected_usage_percent,
    )

```

==== Roboton_Reflekton/decision_trees.py =====

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from .models import LegalForm, TaxForm
```

```
from .strategies import InvalidTaxPolicyError, StrategyContext, StrategyRegistry
```

```
@dataclass(slots=True)
```

```
class DecisionNode:
```

```
    key: str
```

```
    value: str
```

```
    children: list["DecisionNode"] = field(default_factory=list)
```

```

def add_child(self, node: "DecisionNode") -> None:
    self.children.append(node)

def to_dict(self) -> dict:
    return {
        "key": self.key,
        "value": self.value,
        "children": [child.to_dict() for child in self.children],
    }

class CompanyDecisionTree:
    def __init__(self, legal_form: LegalForm, tax_form: TaxForm, *, ksef_active: bool, vat_proportion: float):
        self.legal_form = legal_form
        self.tax_form = tax_form
        self.context = StrategyContext(legal_form=legal_form, ksef_active=ksef_active, vat_proportion=vat_proportion)
        self.registry = StrategyRegistry()

    def validate_and_build(self) -> tuple[dict, dict]:
        strategy = self.registry.select(self.tax_form)
        policy = strategy.policy_payload(self.context)
        if not self.context.ksef_active:
            raise InvalidTaxPolicyError("KSeF musi być aktywny dla procesu konfiguracji.")

        root = DecisionNode("legal_form", self.legal_form)
        root.add_child(DecisionNode("tax_form", self.tax_form))
        root.add_child(DecisionNode("ksef_active", str(self.context.ksef_active).lower()))
        root.add_child(DecisionNode("vat_proportion", str(self.context.vat_proportion)))
        return root.to_dict(), policy

===== Roboton_Reflekton/dunning_engine.py =====
from __future__ import annotations

import uuid
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any, Protocol

class DunningAIAgent(Protocol):
    def generate_dunning_text(self, invoice_data: dict[str, Any], vendor_score: float, level: int) -> str: ...

class DunningEmailProvider(Protocol):
    def send(self, *, to_email: str, subject: str, body: str) -> bool: ...

@dataclass(frozen=True, slots=True)
class DunningGuardrails:
    cooldown_days: int = 7
    min_amount_pln: float = 10.0

class DunningEngine:
    def __init__(self, duckdb_manager: Any, ai_agent: DunningAIAgent, email_provider: DunningEmailProvider):
        self.duckdb = duckdb_manager
        self.ai_agent = ai_agent
        self.email_provider = email_provider
        self.guardrails = guardrails

    def ensure_schema(self) -> None:
        self.duckdb.execute(
            """
            CREATE TABLE IF NOT EXISTS dunning_policy (

```

```

        id UUID,
        level INTEGER,
        days_after_due INTEGER,
        template_id VARCHAR,
        channel VARCHAR
    )
    """
)
self.duckdb.execute(
    """
    CREATE TABLE IF NOT EXISTS dunning_history (
        id UUID,
        invoice_id UUID,
        sent_at TIMESTAMP,
        level_reached INTEGER,
        status VARCHAR,
        message_content TEXT
    )
    """
)

def ensure_collectible_view(self) -> None:
    self.duckdb.execute(
        """
        CREATE OR REPLACE VIEW v_collectible_invoices AS
        WITH last_dunning AS (
            SELECT invoice_id, MAX(sent_at) AS last_dunning_date
            FROM dunning_history
            GROUP BY 1
        )
        SELECT
            i.id,
            i.number,
            i.contractor_nip,
            i.amount_gross,
            i.currency,
            i.due_date,
            i.status,
            i.customer_email,
            i.vendor_score,
            COALESCE(i.balance_due, i.amount_gross) AS balance_due,
            ld.last_dunning_date,
            date_diff('day', i.due_date, current_date) AS days_overdue
        FROM invoices_replica i
        LEFT JOIN last_dunning ld ON ld.invoice_id = i.id
        WHERE COALESCE(i.balance_due, i.amount_gross) >= {self.guardrails.min_amount_pln}
            AND COALESCE(i.balance_due, i.amount_gross) > 0
            AND i.due_date < current_date
            AND (
                ld.last_dunning_date IS NULL
                OR ld.last_dunning_date < current_date - INTERVAL {self.guardrails.cooldown_days} DAY
            )
        """
    )

@staticmethod
def determine_level(days_overdue: int) -> int | None:
    if days_overdue >= 30:
        return 3
    if days_overdue >= 10:
        return 2
    if days_overdue >= 3:
        return 1
    return None

```

```

async def run_daily_dunning_check(self) -> dict[str, int]:
    self.ensure_schema()
    self.ensure_collectible_view()

    rows = self.duckdb.execute(
        """
        SELECT id, number, contractor_nip, balance_due, customer_email, vendor_score, days_overdue
        FROM v_collectible_invoices
        """
    )

    sent = 0
    failed = 0
    for invoice_id, number, contractor_nip, balance_due, customer_email, vendor_score, days_overdue in rows:
        level = self.determine_level(int(days_overdue))
        if level is None:
            continue

        payload = {
            "invoice_id": str(invoice_id),
            "invoice_number": number,
            "contractor_nip": contractor_nip,
            "balance_due": float(balance_due),
            "days_overdue": int(days_overdue),
        }
        content = self.ai_agent.generate_dunning_text(payload, float(vendor_score or 0.5), level)
        delivered = self.email_provider.send(
            to_email=str(customer_email or ""),
            subject=f"Przypomnienie o p?atno?ci FV {number}",
            body=content,
        )
        status = "SENT" if delivered else "FAILED"
        sent += 1 if delivered else 0
        failed += 0 if delivered else 1
        self.duckdb.execute(
            """
            INSERT INTO dunning_history (id, invoice_id, sent_at, level_reached, status, message_content)
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            (
                str(uuid.uuid4()),
                str(invoice_id),
                datetime.now(timezone.utc),
                level,
                status,
                content,
            ),
        )

    return {"checked": len(rows), "sent": sent, "failed": failed}

```

```

===== Roboton_Reflekton/forex_engine.py =====
from __future__ import annotations

```

```

from dataclasses import dataclass
import uuid
from datetime import date, timedelta
from decimal import Decimal, ROUND_HALF_UP
import json
from urllib import request, error
from typing import Any

from .ledger_client import TigerBeetleClient

```

```
@dataclass(frozen=True, slots=True)
```

```
class FXResult:
```

```
    invoice_id: str
    payment_id: str
    currency_code: str
    amount_foreign: Decimal
    historical_rate: Decimal
    settlement_rate: Decimal
    fx_diff_pln: Decimal
    direction: str
```

```
class ForexEngine:
```

```
    def __init__(self, duckdb_manager: Any, tb_client: TigerBeetleClient, account_receivable: int, account_payable: int):
        self.duckdb = duckdb_manager
        self.tb_client = tb_client
        self.account_receivable = account_receivable
        self.account_fx_gain = account_fx_gain
        self.account_fx_loss = account_fx_loss
```

```
    def ensure_exchange_rate_schema(self) -> None:
```

```
        self.duckdb.execute(
            """
            CREATE TABLE IF NOT EXISTS exchange_rates (
                currency_code VARCHAR,
                rate_date DATE,
                avg_rate DOUBLE,
                table_no VARCHAR,
                PRIMARY KEY (currency_code, rate_date)
            )
            """
        )
```

```
    def fetch_nbp_rate(self, target_date: date, currency: str, max_lookback_days: int = 5) -> Decimal:
```

```
        self.ensure_exchange_rate_schema()
        currency_code = currency.upper()

        for offset in range(max_lookback_days + 1):
            rate_day = target_date - timedelta(days=offset)
            cached = self.duckdb.execute(
                "SELECT avg_rate FROM exchange_rates WHERE currency_code = ? AND rate_date = ?",
                (currency_code, rate_day),
            )
            if cached:
                return Decimal(str(cached[0][0]))

            url = f"https://api.nbp.pl/api/exchangerates/rates/A/{currency_code}/{rate_day.isoformat()}?format=json"
            try:
                with request.urlopen(url, timeout=10) as response:
                    payload = json.loads(response.read().decode("utf-8"))
            except (error.HTTPError, error.URLError, TimeoutError):
                payload = None

            if payload is not None:
                avg_rate = Decimal(str(payload["rates"][0]["mid"]))
                table_no = str(payload["rates"][0]["no"])
                self.duckdb.execute(
                    """
                    INSERT OR REPLACE INTO exchange_rates(currency_code, rate_date, avg_rate, table_no)
                    VALUES (?, ?, ?, ?)
                    """
                    ,
                    (currency_code, rate_day, float(avg_rate), table_no),
                )
                return avg_rate
```

```

        raise ValueError(f"NBP rate not found for {currency_code} within {max_lookback_days} days before {
async def process_fx_settlement(self, invoice_uuid: str, payment_uuid: str) -> FXResult:
    inv_rows = self.duckdb.execute(
        """
        SELECT currency_code, amount_foreign, historical_rate
        FROM invoices_fx
        WHERE invoice_id = ?
        """,
        (invoice_uuid,),
    )
    pay_rows = self.duckdb.execute(
        """
        SELECT settlement_rate
        FROM bank_transactions_fx
        WHERE payment_id = ?
        """,
        (payment_uuid,),
    )
    if not inv_rows or not pay_rows:
        raise ValueError("Missing invoice or payment FX data")

    currency_code, amount_foreign_raw, historical_rate_raw = inv_rows[0]
    settlement_rate_raw = pay_rows[0][0]

    amount_foreign = Decimal(str(amount_foreign_raw))
    historical_rate = Decimal(str(historical_rate_raw))
    settlement_rate = Decimal(str(settlement_rate_raw))

    fx_diff = (amount_foreign * settlement_rate) - (amount_foreign * historical_rate)
    fx_diff = fx_diff.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    minor = int((abs(fx_diff) * 100).to_integral_value(rounding=ROUND_HALF_UP))

    direction = "NONE"
    if minor > 0:
        if fx_diff > 0:
            direction = "GAIN"
            pending = await self.tb_client.create_two_phase_transfer(
                debit_account=self.account_receivable,
                credit_account=self.account_fx_gain,
                amount_minor=minor,
                source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"fx:{invoice_uuid}:{payment_uuid}")
            )
        else:
            direction = "LOSS"
            pending = await self.tb_client.create_two_phase_transfer(
                debit_account=self.account_fx_loss,
                credit_account=self.account_receivable,
                amount_minor=minor,
                source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"fx:{invoice_uuid}:{payment_uuid}")
            )
        await self.tb_client.post_pending_transfer(pending.pending_id)

    return FXResult(
        invoice_id=invoice_uuid,
        payment_id=payment_uuid,
        currency_code=str(currency_code),
        amount_foreign=amount_foreign,
        historical_rate=historical_rate,
        settlement_rate=settlement_rate,
        fx_diff_pln=fx_diff,
        direction=direction,
    )

```

===== Roboton_Reflekton/ledger_client.py =====

```
from __future__ import annotations
```

```
import hashlib
import os
import uuid
from dataclasses import dataclass
```

```
@dataclass(slots=True)
class TwoPhaseTransfer:
    pending_id: int
    debit_account: int
    credit_account: int
    amount_minor: int
    source_document_id: uuid.UUID
    user_data_128: int = 0
```

```
class TigerBeetleMapper:
    """Converts Polish account symbols (e.g. 401-02) into deterministic uint128-like ints."""
```

```
    @staticmethod
    def account_to_uint128(account_symbol: str) -> int:
        digest = hashlib.blake2b(account_symbol.encode("utf-8"), digest_size=16).digest()
        return int.from_bytes(digest, byteorder="big", signed=False)
```

```
    def build_map(self, accounts: list[str]) -> dict[str, int]:
        return {acc: self.account_to_uint128(acc) for acc in accounts}
```

```
class TigerBeetleClient:
    """Wrapper for TigerBeetle operations (interface-ready, safe stub for local dev)."""
```

```
    def __init__(self, cluster_id: int | None = None, replica_addresses: list[str] | None = None) -> None:
        self.cluster_id = cluster_id or int(os.getenv("TB_CLUSTER_ID", "0"))
        self.replica_addresses = replica_addresses or os.getenv("TB_REPLICA_ADDRESSES", "3000").split(",")
        self._pending_transfers: dict[int, TwoPhaseTransfer] = {}
        self._account_credits_posted: dict[int, int] = {}
```

```
    async def create_accounts(self, account_ids: list[int]) -> dict[str, int]:
        created = len(set(account_ids))
        return {"created": created, "cluster_id": self.cluster_id}
```

```
    async def create_two_phase_transfer(
        self,
        *,
        debit_account: int,
        credit_account: int,
        amount_minor: int,
        source_document_id: uuid.UUID,
        user_data_128: int = 0,
    ) -> TwoPhaseTransfer:
        pending_id = uuid.uuid4().int >> 64
        transfer = TwoPhaseTransfer(
            pending_id=pending_id,
            debit_account=debit_account,
            credit_account=credit_account,
            amount_minor=amount_minor,
            source_document_id=source_document_id,
            user_data_128=user_data_128,
        )
        self._pending_transfers[pending_id] = transfer
        return transfer
```

```
    async def post_pending_transfer(self, pending_id: int) -> bool:
```

```

        transfer = self._pending_transfers.pop(pending_id, None)
        if transfer is None:
            return False
        self._account_credits_posted[transfer.credit_account] = (
            self._account_credits_posted.get(transfer.credit_account, 0) + transfer.amount_minor
        )
        return True

    async def get_account_credits_posted(self, account_id: int) -> int:
        return self._account_credits_posted.get(account_id, 0)

===== Roboton_Reflekton/ledger_initializer.py =====
from __future__ import annotations

from .ledger_client import TigerBeetleClient, TigerBeetleMapper
from .models import LegalForm, TaxForm

class LedgerInitializer:
    def __init__(self, tb_client: TigerBeetleClient | None = None, mapper: TigerBeetleMapper | None = None):
        self.tb_client = tb_client or TigerBeetleClient()
        self.mapper = mapper or TigerBeetleMapper()

    def _chart_of_accounts(self, *, legal_form: LegalForm, tax_form: TaxForm) -> list[str]:
        if legal_form in {LegalForm.SP_ZOO, LegalForm.PSA}:
            base = ["100", "130", "201", "202", "221", "225", "401-01", "490", "700"]
            if tax_form is TaxForm.CIT_ESTONIAN:
                base.extend(["820", "821"])
            return base
        if tax_form is TaxForm.LUMP_SUM:
            return ["100", "130", "700", "720", "221"]
        return ["100", "130", "201", "221", "401-01", "700"]

    async def configure_ledger(self, *, legal_form: LegalForm, tax_form: TaxForm) -> dict[str, int]:
        accounts = self._chart_of_accounts(legal_form=legal_form, tax_form=tax_form)
        mapped = self.mapper.build_map(accounts)
        await self.tb_client.create_accounts(list(mapped.values()))
        return mapped

===== Roboton_Reflekton/models.py =====
from __future__ import annotations

import uuid
from datetime import datetime
from decimal import Decimal
from enum import Enum

class StrEnum(Enum):
    pass

from sqlalchemy import JSON, Boolean, DateTime, Enum, ForeignKey, Numeric, String, UniqueConstraint
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class LegalForm(StrEnum):
    JDG = "jdg"
    CIVIL_PARTNERSHIP = "civil_partnership"
    SP_ZOO = "sp_zoo"
    PSA = "psa"

```

```
class TaxForm(StrEnum):
    LUMP_SUM = "lump_sum"
    SCALE = "scale"
    LINEAR = "linear"
    CIT_STANDARD = "cit_standard"
    CIT_ESTONIAN = "cit_estonian"
```

```
class TransferStatus(StrEnum):
    PENDING = "pending"
    POSTED = "posted"
    REJECTED = "rejected"
```

```
class FinancialPeriodStatus(StrEnum):
    OPEN = "open"
    SOFT_CLOSED = "soft_closed"
    HARD_CLOSED = "hard_closed"
```

```
class CompanyProfile(Base):
    __tablename__ = "company_profiles"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    nip: Mapped[str] = mapped_column(String(10), unique=True, nullable=False)
    legal_form: Mapped[LegalForm] = mapped_column(Enum(LegalForm, name="legal_form_enum"), nullable=False)
    ksef_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    ksef_token: Mapped[str | None] = mapped_column(String(512), nullable=True)
    vat_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    vat_proportion: Mapped[Decimal] = mapped_column(Numeric(5, 4), default=Decimal("1.0"), nullable=False)
    tigerbeetle_ledger_map: Mapped[dict[str, int]] = mapped_column(JSONB, default=dict, nullable=False)
    company_policy: Mapped[dict] = mapped_column(JSON, default=dict, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=datetime.utcnow, nullable=False)

    partners: Mapped[list[CompanyPartner]] = relationship(back_populates="company", cascade="all, delete-orphan")
    tax_policy: Mapped[TaxPolicy | None] = relationship(back_populates="company", uselist=False, cascade="all, delete-orphan")
```

```
class CompanyPartner(Base):
    __tablename__ = "company_partners"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    company_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("company_profiles.id", ondelete="CASCADE"), nullable=False)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    tax_id: Mapped[str] = mapped_column(String(10), nullable=False)
    share_ratio: Mapped[Decimal] = mapped_column(Numeric(5, 4), nullable=False)

    company: Mapped[CompanyProfile] = relationship(back_populates="partners")
```

```
class TaxPolicy(Base):
    __tablename__ = "tax_policies"
    __table_args__ = (UniqueConstraint("company_id", name="uq_tax_policy_company"),)

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    company_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("company_profiles.id", ondelete="CASCADE"), nullable=False)
    tax_form: Mapped[TaxForm] = mapped_column(Enum(TaxForm, name="tax_form_enum"), nullable=False)
    pit_costs_enabled: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    requires_full_ledger: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    vat_settlement_cycle: Mapped[str] = mapped_column(String(32), default="monthly", nullable=False)
    effective_from: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=datetime.utcnow, nullable=False)
```

```

company: Mapped[CompanyProfile] = relationship(back_populates="tax_policy")

class LedgerTransfer(Base):
    __tablename__ = "ledger_transfers"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    company_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("company_profiles.id", ondelete="CASCADE"), nullable=False)
    source_account: Mapped[int] = mapped_column(nullable=False)
    target_account: Mapped[int] = mapped_column(nullable=False)
    amount_minor: Mapped[int] = mapped_column(nullable=False)
    currency: Mapped[str] = mapped_column(String(3), default="PLN", nullable=False)
    source_document_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), nullable=False)
    status: Mapped[TransferStatus] = mapped_column(Enum(TransferStatus, name="transfer_status_enum"), default=TransferStatus.PENDING)
    meta: Mapped[dict] = mapped_column(JSON, default=dict, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=datetime.utcnow, nullable=False)

class FinancialPeriod(Base):
    __tablename__ = "financial_periods"

    period_id: Mapped[str] = mapped_column(String(7), primary_key=True)
    company_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("company_profiles.id", ondelete="CASCADE"), nullable=False)
    status: Mapped[FinancialPeriodStatus] = mapped_column(
        Enum(FinancialPeriodStatus, name="financial_period_status_enum"),
        default=FinancialPeriodStatus.OPEN,
        nullable=False,
    )
    closed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
    vat_declaration_id: Mapped[str | None] = mapped_column(String(128), nullable=True)

===== Roboton_Reflekton/reconciliation_engine.py =====
from __future__ import annotations

import asyncio
import json
import uuid
from collections.abc import AsyncIterator
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from typing import Any

import nats
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from .ledger_client import TigerBeetleClient
from .models import CompanyProfile, LedgerTransfer, TransferStatus

@dataclass(slots=True)
class MissingInvoiceAlert:
    company_id: uuid.UUID
    contractor_nip: str
    amount_minor: int
    transaction_id: str
    bank_posted_at: datetime
    reason: str = "No pending transfer match after 15 days"

    def to_sse_event(self) -> dict[str, Any]:
        return {
            "type": "MissingInvoiceAlert",
            "company_id": str(self.company_id),
            "contractor_nip": self.contractor_nip,
            "amount_minor": self.amount_minor,

```

```

        "transaction_id": self.transaction_id,
        "bank_posted_at": self.bank_posted_at.isoformat(),
        "reason": self.reason,
    }

```

@dataclass

```

class AlertHub:
    _subscribers: set[asyncio.Queue[dict[str, Any]]] = field(default_factory=set)

```

```

    async def publish(self, event: dict[str, Any]) -> None:
        for queue in list(self._subscribers):
            await queue.put(event)

```

```

    async def subscribe(self) -> AsyncIterator[dict[str, Any]]:
        queue: asyncio.Queue[dict[str, Any]] = asyncio.Queue()
        self._subscribers.add(queue)
        try:
            while True:
                yield await queue.get()
        finally:
            self._subscribers.discard(queue)

```

class ReconciliationEngine:

```

    def __init__(
        self,
        session_factory: async_sessionmaker[AsyncSession],
        tb_client: TigerBeetleClient,
        alert_hub: AlertHub,
        nats_url: str = "nats://127.0.0.1:4222",
        subject: str = "bank.transactions.raw",
    ) -> None:
        self.session_factory = session_factory
        self.tb_client = tb_client
        self.alert_hub = alert_hub
        self.nats_url = nats_url
        self.subject = subject
        self._nc = None

```

```

    async def start(self) -> None:
        self._nc = await nats.connect(self.nats_url)
        await self._nc.subscribe(self.subject, cb=self._on_nats_message)

```

```

    async def stop(self) -> None:
        if self._nc is not None:
            await self._nc.close()
            self._nc = None

```

```

    async def _on_nats_message(self, msg: Any) -> None:
        payload = json.loads(msg.data.decode("utf-8"))
        await self.process_bank_transaction(payload)

```

```

    async def process_bank_transaction(self, payload: dict[str, Any]) -> bool:
        company_id = uuid.UUID(payload["company_id"])
        amount_minor = int(payload["amount_minor"])
        contractor_nip = str(payload["contractor_nip"])
        posted_at = datetime.fromisoformat(payload["posted_at"])
        transaction_id = str(payload.get("transaction_id", ""))

```

```

    async with self.session_factory() as session:
        company = await session.scalar(select(CompanyProfile).where(CompanyProfile.id == company_id))
        if company is None:
            return False

```

```

pending_transfers = (
    (
        await session.execute(
            select(LedgerTransfer).where(
                LedgerTransfer.company_id == company_id,
                LedgerTransfer.status == TransferStatus.PENDING,
                LedgerTransfer.amount_minor == amount_minor,
            )
        )
    )
    .scalars()
    .all()
)

matched_transfer: LedgerTransfer | None = None
for transfer in pending_transfers:
    transfer_nip = str(transfer.meta.get("classification", {}).get("contractor_nip", ""))
    if transfer_nip == contractor_nip:
        credits_posted = await self.tb_client.get_account_credits_posted(transfer.target_account_id)
        if credits_posted >= amount_minor:
            matched_transfer = transfer
            break

if matched_transfer is not None:
    pending_id = int(matched_transfer.meta.get("tb_pending_id"))
    approved = await self.tb_client.post_pending_transfer(pending_id)
    if approved:
        matched_transfer.status = TransferStatus.POSTED
        matched_transfer.meta = {
            **matched_transfer.meta,
            "reconciliation": {
                "status": "auto-confirmed",
                "bank_transaction_id": transaction_id,
                "matched_at": datetime.now(timezone.utc).isoformat(),
            },
        }
        await session.commit()
    return approved

if posted_at <= datetime.now(posted_at.tzinfo or timezone.utc) - timedelta(days=15):
    alert = MissingInvoiceAlert(
        company_id=company_id,
        contractor_nip=contractor_nip,
        amount_minor=amount_minor,
        transaction_id=transaction_id,
        bank_posted_at=posted_at,
    )
    await self.alert_hub.publish(alert.to_sse_event())
return False

```

```

@dataclass(frozen=True, slots=True)

```

```

class ClearingAccountsConfig:
    account_bank_main: int
    account_expense_fees: int
    account_receivable: int
    provider_clearing_accounts: dict[str, int]

```

```

class ClearingAccountsEngine:

```

```

    """Implements provider fee extraction and bank payout reconciliation with idempotency."""

```

```

    def __init__(self, tb_client: TigerBeetleClient, config: ClearingAccountsConfig) -> None:
        self.tb_client = tb_client
        self.config = config

```

```

self._processed_operations: set[str] = set()

def _provider_account(self, provider_id: str) -> int:
    key = provider_id.strip().lower()
    account = self.config.provider_clearing_accounts.get(key)
    if account is None:
        raise ValueError(f"Unknown provider_id={provider_id!r}")
    return account

async def process_provider_fees(self, *, provider_id: str, fee_amount: int, operation_id: str) -> dict:
    """Step 2: Debit fees expense / Credit provider clearing account."""
    if fee_amount <= 0:
        raise ValueError("fee_amount must be > 0")
    if operation_id in self._processed_operations:
        return {"status": "idempotent-replay", "operation_id": operation_id}

    provider_account = self._provider_account(provider_id)
    pending = await self.tb_client.create_two_phase_transfer(
        debit_account=self.config.account_expense_fees,
        credit_account=provider_account,
        amount_minor=fee_amount,
        source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"fees:{provider_id}:{operation_id}"),
    )
    posted = await self.tb_client.post_pending_transfer(pending.pending_id)
    if posted:
        self._processed_operations.add(operation_id)
    return {
        "status": "posted" if posted else "failed",
        "operation_id": operation_id,
        "pending_id": pending.pending_id,
    }

async def reconcile_bank_payout(self, *, provider_id: str, payout_amount: int, operation_id: str) -> dict:
    """Step 3: Debit main bank / Credit provider clearing account."""
    if payout_amount <= 0:
        raise ValueError("payout_amount must be > 0")
    if operation_id in self._processed_operations:
        return {"status": "idempotent-replay", "operation_id": operation_id}

    provider_account = self._provider_account(provider_id)
    pending = await self.tb_client.create_two_phase_transfer(
        debit_account=self.config.account_bank_main,
        credit_account=provider_account,
        amount_minor=payout_amount,
        source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"payout:{provider_id}:{operation_id}"),
    )
    posted = await self.tb_client.post_pending_transfer(pending.pending_id)
    if posted:
        self._processed_operations.add(operation_id)
    clearing_balance = await self.tb_client.get_account_credits_posted(provider_account)
    return {
        "status": "posted" if posted else "failed",
        "operation_id": operation_id,
        "pending_id": pending.pending_id,
        "clearing_credits_posted": clearing_balance,
    }

```

```

@dataclass(frozen=True, slots=True)
class OpenInvoice:
    invoice_id: str
    amount_due_minor: int
    due_date: datetime

```

```

@dataclass(frozen=True, slots=True)
class BankReconciliationConfig:
    account_bank_main: int
    account_receivable: int
    account_rounding_differences: int
    rounding_threshold_minor: int = 10

class BankReconciliationEngine:
    def __init__(self, tb_client: TigerBeetleClient, config: BankReconciliationConfig) -> None:
        self.tb_client = tb_client
        self.config = config

    async def reconcile_bulk_payment(
        self,
        *,
        vendor_id: str,
        payment_amount_minor: int,
        received_date: datetime,
        open_invoices: list[OpenInvoice],
    ) -> dict[str, Any]:
        if payment_amount_minor <= 0:
            raise ValueError("payment_amount_minor must be > 0")

        remaining = payment_amount_minor
        allocations: list[dict[str, Any]] = []
        sorted_invoices = sorted(open_invoices, key=lambda item: item.due_date)

        for invoice in sorted_invoices:
            if remaining <= 0:
                break
            allocated = min(invoice.amount_due_minor, remaining)
            if allocated <= 0:
                continue
            pending = await self.tb_client.create_two_phase_transfer(
                debit_account=self.config.account_bank_main,
                credit_account=self.config.account_receivable,
                amount_minor=allocated,
                source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"bulk:{vendor_id}:{invoice.invoice_id}"),
            )
            posted = await self.tb_client.post_pending_transfer(pending.pending_id)
            allocations.append({"invoice_id": invoice.invoice_id, "amount_minor": allocated, "posted": posted})
            remaining -= allocated

        rounding_adjustment_posted = False
        if 0 < remaining < self.config.rounding_threshold_minor:
            rounding_pending = await self.tb_client.create_two_phase_transfer(
                debit_account=self.config.account_bank_main,
                credit_account=self.config.account_rounding_differences,
                amount_minor=remaining,
                source_document_id=uuid.uuid5(uuid.NAMESPACE_URL, f"rounding:{vendor_id}:{received_date.isoformat()}"),
            )
            rounding_adjustment_posted = await self.tb_client.post_pending_transfer(rounding_pending.pending_id)
            remaining = 0

        return {
            "vendor_id": vendor_id,
            "received_date": received_date.isoformat(),
            "payment_amount_minor": payment_amount_minor,
            "allocations": allocations,
            "rounding_adjustment_posted": rounding_adjustment_posted,
            "remaining_unallocated_minor": remaining,
        }

```

==== Roboton_Reflekton/roboton_worker.py =====

```

from __future__ import annotations

import uuid
from dataclasses import dataclass
from datetime import date
from typing import Protocol

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from .ledger_client import TigerBeetleClient
from .models import CompanyProfile, FinancialPeriod, FinancialPeriodStatus, LedgerTransfer, TransferStatus

class PeriodLockedException(Exception):
    pass

class TaxClassifierAgent(Protocol):
    async def classify(self, payload: dict, company_policy: dict) -> dict: ...

@dataclass(slots=True)
class SimpleRuleBasedAgent:
    async def classify(self, payload: dict, company_policy: dict) -> dict:
        amount_minor = int(payload.get("amount_minor", 0))
        mixed_vehicle = bool(payload.get("mixed_vehicle_use", False))
        kup_ratio = 0.75 if mixed_vehicle else 1.0
        return {
            "debit_symbol": payload.get("debit_symbol", "401-01"),
            "credit_symbol": payload.get("credit_symbol", "202"),
            "amount_minor": amount_minor,
            "kup_ratio": kup_ratio,
            "reasoning": "rule-based fallback for CrewAI",
            "source_document_id": payload.get("source_document_id"),
            "contractor_nip": payload.get("contractor_nip", ""),
        }

class RobotonWorker:
    def __init__(self, session: AsyncSession, tb_client: TigerBeetleClient, agent: TaxClassifierAgent) ->
        self.session = session
        self.tb_client = tb_client
        self.agent = agent

    async def _apply_financial_period_lock(self, company_id: uuid.UUID, event: dict) -> dict:
        issue_date = date.fromisoformat(str(event["date_of_issue"]))
        tax_point_date = date.fromisoformat(str(event.get("tax_point_date", event["date_of_issue"])))
        period_id = issue_date.strftime("%Y-%m")

        period = await self.session.scalar(
            select(FinancialPeriod).where(FinancialPeriod.company_id == company_id, FinancialPeriod.period_id == period_id)
        )
        if period is None or period.status != FinancialPeriodStatus.HARD_CLOSED:
            event.setdefault("posting_date", issue_date.isoformat())
            event.setdefault("tax_point_date", tax_point_date.isoformat())
            return event

        open_period = await self.session.scalar(
            select(FinancialPeriod)
            .where(FinancialPeriod.company_id == company_id, FinancialPeriod.status == FinancialPeriodStatus.OPEN)
            .order_by(FinancialPeriod.period_id.asc())
        )
        if open_period is None:
            raise PeriodLockedException("No OPEN financial period available for HARD_CLOSED shift")

```

```

shifted_date = date.fromisoformat(f"{open_period.period_id}-01")
event["posting_date"] = shifted_date.isoformat()
event["tax_point_date"] = shifted_date.isoformat()
metadata = dict(event.get("metadata", {}))
metadata["late_submission_shifted"] = True
metadata["original_period_id"] = period_id
metadata["shifted_to_period_id"] = open_period.period_id
event["metadata"] = metadata
return event

```

```

async def process_invoice_extracted(self, event: dict) -> LedgerTransfer:
    company_id = uuid.UUID(event["company_id"])
    event = await self._apply_financial_period_lock(company_id, event)
    company = await self.session.scalar(select(CompanyProfile).where(CompanyProfile.id == company_id))
    if company is None:
        raise ValueError("Nie znalezione CompanyProfile dla eventu.")

    classification = await self.agent.classify(event, company.company_policy)
    ledger_map = company.tigerbeetle_ledger_map
    debit_account = ledger_map[classification["debit_symbol"]]
    credit_account = ledger_map[classification["credit_symbol"]]
    source_document_id = uuid.UUID(classification["source_document_id"])

    pending = await self.tb_client.create_two_phase_transfer(
        debit_account=debit_account,
        credit_account=credit_account,
        amount_minor=int(classification["amount_minor"]),
        source_document_id=source_document_id,
    )

    classification.setdefault("contractor_nip", event.get("contractor_nip", ""))

    transfer = LedgerTransfer(
        company_id=company_id,
        source_account=debit_account,
        target_account=credit_account,
        amount_minor=int(classification["amount_minor"]),
        source_document_id=source_document_id,
        status=TransferStatus.PENDING,
        meta={"tb_pending_id": pending.pending_id, "classification": classification},
    )
    self.session.add(transfer)
    await self.session.commit()
    await self.session.refresh(transfer)
    return transfer

```

===== Roboton_Reflekton/shadow_ledger.py =====

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from datetime import date
```

```
from typing import Any
```

```
from .models import LegalForm, TaxForm
```

```
from .strategies import StrategyContext, StrategyRegistry
```

```
@dataclass(slots=True)
```

```
class ShadowLedgerInput:
```

```
    company_id: str
```

```
    legal_form: LegalForm
```

```
    vat_proportion: float
```

```
    month_start: date
```

```
    month_end: date
```

```

class TaxSimulator:
    """Predictive tax simulator powered by DuckDB + Polars shadow ledgers."""

    def __init__(self, *, duckdb_path: str = "nexus.duckdb", strategy_registry: StrategyRegistry | None = None):
        self.duckdb_path = duckdb_path
        self.strategy_registry = strategy_registry or StrategyRegistry()

    async def run_shadow_simulation(self, current_month_data: Any, *, legal_form: LegalForm, vat_proportion: float):
        """Return Polars DataFrame comparing tax burden for all valid strategies.

        Expected columns in 'current_month_data':
        - 'kind': 'revenue' or 'expense'
        - 'net': numeric
        - 'vat_amount': numeric
        """

        pl = self._require_polars()
        frame = self._ensure_polars_frame(current_month_data, pl)
        metrics = self._aggregate_month_metrics(frame, pl)

        rows: list[dict[str, Any]] = []
        for tax_form, strategy in self.strategy_registry._strategies.items(): # noqa: SLF001 - registry is private
            if not self._is_strategy_compatible(tax_form=tax_form, legal_form=legal_form):
                continue

            policy = strategy.policy_payload(
                StrategyContext(
                    legal_form=legal_form,
                    ksef_active=True,
                    vat_proportion=vat_proportion,
                )
            )
            rows.append(self._simulate_policy_row(metrics=metrics, policy=policy, tax_form=tax_form, pl=pl))

        if not rows:
            return pl.DataFrame(schema={
                "tax_form": pl.Utf8,
                "income_tax_due": pl.Float64,
                "vat_due": pl.Float64,
                "effective_tax_rate": pl.Float64,
                "cash_left": pl.Float64,
            })

        return pl.DataFrame(rows).sort("cash_left", descending=True)

    async def load_current_month_data(self, params: ShadowLedgerInput) -> Any:
        """Load monthly journal entries from DuckDB and return Polars DataFrame."""

        duckdb = self._require_duckdb()
        _ = self._require_polars()

        with duckdb.connect(self.duckdb_path, read_only=True) as conn:
            relation = conn.execute(
                """
                SELECT
                    kind,
                    net_amount AS net,
                    COALESCE(vat_amount, 0) AS vat_amount,
                    COALESCE(vat_deductible_ratio, 1.0) AS vat_deductible_ratio
                FROM accounting_entries
                WHERE company_id = ?
                   AND posted_at >= ?
                   AND posted_at <= ?
            """
            )

```

```

        """
        [params.company_id, params.month_start.isoformat(), params.month_end.isoformat()],
    )
    return relation.pl()

@staticmethod
def _aggregate_month_metrics(frame: Any, pl: Any) -> dict[str, float]:
    aggregated = frame.select(
        [
            pl.col("net").filter(pl.col("kind") == "revenue").sum().fill_null(0.0).alias("revenue_net")
            pl.col("net").filter(pl.col("kind") == "expense").sum().fill_null(0.0).alias("expense_net")
            pl.col("vat_amount").filter(pl.col("kind") == "revenue").sum().fill_null(0.0).alias("output_vat")
            (
                pl.col("vat_amount")
                .mul(pl.col("vat_deductible_ratio").fill_null(1.0))
                .filter(pl.col("kind") == "expense")
                .sum()
                .fill_null(0.0)
                .alias("input_vat")
            ),
        ]
    )
    return aggregated.to_dicts()[0]

@staticmethod
def _simulate_policy_row(*, metrics: dict[str, float], policy: dict[str, Any], tax_form: TaxForm, pl: Any) -> dict[str, float]:
    revenue = float(metrics["revenue_net"])
    expense = float(metrics["expense_net"])
    taxable_income = max(revenue - expense, 0.0)

    if tax_form == TaxForm.LUMP_SUM:
        rate = float(policy.get("revenue_rates", [0.12])[4])
        income_tax_due = revenue * rate
    elif tax_form == TaxForm.LINEAR:
        income_tax_due = taxable_income * float(policy.get("pit_rate", 0.19))
    elif tax_form == TaxForm.CIT_STANDARD:
        income_tax_due = taxable_income * float(policy.get("cit_rates", {}).get("small", 0.09))
    elif tax_form == TaxForm.CIT_ESTONIAN:
        income_tax_due = taxable_income * float(policy.get("distribution_tax_rate", 0.2))
    else:
        income_tax_due = 0.0

    vat_due = max(float(metrics["output_vat"]) - float(metrics["input_vat"]), 0.0)
    total_tax = income_tax_due + vat_due
    cash_left = max(revenue - expense - total_tax, 0.0)
    effective_tax_rate = (total_tax / revenue) if revenue > 0 else 0.0

    return {
        "tax_form": tax_form.value,
        "income_tax_due": round(income_tax_due, 2),
        "vat_due": round(vat_due, 2),
        "effective_tax_rate": round(effective_tax_rate, 4),
        "cash_left": round(cash_left, 2),
    }

@staticmethod
def _is_strategy_compatible(*, tax_form: TaxForm, legal_form: LegalForm) -> bool:
    if tax_form in {TaxForm.LUMP_SUM, TaxForm.LINEAR}:
        return legal_form in {LegalForm.JDG, LegalForm.CIVIL_PARTNERSHIP}
    if tax_form in {TaxForm.CIT_STANDARD, TaxForm.CIT_ESTONIAN}:
        return legal_form in {LegalForm.SP_ZOO, LegalForm.PSA}
    return False

@staticmethod
def _ensure_polars_frame(current_month_data: Any, pl: Any) -> Any:

```

```

        if hasattr(current_month_data, "lazy") and hasattr(current_month_data, "select"):
            return current_month_data
        return pl.DataFrame(current_month_data)

    @staticmethod
    def _require_polars() -> Any:
        try:
            import polars as pl
        except Exception as exc: # pragma: no cover - optional dependency in CI image
            raise RuntimeError("Polars is required for PredictiveTaxEngine. Install 'polars'.") from exc
        return pl

    @staticmethod
    def _require_duckdb() -> Any:
        try:
            import duckdb
        except Exception as exc: # pragma: no cover - optional dependency in CI image
            raise RuntimeError("DuckDB is required for PredictiveTaxEngine. Install 'duckdb'.") from exc
        return duckdb

===== Roboton_Reflekton/strategies.py =====
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Protocol

from .models import LegalForm, TaxForm

class InvalidTaxPolicyError(ValueError):
    """Raised when tax policy is incompatible with legal form."""

class LedgerInitializer(Protocol):
    def initialize(self, *, legal_form: LegalForm, tax_form: TaxForm) -> dict[str, int]: ...

@dataclass(slots=True)
class StrategyContext:
    legal_form: LegalForm
    ksef_active: bool
    vat_proportion: float

class TaxStrategy(ABC):
    tax_form: TaxForm

    @abstractmethod
    def policy_payload(self, context: StrategyContext) -> dict:
        raise NotImplementedError

class JdgLumpSumStrategy(TaxStrategy):
    tax_form = TaxForm.LUMP_SUM

    def policy_payload(self, context: StrategyContext) -> dict:
        if context.legal_form not in {LegalForm.JDG, LegalForm.CIVIL_PARTNERSHIP}:
            raise InvalidTaxPolicyError("Rycza?t dost?pny tylko dla JDG i sp?ki cywilnej.")
        return {
            "tax_form": self.tax_form,
            "pit_costs_enabled": False,
            "requires_full_ledger": False,
            "vat_proportion": context.vat_proportion,
            "ksef_active": context.ksef_active,

```

```

        "revenue_rates": [0.02, 0.03, 0.055, 0.085, 0.12, 0.14, 0.15, 0.17],
    }

class JdgLinearStrategy(TaxStrategy):
    tax_form = TaxForm.LINEAR

    def policy_payload(self, context: StrategyContext) -> dict:
        if context.legal_form not in {LegalForm.JDG, LegalForm.CIVIL_PARTNERSHIP}:
            raise InvalidTaxPolicyError("Podatek liniowy dost?pnny tylko dla JDG i sp?ki cywilnej.")
        return {
            "tax_form": self.tax_form,
            "pit_costs_enabled": True,
            "requires_full_ledger": False,
            "pit_rate": 0.19,
            "vat_proportion": context.vat_proportion,
            "ksef_active": context.ksef_active,
        }

class CorpFullLedgerStrategy(TaxStrategy):
    tax_form = TaxForm.CIT_STANDARD

    def policy_payload(self, context: StrategyContext) -> dict:
        if context.legal_form not in {LegalForm.SP_ZOO, LegalForm.PSA}:
            raise InvalidTaxPolicyError("CIT standardowy dost?pnny tylko dla sp?ek kapita?owych.")
        return {
            "tax_form": self.tax_form,
            "pit_costs_enabled": False,
            "requires_full_ledger": True,
            "cit_rates": {"small": 0.09, "standard": 0.19},
            "ksef_active": context.ksef_active,
        }

class CitEstonianStrategy(TaxStrategy):
    tax_form = TaxForm.CIT_ESTONIAN

    def policy_payload(self, context: StrategyContext) -> dict:
        if context.legal_form not in {LegalForm.SP_ZOO, LegalForm.PSA}:
            raise InvalidTaxPolicyError("CIT Esto?ski dost?pnny tylko dla sp?ek kapita?owych.")
        return {
            "tax_form": self.tax_form,
            "pit_costs_enabled": False,
            "requires_full_ledger": True,
            "deferred_tax": True,
            "distribution_tax_rate": 0.2,
            "ksef_active": context.ksef_active,
        }

class StrategyRegistry:
    def __init__(self) -> None:
        self._strategies: dict[TaxForm, TaxStrategy] = {
            TaxForm.LUMP_SUM: JdgLumpSumStrategy(),
            TaxForm.LINEAR: JdgLinearStrategy(),
            TaxForm.CIT_STANDARD: CorpFullLedgerStrategy(),
            TaxForm.CIT_ESTONIAN: CitEstonianStrategy(),
        }

    def select(self, tax_form: TaxForm) -> TaxStrategy:
        try:
            return self._strategies[tax_form]
        except KeyError as exc:
            raise InvalidTaxPolicyError(f"Brak strategii dla tax_form={tax_form}") from exc

```

```

===== Roboton_Reflekton/vat_reconciliation.py =====
from __future__ import annotations

import uuid
from dataclasses import dataclass
from decimal import Decimal, ROUND_HALF_UP
from typing import Any

from .ledger_client import TigerBeetleClient

_ALLOWED_RATES = {"23", "8", "5", "0", "np", "zw"}

@dataclass(frozen=True, slots=True)
class VATBreakdown:
    rate: str
    net_amount: Decimal
    vat_amount: Decimal
    gross_amount: Decimal

@dataclass(frozen=True, slots=True)
class VATIntegrityResult:
    invoice_id: str
    status: str
    errors: list[str]

@dataclass(frozen=True, slots=True)
class ReconciliationAlert:
    status: str
    vat_in_ledger: Decimal
    vat_out_ledger: Decimal
    vat_in_register: Decimal
    vat_out_register: Decimal
    missing_in_duckdb: list[str]
    missing_in_ledger: list[str]

class VATReconciliationEngine:
    def __init__(self, duckdb_manager: Any, tb_client: TigerBeetleClient, account_vat_in: int, account_vat_out: int):
        self.duckdb = duckdb_manager
        self.tb_client = tb_client
        self.account_vat_in = account_vat_in
        self.account_vat_out = account_vat_out

    def ensure_tax_rates_schema(self) -> None:
        self.duckdb.execute("""
            CREATE TABLE IF NOT EXISTS tax_rates (rate VARCHAR PRIMARY KEY)
        """)
        for rate in ["23", "8", "5", "0", "np", "zw"]:
            self.duckdb.execute("INSERT OR IGNORE INTO tax_rates(rate) VALUES (?)", (rate,))

    def _to_breakdown(self, row: dict[str, Any]) -> VATBreakdown:
        rate = str(row.get("rate", "")).lower()
        if rate not in _ALLOWED_RATES:
            raise ValueError(f"Unsupported VAT rate: {rate}")
        return VATBreakdown(
            rate=rate,
            net_amount=Decimal(str(row["net_amount"])),
            vat_amount=Decimal(str(row["vat_amount"])),
            gross_amount=Decimal(str(row["gross_amount"])),
        )

```

```

def check_vat_integrity(self, invoice_id: str, ocr_results: dict[str, Any]) -> VATIntegrityResult:
    self.ensure_tax_rates_schema()
    errors: list[str] = []
    breakdown_rows = ocr_results.get("vat_breakdown", [])
    breakdown = [self._to_breakdown(row) for row in breakdown_rows]

    total_net = Decimal("0.00")
    total_vat = Decimal("0.00")
    total_gross = Decimal("0.00")
    for item in breakdown:
        rate_decimal = Decimal("0") if item.rate in {"np", "zw"} else Decimal(item.rate)
        expected_vat = (item.net_amount * rate_decimal / Decimal("100")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        if abs(expected_vat - item.vat_amount) > Decimal("0.01"):
            errors.append(f"VAT_MISMATCH:{item.rate}")
        if abs((item.net_amount + item.vat_amount) - item.gross_amount) > Decimal("0.01"):
            errors.append(f"MATH_ERROR_LINE:{item.rate}")
        total_net += item.net_amount
        total_vat += item.vat_amount
        total_gross += item.gross_amount

    reported_total_gross = Decimal(str(ocr_results.get("total_gross", total_gross)))
    if abs((total_net + total_vat) - reported_total_gross) > Decimal("0.01"):
        errors.append("MATH_ERROR_TOTAL")

    return VATIntegrityResult(
        invoice_id=invoice_id,
        status="OK" if not errors else "FAILED",
        errors=errors,
    )

async def reconcile_with_tigerbeetle(self) -> ReconciliationAlert:
    vat_in_ledger = Decimal(await self.tb_client.get_account_credits_posted(self.account_vat_in)) / Decimal("100")
    vat_out_ledger = Decimal(await self.tb_client.get_account_credits_posted(self.account_vat_out)) / Decimal("100")

    vat_rows = self.duckdb.execute(
        """
        SELECT
            COALESCE(SUM(CASE WHEN type = 'PURCHASE' THEN vat_amount ELSE 0 END), 0),
            COALESCE(SUM(CASE WHEN type = 'SALE' THEN vat_amount ELSE 0 END), 0)
        FROM invoices_vat
        """
    )
    vat_in_register = Decimal(str(vat_rows[0][0])) if vat_rows else Decimal("0")
    vat_out_register = Decimal(str(vat_rows[0][1])) if vat_rows else Decimal("0")

    diff_rows = self.duckdb.execute(
        """
        SELECT source, invoice_id FROM vat_reconciliation_candidates
        WHERE source IN ('DUCKDB_ONLY', 'LEDGER_ONLY')
        """
    )
    missing_in_ledger = [str(invoice_id) for source, invoice_id in diff_rows if source == "DUCKDB_ONLY"]
    missing_in_duckdb = [str(invoice_id) for source, invoice_id in diff_rows if source == "LEDGER_ONLY"]

    status = "OK" if (vat_in_ledger == vat_in_register and vat_out_ledger == vat_out_register and not missing_in_duckdb and not missing_in_ledger) else "FAILED"
    return ReconciliationAlert(
        status=status,
        vat_in_ledger=vat_in_ledger,
        vat_out_ledger=vat_out_ledger,
        vat_in_register=vat_in_register,
        vat_out_register=vat_out_register,
        missing_in_duckdb=missing_in_duckdb,
        missing_in_ledger=missing_in_ledger,
    )

```

```
===== Roboton_Reflekton/vision_agent.py =====
```

```
from __future__ import annotations
```

```
import asyncio
```

```
import json
```

```
import re
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
from typing import Any
```

```
@dataclass(slots=True)
```

```
class VisionExtraction:
```

```
    vendor_nip: str | None
```

```
    total_gross: float | None
```

```
    vat_rate: float | None
```

```
    payment_status: str
```

```
    visual_anomalies_detected: bool
```

```
    handwritten_notes_summary: str
```

```
    source: str
```

```
def _to_float(value: Any) -> float | None:
```

```
    if value is None:
```

```
        return None
```

```
    try:
```

```
        return float(str(value).replace(" ", "").replace(",", "."))
```

```
    except ValueError:
```

```
        return None
```

```
class VisionAgent:
```

```
    """Visual Reasoning layer for invoice images (Qwen2.5-VL first, heuristic fallback)."""
```

```
PROMPT = (
```

```
    "Extract JSON: {vendor_nip, total_gross, vat_rate, payment_status, "
```

```
    "visual_anomalies_detected, handwritten_notes_summary}."
```

```
)
```

```
def __init__(
```

```
    self,
```

```
    model_name: str = "Qwen/Qwen2.5-VL-2B-Instruct",
```

```
    use_4bit: bool = True,
```

```
    max_new_tokens: int = 220,
```

```
) -> None:
```

```
    self.model_name = model_name
```

```
    self.use_4bit = use_4bit
```

```
    self.max_new_tokens = max_new_tokens
```

```
    self._processor = None
```

```
    self._model = None
```

```
    self._enabled = False
```

```
    self._load_error: str | None = None
```

```
    self._try_load_runtime()
```

```
def _try_load_runtime(self) -> None:
```

```
    try:
```

```
        from transformers import AutoProcessor, BitsAndBytesConfig, Qwen2_5_VLForConditionalGeneration
```

```
        import torch
```

```
    except Exception as exc: # pragma: no cover - optional runtime dependency
```

```
        self._load_error = str(exc)
```

```
        return
```

```
    try: # pragma: no cover - model loading is environment dependent
```

```
        quant_config = (
```

```
            BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.float16) if self.use_4b
```

```

    )
    kwargs = {"device_map": "auto", "trust_remote_code": True}
    if quant_config is not None:
        kwargs["quantization_config"] = quant_config

    self._processor = AutoProcessor.from_pretrained(self.model_name, trust_remote_code=True)
    self._model = Qwen2_5_VLForConditionalGeneration.from_pretrained(self.model_name, **kwargs)
    self._enabled = True
except Exception as exc:
    self._load_error = str(exc)
    self._enabled = False

async def analyze(self, image_path: Path, ocr_text: str) -> VisionExtraction:
    if self._enabled:
        result = await asyncio.to_thread(self._run_model, image_path)
        if result is not None:
            return result
    return self._fallback_from_ocr(ocr_text)

def _run_model(self, image_path: Path) -> VisionExtraction | None:
    try: # pragma: no cover - requires VL runtime + model files
        from PIL import Image
        import torch
    except Exception:
        return None

    assert self._processor is not None
    assert self._model is not None

    try: # pragma: no cover - requires VL runtime + model files
        image = Image.open(image_path).convert("RGB")
        messages = [
            {
                "role": "user",
                "content": [
                    {"type": "image", "image": image},
                    {"type": "text", "text": self.PROMPT},
                ],
            }
        ]
        text = self._processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
        inputs = self._processor(text=[text], images=[image], return_tensors="pt")
        inputs = {k: v.to(self._model.device) if hasattr(v, "to") else v for k, v in inputs.items()}
        with torch.inference_mode():
            outputs = self._model.generate(**inputs, max_new_tokens=self.max_new_tokens)
            decoded = self._processor.batch_decode(outputs, skip_special_tokens=True)[0]
            json_blob = self._extract_json_blob(decoded)
            if json_blob is None:
                return None
            data = json.loads(json_blob)
            return VisionExtraction(
                vendor_nip=data.get("vendor_nip"),
                total_gross=_to_float(data.get("total_gross")),
                vat_rate=_to_float(data.get("vat_rate")),
                payment_status=str(data.get("payment_status", "unknown")),
                visual_anomalies_detected=bool(data.get("visual_anomalies_detected", False)),
                handwritten_notes_summary=str(data.get("handwritten_notes_summary", "")),
                source="qwen2.5-vl-2b-4bit",
            )
    except Exception:
        return None

@staticmethod
def _extract_json_blob(text: str) -> str | None:
    start = text.find("{")

```

```

end = text.rfind("}")
if start < 0 or end < start:
    return None
return text[start : end + 1]

```

```
@staticmethod
```

```
def _fallback_from_ocr(raw_text: str) -> VisionExtraction:
```

```

    normalized = raw_text.replace("\n", " ")
    nip_match = re.search(r"\b\d{10}\b", normalized)
    gross_match = re.search(r"(?:brutto|total|razem)\D{0,12}(\d[\d\s]*[.],\d{2})", normalized, re.IGNORECASE)
    vat_match = re.search(r"(?:VAT|PTU)\D{0,6}(\d{1,2})\s?%", normalized, re.IGNORECASE)
    paid = bool(re.search(r"\b(zap?acono|paid|op?acono)\b", normalized, re.IGNORECASE))
    anomalies = bool(re.search(r"\b(korekta|duplikat|anulowano)\b", normalized, re.IGNORECASE))
    handwritten_hint = "possible handwritten note detected" if "odr?cz" in normalized.lower() else ""

```

```
total_gross = None
```

```
if gross_match:
```

```
    total_gross = _to_float(gross_match.group(1))
```

```
return VisionExtraction(
```

```
    vendor_nip=nip_match.group(0) if nip_match else None,
```

```
    total_gross=total_gross,
```

```
    vat_rate=_to_float(vat_match.group(1)) if vat_match else None,
```

```
    payment_status="paid" if paid else "unknown",
```

```
    visual_anomalies_detected=anomalies,
```

```
    handwritten_notes_summary=handwritten_hint,
```

```
    source="ocr-fallback",
```

```
)
```

```
===== luz/build_nexus.py.py =====
```

```
# build_nexus.py
```

```
import subprocess
```

```
import sys
```

```
def build_executable():
```

```
    """Kompiluje aplikacj? do natywnego pliku .exe za pomoc? Nuitka."""
```

```
    command = [
```

```
        sys.executable, "-m", "nuitka",
```

```
        "--standalone", # Tworzy niezale?ny folder z plikiem .exe
```

```
        "--onefile", # Opcjonalnie: pakuje wszystko do jednego pliku
```

```
        "--enable-plugin=pydantic", # Rozwi?zuje problemy z Pydantic v2
```

```
        "--enable-plugin=torch", # Do??cza odpowiednie DLL-ki dla AI
```

```
        "--enable-plugin=numpy",
```

```
        "--include-data-dir=app_data=app_data", # Do??cza puste foldery na bazy
```

```
        "--output-dir=dist",
```

```
        "main.py"
```

```
    ]
```

```
    print("Rozpoczynam kompilacj? Nuitka. To mo?e potrwa? kilkadziesi?t minut...")
```

```
    subprocess.run(command, check=True)
```

```
if __name__ == "__main__":
```

```
    build_executable()
```

```
# Sekwencja komend do CI/CD lub uruchamiania lokalnego
```

```
# 1. Zablokowanie wersji za pomoc? 'uv' (szybsza alternatywa dla pip-tools)
```

```
# uv pip compile pyproject.toml -o requirements.txt
```

```
# 2. Audyt bezpiecze?stwa paczek Pythona
```

```
# uv pip audit requirements.txt
```

```
# 3. Wygenerowanie SBOM w standardzie CycloneDX (u?ywaj?c narz?dzia syft/trivy)
```

```

# trivy fs --format cyclonedx --output nexus_sbom.json .

# 4. Skanowanie wygenerowanego SBOM pod kątem krytycznych luk (CVE)
# trivy sbom nexus_sbom.json --severity CRITICAL,HIGH

===== luz/main.py =====
import os
import sys
import socket
import subprocess
import time
import secrets
from pathlib import Path
import flet as ft

# --- KONFIGURACJA OFFLINE AI ---
if getattr(sys, 'frozen', False):
    # Jeśli aplikacja jest skompilowana jako .exe (PyInstaller)
    base_path = Path(sys._MEIPASS)
else:
    # Jeśli uruchamiamy z kodu źródłowego
    base_path = Path(__file__).parent

models_cache_dir = base_path / "models"
os.environ["HF_HOME"] = str(models_cache_dir)
os.environ["TRANSFORMERS_OFFLINE"] = "1" # Twarde wymuszenie trybu offline

def get_free_port() -> int:
    """Dynamicznie znajduje wolny, bezpieczny port na komputerze użytkownika."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind(('', 0)) # Bindowanie do portu 0 zmusza system do przydzielenia wolnego portu
        return s.getsockname()[1]

def launch_app():
    print("[Launcher] Inicjalizacja Nexus Accounting...")

    # 1. Generowanie jednorazowego tokena bezpieczeństwa (Handshake)
    bootstrap_token = secrets.token_urlsafe(32)

    # 2. Znalezienie wolnego portu
    backend_port = get_free_port()
    print(f"[Launcher] Przydzielono port: {backend_port}")

    # 3. Przygotowanie środowiska dla backendu
    backend_env = os.environ.copy()
    backend_env["NEXUS_PORT"] = str(backend_port)
    backend_env["NEXUS_TOKEN"] = bootstrap_token
    backend_env["PYTHONPATH"] = str(Path.cwd())

    # 4. Uruchomienie Serwera Litestar (przez uvicorn)
    print("[Launcher] Uruchamianie serwera API w tle...")
    backend_proc = subprocess.Popen(
        [sys.executable, "-m", "uvicorn", "api.server:app", "--port", str(backend_port)],
        env=backend_env,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE
    )

    # Dajemy serwerowi chwilę na wstanie
    time.sleep(1.5)

    if backend_proc.poll() is not None:
        print("[Błąd] Backend nie mógł wystartować!")
        sys.exit(1)

```

```

# 5. Uruchomienie okna interfejsu Flet
from frontend.main_ui import main as start_ui

def ui_wrapper(page: ft.Page):
    page.title = "Nexus Accounting"
    page.window_width = 1200
    page.window_height = 800

    # Zapisujemy dane po??czeniowe w sesji
    page.session.set("api_port", backend_port)
    page.session.set("api_token", bootstrap_token)

    start_ui(page)

try:
    print("[Launcher] Otwieranie interfejsu u?ytkownika...")
    ft.app(target=ui_wrapper)
finally:
    # 6. Krytyczne sprz?tanie (Graceful Shutdown)
    print("[Launcher] Zamykanie aplikacji. Ubijanie procesw w tle...")
    backend_proc.terminate()
    try:
        backend_proc.wait(timeout=3.0)
    except subprocess.TimeoutExpired:
        backend_proc.kill()
    print("[Launcher] Zako?czono pomy?lnie.")

if __name__ == "__main__":
    launch_app()

import os
import sys
import asyncio
import subprocess
import logging
import flet as ft
from pathlib import Path

# Importy wewn?trzne
from core.config import AppConfig
from scripts.setup_env import bootstrap_system
from scripts.doctor import run_diagnostics

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler("nexus_main.log")]
)
logger = logging.getLogger("nexus.main")

class NexusOrchestrator:
    def __init__(self):
        self.config = AppConfig()
        self.worker_process = None
        self.nats_process = None

    async def start_nats(self):
        """Uruchamia lokalny serwer NATS."""
        nats_path = self.config.base_dir / ("nats-server.exe" if os.name == "nt" else "nats-server")
        if nats_path.exists():
            logger.info("Uruchamianie serwera NATS...")
            self.nats_process = subprocess.Popen(
                [str(nats_path), "-p", "4222", "-js"],
                stdout=subprocess.DEVNULL,

```

```

        stderr=subprocess.DEVNULL
    )
    await asyncio.sleep(2)
else:
    logger.error("Nie znalezione binarki NATS!")

async def start_backend_api(self):
    """Uruchamia serwer API (Litestar) w p?tli zdarze?."""
    from api.app import app
    import uvicorn
    config = uvicorn.Config(app, host="127.0.0.1", port=8000, log_level="info")
    server = uvicorn.Server(config)
    logger.info("Inicjalizacja Backend API na http://127.0.0.1:8000")
    return asyncio.create_task(server.serve())

async def start_worker(self):
    """Uruchamia proces roboczy Taskiq (OCR/AI)."""
    logger.info("Uruchamianie Workera AI...")
    cmd = [sys.executable, "-m", "taskiq", "worker", "worker:broker", "--fs-startup"]
    self.worker_process = subprocess.Popen(cmd)

def cleanup(self):
    logger.info("Zamykanie komponentw Nexus AI...")
    if self.worker_process: self.worker_process.terminate()
    if self.nats_process: self.nats_process.terminate()

async def main_ui(page: ft.Page):
    from ui.app import NexusUI
    page.title = "Nexus AI - Zarz?dzanie Fakturami"
    page.theme_mode = ft.ThemeMode.DARK
    ui = NexusUI(page)
    await ui.build()

async def start_app():
    orchestrator = NexusOrchestrator()
    print(">>> Nexus AI: Sprawdzenie integralno?ci systemu...")
    await bootstrap_system()
    run_diagnostics()

    await orchestrator.start_nats()
    await orchestrator.start_worker()
    api_task = await orchestrator.start_backend_api()

    try:
        await ft.app_async(target=main_ui)
    except Exception as e:
        logger.error(f"B??d krytyczny UI: {e}")
    finally:
        orchestrator.cleanup()
        api_task.cancel()

if __name__ == "__main__":
    if os.name == "nt":
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
    try:
        asyncio.run(start_app())
    except KeyboardInterrupt:
        pass

import os
import sys
import asyncio
import subprocess
import socket

```

```

import logging
import time
from pathlib import Path
from datetime import datetime
import flet as ft

# Importy pomocnicze
from core.config import AppConfig
from scripts.setup_env import bootstrap_system

# --- ZAAWANSOWANE LOGOWANIE ---
log_dir = Path("logs")
log_dir.mkdir(exist_ok=True)
log_file = log_dir / f"nexus_{datetime.now().strftime('%Y%m%d')}.log"

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler(log_file, encoding="utf-8")]
)
logger = logging.getLogger("nexus.main")

# --- MECHANIZM: SINGLE INSTANCE LOCK ---
def is_already_running(port=47999):
    """Zapobiega uruchomieniu wielu instancji aplikacji."""
    try:
        lock_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        lock_socket.bind(("127.0.0.1", port))
        globals()[ "_lock_socket" ] = lock_socket
        return False
    except socket.error:
        return True

# --- ORKIESTRATOR PROCESW (WATCHDOG) ---
class NexusProcessManager:
    def __init__(self, config: AppConfig):
        self.config = config
        self.nats_proc = None
        self.worker_proc = None
        self._running = True

    async def start_services(self):
        # 1. NATS Server
        nats_bin = "nats-server.exe" if os.name == "nt" else "nats-server"
        nats_path = self.config.base_dir / nats_bin
        if nats_path.exists():
            self.nats_proc = subprocess.Popen(
                [str(nats_path), "-p", "4222", "-js"],
                stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
            )

        # 2. Worker AI
        logger.info("Uruchamianie Workera AI...")
        cmd = [sys.executable, "-m", "taskiq", "worker", "worker:broker", "--fs-startup"]
        self.worker_proc = subprocess.Popen(cmd)

    async def watchdog(self):
        """Monitoruje zdrowie procesw i restartuje je w razie awarii."""
        while self._running:
            if self.worker_proc and self.worker_proc.poll() is not None:
                logger.warning("Worker przesta? odpowiada?. Restartowanie...")
                await self.start_services()
                await asyncio.sleep(15)

    def stop_all(self):

```

```

        self._running = False
        if self.worker_proc: self.worker_proc.terminate()
        if self.nats_proc: self.nats_proc.terminate()
        logger.info("Infrastruktura: Bezpiecznie zatrzymana.")

# --- UI SPLASH SCREEN & MAIN ---
async def start_nexus_app(page: ft.Page):
    manager = NexusProcessManager(AppConfig())

    # Konfiguracja okna Splash Screen
    page.title = "Nexus AI"
    page.window_width, page.window_height = 450, 600
    page.window_center()
    page.window_resizable = False
    page.theme_mode = ft.ThemeMode.DARK
    page.window_always_on_top = True

    status_text = ft.Text("Przygotowywanie systemw...", size=14, italic=True)
    pb = ft.ProgressBar(width=350, color="blue", bgcolor="#1e1e1e")

    page.add(
        ft.Container(
            content=ft.Column([
                ft.Image(src="assets/logo_splash.png", width=150),
                ft.Divider(height=40, color="transparent"),
                status_text,
                pb
            ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
            expand=True, alignment=ft.alignment.center
        )
    )

# KROKI STARTOWE
try:
    # 1. Bootstrap
    status_text.value = "Krok 1/4: Inicjalizacja bazy danych..."
    page.update()
    await bootstrap_system()

    # 2. Start Us?ug
    status_text.value = "Krok 2/4: Uruchamianie silnikw AI..."
    page.update()
    await manager.start_services()

    # 3. Start API
    status_text.value = "Krok 3/4: Konfiguracja API..."
    page.update()
    from api.app import app
    import uvicorn
    api_config = uvicorn.Config(app, host="127.0.0.1", port=8000, log_level="error")
    api_server = uvicorn.Server(api_config)
    asyncio.create_task(api_server.serve())
    asyncio.create_task(manager.watchdog())

    # 4. Finalizacja
    status_text.value = "Krok 4/4: Synchronizacja interfejsu..."
    page.update()
    await asyncio.sleep(1.5)

    # Przej?cie do g?wnej aplikacji
    page.clean()
    page.window_always_on_top = False
    page.window_resizable = True
    page.window_width, page.window_height = 1280, 800
    page.update()

```

```

        from ui.root import NexusRootUI
        app_ui = NexusRootUI(page, manager)
        await app_ui.build()

    except Exception as e:
        logger.critical(f"B??D STARTU: {e}")
        page.add(ft.Text(f"B??d krytyczny: {e}", color="red"))
        page.update()

async def main():
    if is_already_running():
        print("Nexus AI ju? dzia?a.")
        sys.exit(0)

    try:
        await ft.app_async(target=start_nexus_app, assets_dir="assets")
    except KeyboardInterrupt:
        logger.info("Przerwanie przez u?ytkownika.")
    finally:
        logger.info("Zwalnianie zasobw przed zamkni?ciem...")

if __name__ == "__main__":
    if os.name == "nt":
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
    asyncio.run(main())


import os
import sys
import asyncio
import subprocess
import socket
import logging
import signal
import time
import secrets
from pathlib import Path
from datetime import datetime
import flet as ft

# --- 1. KONFIGURACJA ?RODOWISKA OFFLINE AI (KRYTYCZNE: Przed importem modeli) ---
if getattr(sys, 'frozen', False):
    # Je?li aplikacja jest skompilowana jako .exe (PyInstaller)
    base_path = Path(sys._MEIPASS)
else:
    # Je?li uruchamiamy z kodu ?rd?owego
    base_path = Path(__file__).parent

models_cache_dir = base_path / "models"
os.environ["HF_HOME"] = str(models_cache_dir)
os.environ["TRANSFORMERS_OFFLINE"] = "1" # Twarde wymuszenie trybu offline

# --- 2. KONFIGURACJA LOGOWANIA ---
log_dir = Path("logs")
log_dir.mkdir(exist_ok=True)
log_file = log_dir / f"nexus_{datetime.now().strftime('%Y%m%d')}.log"

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[

```

```

        logging.StreamHandler(),
        logging.FileHandler(log_file, encoding="utf-8")
    ]
)
logger = logging.getLogger("nexus.main")

# --- 3. IMPORTY WEWN?TRZNE (Po konfiguracji ?rodowiska) ---
from core.config import AppConfig
from scripts.setup_env import bootstrap_system
from scripts.doctor import run_diagnostics

# --- 4. FUNKCJE POMOCNICZE I BLOKADA INSTANCJI ---
def get_free_port() -> int:
    """Dynamicznie znajduje wolny port na localhost."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind('', 0)
        return s.getsockname()[1]

def is_already_running(port=47999):
    """Zapobiega uruchomieniu wielu instancji aplikacji (Single Instance Lock)."""
    try:
        lock_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        lock_socket.bind(("127.0.0.1", port))
        globals()[ "_lock_socket" ] = lock_socket
        return False
    except socket.error:
        return True

# --- 5. ORKIESTRATOR PROCESW (WATCHDOG) ---
class NexusOrchestrator:
    """Zarz?dza NATS, Workerem i API, gwarantuj?c ich restart i poprawne zamkni?cie."""
    def __init__(self):
        self.config = AppConfig()
        self.nats_process = None
        self.worker_process = None
        self.api_task = None
        self.bootstrap_token = secrets.token_urlsafe(32)

    async def start_nats(self):
        """Uruchamia lokalny serwer NATS z obs?ug? JetStream."""
        nats_bin = "nats-server.exe" if os.name == "nt" else "nats-server"
        nats_path = self.config.base_dir / nats_bin

        if nats_path.exists():
            logger.info("Uruchamianie infrastruktury NATS JetStream...")
            self.nats_process = subprocess.Popen(
                [str(nats_path), "-p", "4222", "-js"],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL
            )
            await asyncio.sleep(2)
        else:
            logger.error("Nie znaleziono binarki NATS w folderze g?wnym!")

    async def start_worker(self):
        """Uruchamia proces roboczy Taskiq (OCR/AI)."""
        logger.info("Uruchamianie silnika AI (Worker)...")
        cmd = [sys.executable, "-m", "taskiq", "worker", "worker:broker", "--fs-startup"]
        self.worker_process = subprocess.Popen(cmd)

    async def start_backend_api(self, port: int):
        """Uruchamia serwer API (Litestar) w p?tli zdarze?."""
        from api.app import app
        import uvicorn

```

```

backend_env = os.environ.copy()
backend_env["NEXUS_PORT"] = str(port)
backend_env["NEXUS_TOKEN"] = self.bootstrap_token
backend_env["PYTHONPATH"] = str(Path.cwd())

config = uvicorn.Config(app, host="127.0.0.1", port=port, log_level="error")
server = uvicorn.Server(config)
logger.info(f"Inicjalizacja API na http://127.0.0.1:{port}")
self.api_task = asyncio.create_task(server.serve())

def cleanup(self):
    """Krytyczne sprz?tanie procesw przy wyj?ciu z aplikacji."""
    logger.info("Zamykanie komponentw Nexus AI...")
    if self.worker_process:
        self.worker_process.terminate()
    if self.nats_process:
        self.nats_process.terminate()
    if self.api_task:
        self.api_task.cancel()
    logger.info("System zamkni?ty pomy?lnie.")

# --- 6. LOGIKA INTERFEJSU (Flet) ---
async def main_ui(page: ft.Page, orchestrator: NexusOrchestrator, port: int):
    """G?wny punkt wej?cia dla interfejsu graficznego ze Splash Screenem."""
    page.title = "Nexus AI - System Ksi?gowy"
    page.window_width = 450
    page.window_height = 600
    page.window_resizable = False
    page.theme_mode = ft.ThemeMode.DARK
    page.window_always_on_top = True

    # Splash Screen
    status_text = ft.Text("Przygotowywanie systemw...", size=14, italic=True)
    pb = ft.ProgressBar(width=350, color="blue", bgcolor="#1e1e1e")

    page.add(
        ft.Container(
            content=ft.Column([
                ft.Image(src="assets/logo_splash.png", width=150),
                ft.Divider(height=40, color="transparent"),
                status_text,
                pb
            ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
            expand=True, alignment=ft.alignment.center
        )
    )
    page.update()

    # Krok 1: Integralno?? i Bazy
    status_text.value = "Krok 1/4: Sprawdzanie bazy danych..."
    page.update()
    await bootstrap_system()
    run_diagnostics()

    # Krok 2: Infrastruktura (NATS)
    status_text.value = "Krok 2/4: Uruchamianie magistrali danych..."
    page.update()
    await orchestrator.start_nats()

    # Krok 3: Silniki AI i API
    status_text.value = "Krok 3/4: Budzenie silnikw AI..."
    page.update()
    await orchestrator.start_worker()
    await orchestrator.start_backend_api(port)

```

```

# Krok 4: Finalizacja i przejście do głównego UI
status_text.value = "Krok 4/4: Synchronizacja interfejsu..."
page.update()
await asyncio.sleep(1.5)

# Przekazanie na właściwy interfejs
page.clean()
page.window_always_on_top = False
page.window_resizable = True
page.window_width = 1280
page.window_height = 850

# Przekazanie powiadczeń do sesji UI
page.session.set("api_port", port)
page.session.set("api_token", orchestrator.bootstrap_token)

from ui.root import NexusRootUI
app_ui = NexusRootUI(page, orchestrator)
await app_ui.build()
page.update()

# --- 7. GŁÓWNY START SYSTEMU ---
async def start_app():
    if is_already_running():
        print("Nexus AI już działa.")
        sys.exit(1)

    orchestrator = NexusOrchestrator()
    backend_port = get_free_port()

    try:
        # Flet działa we własnej pętli zdarzeń
        await ft.app_async(
            target=lambda page: main_ui(page, orchestrator, backend_port)
        )
    except Exception as e:
        logger.critical(f"BYŁO KRYTYCZNY STARTU: {e}")
    finally:
        orchestrator.cleanup()

if __name__ == "__main__":
    if os.name == "nt":
        # Wymagane dla poprawnego działania asyncio i procesów na Windows
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

    try:
        asyncio.run(start_app())
    except KeyboardInterrupt:
        pass

===== luz/worker.py =====
"""Taskiq worker bootstrap and local connectivity smoke test."""
from __future__ import annotations

import asyncio
import gc
import logging
import os
import platform
import signal
import sys
from datetime import datetime, timezone
from pathlib import Path

import psutil

```

```

try:
    import torch
except Exception: # pragma: no cover - torch may be optional in some envs
    torch = None

from taskiq import TaskiqEvents

from Code.CORE.broker import broker
from Code.CORE.config import AppConfig
import Code.CORE.tasks # noqa: F401 # required to register @broker.task handlers
from Roboton_Reflekton.vision_agent import VisionAgent

if getattr(sys, "frozen", False):
    BASE_PATH = Path(sys._MEIPASS)
else:
    BASE_PATH = Path(__file__).resolve().parent

MODELS_CACHE_DIR = BASE_PATH / "models"
os.environ.setdefault("HF_HOME", str(MODELS_CACHE_DIR))
os.environ.setdefault("TRANSFORMERS_OFFLINE", "1")
os.environ.setdefault("TORCH_HOME", str(MODELS_CACHE_DIR / "torch"))
os.environ.setdefault("OMP_NUM_THREADS", "4")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")
os.environ.setdefault("MKL_NUM_THREADS", "4")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s (PID:%(process)d): %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("nexus.worker")

shutdown_flag = asyncio.Event()

class WorkerGuard:
    """Simple watchdog for worker process health and memory usage."""

    def __init__(self, ram_limit_gb: float = 6.0) -> None:
        self.process = psutil.Process(os.getpid())
        self.ram_limit = int(ram_limit_gb * 1024 * 1024 * 1024)
        self.start_time = datetime.now(timezone.utc)

    def check_resources(self) -> bool:
        current_mem = self.process.memory_info().rss
        if current_mem <= self.ram_limit:
            return False

        logger.warning("RAM alert: %.1f MB. Triggering cleanup...", current_mem / 1024**2)
        if torch is not None and torch.cuda.is_available():
            torch.cuda.empty_cache()
        gc.collect()
        return True

    async def heartbeat(self) -> None:
        while True:
            uptime = datetime.now(timezone.utc) - self.start_time
            ram_mb = self.process.memory_info().rss / 1024**2
            logger.debug("Heartbeat uptime=%s RAM=%.1fMB", uptime, ram_mb)
            await asyncio.sleep(60)

@broker.on_event(TaskiqEvents.WORKER_STARTUP)

```

```

async def on_worker_startup(state) -> None:
    logger.info(">>> Worker startup: loading config and OCR resources...")
    state.config = AppConfig()
    state.guard = WorkerGuard(ram_limit_gb=8.0)
    state.vision_agent = VisionAgent()
    state.heartbeat_task = asyncio.create_task(state.guard.heartbeat())

    gpu = bool(torch is not None and torch.cuda.is_available())
    logger.info(
        ">>> Worker ready. OS=%s, device=%s, vision_agent=%s",
        platform.system(),
        "CUDA/GPU" if gpu else "CPU",
        "qwen2.5-vl-2b-4bit" if getattr(state.vision_agent, "_enabled", False) else "ocr-fallback",
    )

@broker.on_event(TaskiqEvents.TASK_POST_EXECUTION)
async def on_task_post_execution(state, task_result) -> None:
    state.guard.check_resources()
    logger.debug("Task finished: %s", task_result.task_id)

@broker.on_event(TaskiqEvents.WORKER_SHUTDOWN)
async def on_worker_shutdown(state) -> None:
    logger.info(">>> Worker shutdown: releasing resources...")
    if hasattr(state, "heartbeat_task"):
        state.heartbeat_task.cancel()
    if torch is not None and torch.cuda.is_available():
        torch.cuda.empty_cache()
    gc.collect()

def _signal_handler(sig, _frame) -> None:
    logger.info("Received shutdown signal (%s).", sig)
    shutdown_flag.set()

async def main() -> None:
    if platform.system() != "Windows":
        for sig in (signal.SIGINT, signal.SIGTERM):
            signal.signal(sig, _signal_handler)

    try:
        logger.info("Connecting to Taskiq broker...")
        await broker.startup()
        logger.info("Worker is active and listening for tasks.")
        await shutdown_flag.wait()
    except Exception as exc:
        logger.error("Critical worker error: %s", exc)
    finally:
        await broker.shutdown()
        logger.info("Worker stopped.")

if __name__ == "__main__":
    if platform.system() == "Windows":
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
    asyncio.run(main())

===== run_local.py =====
"""Local launcher for NexusAI backend.

```

Examples:

```

python run_local.py
python run_local.py --bootstrap

```

```

python run_local.py --host 0.0.0.0 --port 8080
"""

from __future__ import annotations

import importlib
import os
import subprocess
import sys
from argparse import ArgumentParser, Namespace

REQUIRED_PACKAGES = ("uvicorn", "litestar", "sqlalchemy", "pydantic")
PACKAGE_TO_PIP_NAME = {
    "uvicorn": "uvicorn",
    "litestar": "litestar",
    "sqlalchemy": "sqlalchemy",
    "pydantic": "pydantic",
}

def _check_dependencies() -> list[str]:
    missing: list[str] = []
    for package in REQUIRED_PACKAGES:
        try:
            importlib.import_module(package)
        except ModuleNotFoundError:
            missing.append(package)
    return missing

def _install_dependencies(requirements_file: str = "requirements.txt") -> int:
    command = [sys.executable, "-m", "pip", "install", "-r", requirements_file]
    print("Instalowanie zale?no?ci:", " ".join(command))
    return subprocess.call(command)

def _build_parser() -> ArgumentParser:
    parser = ArgumentParser(description="Uruchamia lokalny backend NexusAI.")
    parser.add_argument(
        "--bootstrap",
        action="store_true",
        help="Automatycznie doinstaluj brakuj?ce zale?no?ci z requirements.txt.",
    )
    parser.add_argument("--host", default="127.0.0.1", help="Host backendu (domy?lnie: 127.0.0.1).")
    parser.add_argument("--port", default="8000", help="Port backendu (domy?lnie: 8000).")
    return parser

def main(argv: list[str] | None = None) -> int:
    args: Namespace = _build_parser().parse_args(argv)
    os.environ["NEXUS_HOST"] = args.host
    os.environ["NEXUS_PORT"] = str(args.port)

    missing = _check_dependencies()
    if missing:
        print("Brakuj?ce zale?no?ci:", " ".join(missing))
        if not args.bootstrap:
            package_list = " ".join(PACKAGE_TO_PIP_NAME[pkg] for pkg in missing if pkg in PACKAGE_TO_PIP_NAME)
            print("Uruchom jedn? z komend:")
            print(" 1) pip install -r requirements.txt")
            print(f" 2) pip install {package_list}")
            print("Albo u?yj: python run_local.py --bootstrap")
            return 1

    rc = _install_dependencies()

```

```

    if rc != 0:
        print("Nie uda?o si? zainstalowa? zale?no?ci.")
        return rc

    missing = _check_dependencies()
    if missing:
        print("Po instalacji nadal brakuje pakietw:", ", ".join(missing))
        return 1

    from api.server import run_backend

    run_backend()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

===== scripts_generate_py_pdf.py =====
from pathlib import Path

ROOT = Path('.')
OUT = ROOT / 'all_python_code.pdf'
py_files = sorted(p for p in ROOT.rglob('*.py') if '.git' not in p.parts)

content_lines = []
for p in py_files:
    rel = p.as_posix()
    content_lines.append(f"===== {rel} =====")
    try:
        text = p.read_text(encoding='utf-8')
    except UnicodeDecodeError:
        text = p.read_text(encoding='latin-1')
    content_lines.extend(text.splitlines())
    content_lines.append('')

# Very simple PDF writer using base14 Courier font
page_w, page_h = 612, 792
left, top, bottom = 40, 760, 40
font_size = 9
line_h = 11
max_lines = (top - bottom) // line_h

# escape PDF string chars
def esc(s: str) -> str:
    return s.replace('\\', '\\\\').replace('(', '\\(').replace(')', '\\)')

pages = []
for i in range(0, len(content_lines), max_lines):
    chunk = content_lines[i:i+max_lines]
    y = top
    cmds = ['BT', f'/F1 {font_size} Tf']
    for line in chunk:
        cmds.append(f'1 0 0 1 {left} {y} Tm ({esc(line)[:180]}) Tj')
        y -= line_h
    cmds.append('ET')
    pages.append('\n'.join(cmds).encode('latin-1', errors='replace'))

objects = []

# 1: Catalog, 2: Pages, 3.. page objs + streams + font
font_obj_num = 3 + len(pages)*2

objects.append(b'<< /Type /Catalog /Pages 2 0 R >>')
```

```

kids = ' '.join(f'{3+i*2} 0 R' for i in range(len(pages)))
objects.append(f'<< /Type /Pages /Count {len(pages)} /Kids [{kids}] >>'.encode())

for idx, stream in enumerate(pages):
    page_obj = f'<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {page_w} {page_h}] /Resources << /Font << /F1
    objects.append(page_obj.encode())
    cont = b'<< /Length ' + str(len(stream)).encode() + b' >>\nstream\n' + stream + b'\nendstream'
    objects.append(cont)

objects.append(b'<< /Type /Font /Subtype /Type1 /BaseFont /Courier >>')

pdf = bytearray(b'%PDF-1.4\n')
offsets = [0]
for i, obj in enumerate(objects, start=1):
    offsets.append(len(pdf))
    pdf.extend(f'{i} 0 obj\n'.encode())
    pdf.extend(obj)
    pdf.extend(b'\nendobj\n')

xref_pos = len(pdf)
pdf.extend(f'xref\n0 {len(objects)+1}\n'.encode())
pdf.extend(b'0000000000 65535 f \n')
for off in offsets[1:]:
    pdf.extend(f'{off:010d} 00000 n \n'.encode())
pdf.extend(f'trailer\n<< /Size {len(objects)+1} /Root 1 0 R >>\nstartxref\n{xref_pos}\nn%%EOF\n'.encode())

OUT.write_bytes(pdf)
print(f'Wrote {OUT} with {len(py_files)} files and {len(pages)} pages')

===== tests/test_autonomous_cfo_final_frontier.py =====
from __future__ import annotations

import sys
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

import asyncio
import uuid
from datetime import date, datetime, timezone
from types import SimpleNamespace

from Roboton_Reflekton.ledger_client import TigerBeetleClient
from Roboton_Reflekton.models import FinancialPeriodStatus
from Roboton_Reflekton.reconciliation_engine import BankReconciliationConfig, BankReconciliationEngine, Op
from Roboton_Reflekton.roboto_worker import RobotonWorker, SimpleRuleBasedAgent

class FakeSession:
    def __init__(self, period_status: FinancialPeriodStatus, open_period_id: str = "2026-04"):
        self.period_status = period_status
        self.open_period_id = open_period_id

    async def scalar(self, query):
        text = str(query)
        if "FROM financial_periods" in text and "period_id =" in text:
            return SimpleNamespace(period_id="2026-03", status=self.period_status)
        if "FROM financial_periods" in text and "status =" in text:
            return SimpleNamespace(period_id=self.open_period_id, status=FinancialPeriodStatus.OPEN)
        return SimpleNamespace(
            id=uuid.uuid4(),

```

```

        company_policy={},
        tigerbeetle_ledger_map={"401-01": 40101, "202": 20200},
    )

def add(self, _item):
    return None

async def commit(self):
    return None

async def refresh(self, _item):
    return None

def test_reconcile_bulk_payment_covers_three_invoices_and_posts_sub_10gr_rounding() -> None:
    async def run() -> None:
        tb = TigerBeetleClient()
        engine = BankReconciliationEngine(
            tb_client=tb,
            config=BankReconciliationConfig(
                account_bank_main=131,
                account_receivable=201,
                account_rounding_differences=760,
            ),
        )
        result = await engine.reconcile_bulk_payment(
            vendor_id="V-1",
            payment_amount_minor=12007,
            received_date=datetime.now(timezone.utc),
            open_invoices=[
                OpenInvoice(invoice_id="FV/1", amount_due_minor=5000, due_date=datetime(2026, 3, 1, tzinfo=timezone.utc)),
                OpenInvoice(invoice_id="FV/2", amount_due_minor=5000, due_date=datetime(2026, 3, 2, tzinfo=timezone.utc)),
                OpenInvoice(invoice_id="FV/3", amount_due_minor=2000, due_date=datetime(2026, 3, 3, tzinfo=timezone.utc)),
            ],
        )
        assert len(result["allocations"]) == 3
        assert result["rounding_adjustment_posted"] is True
        assert result["remaining_unallocated_minor"] == 0
        assert [line["invoice_id"] for line in result["allocations"]] == ["FV/1", "FV/2", "FV/3"]
        assert await tb.get_account_credits_posted(760) == 7

    asyncio.run(run())

def test_period_lock_mutates_posting_and_tax_point_for_hard_closed_period() -> None:
    async def run() -> None:
        worker = RobotonWorker(session=FakeSession(FinancialPeriodStatus.HARD_CLOSED), tb_client=TigerBeetleClient())
        event = {
            "company_id": str(uuid.uuid4()),
            "date_of_issue": "2026-03-17",
            "tax_point_date": "2026-03-17",
            "amount_minor": 1000,
            "source_document_id": str(uuid.uuid4()),
        }
        mutated = await worker._apply_financial_period_lock(uuid.UUID(event["company_id"]), event)
        assert mutated["posting_date"] == "2026-04-01"
        assert mutated["tax_point_date"] == "2026-04-01"
        assert mutated["metadata"]["late_submission_shifted"] is True

    asyncio.run(run())

def test_period_lock_keeps_dates_for_open_period() -> None:
    async def run() -> None:
        worker = RobotonWorker(session=FakeSession(FinancialPeriodStatus.OPEN), tb_client=TigerBeetleClient())

```

```

event = {
    "company_id": str(uuid.uuid4()),
    "date_of_issue": "2026-03-17",
    "tax_point_date": "2026-03-17",
    "amount_minor": 1000,
    "source_document_id": str(uuid.uuid4()),
}
mutated = await worker._apply_financial_period_lock(uuid.UUID(event["company_id"]), event)
assert mutated["posting_date"] == "2026-03-17"
assert mutated["tax_point_date"] == "2026-03-17"
assert "metadata" not in mutated or "late_submission_shifted" not in mutated.get("metadata", {})

asyncio.run(run())

===== tests/test_budgetary_control.py =====
from pathlib import Path
import sys
import asyncio
from datetime import date

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.budgetary_control import BudgetaryControlEngine
from Roboton_Reflekton.ledger_client import TigerBeetleClient

class FakeDuckDBManager:
    def __init__(self, rows):
        self.rows = rows

    def execute(self, query, parameters=None):
        if "SELECT limit_amount" in query:
            account_code, period = parameters
            return self.rows.get((account_code, period), [])
        return []

def test_get_budget_status_warn_and_critical() -> None:
    async def run() -> None:
        period = date(2026, 4, 1)
        db = FakeDuckDBManager({"402", period): [(1000.0, 0.85)]})
        tb = TigerBeetleClient()
        tb._account_credits_posted[402001] = 80000

        engine = BudgetaryControlEngine(db, tb, {"402": 402001})

        warn_status = await engine.get_budget_status("402", 100.0, month_period=period)
        assert warn_status.status == "WARN"
        assert round(warn_status.projected_usage_percent, 1) == 90.0

        critical_status = await engine.get_budget_status("402", 250.0, month_period=period)
        assert critical_status.status == "CRITICAL"
        assert round(critical_status.projected_usage_percent, 1) == 105.0

    asyncio.run(run())

def test_get_budget_status_ok_without_definition() -> None:
    async def run() -> None:
        db = FakeDuckDBManager({})
        tb = TigerBeetleClient()
        engine = BudgetaryControlEngine(db, tb, {"401": 401001})

        status = await engine.get_budget_status("401", 50.0, month_period=date(2026, 4, 1))
        assert status.status == "OK"

```

```

        assert "No budget configured" in status.message

    asyncio.run(run())

===== tests/test_cashflow_projection.py =====
from datetime import date
from pathlib import Path
import sys
import types

sys.path.append(str(Path(__file__).resolve().parents[1] / "Code"))

db_module = types.ModuleType("db")
analytics_module = types.ModuleType("db.analytics")
analytics_module.DuckDBManager = object
analytics_module.DuckDBLimits = object
db_module.analytics = analytics_module
sys.modules.setdefault("db", db_module)
sys.modules.setdefault("db.analytics", analytics_module)

import importlib.util

views_spec = importlib.util.spec_from_file_location("views_module", Path(__file__).resolve().parents[1] /
views_module = importlib.util.module_from_spec(views_spec)
assert views_spec and views_spec.loader
sys.modules["views_module"] = views_module
views_spec.loader.exec_module(views_module)
AnalyticsViewsSetup = views_module.AnalyticsViewsSetup

cfo_spec = importlib.util.spec_from_file_location("cfo_module", Path(__file__).resolve().parents[1] / "Cod
cfo_module = importlib.util.module_from_spec(cfo_spec)
assert cfo_spec and cfo_spec.loader
sys.modules["cfo_module"] = cfo_module
cfo_spec.loader.exec_module(cfo_module)
CashflowForecastService = cfo_module.CashflowForecastService

class FakeDuckDBManager:
    def __init__(self) -> None:
        self.queries: list[str] = []

    def execute(self, query: str):
        self.queries.append(query)
        return self

class FakeConn:
    def execute(self, query: str, params: tuple[date, date]):
        self.query = query
        self.params = params
        return self

    def fetchall(self):
        start = self.params[0]
        return [
            (start, "INFLOW", 100.0, "INVOICE_INFLOW"),
            (start, "OUTFLOW", 40.0, "INVOICE_OUTFLOW"),
        ]

def test_create_cashflow_projection_view_generates_expected_objects() -> None:
    db = FakeDuckDBManager()

    AnalyticsViewsSetup.create_cashflow_projection_view(db)

```

```

joined = "\n".join(db.queries)
assert "CREATE TABLE IF NOT EXISTS manual_cashflow_items" in joined
assert "CREATE OR REPLACE VIEW v_cashflow_projection" in joined
assert "vendor_scorecards" in joined

def test_get_daily_forecast_returns_running_balance_and_confidence() -> None:
    service = CashflowForecastService(FakeConn())

    result = service.get_daily_forecast(days=0, opening_balance=50.0)

    assert len(result) == 1
    day, balance, confidence = result[0]
    assert isinstance(day, date)
    assert balance == 110.0
    assert 0.8 <= confidence <= 0.9

===== tests/test_compliance_analytics.py =====
from __future__ import annotations

import importlib.util
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))

MODULE_PATH = ROOT / "Code" / "SERVICES" / "compliance_analytics.py"
spec = importlib.util.spec_from_file_location("compliance_analytics", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)

infer_ledger_entries = module.infer_ledger_entries
ensure_compliance_analytics_schema = module.ensure_compliance_analytics_schema

class FakeDuckDBManager:
    def __init__(self):
        self.queries = []

    def execute(self, query, parameters=None):
        self.queries.append((query, parameters))
        if "FROM transaction_rules" in query:
            return [("1100", "4100", "VAT_OUTPUT")]
        return []

def test_infer_ledger_entries_maps_rulebook_to_packet() -> None:
    db = FakeDuckDBManager()
    packet = infer_ledger_entries(
        db,
        {
            "transaction_type": "INVOICE_SALE",
            "amount": "120.50",
            "currency": "pln",
            "metadata_tags": {"project": "X"},
        },
    )

    assert packet.transaction_type == "INVOICE_SALE"
    assert packet.debit_account_code == "1100"
    assert packet.credit_account_code == "4100"

```

```

assert packet.tax_impact_code == "VAT_OUTPUT"
assert packet.amount_minor == 12050
assert packet.currency == "PLN"

def test_ensure_schema_executes_view_and_rulebook_setup() -> None:
    db = FakeDuckDBManager()
    ensure_compliance_analytics_schema(db)
    merged_sql = "\n".join(q for q, _ in db.queries)
    assert "CREATE TABLE IF NOT EXISTS gl_accounts" in merged_sql
    assert "CREATE TABLE IF NOT EXISTS transaction_rules" in merged_sql
    assert "CREATE OR REPLACE VIEW v_account_balances" in merged_sql

===== tests/test_dunning_engine.py =====
from pathlib import Path
import sys
sys.path.append(str(Path(__file__).resolve().parents[1]))

import asyncio

from Roboton_Reflekton.dunning_engine import DunningEngine

class FakeAIAgent:
    def generate_dunning_text(self, invoice_data, vendor_score, level):
        return f"L{level}:{invoice_data['invoice_number']}:{vendor_score}"

class FakeEmailProvider:
    def __init__(self):
        self.sent = []

    def send(self, *, to_email: str, subject: str, body: str) -> bool:
        self.sent.append((to_email, subject, body))
        return True

class FakeDuckDBManager:
    def __init__(self):
        self.history_inserts = []

    def execute(self, query, parameters=None):
        normalized = " ".join(query.split())
        if "FROM v_collectible_invoices" in normalized:
            return [("inv-1", "FV/1", "123", 150.0, "a@b.com", 0.9, 12)]
        if "INSERT INTO dunning_history" in normalized:
            self.history_inserts.append(parameters)
            return []
        return []

def test_run_daily_dunning_check_creates_history_and_sends_reminder() -> None:
    db = FakeDuckDBManager()
    email = FakeEmailProvider()
    engine = DunningEngine(db, FakeAIAgent(), email)

    result = asyncio.run(engine.run_daily_dunning_check())

    assert result == {"checked": 1, "sent": 1, "failed": 0}
    assert len(email.sent) == 1
    assert "FV/1" in email.sent[0][1]
    assert len(db.history_inserts) == 1
    assert db.history_inserts[0][1] == "inv-1"
    assert db.history_inserts[0][3] == 2
    assert db.history_inserts[0][4] == "SENT"

```

```

===== tests/test_fixed_assets_depreciation.py =====
from pathlib import Path
import sys
import asyncio
from datetime import date
from decimal import Decimal
import importlib.util
import types

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.ledger_client import TigerBeetleClient

db_module = types.ModuleType("db")
analytics_module = types.ModuleType("db.analytics")
analytics_module.DuckDBManager = object
sys.modules.setdefault("db", db_module)
sys.modules["db.analytics"] = analytics_module

fixed_assets_spec = importlib.util.spec_from_file_location(
    "fixed_assets_module",
    Path(__file__).resolve().parents[1] / "Code" / "SERVICES" / "fixed_assets.py",
)
fixed_assets_module = importlib.util.module_from_spec(fixed_assets_spec)
assert fixed_assets_spec and fixed_assets_spec.loader
sys.modules["fixed_assets_module"] = fixed_assets_module
fixed_assets_spec.loader.exec_module(fixed_assets_module)
FixedAssetsService = fixed_assets_module.FixedAssetsService

class FakeDuckDB:
    def __init__(self):
        self.asset = {
            "id": "11111111-1111-1111-1111-111111111111",
            "asset_name": "Laptop",
            "initial_value": Decimal("12000.00"),
            "salvage_value": Decimal("2000.00"),
            "depreciation_method": "LINEAR",
            "annual_rate": 0.2,
            "purchase_date": date(2026, 1, 15),
            "last_depreciation_date": None,
            "account_id_debit": 400,
            "account_id_credit": 700,
            "status": "ACTIVE",
        }
        self.schedule: list[dict] = []

    def execute(self, query, params=None):
        q = " ".join(query.split())
        if "FROM fixed_assets" in q and "status = 'ACTIVE'" in q and "WHERE id = ?" in q:
            if params[0] != self.asset["id"] or self.asset["status"] != "ACTIVE":
                return []
            a = self.asset
            return [
                a["id"], a["asset_name"], a["initial_value"], a["salvage_value"], a["depreciation_method"],
                a["annual_rate"], a["purchase_date"], a["last_depreciation_date"], a["account_id_debit"],
            ]
        if "DELETE FROM depreciation_schedule" in q:
            self.schedule = [s for s in self.schedule if s["asset_id"] != params[0] or s["is_posted"]]
            return []
        if q.startswith("SELECT COALESCE(SUM(amount), 0) FROM depreciation_schedule") and "is_posted = TRUE":
            total = sum(s["amount"] for s in self.schedule if s["asset_id"] == params[0] and s["is_posted"])
            return [(total,)]
        if "INSERT INTO depreciation_schedule" in q:

```

```

        self.schedule.append({
            "asset_id": params[0], "planned_date": params[1], "amount": Decimal(str(params[2])),
            "is_posted": params[3], "status": params[4], "posted_at": None,
        })
    return []
if "FROM depreciation_schedule ds JOIN fixed_assets fa" in q:
    as_of = params[0]
    out = []
    for s in self.schedule:
        if s["planned_date"] <= as_of and not s["is_posted"] and self.asset["status"] == "ACTIVE":
            out.append((s["asset_id"], s["planned_date"], s["amount"], self.asset["account_id_debi"]
        return sorted(out, key=lambda x: x[1])
if "UPDATE depreciation_schedule SET is_posted = TRUE" in q:
    for s in self.schedule:
        if s["asset_id"] == params[0] and s["planned_date"] == params[1]:
            s["is_posted"] = True
    return []
if "UPDATE fixed_assets SET last_depreciation_date" in q:
    self.asset["last_depreciation_date"] = params[0]
    return []
if "UPDATE fixed_assets SET status = CASE" in q:
    posted = sum(s["amount"] for s in self.schedule if s["asset_id"] == params[0] and s["is_posted"]
    if posted >= Decimal(str(params[1])):
        self.asset["status"] = "FULLY_DEPRECIATED"
    return []
raise AssertionError(f"Unhandled query: {q}")

```

```
def test_generate_schedule_starts_next_month_and_clears_last_month_delta():
```

```

    db = FakeDuckDB()
    service = FixedAssetsService(db, TigerBeetleClient())

    count = service.generate_schedule(db.asset["id"])

    assert count == 60
    assert db.schedule[0]["planned_date"] == date(2026, 2, 28)
    assert db.schedule[-1]["amount"] == Decimal("166.47")
    assert sum(s["amount"] for s in db.schedule) == Decimal("10000.00")

```

```
def test_execute_monthly_depreciation_marks_asset_as_fully_depreciated():
```

```

    async def run_case():
        db = FakeDuckDB()
        tb = TigerBeetleClient()
        service = FixedAssetsService(db, tb)
        service.generate_schedule(db.asset["id"])

        posted = await service.execute_monthly_depreciation(as_of=date(2032, 1, 1))

        assert posted == len(db.schedule)
        assert db.asset["status"] == "FULLY_DEPRECIATED"
        assert db.asset["last_depreciation_date"] == db.schedule[-1]["planned_date"]

    asyncio.run(run_case())

```

```
===== tests/test_forex_engine.py =====
```

```

from pathlib import Path
import sys
import asyncio
from datetime import date

```

```
sys.path.append(str(Path(__file__).resolve().parents[1]))
```

```

from Roboton_Reflekton.forex_engine import ForexEngine
from Roboton_Reflekton.ledger_client import TigerBeetleClient

```

```

class FakeResponse:
    def __init__(self, payload):
        self._payload = payload

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc, tb):
        return False

    def read(self):
        import json
        return json.dumps(self._payload).encode("utf-8")

class FakeDuckDB:
    def __init__(self):
        self.rate_cache = {}
        self.invoice_rows = {"inv-1": [("EUR", 100, 4.30)]}
        self.payment_rows = {"pay-1": [(4.40,)]}

    def execute(self, query, params=None):
        q = " ".join(query.split())
        if "SELECT avg_rate FROM exchange_rates" in q:
            return [(self.rate_cache[(params[0], params[1])],)] if (params[0], params[1]) in self.rate_cache else []
        if "INSERT OR REPLACE INTO exchange_rates" in q:
            self.rate_cache[(params[0], params[1])] = params[2]
            return []
        if "FROM invoices_fx" in q:
            return self.invoice_rows.get(params[0], [])
        if "FROM bank_transactions_fx" in q:
            return self.payment_rows.get(params[0], [])
        return []

def test_fetch_nbp_rate_uses_lookback_and_cache(monkeypatch):
    db = FakeDuckDB()
    tb = TigerBeetleClient()
    engine = ForexEngine(db, tb, 201, 750, 751)

    calls = {"n": 0}

    from urllib.error import HTTPError

    def fake_urlopen(url, timeout):
        calls["n"] += 1
        if "2026-04-27" in url:
            raise HTTPError(url, 404, "not found", hdrs=None, fp=None)
        return FakeResponse({"rates": [{"mid": 4.321, "no": "060/A/NBP/2026"}]})

    monkeypatch.setattr("Roboton_Reflekton.forex_engine.request.urlopen", fake_urlopen)
    rate = engine.fetch_nbp_rate(date(2026, 4, 27), "EUR")
    assert float(rate) == 4.321

    rate_cached = engine.fetch_nbp_rate(date(2026, 4, 26), "EUR")
    assert float(rate_cached) == 4.321
    assert calls["n"] >= 2

def test_fetch_nbp_rate_retries_on_url_error(monkeypatch):
    db = FakeDuckDB()

```

```

tb = TigerBeetleClient()
engine = ForexEngine(db, tb, 201, 750, 751)

from urllib.error import URLError

calls = {"n": 0}

def fake_urlopen(url, timeout):
    calls["n"] += 1
    if "2026-04-27" in url:
        raise URLError("temporary dns failure")
    return FakeResponse({"rates": [{"mid": 4.111, "no": "061/A/NBP/2026"}]})

monkeypatch.setattr("Roboton_Reflekton.forex_engine.request.urlopen", fake_urlopen)
rate = engine.fetch_nbp_rate(date(2026, 4, 27), "EUR")
assert float(rate) == 4.111
assert calls["n"] >= 2

def test_process_fx_settlement_creates_gain_transfer() -> None:
    async def run() -> None:
        db = FakeDuckDB()
        tb = TigerBeetleClient()
        engine = ForexEngine(db, tb, 201, 750, 751)

        result = await engine.process_fx_settlement("inv-1", "pay-1")
        assert result.direction == "GAIN"
        assert float(result.fx_diff_pln) == 10.0
        assert await tb.get_account_credits_posted(750) == 1000

    asyncio.run(run())

===== tests/test_fraud_graph_scanner.py =====

from pathlib import Path
import importlib.util
import sys

MODULE_PATH = Path(__file__).resolve().parents[1] / "Code" / "SERVICES" / "fraud_graph_scanner.py"
spec = importlib.util.spec_from_file_location("fraud_graph_scanner", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)
FraudGraphScanner = module.FraudGraphScanner

class FakeDuckDBManager:
    def __init__(self, rows):
        self.rows = rows

    def execute(self, query, parameters=None):
        return self.rows

def test_detect_shared_iban_between_employee_and_vendor() -> None:
    rows = [
        ("EMPLOYEE", "E-100", "PL0011223344", "Main Street 10"),
        ("VENDOR", "V-200", "PL0011223344", "Warehouse Ave 5"),
    ]
    scanner = FraudGraphScanner(FakeDuckDBManager(rows))
    alerts = scanner.detect_shared_identity_links()
    assert len(alerts) == 1
    alert = alerts[0]
    assert alert.rule_code == "GHOST_VENDOR_SHARED_IBAN"
    assert alert.severity == "CRITICAL"

```

```

def test_detect_shared_address_between_employee_and_vendor() -> None:
    rows = [
        ("EMPLOYEE", "E-101", "PL999", "ul. Kwiatowa 1"),
        ("VENDOR", "V-201", "PL123", "UL. KWIATOWA 1"),
    ]
    scanner = FraudGraphScanner(FakeDuckDBManager(rows))
    alerts = scanner.detect_shared_identity_links()
    assert len(alerts) == 1
    alert = alerts[0]
    assert alert.rule_code == "GHOST_VENDOR_SHARED_ADDRESS"
    assert alert.shared_value == "ul. kwiatowa 1"

def test_no_alert_when_no_shared_identity() -> None:
    rows = [
        ("EMPLOYEE", "E-102", "PL111", "Address A"),
        ("VENDOR", "V-202", "PL222", "Address B"),
    ]
    scanner = FraudGraphScanner(FakeDuckDBManager(rows))
    assert scanner.detect_shared_identity_links() == []

===== tests/test_fx_revaluation.py =====
from __future__ import annotations

import importlib.util
import sys
from decimal import Decimal
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))

MODULE_PATH = ROOT / "Code" / "SERVICES" / "fx_revaluation.py"
spec = importlib.util.spec_from_file_location("fx_revaluation", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)

post_realized_fx_difference = module.post_realized_fx_difference

def test_sales_positive_delta_is_gain() -> None:
    decision = post_realized_fx_difference(
        invoice_id="inv-1",
        invoice_type="SALES",
        payment_amount_foreign=Decimal("100"),
        exchange_rate_at_issue=Decimal("4.00"),
        payment_date_rate=Decimal("4.10"),
    )
    assert decision is not None
    assert decision.is_gain is True
    assert decision.account_code == "750_FX_Income"

def test_purchase_positive_delta_is_loss() -> None:
    decision = post_realized_fx_difference(
        invoice_id="inv-2",
        invoice_type="PURCHASE",
        payment_amount_foreign=Decimal("100"),
        exchange_rate_at_issue=Decimal("4.00"),
        payment_date_rate=Decimal("4.10"),
    )

```

```

    )
    assert decision is not None
    assert decision.is_gain is False
    assert decision.account_code == "751_FX_Expense"

def test_purchase_negative_delta_is_gain() -> None:
    decision = post_realized_fx_difference(
        invoice_id="inv-3",
        invoice_type="PURCHASE",
        payment_amount_foreign=Decimal("100"),
        exchange_rate_at_issue=Decimal("4.00"),
        payment_date_rate=Decimal("3.90"),
    )
    assert decision is not None
    assert decision.is_gain is True
    assert decision.account_code == "750_FX_Income"

===== tests/test_inventory_fifo.py =====
from datetime import date
from decimal import Decimal
from pathlib import Path
import importlib.util
import sys
import types

def _load_module(path: Path, module_name: str):
    spec = importlib.util.spec_from_file_location(module_name, path)
    module = importlib.util.module_from_spec(spec)
    assert spec and spec.loader
    sys.modules[module_name] = module
    spec.loader.exec_module(module)
    return module

fifo = _load_module(Path(__file__).resolve().parents[1] / "Code" / "SERVICES" / "inventory_fifo.py", "fifo")
InventoryBatch = fifo.InventoryBatch
calculate_fifo_cogs = fifo.calculate_fifo_cogs
apply_fifo_consumption = fifo.apply_fifo_consumption
InsufficientStockError = fifo.InsufficientStockError
calculate_and_post_cogs = fifo.calculate_and_post_cogs
DualWriteConsistencyError = fifo.DualWriteConsistencyError

def test_calculate_fifo_cogs_consumes_oldest_batches_first() -> None:
    batches = [
        InventoryBatch("b1", "SKU-1", date(2026, 1, 1), Decimal("3.0000"), Decimal("10.00")),
        InventoryBatch("b2", "SKU-1", date(2026, 1, 5), Decimal("5.0000"), Decimal("12.00")),
    ]

    result = calculate_fifo_cogs("SKU-1", Decimal("6.0000"), batches)

    assert result.total_cogs_net == Decimal("66.00")
    assert [(l.batch_id, l.qty_taken) for l in result.lines] == [
        ("b1", Decimal("3.0000")),
        ("b2", Decimal("3.0000")),
    ]

def test_apply_fifo_consumption_updates_remaining_quantities() -> None:
    batches = [
        InventoryBatch("b1", "SKU-1", date(2026, 1, 1), Decimal("3.0000"), Decimal("10.00")),
        InventoryBatch("b2", "SKU-1", date(2026, 1, 5), Decimal("5.0000"), Decimal("12.00")),
    ]

```

```

consumed = calculate_fifo_cogs("SKU-1", Decimal("6.0000"), batches)

updated = apply_fifo_consumption(batches, consumed)

assert updated[0].remaining_qty == Decimal("0.0000")
assert updated[1].remaining_qty == Decimal("2.0000")

def test_calculate_fifo_cogs_raises_on_insufficient_stock() -> None:
    batches = [InventoryBatch("b1", "SKU-1", date(2026, 1, 1), Decimal("1.0000"), Decimal("10.00"))]

    try:
        calculate_fifo_cogs("SKU-1", Decimal("2.0000"), batches)
        raise AssertionError("Expected InsufficientStockError")
    except InsufficientStockError as exc:
        assert "Insufficient inventory" in str(exc)

def test_inventory_schema_creates_batches_table() -> None:
    db_module = types.ModuleType("db")
    analytics_module = types.ModuleType("db.analytics")
    analytics_module.DuckDBManager = object
    db_module.analytics = analytics_module
    sys.modules.setdefault("db", db_module)
    sys.modules.setdefault("db.analytics", analytics_module)

    telemetry = types.ModuleType("services.telemetry")
    telemetry.ensure_telemetry_schema = lambda _: None
    sys.modules.setdefault("services.telemetry", telemetry)

    fingerprint = types.ModuleType("services.document_fingerprint")
    fingerprint.ensure_fingerprint_schema = lambda _: None
    sys.modules.setdefault("services.document_fingerprint", fingerprint)

    zpk = types.ModuleType("db.zpk_schema")
    zpk.ensure_zpk_schema = lambda _: None
    sys.modules.setdefault("db.zpk_schema", zpk)

    schema = _load_module(Path(__file__).resolve().parents[1] / "Code" / "DB" / "analytics_schema.py", "sc

class FakeMgr:
    def __init__(self):
        self.queries = []

    def execute(self, query: str):
        self.queries.append(query)
        return []

mgr = FakeMgr()
schema.ensure_inventory_schema(mgr)

sql = "\n".join(mgr.queries)
assert "CREATE TABLE IF NOT EXISTS inventory_batches" in sql
assert "CREATE TABLE IF NOT EXISTS inventory_consumption_lines" in sql

def test_calculate_and_post_cogs_rolls_back_duckdb_on_write_error() -> None:
    class FakeTB:
        async def create_two_phase_transfer(self, **kwargs):
            class Pending:
                pending_id = 1
            return Pending()

        async def post_pending_transfer(self, _pending_id):
            return True

```

```

class FailingDuckDBWriter:
    def __init__(self):
        self.events = []

    def begin(self):
        self.events.append("begin")

    def record_cogs_consumption(self, _consumption):
        raise RuntimeError("duckdb write failed")

    def commit(self):
        self.events.append("commit")

    def rollback(self):
        self.events.append("rollback")

async def run() -> None:
    batches = [InventoryBatch("b1", "SKU-1", date(2026, 1, 1), Decimal("1.0000"), Decimal("10.00"))]
    writer = FailingDuckDBWriter()
    try:
        await calculate_and_post_cogs(
            product_id="SKU-1",
            qty_sold=Decimal("1.0000"),
            open_batches=batches,
            tb_client=FakeTB(),
            debit_account_731_cogs=731,
            credit_account_330_inventory=330,
            source_document_id="doc-1",
            duckdb_writer=writer,
        )
        raise AssertionError("Expected DualWriteConsistencyError")
    except DualWriteConsistencyError:
        pass

    assert writer.events == ["begin", "rollback"]

import asyncio

asyncio.run(run())

===== tests/test_liquidity_oracle.py =====
from __future__ import annotations

import importlib.util
import sys
from datetime import date, timedelta
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))

MODULE_PATH = ROOT / "Code" / "SERVICES" / "liquidity_oracle.py"
spec = importlib.util.spec_from_file_location("liquidity_oracle", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)

calculate_liquidity_timeline = module.calculate_liquidity_timeline
ensure_liquidity_schema = module.ensure_liquidity_schema

class FakeTB:

```

```

def __init__(self):
    self._account_credits_posted = {1001: 1000000} # 10,000.00
    self._pending_transfers = {}

class FakeDuckDB:
    def __init__(self):
        self.queries = []

    def execute(self, query, parameters=None):
        self.queries.append((query, parameters))
        today = date.today()
        if "FROM invoices_replica i" in query:
            return [("V1", 3000.0, today + timedelta(days=2), 0.0, 1.0, 80)]
        if "type = 'PURCHASE'" in query:
            return [(1500.0, today + timedelta(days=1))]
        if "FROM recurring_commitments" in query:
            return [(1000.0, (today + timedelta(days=1)).day)]
        if "vat_liability" in query:
            return [(200.0,)]
        return []

def test_ensure_liquidity_schema_creates_view_and_table() -> None:
    db = FakeDuckDB()
    ensure_liquidity_schema(db)
    sql = "\n".join(q for q, _ in db.queries)
    assert "CREATE TABLE IF NOT EXISTS recurring_commitments" in sql
    assert "CREATE OR REPLACE VIEW v_payor_reliability_stats" in sql

def test_calculate_liquidity_timeline_returns_daily_projection() -> None:
    db = FakeDuckDB()
    tb = FakeTB()
    timeline = calculate_liquidity_timeline(db, tb, account_bank_id=1001, days_ahead=3)
    assert len(timeline) == 4
    assert timeline[0].optimistic_balance == "10000.00"
    assert float(timeline[-1].likely_balance) >= 0

===== tests/test_payment_priority_service.py =====
from datetime import date, timedelta
from pathlib import Path
import importlib.util
import sys

cfo_spec = importlib.util.spec_from_file_location("cfo_module", Path(__file__).resolve().parents[1] / "Cod
cfo_module = importlib.util.module_from_spec(cfo_spec)
assert cfo_spec and cfo_spec.loader
sys.modules["cfo_module"] = cfo_module
cfo_spec.loader.exec_module(cfo_module)
PaymentPriorityService = cfo_module.PaymentPriorityService

class FakeConn:
    def __init__(self, rows):
        self.rows = rows

    def execute(self, query: str):
        self.query = query
        return self

    def fetchall(self):
        return self.rows

```

```

def test_calculate_priority_score_skonto_overdue_and_critical() -> None:
    service = PaymentPriorityService(FakeConn([]))
    today = date(2026, 4, 30)
    invoice = {
        "due_date": today - timedelta(days=3),
        "skonto_deadline": today + timedelta(days=1),
        "vendor_priority": "critical",
        "penalty_rate": 12.5,
    }

    score, reasons = service.calculate_priority_score(invoice, today=today)

    assert score == 100
    assert any("Skonto" in reason for reason in reasons)
    assert any("Overdue" in reason for reason in reasons)
    assert any("Critical vendor" in reason for reason in reasons)

def test_suggest_payment_batch_respects_safe_limit_and_sorting() -> None:
    today = date(2026, 4, 30)
    rows = [
        ("inv-critical", today - timedelta(days=5), today + timedelta(days=1), 2.0, "critical", 12.0, 500.0),
        ("inv-normal", today + timedelta(days=7), None, None, "normal", 11.5, 300.0),
        ("inv-flex", today + timedelta(days=14), None, None, "flexible", 11.5, 200.0),
    ]
    service = PaymentPriorityService(FakeConn(rows))

    result = service.suggest_payment_batch(available_cash=1000.0, today=today)

    assert result["safe_limit"] == 900.0
    assert result["used_cash"] == 800.0
    assert [item["invoice_id"] for item in result["recommended_today"]] == ["inv-critical", "inv-normal"]
    assert [item["invoice_id"] for item in result["wait"]] == ["inv-flex"]

===== tests/test_perfect_accounting_architecture.py =====
from __future__ import annotations

import sys
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Code.ARCHITECTURE.perfect_accounting_architecture import (
    REQUIRED_TECHNOLOGIES,
    build_blueprint,
    validate_blueprint_coverage,
)

def test_blueprint_covers_all_required_technologies() -> None:
    blueprint = build_blueprint()
    ok, missing = validate_blueprint_coverage(blueprint)

    assert ok is True
    assert missing == set()

def test_blueprint_contains_rich_component_model() -> None:
    blueprint = build_blueprint()

    assert len(blueprint.components) >= 7
    assert REQUIRED_TECHNOLOGIES.issubset(blueprint.technology_names())
    assert len(blueprint.ocr_pipeline) >= 2
    assert len(blueprint.ml_pipeline) >= 3

```

```

===== tests/test_reconciliation_engine.py =====
from __future__ import annotations

import asyncio
import sys
import uuid
from datetime import datetime, timedelta, timezone
from pathlib import Path
from types import SimpleNamespace

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.ledger_client import TigerBeetleClient
from Roboton_Reflekton.models import TransferStatus
from Roboton_Reflekton.reconciliation_engine import AlertHub, ClearingAccountsConfig, ClearingAccountsEngine

class FakeResult:
    def __init__(self, rows):
        self._rows = rows

    def scalars(self):
        return self

    def all(self):
        return self._rows

class FakeSession:
    def __init__(self, company, transfers):
        self.company = company
        self.transfers = transfers
        self.committed = False

    async def scalar(self, _query):
        return self.company

    async def execute(self, _query):
        return FakeResult(self.transfers)

    async def commit(self):
        self.committed = True

class FakeSessionFactory:
    def __init__(self, session):
        self.session = session

    def __call__(self):
        return self

    async def __aenter__(self):
        return self.session

    async def __aexit__(self, exc_type, exc, tb):
        return False

def test_engine_uses_bank_transactions_subject_by_default() -> None:
    engine = ReconciliationEngine(session_factory=FakeSessionFactory(FakeSession(None, [])), tb_client=TigerBeetleClient)
    assert engine.subject == "bank.transactions.raw"

def test_matching_transaction_auto_confirms_pending_transfer() -> None:

```

```

async def run() -> None:
    company_id = uuid.uuid4()
    tb_client = TigerBeetleClient()
    pending = await tb_client.create_two_phase_transfer(
        debit_account=101,
        credit_account=202,
        amount_minor=12000,
        source_document_id=uuid.uuid4(),
    )
    await tb_client.post_pending_transfer(pending.pending_id)
    pending = await tb_client.create_two_phase_transfer(
        debit_account=101,
        credit_account=202,
        amount_minor=12000,
        source_document_id=uuid.uuid4(),
    )

    transfer = SimpleNamespace(
        target_account=202,
        amount_minor=12000,
        status=TransferStatus.PENDING,
        meta={"tb_pending_id": pending.pending_id, "classification": {"contractor_nip": "1234567890"}}
    )
    session = FakeSession(company=SimpleNamespace(id=company_id), transfers=[transfer])
    engine = ReconciliationEngine(
        session_factory=FakeSessionFactory(session),
        tb_client=tb_client,
        alert_hub=AlertHub(),
    )

    matched = await engine.process_bank_transaction(
        {
            "company_id": str(company_id),
            "amount_minor": 12000,
            "contractor_nip": "1234567890",
            "transaction_id": "tx-001",
            "posted_at": datetime.now(timezone.utc).isoformat(),
        }
    )

    assert matched is True
    assert transfer.status == TransferStatus.POSTED
    assert transfer.meta["reconciliation"]["status"] == "auto-confirmed"
    assert session.committed is True

asyncio.run(run())

```

```

def test_old_unmatched_transaction_emits_missing_invoice_alert() -> None:
    async def run() -> None:
        company_id = uuid.uuid4()
        alert_hub = AlertHub()
        session = FakeSession(company=SimpleNamespace(id=company_id), transfers=[])
        engine = ReconciliationEngine(
            session_factory=FakeSessionFactory(session),
            tb_client=TigerBeetleClient(),
            alert_hub=alert_hub,
        )

        async def next_alert() -> dict:
            async for event in alert_hub.subscribe():
                return event
            raise RuntimeError("stream closed")

        waiter = asyncio.create_task(next_alert())

```

```

await asyncio.sleep(0)

matched = await engine.process_bank_transaction(
    {
        "company_id": str(company_id),
        "amount_minor": 9999,
        "contractor_nip": "99999999999",
        "transaction_id": "tx-404",
        "posted_at": (datetime.now(timezone.utc) - timedelta(days=16)).isoformat(),
    }
)

event = await asyncio.wait_for(waiter, timeout=2)

assert matched is False
assert event["type"] == "MissingInvoiceAlert"
assert event["transaction_id"] == "tx-404"

asyncio.run(run())

def test_clearing_engine_processes_fees_with_idempotency() -> None:
    async def run() -> None:
        tb_client = TigerBeetleClient()
        engine = ClearingAccountsEngine(
            tb_client=tb_client,
            config=ClearingAccountsConfig(
                account_bank_main=100,
                account_expense_fees=402,
                account_receivable=201,
                provider_clearing_accounts={"stripe": 139001},
            ),
        )

        first = await engine.process_provider_fees(provider_id="stripe", fee_amount=200, operation_id="fee")
        second = await engine.process_provider_fees(provider_id="stripe", fee_amount=200, operation_id="fee")

        assert first["status"] == "posted"
        assert second["status"] == "idempotent-replay"
        assert await tb_client.get_account_credits_posted(139001) == 200

    asyncio.run(run())

def test_clearing_engine_reconciles_payout_with_idempotency() -> None:
    async def run() -> None:
        tb_client = TigerBeetleClient()
        engine = ClearingAccountsEngine(
            tb_client=tb_client,
            config=ClearingAccountsConfig(
                account_bank_main=131,
                account_expense_fees=402,
                account_receivable=201,
                provider_clearing_accounts={"terminal": 139002},
            ),
        )

        first = await engine.reconcile_bank_payout(provider_id="terminal", payout_amount=9800, operation_id="payout")
        second = await engine.reconcile_bank_payout(provider_id="terminal", payout_amount=9800, operation_id="payout")

        assert first["status"] == "posted"
        assert first["clearing_credits_posted"] == 9800
        assert second["status"] == "idempotent-replay"
        assert await tb_client.get_account_credits_posted(139002) == 9800

```

```

    asyncio.run(run())

===== tests/test_rules_engine.py =====
from __future__ import annotations

import asyncio
import importlib.util
import sys
import uuid
from decimal import Decimal
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))

MODULE_PATH = Path(__file__).resolve().parents[1] / "Code" / "SERVICES" / "rules_engine.py"
spec = importlib.util.spec_from_file_location("rules_engine", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)
RulesEngine = module.RulesEngine

from Roboton_Reflekton.ledger_client import TigerBeetleClient

class FakeDuckDBManager:
    def __init__(self, template_row=None):
        self.template_row = template_row

    def execute(self, query, parameters=None):
        if "FROM accounting_templates" in query:
            return [self.template_row] if self.template_row else []
        return []

def _proposal_balanced() -> dict:
    return {
        "template": "standard_sale",
        "source_document_id": uuid.uuid4(),
        "entries": [
            {"side": "DEBIT", "account": "201", "amount": "100.00", "tb_account_id": 201001},
            {"side": "CREDIT", "account": "701", "amount": "100.00", "tb_account_id": 701001},
        ],
    }

def test_validate_and_post_happy_path() -> None:
    async def run() -> None:
        db = FakeDuckDBManager(["201", "701"], "total_debit == total_credit")
        engine = RulesEngine(db, TigerBeetleClient())
        result = await engine.validate_and_post(_proposal_balanced())
        assert result["status"] == "VALIDATED"
        assert result["template"] == "standard_sale"
        assert result["total_debit"] == "100.00"
        assert len(result["pending_transfer_ids"]) == 1

    asyncio.run(run())

def test_validate_and_post_rejects_unbalanced() -> None:
    async def run() -> None:
        db = FakeDuckDBManager(["201", "701"], "total_debit == total_credit")
        engine = RulesEngine(db, TigerBeetleClient())

```

```

        proposal = _proposal_balanced()
        proposal["entries"][1]["amount"] = Decimal("99.99")
        try:
            await engine.validate_and_post(proposal)
            assert False, "Expected ValueError"
        except ValueError as exc:
            assert "Unbalanced booking" in str(exc)

    asyncio.run(run())

def test_validate_and_post_rejects_missing_required_account() -> None:
    async def run() -> None:
        db = FakeDuckDBManager(("["401"]", "["701"]", "total_debit == total_credit"))
        engine = RulesEngine(db, TigerBeetleClient())
        try:
            await engine.validate_and_post(_proposal_balanced())
            assert False, "Expected ValueError"
        except ValueError as exc:
            assert "Template mismatch" in str(exc)

    asyncio.run(run())

===== tests/test_run_local.py =====
from __future__ import annotations

import sys
import types
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

import run_local

def test_parser_supports_bootstrap_flag() -> None:
    args = run_local._build_parser().parse_args(["--bootstrap", "--host", "0.0.0.0", "--port", "9000"])
    assert args.bootstrap is True
    assert args.host == "0.0.0.0"
    assert args.port == "9000"

def test_main_returns_error_without_bootstrap_when_missing_dependencies(monkeypatch) -> None:
    monkeypatch.setattr(run_local, "_check_dependencies", lambda: ["uvicorn"])
    code = run_local.main([])
    assert code == 1

def test_main_bootstrap_path_installs_dependencies(monkeypatch) -> None:
    calls: list[str] = []

    def fake_check_dependencies() -> list[str]:
        calls.append("check")
        return ["uvicorn"] if len(calls) == 1 else []

    monkeypatch.setattr(run_local, "_check_dependencies", fake_check_dependencies)
    monkeypatch.setattr(run_local, "_install_dependencies", lambda: 0)

    fake_server = types.ModuleType("api.server")
    fake_server.run_backend = lambda: calls.append("run")
    monkeypatch.setitem(sys.modules, "api.server", fake_server)

    code = run_local.main(["--bootstrap"])

    assert code == 0

```

```

    assert calls == ["check", "check", "run"]

===== tests/test_shadow_ledger_contract.py =====
from __future__ import annotations

import ast
from pathlib import Path

def _parse(path: str) -> ast.Module:
    return ast.parse(Path(path).read_text(encoding="utf-8"))

def test_shadow_ledger_module_contains_tax_simulator() -> None:
    module = _parse("Roboton_Reflekton/shadow_ledger.py")
    class_names = {node.name for node in module.body if isinstance(node, ast.ClassDef)}
    assert "TaxSimulator" in class_names

def test_shadow_ledger_mentions_polars_and_duckdb() -> None:
    source = Path("Roboton_Reflekton/shadow_ledger.py").read_text(encoding="utf-8")
    assert "import polars as pl" in source
    assert "import duckdb" in source
    assert "run_shadow_simulation" in source

===== tests/test_smart_approvals.py =====
from __future__ import annotations

import importlib.util
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT))

MODULE_PATH = ROOT / "Code" / "SERVICES" / "smart_approvals.py"
spec = importlib.util.spec_from_file_location("smart_approvals", MODULE_PATH)
module = importlib.util.module_from_spec(spec)
assert spec.loader is not None
sys.modules[spec.name] = module
spec.loader.exec_module(module)

evaluate_approval_routing = module.evaluate_approval_routing
assert_auto_approved_or_block = module.assert_auto_approved_or_block

class FakeDuckDBManager:
    def __init__(self, invoice_row, history_count=5, avg=1000.0, sentinel_count=0):
        self.invoice_row = invoice_row
        self.history_count = history_count
        self.avg = avg
        self.sentinel_count = sentinel_count
        self.updated = None

    def execute(self, query, parameters=None):
        if "FROM invoices_replica\n          WHERE id" in query:
            return [self.invoice_row]
        if "COUNT(*) FROM invoices_replica" in query:
            return [(self.history_count,)]
        if "AVG(amount_gross)" in query:
            return [(self.avg,)]
        if "FROM audit_log" in query:
            return [(self.sentinel_count,)]
        if query.startswith("UPDATE invoices_replica"):

```

```

        self.updated = parameters
        return []
    return []

def test_auto_approved_when_score_high() -> None:
    db = FakeDuckDBManager(("inv-1", "V1", 1000.0), history_count=10, avg=1000.0, sentinel_count=0)
    decision = evaluate_approval_routing(db, "inv-1")
    assert decision.approval_status == "AUTO_APPROVED"
    assert decision.score == 100

def test_pending_human_when_new_vendor_and_high_amount() -> None:
    db = FakeDuckDBManager(("inv-2", "V2", 2000.0), history_count=0, avg=1000.0, sentinel_count=0)
    decision = evaluate_approval_routing(db, "inv-2")
    assert decision.approval_status == "PENDING_HUMAN"
    assert decision.score == 30

def test_blocker_raises_on_sentinel_flag() -> None:
    db = FakeDuckDBManager(("inv-3", "V3", 1000.0), history_count=10, avg=1000.0, sentinel_count=1)
    try:
        assert_auto_approved_or_block(db, "inv-3")
        assert False, "Expected PermissionError"
    except PermissionError as exc:
        assert "blocked by maker-checker" in str(exc)

===== tests/test_startup_contract.py =====
from __future__ import annotations

import ast
from pathlib import Path

def _parse(path: str) -> ast.Module:
    return ast.parse(Path(path).read_text(encoding="utf-8"))

def test_single_create_app_factory_exists() -> None:
    module = _parse("Code/API/app.py")
    create_app_defs = [n for n in module.body if isinstance(n, ast.FunctionDef) and n.name == "create_app"]
    assert len(create_app_defs) == 1

def test_server_uses_unified_create_app() -> None:
    module = _parse("Code/API/server.py")
    source = Path("Code/API/server.py").read_text(encoding="utf-8")
    assert "from api.app import create_app" in source
    assert "uvicorn.run(" in source

def test_v1_and_v2_paths_defined() -> None:
    source = Path("Code/API/app.py").read_text(encoding="utf-8")
    assert "/api/v2/health" in source
    assert "InvoiceController" in source

===== tests/test_storno_and_edge_cases.py =====
from __future__ import annotations

import asyncio
import importlib.util
import sys
import uuid
from datetime import date
from decimal import Decimal

```

```

from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.audit_storno import LedgerTransferRecord, decimal_to_minor_units, reverse_transaction
from Roboton_Reflekton.ledger_client import TigerBeetleClient

def _load_module(path: Path, module_name: str):
    spec = importlib.util.spec_from_file_location(module_name, path)
    module = importlib.util.module_from_spec(spec)
    assert spec and spec.loader
    sys.modules[module_name] = module
    spec.loader.exec_module(module)
    return module

fifo = _load_module(Path(__file__).resolve().parents[1] / "Code" / "SERVICES" / "inventory_fifo.py", "fifo")
InventoryBatch = fifo.InventoryBatch
InsufficientStockError = fifo.InsufficientStockError
calculate_fifo_cogs = fifo.calculate_fifo_cogs

class FakeDuckDBWriter:
    def __init__(self) -> None:
        self.voided: list[str] = []
        self.drafts: list[str] = []
        self.tx_events: list[str] = []

    def begin(self) -> None:
        self.tx_events.append("begin")

    def commit(self) -> None:
        self.tx_events.append("commit")

    def rollback(self) -> None:
        self.tx_events.append("rollback")

    def mark_invoice_voided(self, invoice_id: str) -> None:
        self.voided.append(invoice_id)

    def create_draft_from_invoice(self, invoice_id: str) -> str:
        draft = f"draft-{invoice_id}"
        self.drafts.append(draft)
        return draft

def test_last_unit_inventory_succeeds_and_extra_unit_fails() -> None:
    batches = [InventoryBatch("b1", "SKU-X", date(2026, 2, 1), Decimal("1.0000"), Decimal("99.99"))]
    ok = calculate_fifo_cogs("SKU-X", Decimal("1.0000"), batches)
    assert ok.total_cogs_net == Decimal("99.99")

    try:
        calculate_fifo_cogs("SKU-X", Decimal("1.0001"), batches)
        raise AssertionError("Expected InsufficientStockError")
    except InsufficientStockError:
        pass

def test_currency_invoice_correction_storno_links_original_transfer() -> None:
    async def run() -> None:
        tb = TigerBeetleClient()
        writer = FakeDuckDBWriter()
        amount_minor = decimal_to_minor_units(Decimal("120.05"))
        original = LedgerTransferRecord(

```

```

        transfer_id=987654321,
        debit_account=201,
        credit_account=700,
        amount_minor=amount_minor,
        currency="PLN",
        source_document_id=uuid.uuid4(),
    )
    result = await reverse_transaction(tb_client=tb, duckdb_writer=writer, original_transfer=original,
    assert result["status"] == "reversed"
    assert result["user_data_128"] == str(original.transfer_id)
    assert writer.voided == ["INV-FX-1"]
    assert writer.drafts == ["draft-INV-FX-1"]
    assert writer.tx_events == ["begin", "commit"]

    asyncio.run(run())

===== tests/test_vat_reconciliation.py =====
from pathlib import Path
import sys
import asyncio

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.ledger_client import TigerBeetleClient
from Roboton_Reflekton.vat_reconciliation import VATReconciliationEngine

class FakeDuckDBManager:
    def __init__(self):
        self.tax_rates = set()

    def execute(self, query, params=None):
        q = " ".join(query.split())
        if "INSERT OR IGNORE INTO tax_rates" in q:
            self.tax_rates.add(params[0])
            return []
        if "FROM invoices_vat" in q:
            return [(23.0, 46.0)]
        if "FROM vat_reconciliation_candidates" in q:
            return [("DUCKDB_ONLY", "inv-a"), ("LEDGER_ONLY", "inv-b")]
        return []

def test_check_vat_integrity_detects_mismatch() -> None:
    engine = VATReconciliationEngine(FakeDuckDBManager(), TigerBeetleClient(), 221001, 222001)

    result = engine.check_vat_integrity(
        "inv-1",
        {
            "total_gross": "123.00",
            "vat_breakdown": [
                {"rate": "23", "net_amount": "100.00", "vat_amount": "22.00", "gross_amount": "122.00"}
            ],
        },
    )
    assert result.status == "FAILED"
    assert "VAT_MISMATCH:23" in result.errors
    assert "MATH_ERROR_TOTAL" in result.errors

def test_reconcile_with_tigerbeetle_returns_alert() -> None:
    async def run() -> None:
        tb = TigerBeetleClient()
        tb._account_credits_posted[221001] = 2300
        tb._account_credits_posted[222001] = 4500

```

```

engine = VATReconciliationEngine(FakeDuckDBManager(), tb, 221001, 222001)
alert = await engine.reconcile_with_tigerbeetle()

assert alert.status == "ALERT"
assert alert.vat_in_ledger == 23
assert alert.vat_out_ledger == 45
assert alert.vat_in_register == 23
assert alert.vat_out_register == 46
assert alert.missing_in_ledger == ["inv-a"]
assert alert.missing_in_duckdb == ["inv-b"]

asyncio.run(run())

===== tests/test_vision_agent.py =====
from __future__ import annotations

import asyncio
import sys
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

from Roboton_Reflekton.vision_agent import VisionAgent

def test_vision_agent_prompt_has_required_json_schema() -> None:
    assert "vendor_nip" in VisionAgent.PROMPT
    assert "visual_anomalies_detected" in VisionAgent.PROMPT
    assert "handwritten_notes_summary" in VisionAgent.PROMPT

def test_vision_agent_fallback_extract_core_fields_from_ocr_text() -> None:
    raw_text = (
        "Faktura VAT\n"
        "NIP 1234567890\n"
        "Razem brutto: 1 250,99 PLN\n"
        "VAT 23%\n"
        "Zapłacono przelewem\n"
        "odroczone adnotacja: zaliczka 200"
    )

    extraction = VisionAgent._fallback_from_ocr(raw_text)

    assert extraction.vendor_nip == "1234567890"
    assert extraction.total_gross == 1250.99
    assert extraction.vat_rate == 23.0
    assert extraction.payment_status == "paid"
    assert extraction.handwritten_notes_summary != ""

def test_vision_agent_analyze_uses_fallback_when_runtime_is_unavailable() -> None:
    async def run() -> None:
        agent = VisionAgent()
        extraction = await agent.analyze(Path("dummy.png"), "NIP 1111111111\nBrutto 99,00")
        assert extraction.vendor_nip == "1111111111"
        assert extraction.total_gross == 99.0
        assert extraction.source in {"ocr-fallback", "qwen2.5-vl-2b-4bit"}

    asyncio.run(run())

```